Embedded Linker

ELD Documentation

A detailed guide to linking, diagnostics, plugins, linker-scripts, etc.

Qualcomm Technologies, Inc.

Table of Contents

ELD User Guide

This document describes usage of ELD

1. Getting started

1.1. About the linker

Linking is the process of collecting and combining various pieces of code and data into a single file that can be loaded (copied) into memory and executed. Linking is performed automatically by programs called linkers, which enable separate compilation.

1.2. Source

eld is available in the git repository:

Note
git clone git://github.com/qualcomm/eld

1.3. Summary of the linker features

- Most important features supported by GNU Linker : - Development going on to add additional support
- LTO : - Supports ThinLTO
  - Supports Full LTO
  - Supports Both flavors of LTO with linker scripts
- Supports Version scripts : - Supports extensive usage of Linker scripts
- Better Diagnostics : - YAML Map Files
  - Easier to read Text Map files
- Support for Linker Plugins
- –reproduce flag to easily reproduce the linking step.

1.4. What the linker can accept as input

The following types of files can be used as input to ld.eld:

- Object files
- Static Libraries
- Shared Libraries
- Symbol definition file
- Extern list
- Binary files

1.5. What the linker outputs

- ELF File
- A partially linked ELF object that can be used as input in a subsequent link step.
- Map file : - YAML Map
  - Text Map
- Tar file : - when using –reproduce flag

2. Supported Linking Modes

A linking model is a group of command-line options and memory maps that control the behavior of the linker. The linking models supported by eld are:

2.1. Static Linking

In this mode, all the symbol definitions/functions are copied in the final executable.

2.2. Dynamic Linking

In Dynamic linking is accomplished by placing the name of a sharable library in the executable image. Actual linking with the library routines does not occur until the image is run, when both the executable and the library are placed in memory. An advantage of dynamic linking is that multiple programs can share a single copy of the library.

2.3. Partial Linking

This model produces a relocatable ELF object suitable for input to the linker in a subsequent link step. The partial object can be used as input to another link step. The linker performs limited processing of input objects to produce a single output object

2.4. PIE

This model produces a Position Independent Executable (PIE). This is an executable that does not need to be executed at a specific address but can be executed at any suitably aligned address. All objects and libraries linked into the image must be compiled to be position independent

2.5. Shared Libraries

A shared library is an object module that, at run time, can be loaded at an arbitrary memory address and linked with a program in memory. The process is known as dynamic linking and is performed by a program called a dynamic linker. Shared libraries are also referred to as shared objects (.so).

Shared libraries are “shared” in two different ways:

there is exactly one .so file for a particular library. The code and data in this .so file are shared by all of the exe obj files that reference the library. a single copy of the .text section of a shared library in memory can be shared by different running processes.

3. Supported Targets

Below are the currently supported targets by the ELD
- Hexagon
- RISC-V
- ARM
- AARCH64

4. Linker Script

4.1. Overview

Linker scripts define how input sections (like .text, .data, .bss) are mapped to output sections and where they are placed in memory. These scripts control:
- Memory regions (e.g., RAM, ROM)
- Section alignment
- Ordering of code and data
- Load vs. execution addresses

This is the standard method used by most linkers like GNU ld.

Linker scripts provide detailed specifications of how files are to be linked. They offer greater control over linking than is available using just the linker command options.

NOTE Linker scripts are optional. In most cases, the default behavior of the linker is sufficient.

Linker scripts control the following properties:

- ELF program header
- Program entry point
- Input and output files and searching paths
- Section memory placement and runtime
- Section removal
- Symbol definition

A linker script consists of a sequence of commands stored in a text file.

The script file can be specified on the command line either with -T, or by specifying the file as an input files.

The linker distinguishes between script files and object files and handles each accordingly.

To generate a map file that shows how a linker script controlled linking, use the M option.

Command	Description
PHDRS	Program Headers
SECTIONS	Section mapping and memory placementELF program header definition
MEMORY	Define memory regions used by >REGION and AT>REGION placement
ENTRY	ELF program headerProgram execution entry point
OUTPUT_FORMAT	Parsed, but no effect on linking
OUTPUT_ARCH	Parsed, but no effect on linking
SEARCH_DIR	Add additional searching directory for libraries
INCLUDE	Include linker script file
OUTPUT	Define output filename
GROUP	Define files that will be searched repeatedly
ASSERT	Linker script assertion
NOCROSSREFS	Check cross references among a group of sections
OVERLAY	GNU ld compatible OVERLAY block (parsing support)

4.2. Basic script syntax

4.2.1. Symbols

Symbol names must begin with a letter, underscore, or period. They can include letters, numbers, underscores, hyphens, or periods.

4.2.2. Comments

Comments can appear in linker scripts.

4.2.3. Strings

Character strings can be specified as parameters with or without delimiter characters.

4.2.4. Expression basics

Expressions are similar to C, and support all C arithmetic operators. They are evaluated as type `long` or `unsigned long`.

4.2.5. Location counter basics

A period is used as a symbol to indicate the current location counter. It is used in the `SECTIONS` command only, where it designates locations in the output section:

```
. = ALIGN(0x1000);
. = . + 0x1000;
```

Assigning a value to the location counter symbol changes the location counter to the specified value. The location counter can be moved forward by arbitrary amounts to create gaps in an output section. It cannot, however, be moved backwards.

4.2.6. Symbol assignment basics

Symbols, including the location counter, can be assigned constants or expressions:

```
__text_start = . + 0x1000;
```

Assignment statements are similar to C, and support all C assignment operators. Terminate assignment statements with a semicolon.

4.3. Script commands

The `SECTIONS` command must be specified in a linker script. All the other script commands are optional.

4.3.1. PHDRS

The `PHDRS` (Program Headers) command in a linker script is used to define program headers in the output binary, particularly for ELF Executable and Linkable Format (ELF) files. These headers are essential for the runtime loader to understand how to load and map the binary into memory.

When and why PHDRS is used?

- Custom memory mapping You use `PHDRS` when you want to explicitly control how sections are grouped into segments in the ELF file. This is especially important for:
  - Embedded systems
  - Custom bootloaders
  - OS kernels
- Fine-grained segment control
  - Assign specific sections to specific segments.
  - Control segment flags (for example, `PT_LOAD`, `PT_NOTE`, `PT_TLS`).
  - Set permissions (`r`, `w`, `x`) for each segment.

Syntax :- `{ name type [FILEHDR][PHDRS][AT (address)][FLAGS (flags)] }`

The `PHDRS` script command sets information in the program headers, also known as the *segment header* of an ELF output file.

- `name` – Specifies the program header in the `SECTIONS` command.
- `type` – Specifies the program header type.
- `PT_LOAD` – Loadable segment.
- `PT_NULL` – Linker does not include section in a segment. No loadable section should be set to `PT_NULL`.
- `PT_DYNAMIC` – Segment where dynamic linking information is stored.
- `PT_INTERP` – Segment where the name of the dynamic linker is stored.
- `PT_NOTE` – Segment where note information is stored.
- `PT_SHLIB` – Reserved program header type.
- `PT_PHDR` – Segment where program headers are stored.
- `FLAGS` – Specifies the `p_flags` field in the program header. The value of flags must be an integer. It is used to set the `p_flags` field of the program header; for instance, `FLAGS(5)` sets `p_flags` to `PF_R | PF_X` and `FLAGS(0x03000000)` sets OS-specific flags.

Note

If the sections in an output file have different flag settings than what is specified in `PHDRS`, the linker combines the two different flags using bitwise OR.

4.3.2. MEMORY

The `MEMORY` command defines named memory regions (for example `FLASH` and `RAM`) that can be used to place output sections without hard-coding absolute addresses.

In ELD, memory regions can be used in three related ways:

1. **VMA placement**: `>REGION` assigns the output section’s VMA to the next available address in `REGION` (starting from `ORIGIN` and advancing by the output section size).
2. **LMA placement**: `AT>REGION` assigns the output section’s LMA to the next available address in `REGION` (tracked independently from VMA placement).
3. **Automatic region selection**: if an allocatable output section does not specify `>REGION`, ELD can select a region by matching the section’s flags against region attributes (for example, place writable sections in a `(w)` region and code in an `(x)` region).

4.3.2.1. Syntax

```
MEMORY {
  <name> [(<attrs>)] : ORIGIN = <expr> , LENGTH = <expr>
  <name> [(<attrs>)] : org = <expr> , len = <expr>
  ...
}
```

Notes:

- `ORIGIN` can also be spelled as `org` or `o`.
- `LENGTH` can also be spelled as `len` or `l`.
- `ORIGIN` and `LENGTH` must be separate tokens (for example, `ORIGIN = 0x1000` is accepted, but `ORIGIN= 0x1000` is rejected).

- The expressions are normal linker-script expressions; you can use constants and arithmetic, and size suffixes like `K`, `M`, and `G` (for example, `LENGTH = 4K`).
- The `MEMORY` block can contain `INCLUDE`/`INCLUDE_OPTIONAL` directives.

4.3.2.2. Memory region names

Region names are identifiers (and may also appear quoted in scripts). Region names are referenced in output section descriptions via `>REGION` and `AT>REGION`.

If a section references an unknown region name, ELD errors out (for example, `Cannot find memory region <name>`).

4.3.2.3. Memory region attributes

`(<attrs>)` is optional and is primarily used for **automatic region selection** (when an allocatable output section does not specify an explicit `>REGION`).

ELD supports the following attribute letters:

- `w`: writable (matches `SHF_WRITE` sections).
- `x`: executable (matches `SHF_EXECINSTR` sections).
- `a`: allocatable (matches `SHF_ALLOC` sections).
- `r`: read-only (matches *non-writable* allocatable sections; ELF sections do not explicitly carry a read flag).
- `i` / `l`: initialized (matches `SHT_PROGBITS` sections; used to avoid matching `SHT_NOBITS` when desired).
- `!`: negation operator, used to say that attributes must **not** be present. For example, `(r!x)` matches non-writable, non-executable sections.

The attribute matching rules apply only to allocatable output sections that do not already have an explicit `>REGION`.

> **Important**
> If you explicitly assign a section to a region using `>REGION`, ELD does not reject the assignment even if the section's flags do not match the region's attributes. Attributes are for auto-selection, not enforcement.

4.3.2.4. Automatic region selection (orphans and missing `>REGION`)

When a linker script uses `MEMORY` and an allocatable output section is not explicitly assigned to a region, ELD attempts to assign it to the first memory region (in `MEMORY` order) whose attributes match the section's flags.

This includes *orphan output sections* (output sections created implicitly because some input sections were not matched by any rule in `SECTIONS`).

If no memory region matches an allocatable output section, ELD errors out (for example, `Error: No memory region assigned to section .data`).

> **Note**
> This behavior can differ from GNU ld and LLD for orphans. For example, some linkers may keep orphan sections in the "current" region/cursor even when region attributes suggest a different region. ELD follows the region attribute matching rules for orphans and other unassigned allocatable output sections. See https://github.com/qualcomm/eld/issues/127 for a concrete example and background.

Common recommendation: if the placement of a section matters (for example, `.rodata`, `.eh_frame`, `.tbss`), add an explicit output section rule and assign it to the intended region rather than relying on orphan heuristics.

4.3.2.5. Interaction with explicit addresses (VMA overrides)

If an output section has an explicit VMA (for example `.text 0x2000 : { ... }` or via `--section-start`), the explicit VMA is honored.

If that output section also has `>REGION`:

- If the explicit VMA lies **within** the region's `[ORIGIN, ORIGIN+LENGTH]` range, the section is still accounted against the region for memory usage checks/reporting.
- If the explicit VMA lies **outside** the region range, the section is not charged against that region (the explicit address is treated as an override).

This is intended to support scripts that use regions as a default placement mechanism while still allowing some sections to be placed at fixed addresses.

4.3.2.6. TLS and `.tbss`

ELD treats TLS `SHT_NOBITS` sections (for example `.tbss`) specially for `MEMORY` region cursors: `.tbss` does not advance the region cursor, so a subsequent non-TLS output section may end up with the same VMA. If your script requires non-overlapping VMAs for TLS and non-TLS sections, place TLS output sections explicitly (for example, in a dedicated region or at a dedicated address range) rather than relying on region cursors.

4.3.2.7. LMA regions and `AT>REGION`

Use `AT>REGION` on an output section description to place the section's **load memory address (LMA)** in a region, independent of its VMA placement:

```
.text : { *(.text*) } >RAM AT>FLASH
```

Rules:
- `AT(<address>)` and `AT>REGION` are mutually exclusive; specifying both is an error.
- If a section has a VMA region (`>REGION`) but no explicit LMA control, ELD defaults the LMA region to the same region as the VMA region.
- If a section has explicit LMA control (`AT(<address>)`), ELD does not automatically align the LMA. (See Controlling Physical addresses.)
- `NOLOAD`/`SHT_NOBITS` sections do not occupy file space, so they are ignored by load-address overlap checks.

4.3.2.8. Builtins: `ORIGIN()` and `LENGTH()`

ELD supports GNU-compatible `ORIGIN(<region>)` and `LENGTH(<region>)` builtins in linker script expressions.

These functions accept a memory region name or a region alias and evaluate to the region's origin and length, respectively. If the region name cannot be resolved, ELD errors out.

4.3.2.9. REGION_ALIAS

`REGION_ALIAS("alias", "region")` defines an alias name for an existing memory region. Aliases can be used anywhere a region name is expected (for example, `>alias` or `ORIGIN(alias)`).

ELD restrictions and diagnostics:
- Creating two aliases with the same alias name is an error.
- An alias name must not collide with a real memory region name.

4.3.2.10. Memory usage checking and reporting

When `MEMORY` is used, ELD tracks memory usage per region during layout:
- If the total size placed in a region exceeds `LENGTH`, ELD errors out and reports the overflow amount per output section.
- With `-Wlinker-script-memory`, ELD can warn on zero-length regions and on regions that end up with zero used size after layout.

Additionally:
- The text map format (`-MapStyle txt`) includes a `# MEMORY` block that prints the parsed `MEMORY` regions and annotates each output section line with `Memory : [<VMARegion>, <LMARegion>]` when available.
- `--print-memory-usage` prints a summary table of used size and percentage per memory region when a `MEMORY` directive is present.

4.3.2.11. PHDRS interaction and program header loading

`MEMORY` controls **where sections go** (VMA via `>REGION` and LMA via `AT>REGION`). `PHDRS` controls **which segments exist** and which output sections are assigned to those segments (via `:phdr` on output section descriptions).

Key points:
- If you do not specify `PHDRS`, ELD creates segments using its default heuristics. A change in memory region (for example, `>FLASH` to `>RAM`) can force ELD to start a new `PT_LOAD` segment.
- If you specify `PHDRS`, ELD only creates the program headers declared in the script. In this mode, region changes do *not* create extra segments; you must assign each output section to the intended segment(s) using `:phdr`.
- Runtime loaders use the ELF program headers (and sometimes the file header) to decide what to map or copy. If your loader needs the file header and/or program headers to be part of the first loadable segment, use `FILEHDR` and `PHDRS` keywords on the corresponding `PT_LOAD` in the `PHDRS` command.

Example (first segment includes file+program headers):

```
PHDRS {
  text PT_LOAD FILEHDR PHDRS;
  data PT_LOAD;
}
```

If you include headers in a loadable segment, you typically must also reserve space for them in the image layout (see Using SIZEOF_HEADERS with MEMORY).

4.3.2.12. Using SIZEOF_HEADERS with MEMORY

`SIZEOF_HEADERS` evaluates to the total size (in bytes) of the ELF file header and program header table that ELD emits for the output.

Nuance: in ELD, using `SIZEOF_HEADERS` *before the first output section is seen* also acts as a signal that the script is intentionally accounting for headers; this impacts whether ELD considers the headers as occupying space at the start of the image when a linker script is present.

4.3.2.13. Common MEMORY pitfalls and errors

- **Tokenization**: `ORIGIN`/`LENGTH` must be separate tokens (`ORIGIN = ...`); scripts like `ORIGIN= 0x1000` are rejected.
- **Unknown region name**: `>REGION` or `AT>REGION` refers to a region not defined in `MEMORY` (error: `Cannot find memory region ...`).
- **No region assigned**: if `MEMORY` is present and an allocatable output section does not have an explicit region and does not match any region attributes (error: `No memory region assigned to section ...`).
- **Overflow**: placed size exceeds `LENGTH` (error: `Memory region <name> exceeded limit ... overflowed by ...`). Remember to account for alignment, page alignment, and headers if `FILEHDR PHDRS` are used.
- **Mixing ``AT(address)`` with ``AT>REGION``**: these are mutually exclusive. If you need a fixed LMA base but still want region-based placement for the rest of the image, prefer shifting the region origin using expressions (see Using SIZEOF_HEADERS with MEMORY) or keep all LMAs explicit.
- **Orphan placement surprises**: orphan output sections and other unassigned allocatable sections are auto-assigned using region attributes; if a specific section must be in a particular region, add an explicit output section rule.

4.3.3. SECTIONS

Syntax :- `SECTIONS { section_statement section_statement ... }`

The `SECTIONS` script command specifies how input sections are mapped to output sections, and where output sections are located in memory. The `SECTIONS` command must be specified once, and only once, in a linker script.

4.3.3.1. Section statements

A `SECTIONS` command contains one or more section statements, each of which can be one of the following:
- An `ENTRY` command.
- A symbol assignment statement to set the location counter. The location counter specifies the default address in subsequent section-mapping statements that do not explicitly specify an address.
- An output section description to specify one or more input sections in one or more library files, and map those sections to an output section. The virtual memory address of the output section can be specified using attribute keywords.

4.3.4. ENTRY

Syntax :- `ENTRY(symbol)`
- The `ENTRY` script command specifies the program execution entry point.
- The entry point is the first instruction that is executed after a program is loaded.
- This command is equivalent to the linker command-line option `-e`.

4.3.5. OUTPUT_FORMAT

Syntax :- `OUTPUT_FORMAT(string)`
- The `OUTPUT_FORMAT` script command specifies the output file properties.
- For compatibility with the GNU linker, this command is parsed but has no effect on linking.

4.3.6. OUTPUT_ARCH

Syntax :- `OUTPUT_ARCH("aarch64")`

- The `OUTPUT_ARCH` script command specifies the target processor architecture.
- For compatibility with the GNU linker, this command is parsed but has no effect on linking.

4.3.7. SEARCH_DIR

Syntax :- `SEARCH_DIR(path)`

- The `SEARCH_DIR` script command adds the specified path to the list of paths that the linker uses to search for libraries.
- This command is equivalent to the linker command-line option `-L`.

4.3.8. INCLUDE

Syntax :- `INCLUDE(file)`

- The `INCLUDE` script command specifies the contents of the text file at the current location in the linker script.
- The specified file is searched for in the current directory and any directory that the linker uses to search for libraries.

Note
Include files can be nested.

4.3.9. OUTPUT

Syntax :- `OUTPUT(file)`

- The `OUTPUT` script command defines the location and file where the linker will write output data.
- Only one output is allowed per linking.

4.3.10. GROUP

Syntax :- `GROUP(file, file, …)`

- The `GROUP` script command includes a list of archive file names.
- The archive names defined in the list are searched repeatedly until all defined references are resolved.

4.3.11. ASSERT

Syntax :- `ASSERT(expression, string)`

- The `ASSERT` script command adds an assertion to the linker script.

4.4. Expressions

Expressions in linker scripts are identical to C expressions

Function	Description
.	Return the location counter value representing the current virtual address.
ABSOLUTE(expression)	Return the absolute value of the expression.
ADDR(string)	Return the virtual address of the symbol or section. . is supported.
ALIGN(expression)	Return the value of the location counter when aligned to the next expression boundary.
ALIGN(expression1, expression2)	Return expression1 rounded up to the next expression2 boundary.
ALIGNOF(string)	Return the alignment of the symbol or section. NEXT_SECTION returns the next allocated output section alignment (only supported with ALIGNOF/SIZEOF).
ASSERT(expression, string)	Throw an assertion if the expression evaluates to zero.
BLOCK(expression)	Synonym for ALIGN(expression).
DATA_SEGMENT_ALIGN(maxpagesize, commonpagesize)	Equivalent to ALIGN(maxpagesize) + (. & (maxpagesize - 1)) or ALIGN(maxpagesize) + (. & (maxpagesize - common-pagesize)) (the larger of the two).
DATA_SEGMENT_END(expression)	Return the value of the expression.
DATA_SEGMENT_RELRO_END(expression)	Return the value of the expression.
DEFINED(symbol)	Return 1 if the symbol is defined in the global symbol table.
LOADADDR(string)	Synonym for ADDR.
MAX(expression1, expression2)	Return the maximum of two expressions.
MIN(expression1, expression2)	Return the minimum of two expressions.
SEGMENT_START(string, expression)	If the string matches a known segment, return its start address; otherwise return the expression.
SIZEOF(string)	Return the size of the symbol, section, or segment. NEXT_SECTION returns the size of the next allocated output section (only with ALIGNOF/SIZEOF).
SIZEOF_HEADERS	Return the total size (bytes) of the ELF file header plus program headers.
CONSTANT(MAXPAGESIZE)	Return the ABI-defined maximum page size.
CONSTANT(COMMONPAGESIZE)	Return the ABI-defined common page size.

4.5. Symbol assignments

Linker scripts can define symbols that are used by the link to create the output image. Linker script symbols can be referenced by the program source code. One typical use of linker script symbols is to define the size and start / stop address of an output section.

Linker script symbol assignments support most C operators and follow C-like rules. For example, all the assignments must end with a semi-colon, and the C arithmetic operators are supported. The below mathematical operators are supported in linker script assignments:

```
u = v + w;
u = v - w;
u = v * w;
u = v / w;
u = v & w;
u = v | w;
u = v << w;
u = v >> w;
```

Additionally, linker script also supports compound assignment operators:

```
u += v;
u -= v;
u *= v;
u /= v;
```

```
u &= v;
u |= v;
u <<= v;
u >>= v;
```

The left-hand side symbol must already be defined when using compound assignment operator.

4.5.1. Symbol assignment types

Linker script symbol assignments are of 4 key types:

- `symbol = expression;` Defines a `GLOBAL` symbol.
- `HIDDEN(symbol = expression);` Defines a `GLOBAL` symbol with `HIDDEN` visibility.
- `PROVIDE(symbol = expression);` Defines the `symbol` only if it is required. If defined, the symbol will have `GLOBAL` symbol binding.
- `PROVIDE_HIDDEN(symbol = expression);` Defines the `symbol` only if it is required. If defined, the symbol will be a `GLOBAL` symbol with `HIDDEN` visibility.

4.5.1.1. Multiple PROVIDE commands for the same symbol

When multiple `PROVIDE` (or `PROVIDE_HIDDEN`) commands provide the same symbol, the linker honors the **first** command and ignores all subsequent redeclarations of that symbol. This applies regardless of whether the later commands appear in the same linker script or in a different one, and regardless of whether they use `PROVIDE` or `PROVIDE_HIDDEN`.

When `-Wlinker-script` is in effect, the linker emits a warning of the form:

```
<context>: Ignoring provide symbol '<symbol>' redeclaration
```

for each ignored redeclaration.

```
// script.t
PROVIDE(foo = 0x100);        // Wins: foo is provided as 0x100 (if used).
PROVIDE(foo = 0x200);        // Ignored.
PROVIDE_HIDDEN(foo = 0x300); // Ignored.
```

4.5.2. Section of linker script symbols

Linker script symbols defined outside an output section directive are called *absolute* symbols. That is, they do not belong to any output section. The section index of such symbols is a sentinel value `SHN_ABS`.

Linker script symbols that are defined within an output section directive have the output section index as their section index.

```
// script.t
u = 0x100;  // ABS symbol
SECTIONS {
  v = 0x300; // ABS symbol
  foo : {
    *(.text.foo)
    w = 0x500;  // Section of the symbol is foo
  }
}
e = 0x700; // ABS symbol
```

4.5.3. Location counter

`.` symbol is a special linker symbol that always contains the current output location address. Assigning to it changes the current output location address and can be used to create holes in the output image. This special dot symbol is called the location counter. It may also be referred to as the dot counter and dot symbol.

4.5.4. Assignment evaluation order

Understanding symbol assignment evaluation order is the key to understanding linker scripts and a necessary skill to debug linker script related issues. For the most part, the linker script assignments behave how you expect, but things become interesting when forward references are involved.

4.5.4.1. Basic case: No forward reference

Let's start with a basic case that does not have any forward references.

```
u1 = 0x100;  // A1
SECTIONS {
  u2 = 0x300; // A2
  foo : {
    *(.text.foo)
    u3 = 0x500;  // A3
  }
  u4 = 0x700;  // A4
  bar : {
    u5 = 0x900;  // A5
    *(.text.bar)
  }
  u5 = 0x1100; // A6
}
u6 = 0x1300;  // A7
```

The linker evaluates the assignment during the layout phase. The assignments , when they do not contain any forward reference, are evaluated in the script order. Thus, in this case, the linker script assignment evaluation order is: `[A1, A2, A3, A4, A5, A6, A7]`.

4.5.4.2. Forward reference

We will now see a more complex example that has forward references. But before we dive into the example, let's understand how linker evaluates an individual assignment that has forward reference.

```
u = v + w;
```

For the above assignment, let's say that `v` is defined before this assignment as per the linker script and `w` gets defined later. In such case, the *final* value of the symbol (`w` in this case) is used to evaluate the assignment. Let's understand this with a more concrete example:

```
v = 0x100; // A1
u = v + w; // A2
w = 0x200; // A3
foo = w;   // A4
w = 0x400; // A5
v = 0x600; // A6
```

When A2 (`u = v + w`) is evaluated, `v` is already defined and is thus evaluated to its current value `0x100`. On the other hand, `w` is not defined when A2 is executed and thus the final value of `w` (i.e., 0x400) is used to evaluate A2. Thus, after all assignments are processed, the final values are:

- `v = 0x600`
- `u = 0x500`
- `foo = 0x200`
- `w = 0x400`

Now let's look at a more complex example containing forward references.

```
u = v; // A1
SECTIONS {
 v = v1; // A2
 foo : {
   *(.text.foo)
   v1 = v2; // A3
 }
 v2 = v3; // A4
```

```
}
v3 = SIZEOF(foo); // A5
```

The linker again tries to evaluate the assignment in order, however, the assignments A1, A2, A3 and A4 cannot be completely evaluated because the variables on their right hand side are not evaluated yet. The linker marks the assignments that cannot be completely evaluated as pending assignments. It also records which nodes were unevaluated in the assignment. After the layout is complete, but before the relaxations begin, the linker recursivly evaluates the pending assignments until all assignments are resolved or a circular dependency is encountered. During re-evaluation of an assignment, only the previously unevaluated nodes are reevaluated.

The assignment evaluation sequence for this example is:

1. Evaluate [A1, A2, A3, A4, A5]: PendingAssignments = [A1, A2, A3, A4], CompletedAssignments = [A5]
2. Re-evaluate [A1, A2, A3, A4]: PendingAssignments = [A1, A2, A3], CompletedAssignments = [A5, A4]
3. Re-evalaute [A1, A2, A3]: PendingAssignments = [A1, A2], CompletedAssignments = [A5, A4, A3]
4. Re-evaluate [A1, A2]: PendingAssignments = [A1], CompletedAssignments = [A5, A4, A3, A2]
5. Re-evaluate [A1]: PendingAssignments = [], CompletedAssignments = [A5, A4, A3, A2, A1]

4.5.4.3. Circular dependency

What happens if the linker script contains circular dependency among variables?

```
u = v + 0x1; // A1
v = w + 0x1; // A2
w = u + 0x1; // A3
```

What would be the values of `u`, `v`, and `w` here?

In such a case, the linker would report a warning and stop evaluating symbol assignments once a circular dependency is detected. The final values of the symbols here will be:

- `u = 0x2`
- `v = 0x2`
- `w = 0x2`

Before analyzing this behavior, it’s important to note that a circular dependency represents an erroneous condition and should be considered undefined behavior. Once a layout enters an undefined state, all guarantees regarding its structure and consistency no longer apply.

With this warning in-place, let’s reason how linker arrives at these final values. The assignment evaluation sequence for this example is:

1. Evaluate [A1, A2, A3]: PendingAssignments = [A1, A2, A3], CompletedAssignments = []
2. Re-evaluate [A1, A2, A3]: PendingAssignments = [A1, A2, A3], CompletedAssignments = [], Circular dependency detected, stop evaluation.

The linker stops evaluating assignments when it detects a circular dependency. The linker assigns the value `0x1` to the symbols in the first assignment evaluation iteration, then in the second evaluation iteration it assigns the value `0x2` to the symbols. The linker then detects circular dependency and does not evaluate assignments further, as the layout may never converge due to circular dependencies.

4.5.4.4. Forward references in dot-assignments

Forward references in dot-assignments are more complex than those in non-dot assignments because dot-assignments directly influence the layout. If a dot-assignment cannot be evaluated correctly, the layout itself cannot be computed.

The fundamental rule remains unchanged: whenever a forward reference is encountered, the final value of the symbol is used in the expression.

In eld, when a forward reference appears in a dot-assignment, the layout is computed in two passes:

- First pass: The forward reference symbol is temporarily treated as 0 during layout computation.
- Second pass: After the initial layout is complete, eld recomputes the layout using the actual final values of all forward reference symbols.

4.5.5. –defsym

`--defsym sym=expr` is treated akin to a linker script just with one symbol assignment. For example:

```
ld.eld -o 1.out 1.o --defsym u=0x10 --defsym v=0x30 --defsym w=0x50
```

The above link command is equivalent to:

```
# The scripts consist of the following content:
# script1.t: "u=0x10"
# script2.t: "v=0x30"
# script3.t: "w=0x50"
ld.eld -o 1.out 1.o script1.t script2.t script3.t
```

Function	Description
HIDDEN(symbol = expression)	Hide the defined symbol so it is not exported.
FILL(expression)	Specify a fill pattern for the current output section. The fill element size can be 1, 2, 4, or 8 bytes (chosen by the linker). A FILL covers memory locations from the point at which it occurs to the end of the current output section; multiple FILL statements can be used within one output section.
ASSERT(expression, string)	If the specified expression is zero, the linker errors out with the specified message.
PROVIDE(symbol = expression)	Similar to a symbol assignment, but does not error if the symbol is already defined elsewhere.
PROVIDE_HIDDEN(symbol = expression)	Like PROVIDE, but the defined symbol is hidden (not exported).
PRINT("format-string", expr, …)	Print a formatted message while parsing the script. See the PRINT Command section for format string syntax and supported conversions.

4.6. NOCROSSREFS

- The NOCROSSREFS command takes a list of space-separated output section names as its arguments.
- Any cross references among these output sections will result in link editor failure.
- The list can also contain an orphan section that is not specified in the linker script.
- A linker script can contain multiple NOCROSSREFS commands.
- Each command is treated as an independent set of output sections that are checked for cross references.

4.7. OVERLAY

In GNU ld linker scripts, the `OVERLAY` command is used to define multiple sections that all execute (run) at the same virtual memory address (VMA), but are stored (loaded) at different load memory addresses (LMA). At runtime, only one of those sections is present in memory at a time, and software is responsible for copying the desired section into the overlay region before executing it.

4.7.1. What `OVERLAY` does in GNU ld

1. Same execution address (VMA) All sections inside an `OVERLAY` block are linked to start at the same address. From the CPU's point of view, they occupy the same memory region.
2. Different load addresses (LMA) Each overlaid section has a distinct load address in the output image (often in flash or ROM). These LMAs are laid out back-to-back starting at the overlay's `AT(...)` address.
3. Runtime-managed swapping The linker does not generate code to manage overlays. Your runtime code (overlay manager) must copy the desired section from its LMA into the overlay execution region before calling into it.

4.7.2. Auto-generated symbols in GNU ld

For each section inside an overlay, GNU ld defines:

- `__load_start_<section>`
- `__load_stop_<section>`

These symbols let your code know where to copy from.

4.7.3. `NOCROSSREFS`

GNU ld accepts an optional `NOCROSSREFS` keyword in the overlay header, e.g.:

```
OVERLAY 0x1000 : NOCROSSREFS AT(0x4000) { ... }
```

This causes GNU ld to error out if one overlaid section references another.

4.7.4. Effect on the location counter (`.`) in GNU ld

After an `OVERLAY` block, `.` is advanced by the size of the largest overlay member. This ensures the overlay region reserves enough execution memory for the largest case.

4.7.5. Syntax

```
OVERLAY [<start>] :
    [NOCROSSREFS] [AT(<lma_start>)]
{
    <overlay-member>...
} [><region>] [AT><lma_region>] [:<phdr>...] [=<fillexp>]

<overlay-member> :=
    <output-section-name> { <input-section-description>... }
```

> **Important**
> Overlay member sections are parsed as *name + body only*. Individual overlay members must not use the normal output section description prologue/epilogue syntax (for example, no `:` prologue, no member-level `AT(...)`, no member-level `>REGION`/`AT>REGION`, no `:PHDR` list, and no `=<fill>`). eld errors out if these constructs are used on overlay members.

4.7.6. eld support status

eld currently supports parsing `OVERLAY` blocks and printing them into the text map file (`-MapStyle txt`) as comments. The GNU ld overlay *semantics* described above (LMA/VMA overlay placement, generated symbols, overlay-member swapping behavior, overlay-specific `NOCROSSREFS` enforcement, and location counter advancement rules) are not implemented yet.

4.8. Output Section Description

A `SECTIONS` command can contain one or more output section descriptions.

```
<section-name> [<virtual_addr>][(<type>)] :
[AT(<load_addr>)] [ALIGN(<section_align>) | ALIGN_WITH_INPUT]
[SUBALIGN(<subsection_align>)] [<constraint>]
{
   ...
   <output-section-command> <output-section-command>
}[><region>][AT><lma_region>][:<phdr>...][
=<fillexp>][INSERT AFTER <section-name> | INSERT BEFORE <section-name>]
```

4.9. Syntax

<section-name>

: Specifies the name of the output section.

<virtual_addr>

: Specifies the virtual address of the output section (optional). The address value can be an expression (see Expressions).

<type>

: Specifies the section load property (optional).
- NOLOAD: Marks a section as not loadable.
- INFO: Parsed only; has no effect on linking.

<load_addr>

: Specifies the load address of the output section (optional). The address value can be specified as an expression (see Expressions).

<section_align>

: Specifies the section alignment of the output section (optional). The alignment value can be an expression (see Expressions).

<subsection_align>

: Specifies the subsection alignment of the output section (optional). The alignment value can be an expression (see Expressions).

<constraint>

: Specifies the access type of the input sections (optional).
- NOLOAD: All input sections are read-only.

<output-section-command>

: Specifies an output section command (see Output section commands). An output section description contains one or more output section commands.

<region>

: Specifies the VMA placement memory region (optional) using `>REGION`.

If the output section does not have an explicit VMA, ELD uses the next available address in the region (starting from `ORIGIN` and respecting alignment) as the section's VMA.

If the output section has an explicit VMA, the explicit address is honored. The region is used for memory usage accounting only when the explicit VMA is within the region bounds.

<lma-region>

: Specifies the LMA placement memory region (optional) using `AT>REGION`.

ELD places the section's LMA at the next available address in the selected region, tracked independently from the VMA region cursor. `AT(<address>)` and `AT>REGION` are mutually exclusive.

<fillexp>

: Specifies the fill value of the output section (optional). The value can be an expression. This option is parsed, but it has no effect on linking.

<phdr>

: Specifies a program segment for the output section (optional). To assign multiple program segments to an output section, this option can appear more than once in an output section description.

INSERT AFTER <section-name> | INSERT BEFORE <section-name>

: Requests placing this output section after/before the named output section. The anchor section must exist, or the linker emits an error. Overlay member sections do not support output section epilogues, so INSERT is not allowed inside OVERLAY member blocks.

4.10. Output Section Data

Within an output section description, you can use data-emitting commands to insert raw bytes directly into the output section at the current location counter. These commands are useful for embedding version strings, magic numbers, padding constants, or any other fixed data into the linked image without requiring a separate object file.

ELD supports the following output section data commands:

Command	Description
BYTE(expression)	Emit one byte (8 bits). The expression is truncated to 8 bits.
SHORT(expression)	Emit two bytes (16 bits). The expression is truncated to 16 bits.
LONG(expression)	Emit four bytes (32 bits). The expression is truncated to 32 bits.
QUAD(expression)	Emit eight bytes (64 bits) as an unsigned value.
SQUAD(expression)	Emit eight bytes (64 bits) as a signed value.
ASCIZ “string”	Emit a null-terminated ASCII string. The string literal (enclosed in double quotes) supports a small set of escape sequences and is then written followed by a NUL byte (\0). The size contributed is strlen(unescaped_string) + 1.

✎ **Note**

The numeric commands (`BYTE` through `SQUAD`) accept any linker script expression, including symbol references and arithmetic. Values that do not fit in the target width are truncated; with `-Wlinker-script`, the linker emits a warning when truncation occurs.

`ASCIZ` accepts only a quoted string literal, not an arbitrary expression.

Supported `ASCIZ` escape sequences are:
- `\n` (line feed)
- `\r` (carriage return)
- `\t` (horizontal tab)
- Octal escapes with up to 3 octal digits (for example `\0`, `\12`, `\101`)

Hex escapes (for example `\x41`) are not supported.

✎ **Note**

These commands are documented in the GNU Binutils ld manual under Output Section Data.

4.10.1. Example

The following linker script embeds a magic number, a version constant, and a build-info string into the `.rodata` output section:

```
SECTIONS {
  .text : { *(.text*) }

  .rodata : {
    *(.rodata*)

    /* 4-byte magic number */
    LONG(0xDEADBEEF)

    /* 2-byte protocol version */
    SHORT(3)

    /* Null-terminated build-info string */
    ASCIZ "eld linker v1.0"
  }
}
```

After linking, the `.rodata` section will contain (in order):
- The contents of all matched `.rodata*` input sections.
- Four bytes `ef be ad de` (little-endian `0xDEADBEEF`).
- Two bytes `03 00` (little-endian `3`).
- Sixteen bytes for `"eld linker v1.0\0"` (15 characters + NUL).
- Fifteen bytes for `"built with ELD\0"` (14 characters + NUL).

4.11. Sorting input sections

Linker scripts can control input-section ordering within an output section using sorting directives in input section descriptions. These directives include `SORT`, `SORT_BY_NAME`, `SORT_BY_ALIGNMENT`, and `SORT_BY_INIT_PRIORITY`.

ELD also supports the GNU linker shorthand `SORT(CONSTRUCTORS)` for compatibility with other linkers.

`EXCLUDE_FILE` can be used inside `SORT_*` wrappers, including nested sort expressions, for GNU-compatible forms such as: `SORT_NONE(EXCLUDE_FILE(*crtend.o) .eh_frame)` and `SORT_BY_NAME(SORT_BY_ALIGNMENT(EXCLUDE_FILE(*2.o) .text))`.

4.12. Controlling Physical addresses

In GNU linker scripts, the AT command is used to control the Load Memory Address (LMA) of a section, while the section's placement in memory during execution is defined by its Virtual Memory Address (VMA).

! **Important**
When `AT>REGION` is specified on an output section, the linker does **not** automatically align the LMA to match the natural alignment of the input sections. The LMA region cursor is authoritative in that case.

However, if the output section description contains an explicit `ALIGN(n)` keyword *after* the colon (the section address expression form, e.g. `.sec : ALIGN(n) { }`), ELD applies that alignment to both the VMA and the LMA, even when `AT>REGION` is present.

The `ALIGN(n)` keyword *before* the colon (the prolog/address-expression form, e.g. `.sec ALIGN(n) : { }`) sets only the VMA; the LMA stays at the LMA region cursor and is not aligned.

Summary of LMA alignment behavior with `AT>REGION`:

Syntax	VMA aligned?	LMA aligned?
.sec : { } (no ALIGN)	natural alignment	no (cursor only)
.sec ALIGN(n) : { } (prolog form)	yes, to n	no (cursor only)
.sec : ALIGN(n) { } (section-description form)	yes, to n	yes, to n
.sec : ALIGN_WITH_INPUT { }	natural alignment	tracks VMA delta

ALIGN_WITH_INPUT attribute on an output section preserves the VMA-to-LMA offset from the previous output section when both sections use the same VMA region and the same LMA region. If either region changes, the linker does not reuse the prior offset and instead computes the LMA from the current output section's placement rules. This matches GNU behavior when combining explicit LMA control with region-based placement.

Behavior summary:
- VMA placement is governed by the output section's virtual address rules and the selected VMA region.
- LMA placement is governed by the AT/AT> directives and the selected LMA region, and it is tracked independently from the VMA placement.
- Explicit `ALIGN(n)` in the section description (after the colon) aligns both VMA and LMA, even with `AT>REGION`.
- Prolog `ALIGN(n)` (before the colon) aligns only the VMA; LMA follows the LMA region cursor.
- ALIGN_WITH_INPUT preserves the prior VMA-to-LMA delta, but only while both regions remain the same.
- When VMA and LMA resolve to the same region (for example, no `AT>REGION`), ELD advances that shared region cursor once per output section.

See also:
- `test/Common/standalone/linkerscript/AlignWithInput/NoPhdrs/AlignWithInput.test`
- `test/Common/standalone/linkerscript/AlignWithInput/TLS/TLS.test`
- `test/Common/standalone/linkerscript/AlignWithInput/NoLoadATRAM/NoLoadATRAM.test`
- `test/Common/standalone/linkerscript/MEMORY/ATRegionAlign/ATRegionAlign.test`
- `test/Common/standalone/linkerscript/MEMORY/ATRegionAlignPHDRS/ATRegionAlignPHDRS.test`

4.13. GNU-compatibility

The eld linker script syntax and semantics are GNU-compliant. This means that any linker script that works with the GNU linker should also work with eld, with the exception of a few GNU linker script features that are not yet supported by eld.

Previously, eld supported two extensions to the GNU linker script syntax. **These extensions are no longer supported.** Any scripts using these extensions must be updated to maintain compatibility with eld. These extensions are:

1. Assignment without space between the symbol and `=`

Previously supported:

```
symbol=<expr>
```

GNU-compliant syntax (required now):

```
symbol = <expr>
```

GNU requires a space between the symbol and the assignment operator. eld now enforces this requirement. Scripts must be updated accordingly.

1. Output section description without space between the output section name and `:`

Previously supported:

```
SECTIONS {
  FOO: {
    *(.text.foo)
  }
}
```

GNU-compliant syntax (required now):

```
SECTIONS {
  FOO : {
    *(.text.foo)
  }
}
```

GNU requires a space between the output section name and the colon. eld now enforces this requirement for full GNU compatibility.

4.13.1. Why cannot eld support these extensions along with GNU-compatibility?

eld cannot support these extensions along with GNU-compatibility because they directly conflict with the GNU linker script syntax. For example, GNU ld allows `:` in section names and allows `=` in symbol names. The core issue is that GNU ld uses the same lexing state to parse symbol and section names to keep the parser simple and efficient. Due to this, GNU ld also allows other non-trivial characters in symbol names such as `+`, `-`, `:` and so on. For example, for the below linker script snippet, gnu ld creates a symbol of the name `a+=`:

```
a+= = b # lhs symbol is a+=
```

eld cannot easily add exception to the two cases that were supported by eld extensions while keeping everything else the same to keep the linker script parser efficient. To support these as an exception, the parser needs to lookahead two tokens to resolve ambiguities. Let’s understand this with the help of an example:

```
SECTIONS {
  FOO: {
    *(.text.foo)
  }
  u=v;
}
```

When parsing the `SECTIONS` commands, the parser does not know in which LexState to parse the command. If the command is an output section description, `FOO:`, then the parser should parse the token in `LexState::default`, whereas if the command is an assignment, then the parser should parse the token in `LexState::Expr`. `LexState::default` allows some characters in tokens that are not appropriate when parsing an expression. These characters include `+`, `-`, `=` and more.

To correctly determine which `LexState` to use, the parser needs to peek (lookahead) two tokens in `LexState::Expr`. With the two tokens peek, the parser can determine whether the command is an assignment command or not.

This simple change requires a lot of changes in the parser. The parser needs to change from LL(1) (Simple and efficient) to LL(2) (Complex and less efficient).

4.14. PRINT Command

4.14.1. Overview

The `PRINT` command lets a linker script emit formatted messages on the linker's standard output stream while the script is being parsed and evaluated. This is useful for debugging script expressions, inspecting symbol values, or annotating map files during development.

4.14.2. Syntax

The `PRINT` command has the following syntax:

```
PRINT("format-string", expression, ...)
```

Where:
- `format-string` is a C `printf`-style format string.
- Each `expression` is a regular linker script expression whose value is substituted into the format string.

The command does not define any symbols and does not affect the layout or contents of the linked image; it only produces textual output.

4.14.3. Format String Semantics

4.14.4. Basics

The format string closely follows C `printf` semantics with a limited set of conversion specifiers:
- `%%` – prints a literal `%` character; consumes no argument.
- `%d` – signed decimal integer.
- `%i` – signed decimal integer (alias for `%d`).
- `%u` – unsigned decimal integer.
- `%o` – unsigned octal integer.
- `%x` – unsigned hexadecimal integer (lowercase digits).
- `%X` – unsigned hexadecimal integer (uppercase digits).
- `%c` – single character; the low 8 bits of the numeric expression.
- `%s` – symbol name corresponding to a *symbol expression*.

All numeric expressions are evaluated as 64-bit integers. Length modifiers are accepted for compatibility but ignored; the value is always normalized to 64 bits before formatting.

4.14.5. Flags, Width, and Precision

The following flag characters are recognized:
- `-` – left-justify within the field width.
- `+` – always include a sign for signed conversions.
- `` `` `` (space) – prefix a space for positive signed conversions.
- `#` – alternate form (for example, add `0x` for hex).
- `0` – pad numeric fields with leading zeros.

Field width and precision have the usual `printf` syntax:

```
%[flags][width][.precision][length]conversion
```

with the following constraints:
- `width` and `precision` must be decimal integer constants.
- `*` is **not** supported for either width or precision; using it produces a diagnostics error.
- Supported length modifiers (`hh`, `h`, `l`, `ll`, `j`, `z`, `t`, `L`) are parsed but ignored – values are always treated as 64-bit.

4.14.6. Escape Sequences

The format string supports a small set of C-style escape sequences, which are unescaped before formatting:
- `\\n` – newline
- `\\t` – horizontal tab
- `\\r` – carriage return
- `\\\\` – backslash
- `\\\"` – double quote

Any other escape sequence is left unchanged (the leading backslash is kept as-is).

4.14.7. Argument Matching and Errors

The linker validates the correspondence between conversion specifiers in the format string and the expressions supplied to `PRINT`. The following conditions are diagnosed as `PRINT` errors:

- Not enough arguments:
  - The format string requires more values than the number of expressions supplied (for example, `PRINT("%d %u", 1);`).
- Too many arguments:
  - More expressions are supplied than conversion specifiers in the format string (for example, `PRINT("%d", 1, 2);`).
- Unterminated or malformed format specifier:
  - A `%` at the end of the string (for example, `"value %"`).
  - A partially specified format that never reaches a conversion character.
- Unsupported conversion:
  - Any conversion character other than `d`, `i`, `u`, `o`, `x`, `X`, `c`, `s`, or `%` (for example, `%f`) is rejected.
- Unsupported width or precision:
  - Using `*` for width (for example, `"%*d"`).
  - Using `*` for precision (for example, `"%.*d"`).
- Invalid `%s` argument:
  - The expression corresponding to a `%s` conversion must be a *symbol expression* (such as `foo`). Passing an arbitrary numeric expression (for example, `1 + 2`) is rejected.

On any of these conditions the linker emits an `error_printcmd` diagnostic and treats the `PRINT` invocation as a fatal error for that link.

4.14.8. Examples

4.14.9. Printing Numeric Expressions

```
/* Signed and unsigned forms */
PRINT("value=%d (0x%x)\\n", 42, 42);

SECTIONS {
  .text : { *(.text*) }
}
```

This prints a line similar to:

```
value=42 (0x2a)
```

4.14.10. Using `%c` and `%s`

```
/* Assume 'foo' is a symbol defined by an input object. */
PRINT("symbol %s at '%c' section start\\n", foo, 'T');
```

The `%s` conversion prints the symbol name (`foo`) while `%c` prints the low 8 bits of the numeric expression.

4.14.11. Using Width, Precision, and Flags

```
PRINT("val=%+08d hex=%#06x\n", 42, 42);

SECTIONS {
  .text : { *(.text*) }
}
```

This uses:

- `+` to always show the sign for the decimal value.
- `0` and a width of `8` to zero-pad the decimal field (for example, `+0000042`).
- `#` and a width of `6` for hexadecimal, causing a leading `0x` and zero-padding in the remaining field (for example, `0x002a`).

4.14.12. Error Examples

```
/* Not enough arguments */
PRINT("x=%d y=%d\\n", 1);

/* Unsupported conversion */
PRINT("value=%f\\n", 1);

/* Invalid %s argument */
PRINT("value=%s\\n", 1 + 2);
```

Each of these invocations produces a `PRINT`-related diagnostic and prevents the link from succeeding.

4.14.13. Version Scripts

A *version script* controls the symbol visibility and defined versioned nodes of a shared library (or, more generally, of the symbols exported into the dynamic symbol table). It lets you:

- Choose which global symbols are exported (`global`) and which are hidden (`local`).
- Attach *version nodes* (such as `LIBFOO_1.0`) to exported symbols so that multiple versions of the same symbol can coexist in a single library.

```
ld.eld -shared -o libfoo.so foo.o --version-script=version.map
```

4.14.14. Syntax

A version script is a sequence of *version nodes*. Each node has an optional name and contains `global:` and/or `local:` blocks. Each block lists symbol patterns terminated by `;`.

```
/* Anonymous (unnamed) version node. */
{
  global:
    foo;
    bar*;
  local:
    *;
};
```

Symbol patterns may be:

- **Exact names** — for example `foo`. These match a single symbol.
- **Wildcard patterns** — glob patterns such as `foo*` that match a set of symbols.

Only symbols that are **defined by the objects being linked** participate in version-script matching. Symbols that come from shared libraries are ignored, because it does not make sense to assign a version node to a symbol that is defined elsewhere.

4.14.15. Pattern matching rules

eld follows the same version script pattern matching rules as GNU `ld`. This is more subtle than a simple “first pattern in the file wins”: the *kind* of pattern determines its priority, and only when two patterns are of the same kind does their order in the file matter. Matching proceeds in three phases:

Phase 1 — Exact patterns (first match wins). All exact (non-wildcard) patterns are considered first, in the order the nodes and blocks appear in the file. Within a node the `global:` block is examined before the `local:` block. The first exact pattern that matches a symbol wins.

Phase 2 — Non-``*`` wildcards (last match wins). Symbols still unassigned after Phase 1 are matched against wildcard patterns other than the bare `*` (for example `foo*`). The *last* pattern that matches a symbol wins.

Phase 3 — The ``*`` catch-all (last match wins). Finally, any symbols that are still unassigned are matched against the bare `*` pattern. The *last* pattern that matches a symbol wins.

The practical consequences of this ordering are:

- An exact match always beats a wildcard match, regardless of where each appears in the file.
- A specific wildcard (`foo*`) always beats the catch-all `*`.
- Among patterns of the same kind, exact patterns favor the *first* occurrence while wildcards favor the *last*.

4.14.16. Examples

Exact match takes precedence over a wildcard. Here `foo` is listed as a wildcard (`foo*`) in `global` but as an exact name in `local`. The exact pattern wins, so `foo` becomes local (hidden) while `foo1` and `foo2` remain global (exported), matched by `foo*`:

```
V1 {
  global:
    foo*;
  local:
    foo;
};
```

``*`` has the lowest priority. `foo*` beats the catch-all `*`. Symbols `foo`, `foo1`, `foo2` become local, while every other exported symbol (`bar`, `bar1`, `baz`, `qux`, …) is matched by `*` and stays global:

```
V1 {
  global:
    *;
  local:
    foo*;
};
```

Wildcards: last match wins across version nodes. `V1` marks `foo*` as global, but `V2` (processed later) marks the same `foo*` as local. Because wildcards favor the last match, `foo`, `foo1` and `foo2` end up local:

```
V1 {
  global:
    foo*;
};

V2 {
  local:
    foo*;
} V1;
```

All three phases together. The following script combines every rule. `foo1` matches the exact pattern in `V3` (Phase 1) and is exported as `foo1@@V3`. `foo` and `foo2` match the `foo*` wildcard in `V2`, which beats the `*` in `V1` (Phase 2), so they are exported as `foo@@V2` and `foo2@@V2`. Everything else (`bar`, `bar1`, `baz`, `qux`) falls through to Phase 3, where `V3`'s `local: *` — the last `*` in the file — wins over `V1`'s `global: *`, hiding those symbols:

```
V1 {
  global:
    *;
};

V2 {
  global:
    foo*;
} V1;

V3 {
  global:
    foo1;
  local:
    *;
} V2;
```

5. Linker Diagnostics

This chapter documents diagnostics produced by the linker.

5.1. Diagnostic Reports

Linker diagnostics refer to the messages and reports generated by a linker to help developers identify and resolve issues during the linking phase of compilation. These diagnostics can include:

- Warnings and errors about malformed or unsupported linker scripts
- Misplaced or overlapping sections
- Unresolved symbols
- Alignment issues
- Plugin-related inconsistencies

5.1.1. What `-Wall` and `-Werror` Mean for ELD

5.1.1.1. `-Wall`: Enable All Warnings

In the eld linker, `-Wall` enables a broad set of diagnostic warnings during the linking process. These warnings help developers catch:

- Misuse of linker script commands
- Compatibility issues during linking
- Linker script forward references
- Suspicious section overlaps or misalignments
- Deprecated or unsupported syntax
- Potentially undefined behavior in symbol resolution
- Command line usage

This flag is particularly useful in embedded development, where layout precision and deterministic behavior are critical.

5.1.1.2. `-Werror`: Treat Warnings as Errors

When `-Werror` is used, any warning promoted by `-Wall` (or other `-W` flags) is treated as a **fatal error**, halting the linking process. This ensures:

- No warnings are ignored during CI/CD builds
- Developers are forced to resolve all issues before producing a final binary
- Consistency across builds, especially when linker behavior may vary across toolchain versions

This is especially important in embedded workflows, where eld is used to build critical components like firmware and device drivers.

5.1.1.3. Why It Matters in ELD

- **Embedded Safety**: In embedded systems, even minor layout issues can lead to runtime faults. `-Wall -Werror` ensures these are caught early.
- **Plugin Infrastructure**: ELD supports linker plugins. These can emit custom diagnostics, and `-Wall` ensures they're surfaced; `-Werror` enforces them.
- **Script Compatibility**: ELD aims for GNU linker script compatibility. These flags help validate script correctness during migration from GNU ld.

5.1.1.4. Example Usage

```
eld -T script.ld -o firmware.elf main.o -Wall -Werror
```

This command will:

- Use `script.ld` for layout
- Link `main.o` into `firmware.elf`
- Emit all warnings (`-Wall`)
- Fail the build if any warning is encountered (`-Werror`)

5.1.2. ELD Linker Warning Flags

This document provides a categorized list of warning flags supported by the ELD linker.

5.1.2.1. General Warning Flags

Flag	Description
-Wall	Enables all standard warnings supported by ELD. Useful for catching script and layout issues early.
-Warchive-file	Warns about issues related to archive (.a) when they contain duplicate members.
-Wattribute-mix	Flags inconsistent or conflicting section attributes across input files.
-Wbad-dot-assignments	Warns when the . (location counter) is used in a way that may cause layout issues or undefined behavior.
-Wcommand-line	Enables warnings for malformed or conflicting command-line options.
-Werror	Treats all warnings as errors, halting the link process.
-Wlinker-script	Enables warnings specific to linker script issues, such as malformed directives, deprecated syntax, or unsupported constructs.
-Wlinker-script-memory	Focuses on memory region definitions in linker scripts, such as overlaps or undefined regions.
-Wwhole-archive	Warns when –whole-archive is used inappropriately or causes symbol bloat.
-Wzero-sized-sections	Flags sections that are defined but have zero size, which may indicate a script or input error.
-Wosabi	Generates a warning when an input file's OS/ABI value differs from those encountered in previously processed files.

5.1.2.2. Suppression Flags

Flag	Description
-Wno-archive-file	Suppresses archive file-related warnings.
-Wno-attribute-mix	Suppresses attribute mismatch warnings.
-Wno-error	Allows warnings to be emitted without halting the link process.
-Wno-linker-script	Disables linker script-related warnings.
-Wno-osabi	Disables OS/ABI warnings.
-Wno-whole-archive	Disables whole archive warnings.

5.1.3. Archive Member Reports

In addition to map-file text output, `--archive-member-report` emits a JSON report with one record per archive-member inclusion event. This report is generated even when `-Map` is not requested.

Example linker invocation:

```
clang hw.o -Wl,--archive-member-report,report.json --ld-path=ld.eld
```

Schema (per `ArchiveMembers` entry):

- `MemberPath`: Decorated member path such as `libc.a(malloc.o)`.
- `Archive` / `Member`: Archive and member split fields for archive members.
- `ReferrerObjectFile`: Requester when an object file caused the pull-in.
- `ReferrerArchive` / `ReferrerMember`: Requester when another archive member caused the pull-in.
- `Symbol`: Symbol that triggered inclusion for non-whole-archive pulls.
- `WholeArchive` and `Reason`: Present when `--whole-archive` caused inclusion (for example `Reason: --whole-archive`).
- `MemberReferencedSymbols`: Symbols referenced by the included member; this enables tracing symbol chains such as `sys_init_tls -> malloc`.
- `MemberSymbols`: Symbols defined by the included member.
- `MemberIsArchiveMember` / `ReferrerIsArchiveMember`: Explicit booleans describing whether each side of the edge is an archive member.
- `MemberIsBitcode` / `ReferrerIsBitcode`: Bitcode markers for mixed native/bitcode links.

Behavior notes:

- The report is emitted even when map files are not requested.
- Both regular archives and thin archives are supported.
- For `--whole-archive` inclusion, `WholeArchive` is `true` and `Reason` is emitted as `--whole-archive`.

Example JSON snippet:

```
{
  "ArchiveMembers": [
    {
      "MemberPath": "libc.a(malloc.o)",
      "Archive": "libc.a",
      "Member": "malloc.o",
      "ReferrerArchive": "libstandalone.a",
      "ReferrerMember": "sys_init_tls.o",
      "Symbol": "malloc",
      "MemberReferencedSymbols": ["sbrk", "errno"]
    }
  ]
}
```

5.1.3.1. Tracing behavior with examples

Use `utils/archive_member_report/archive_member_trace.py` to understand *why* members were included and to follow chains through archive members.

Examples:

```
# Why was malloc needed?
python3 utils/archive_member_report/archive_member_trace.py report.json --symbol malloc --why-needed

# Trace all members matching a glob.
python3 utils/archive_member_report/archive_member_trace.py report.json --pattern "*libc.a(*malloc*.o)"

# Show referenced symbols for matching archives.
python3 utils/archive_member_report/archive_member_trace.py report.json --referenced-symbols-archive "*libc.a"

# Show defined symbols for matching members.
python3 utils/archive_member_report/archive_member_trace.py report.json --defined-symbols-member "*tls*.o"

# Thin archive works the same way.
clang main.o -Wl,--archive-member-report,report.json thinlib.a
python3 utils/archive_member_report/archive_member_trace.py report.json --symbol foo
```

The `--symbol` and `--pattern` options are mutually exclusive. For `--why-needed`, the output highlights the requesting file/member and prints a readable chain so each pull-in reason is visible as a separate trace.

5.2. Linker Map Files

A map file provides detailed information about the link and helps analyze output images, debug build issues, and better understand linker decisions. It contains detailed information for input files, linker scripts, memory regions, image memory layout, and so much more. Effectively using map files can significantly reduce the time spent analyzing and debugging builds.

Use `-Map` to generate map files for your builds.

5.2.1. Map File Styles

Map files are available in three distinct styles:

1. Text
2. YAML
3. Binary

Use option `-MapStyle` to specify the map file style. Text map file is the default map style.

5.2.2. Navigating Text Map File

5.2.2.1. Map file contents

Section	Description
Tools Version, and System and Link features	Information about tools version, and system and link features.
Link Stats	Quick overview of key link information
Archive Member Information	Specifies which archive members were included and why.
Allocating Common Symbols	Lists the common symbols which got allocated
Linker Script and Memory Map	Specifies input object files
Build statistics	
Linker scripts used	Specifies the linker scripts used by the link.
Linker plugin information	Details plugin information.
Output Section and Layout	Delineates output image layout

Let's explore each of these sections in detail!

5.2.2.2. Tools Version, and System and Link Features

As the title suggests, this consists information about tools version, and system and link features. An example of this section is shown below:

```
# Linker from QuIC LLVM Hexagon Clang version Version 19.0-alpha2 Engineering Release: hexagon-clang-190-229
# Linker based on LLVM version: 19.0
# Linker: d8a255719548f774e7e73cd97959ca04fd780b54
# LLVM: 9ab18b8e7547380db4fb83b2ada1d7704876b334
# Notable linker command/script options:
# CPU Architecture Version:
# Target triple environment for the link: unknown
# Maximum GP size: 8
# Link type: Static
# ABI Page Size: 0x1000
# CommandLine : hexagon-link -o main.elf main.o libs/libfoo.a libs/libbar.a -Map main.map.txt
```

5.2.2.3. Link Stats

'Link Stats' provides a quick overview of key link information. **Link stats are not displayed if their value is 0.**

An example of 'Link Stats' is shown below:

```
# LinkStats Begin
# ObjectFiles : 2
# LinkerScripts : 2
# ThreadCount : 64
# NumInputSections : 37
# ZeroSizedSections : 28
# Total Symbols: 6 { Global: 3, Local: 2, Weak: 0, Absolute: 3, Hidden: 1, Protected: 0 }
# Discarded Symbols: 1 { Global: 1, Local: 0, Weak: 0, Absolute: 0, Hidden: 1, Protected: 0 }
# NumLinkerScriptRules : 2
# NumOutputSections : 7
# NumOrphans : 5
# NoRuleMatches : 3
# NumSegments : 1
# OutputFileSize : 4920 bytes
# LinkStats End
```

There might be more or less link stats than what is shown here depending on your tools version. Remember that only non-zero stats are displayed.

5.2.2.4. Archive Member Information

'Archive Member Information' tells which archive members were included in the link and due to which symbol. Archive members are only included in the link if they satisfy an undefined symbol reference. The format of each entry of 'Archive Member Information' is:

```
<ArchiveFile> (<ArchiveMember>)
     <CompilationUnitReferringArchiveMember> (<ReferencedSymbol>)
```

The below example shows that archive members 'libs/libfoo.a(Foo.o)' and 'libs/libbar.a(Bar.o)' are included in the link because 'main.o' referenced symbols 'foo' and 'bar' from the archive members.

```
Archive member included because of file (symbol)
libs/libfoo.a(Foo.o)
                main.o (foo)
libs/libbar.a(Bar.o)
                main.o (bar)
```

5.2.2.5. Allocating Common Symbols

- 'Allocating Common Symbols' list the common symbols that were allocated by the linker. Each entry in the common symbols section contains the following items:

    - The name of the symbol
    - The size of the memory area allocated for the symbol
    - The full pathname of the archive file accessed by the linker
    - The name of the archived object file that defines the symbol (in parentheses)

In the following example, the __eh_nodes symbol has size of **0x4** and is defined in the the object file: **xregister.o**

```
Common symbol       size    file

__eh_nodes   0x4     $PATH/../target/hexagon/lib/v65/G0/libc.a(xregister.o)
```

5.2.2.6. Linker Script and Memory Map Section

- The linker script section of a link map lists the complete linker script that is specified for the link.

> **Note**
> Linker scripts are optional. If a script is not specified on the linker command line, the link map does not include a linker script.

- The following example shows the initial lines of a linker script section:

```
Linker Script and memory map
LOAD $PATH/../target/hexagon/lib/v65/G0/crt0_standalone.o[hexagonv65]
LOAD target/main.o[hexagonv65]
START GROUP
LOAD $PATH/../target/hexagon/lib/v65/G0/libstandalone.a(_exit.o)[hexagonv65]
...
LOAD $PATH/../target/hexagon/lib/v65/G0/libgcc.a(vmemcpy_h.o)[hexagonv65]
END GROUP
LOAD $PATH/../target/hexagon/lib/v65/G0/fini.o[hexagonv65]
```

- `LOAD` entries can include trailing comments (`# ...`) with input selection notes. For example, when `--fat-lto-objects` selects a fat LTO payload, the input line is annotated with `# Fat LTO object selected`.

5.2.2.7. Linker Scripts Used

‘Linker Scripts Used’ specifies the linker scripts used by the link.

```
Linker scripts used (including INCLUDE command)
target/linker.script
```

Add ‘Linker Plugin Information’ section docs!

Add ‘Build statistics’ section docs!

5.2.2.8. Output Section and Layout

‘Output Section and Layout’ describes the image layout. It tells where output sections are placed and what they consist of. It presents information in an elaborate format loosely based on the linker script `SECTIONS` command. This format allows users to easily visualize and understand how `SECTIONS` subcommands, linker script expressions and assignment to the location counter affect the output layout.

‘Output Section and Layout’ contains detailed information about output sections, input section descriptions (commonly called rules), input sections, symbols, padding and linker script expressions. It is the most complex part of the map file. Let’s explore its structure to better understand its content.

5.2.2.8.1. Simplified ‘Output Section and Layout’ Structure

We will first see its simplified structure. Padding and linker script expressions are missing from this simplified structure. As we move forward in the documentation, these missing components will be gradually covered to make the structure complete.

5.2.2.8.1.1. <OutputSectionAndLayout>

```
# Output Section and Layout
<OutputSection_1>
<OutputSection_2>
...
<OutputSection_N>
```

Each <OutputSection> describes one output section. Let’s see what it consists of.

5.2.2.8.1.2. <OutputSection>

<OutputSection> is represented as:

```
<OutputSectionName> <size> <VMA> # Offset: <offset>, LMA: <LMA>, Alignment: <alignment>, Flags: <SectionFlags>,
Type: <SectionType>, Segments: <segments>
# INSERT AFTER|BEFORE <anchor-section> # <script:line>
<Rule_1>
<Rule_2>
...
<Rule_N>
```

The INSERT line is present only when the output section was positioned using `INSERT AFTER` or `INSERT BEFORE` in a linker script. The trailing script context shows the originating linker script location.

<OutputSection> tells output section properties and contains a list of `Rule` entries. Placement of rules conveys the placement of the rule contents in the output image layout. The contents of *Rule_1* are placed before the contents of *Rule_2* and so on. The contents of a rule is the merged content of all the sections that were matched to the rule. Now, let’s see what does <Rule> consist of.

5.2.2.8.1.3. <Rule>

<Rule> (input section description) is represented as:

```
<InputSectionDesc> #Rule <RuleNumber>, <InputFile>
<InputSection_1>
```

```
<InputSection_2>
...
<InputSection_N>
```

`RuleNumber` makes it easier to refer to a particular input section description. <Rule> contains a list of <InputSection> that got matched to this rule.

5.2.2.8.1.4. <InputSection>

<InputSection> is typically represented as:

```
<InputSectionName> <VMA> <size> <InputFile> #<SectionType>,<SectionFlags>,<SectionAlignment>
<Symbol_1>
<Symbol_2>
...
<Symbol_N>
```

However, garbage-collected input sections are prefixed with '#', are annotated with `<GC>` and do not have `<VMA>` component. The <InputSection> for garbage-collected input sections is represented as:

```
# <InputSectionName> <GC> <VMA> <InputFile> #<SectionType>,<SectionFlags>,<SectionAlignment>
<Symbol_1>
<Symbol_2>
...
<Symbol_N>
```

5.2.2.8.1.5. <Symbol>

<Symbol> is typically represented as:

```
<VMA> <SymbolName>
```

However, garbage-collected symbols are prefixed with '#', and do not have `VMA` component. The <Symbol> for garbage-collected symbol is represented as:

```
# <SymbolName>
```

5.2.2.8.1.6. Example

We have covered the simplified <OutputSectionAndLayout> structure. Now, let's see a practical example.

Foo.c

```
int foo() { return 1; }
```

Bar.c

```
int bar() { return 3; }
```

script.t

```
SECTIONS {
  FOO : { *(*foo*) }
  BAR : { *(*bar*) }
}
```

Generate the map-file:

```
hexagon-clang -o Foo.o Foo.c -c -ffunction-sections
hexagon-clang -o Bar.o Bar.c -c -ffunction-sections
hexagon-link -o FooBar.elf Foo.o Bar.o -T script.t -Map FooBar.map.txt
```

Output Section and Layout excerpt from the map-file

```
# Output Section and Layout
FOO    0x0    0xc # Offset: 0x1000, LMA: 0x0, Alignment: 0x10, Flags: SHF_ALLOC|SHF_EXECINSTR, Type: SHT_PROGBITS
*(*foo*) #Rule 1, script.t [1:0]
.text.foo       0x0     0xc     Foo.o   #SHT_PROGBITS,SHF_ALLOC|SHF_EXECINSTR,16
        0x0             foo
*(FOO) #Rule 2, Internal-LinkerScript (Implicit rule inserted by Linker) [0:0]

BAR    0x10   0xc # Offset: 0x1010, LMA: 0x10, Alignment: 0x10, Flags: SHF_ALLOC|SHF_EXECINSTR, Type: SHT_PROGBITS
*(*bar*) #Rule 3, script.t [1:0]
.text.bar       0x10    0xc     Bar.o   #SHT_PROGBITS,SHF_ALLOC|SHF_EXECINSTR,16
        0x10            bar
*(BAR) #Rule 4, Internal-LinkerScript (Implicit rule inserted by Linker) [0:0]
```

We can see precisely how output sections are laid in the final layout and what they are composed of. For instance, the `FOO` output section contains the `.text.foo` input section because the `.text.foo` matches the rule `*(*foo*)`. Under the input section information, we can also see where each symbol of the section is placed in the output layout.

5.2.2.8.2. Other <OutputSectionAndLayout> substructures

5.2.2.8.2.1. <LinkerScriptExpression>

<LinkerScriptExpression> entries are interspersed in <OutputSectionAndLayout>, <OutputSection> and <Rule> and indicates their placement in the linker script. <LinkerScriptExpression> contains the linker script expression annotated with the values.

For example, if the linker script contains:

```
SECTIONS {
  FOO : {
    v = . + 0x5;
    *(*foo*)
  }
}
```

Then the map-file will contain:

```
FOO      0x0      0x100c # Offset: 0x1000, LMA: 0x0, Alignment: 0x10, Flags: SHF_ALLOC|SHF_EXECINSTR, Type:
SHT_PROGBITS, Segments : [ FOO ]
        v(0x5) = .(0x0) + 0x5; # v = . + 0x5; script.t
*(*foo*) #Rule 1, script.t [1:0]
.text.foo       0x0     0xc     1.eld.o        #SHT_PROGBITS,SHF_ALLOC|SHF_EXECINSTR,16
        0x0             foo
```

Note the `v(0x5) = .(0x0) + 0x5;` line under `FOO` output section.

5.2.2.8.2.2. <Padding>

<Padding> describes a padding in the output image layout. Paddings are either requested explicitly by the user or are added automatically by the linker to satisfy alignment constraints. Like <LinkerScriptExpression>, <padding> is also interspersed in <OutputSectionAndLayout>, <OutputSection> and <Rule>. The position of <padding> entry in the map-file conveys the position of padding in the memory layout.

Let’s see an example of explicitly requested padding.

If the linker script contains:

```
SECTIONS {
 FOO : {
   . = . + 0x10;
   *(*foo*)
 }
}
```

Then the map-file will contain:

```
# Output Section and Layout

FOO    0x0    0x1c # Offset: 0x1000, LMA: 0x0, Alignment: 0x10, Flags: SHF_ALLOC|SHF_EXECINSTR, Type: SHT_PROGBITS
PADDING 0x0     0x10    0x0
        .(0x10) = .(0x0) + 0x10; # . = . + 0x10; script.t
*(*foo*) #Rule 1, script.t [1:0]
.text.foo       0x10    0xc     Foo.o   #SHT_PROGBITS,SHF_ALLOC|SHF_EXECINSTR,16
        0x10            foo
*(FOO) #Rule 2, Internal-LinkerScript (Implicit rule inserted by Linker) [0:0]
```

Please note the `PADDING` details under the `FOO` output section.

5.2.2.8.3. Link map in YAML format

- If you use the MapStyle option to specify YAML-style output for the map file, the linker generates a YAML file instead of the text file that shows the memory map for the program.
- To derive statistics from the YAML map file produced by the linker, use the **YAMLMapParser.py** tool and include any of the following options:

  - **-info=architecture** – List the architecture
  - **sizes** – List the code and data sizes for the objects in the image
  - **summarysizes** – List the code and data sizes of the image
  - **totals** – List the total size of all objects in the image
  - **unused** – List the sections eliminated from the image
  - **unusedsymbols** – List the symbols eliminated from the image
  - **-map** – Display the memory map of the images
  - **-xref** – List the cross references between input sections
  - **list** – Redirect the output to a file

  The following table describes the sections of a YAML file

Section	Description
Header	Top level information of the program that was built
Version	Information Tools version information
Archive Records	Archive files pulled in by the linker
Inputs	Inputs that were used
InputInfo	Inputs that were used and not used
Linker script used	Linker script file that was used
BuildType	Type of build
OutputFile	Name of the output file
EntryAddress	Entry address for the image built
CommandLine	information on how the linker was called
CommandLineDefaults	Various defaults applied in the linker
OutputSections	All output sections
DiscardedComdats	COMDAT C++ section groups that discarded
DiscardedSections	Sections discarded by garbage collection
DiscardedCommonSymbols	Common symbols that were discarded
LoadRegions	Segments that the program loader will load
CrossReferences	Cross-reference table for the program

6. ELD (debugging guide)

This document describes the high-level flow of how ELD executes a link, with call-site pointers and practical tips for debugging failures.

6.1. Big picture

At a high level, a link invocation looks like this:

1. **eld main** expands response files and selects a driver flavor/target.
2. **Driver** parses options and builds an ordered list of input actions.
3. **Linker prepare** initializes target/emulation, inputs, and plugins, then reads and normalizes input files (and may run LTO).
4. **Linker link** resolves symbols/relocations, lays out output sections, then emits the final ELF and optional map files.
5. **Optional diagnostics**: reproduce tarball/mapping file, plugin activity log, timing stats, summary, etc.

6.2. Entry point and driver selection

The executable entry point is `tools/eld/eld.cpp`:
- Expands `@response` files via `llvm::cl::ExpandResponseFiles(...)`.
- Creates a `Driver` and calls `Driver::setDriverFlavorAndInferredArchFromLinkCommand(...)`.
- Creates a GNU-ld-compatible driver (`GnuLdDriver`) and calls `GnuLdDriver::link(...)`.

Driver flavor selection is implemented in `lib/LinkerWrapper/Driver.cpp`:
- First tries to infer a target from the program name (e.g. `arm-link`, `aarch64-link`, `hexagon-link`).
- Otherwise inspects early arguments like `-m <emulation>` or `-march` to select a target-specific driver.

Environment hooks that affect arguments:
- `ELDFLAGS`: appended to the link command by the driver (useful for always-on debug flags).

6.3. Argument parsing and preprocessing

The top-level flow of option parsing and dispatch is in `lib/LinkerWrapper/GnuLdDriver.cpp` (`GnuLdDriver::link(...)`):

1. `parseOptions(...)`
2. `processLLVMOptions(...)` (parses `-mllvm ...` arguments)
3. `processTargetOptions(...)` (handles `-mtriple`, `-march`, `-mabi`, `-m <emulation>`, etc.)
4. `processOptions(...)` (general linker options)
5. `checkOptions(...)` and `overrideOptions(...)`
6. `createInputActions(...)` to build the ordered action list
7. `doLink(...)` to run the actual link pipeline

If you suspect argument/option issues, start with:
- `--verbose` (or `--verbose=<level>`)
- `--trace=command-line`
- `--trace=files` or `-t` (prints processed files)
- `--error-style=GNU` or `--error-style=LLVM` (if output formatting matters)

6.4. Input actions (what gets fed to the linker)

After parsing options, the driver builds a sequence of actions that are later “activated” to create inputs:
- `-T <script>` / `--default-script <script>` -> `ScriptAction`
- `-R <file>` -> `JustSymbolsAction`
- `--defsym <sym>=<expr>` -> `DefSymAction`
- `-l <namespec>` -> `NamespecAction` (library search)
- Plain inputs -> `InputFileAction`
- State toggles like `--whole-archive`, `--as-needed`, `--start-group/--end-group`, `--start-lib/--end-lib` -> corresponding actions that affect how subsequent inputs are treated

This happens in `GnuLdDriver::createInputActions(...)` in `lib/LinkerWrapper/GnuLdDriver.cpp`.

Debug tip: if you see failures about mismatched groups/libs, the error is detected here (before any ELF parsing starts).

6.5. Link pipeline overview (doLink -> Linker)

Once actions are created, `GnuLdDriver::doLink(...)` does the setup:
- Looks up the LLVM target and the ELD target based on the chosen triple.
- Creates a `Module` (`lib/Core/Module.cpp`) and optional `LayoutInfo` for map printing.
- Selects map printers based on `--MapStyle=...` (or defaults) and prepares layout printers.
- Constructs an `eld::Linker` and runs: * `linker.prepare(actions, target)` * `linker.link()` (unless running “LTO-only” modes) * `linker.printLayout()` (map file emission)
- Runs plugin teardown hooks, unloads plugins, emits stats, and finalizes the diagnostic engine summary.

6.6. Prepare phase (Linker::prepare)

`Linker::prepare(...)` (`lib/Core/Linker.cpp`) is responsible for:

1. **Target/emulation + backend**
   - Initializes emulator and backend for the selected target.
2. **Initialize inputs**
   - Builds the input tree, creates internal linker-generated inputs, activates the action list, and reads linker scripts.
3. **Universal plugins**
   - Reads plugin configuration, loads universal plugins from the script, stores them, and runs plugin init hooks.
4. **Read/normalize inputs**
   - Reads all input files, sections, symbols, and (optionally) runs LTO-related preprocessing.

Common debug levers for this phase:
- `--trace=linker-script` or `--trace-linker-script` (script parsing)
- `--trace=threads` (parallel input/section reading behavior)
- `--trace=LTO` or `--trace-lto` (LTO stage boundaries)
- `--plugin-config=<config-file>` and `--no-default-plugins` (plugin triage)

6.7. Normalize phase (Linker::normalize)

`Linker::normalize()` (`lib/Core/Linker.cpp`) performs:
- Optional command-line header/summary printing when `--trace=command-line` is enabled (via `LinkerConfig::printOptions(...)` in `lib/Config/LinkerConfig.cpp`).
- Reading all input files via `ObjLinker->normalize()`: * Parses ELF objects/archives/shared libraries/bitcode inputs. * Populates symbol tables and initial symbol resolution.
- Loads non-universal plugins.
- Computes code position (static/dynamic/PIE) and validates incompatible options (e.g. combining shared-library output with static-only flags).
- Parses external scripts:
  - Version scripts
  - Dynamic list (when building dynamic artifacts)
- Adds linker-script-defined symbols.
- LTO steps (when needed):
  - Creates an LTO object from bitcode inputs.
  - Re-runs normalization post-LTO after replacing bitcode with generated objects.

Debug tip: LTO-related failures often reproduce reliably with `--reproduce` because the tarball will include bitcode inputs and any generated LTO objects recorded by the linker.

6.8. Resolve + layout + emit (Linker::link)

`Linker::link()` in `lib/Core/Linker.cpp` is the main “work” phase:

1. **Standard sections**
   - Initializes default sections (and per-file synthetic dynamic sections when producing dynamic outputs).
2. **Resolve (symbol/reloc processing)**
   - Reads relocations.
   - Allocates common symbols.
   - Assigns output sections using default + linker script rules.
   - Runs plugin hooks around rule matching/layout.
   - Processes target-specific input handling.
   - Optionally performs garbage collection / stripping.
   - Scans relocations, finalizes scan results, and builds output/dynamic symbol tables.
   - Merges sections and creates section symbols.
3. **Layout**
   - Initializes stubs/trampolines, prelayout, merge-strings optimization.
   - Establishes final layout and postlayout output section table.
   - Finalizes symbol values, runs output-section iterator plugins, applies relocations, and finalizes output state.
4. **Emit**
   - Computes output file size and creates an `llvm::FileOutputBuffer`.

- Writes section contents, performs post-processing, emits Build ID, commits the output, and optionally verifies the output size on disk.

If a failure happens late (layout/emit), map files are usually the fastest way to pinpoint the problematic section/segment/relocation.

6.9. Internal inputs and "internal sections"

ELD creates a number of linker-generated inputs up front so later stages can treat them uniformly as normal inputs/sections/symbols.

Creation happens in `Module::createInternalInputs()` (`lib/Core/Module.cpp`). Each internal input corresponds to a named `Input`/`InputFile` (for example `Attributes`, `CommonSymbols`, `DynamicSections`, `Trampoline`, and others) and is used to host sections/fragments that are not sourced from a user object file.

Two other "internal" concepts are easy to confuse:
- **Linker-internal input sections**: sections owned by an internal input file (`Input::Internal`). These typically have `LDFileFormat::Internal` kind and may carry relocations to be applied later.
- **Output-format sections**: sections that come from the backend/output format (not from a user input file), for example dynamic tables/headers. These are treated as output sections and can be matched/discarded via linker-script rules; see `ObjectLinker::markDiscardFileFormatSections()` in `lib/Object/ObjectLinker.cpp`.

Debug tips:
- If you suspect an unexpected section exists (or is missing), prefer a text map: `-M --MapStyle=Text --Map=<file>`.
- If a section is unexpectedly discarded, use `--trace-section <name>` and check whether it matched a `/DISCARD/` rule.

6.10. Section merging (input sections -> output sections)

The "merge sections" name in ELD means: take the *input* section graph (from object files + internal inputs), and populate the *output* section layout according to default rules + linker-script rules + plugins.

There are three distinct sub-steps to keep straight:

1. **Rule matching / output section assignment**
   - `ObjectLinker::assignOutputSections(...)` (`lib/Object/ObjectLinker.cpp`) uses an `ObjectBuilder` to match input sections against linker-script rules (including wildcards, sorting policies, `EXCLUDE_FILE`, etc).
2. **Input section merging**
   - `ObjectLinker::mergeSections()` calls `mergeInputSections(...)` which iterates all input sections and merges them into output sections, with special handling for some section kinds (`.eh_frame`, `.sframe`, target overrides, linkonce/reloc sections, etc).
3. **Finalize output sections**
   - `createOutputSection(...)` builds each output section's fragment list, computes flags/alignment, assigns fragment offsets, and inserts the output sections into the module's output section table.

6.10.1. Special-case section handling during merging

`ObjectLinker::mergeInputSections(...)` (`lib/Object/ObjectLinker.cpp`) handles some input section kinds specially:
- `LDFileFormat::Relocation` and `LDFileFormat::LinkOnce`: if the "link" section is discarded/ignored, the relocation/linkonce section is ignored too.
- `LDFileFormat::Target`: backends may override merging via `GNULDBackend::DoesOverrideMerge(...)` and `GNULDBackend::mergeSection(...)`.
- `LDFileFormat::EhFrame`:
  - Splits and re-chunks `.eh_frame` into CIE/FDE fragments.
  - If enabled, registers content for `.eh_frame_hdr` and creates filler/hdr fragments in the backend.
- `LDFileFormat::SFrame`:
  - Parses the section and may create an `SFrame` header fragment when configured.

Everything else typically flows through `ObjectBuilder::mergeSection(...)` and ends up contributing fragments to an output section.

6.10.2. Output section construction and offsets

Once merging decides *which* fragments belong to a particular output section, `ObjectLinker::createOutputSection(...)` and `ObjectLinker::assignOffset(...)` lay them out:
- Output section `ALIGN` / input section `SUBALIGN` (from linker script) is enforced when present.

- “Dirty” rules (modified by plugins) trigger a recomputation of input section flags/type/align based on the fragments that ended up in the rule.
- Fragment offsets are assigned linearly; per-fragment padding/alignment is applied by `Fragment::paddingSize()` (`lib/Fragment/Fragment.cpp`).

Debug tips:
- If you see “offset not assigned” diagnostics, the fragment/section likely never got placed into an output section (or got discarded). The diagnostic plumbing is in `Fragment::getOffset(...)` (`lib/Fragment/Fragment.cpp`).
- If rule sorting changes layout unexpectedly, check whether the linker script wildcard includes a sort policy, or whether `--sort-section=...` is enabled.

6.11. String merging (MergeString fragments)

String merging is a dedicated optimization pass that runs during layout, before final output layout is established:
- `ObjectLinker::doMergeStrings()` calls:
  - `mergeIdenticalStrings()`: merges `MergeStringFragment` content (can be threaded across output sections; global non-alloc merge is done single-threaded).
  - `fixMergeStringRelocations()`: updates relocations that refer into merged strings via `Relocator::doMergeStrings(...)`.

The output offset calculation for merged strings is special-cased in `FragmentRef::getOutputOffset(...)` (`lib/Fragment/FragmentRef.cpp`), because multiple input strings may map to a shared output string (including suffix merging).

Debug tips:
- Use `--trace=merge-strings` / `--trace-merge-strings=<option>` to see why strings were merged and how offsets were computed.

6.12. Relocations: read -> scan -> apply -> (optional) emit reloc sections

There are multiple relocation passes, and confusing them is a common source of “where did this relocation come from?” debugging pain.

6.12.1. Read relocations

`ObjectLinker::readRelocations()` reads relocations from input objects (`lib/Object/ObjectLinker.cpp`):
- Skips non-object inputs, and skips inputs marked “just symbols”.

6.12.2. Scan relocations (reservation / planning)

`ObjectLinker::scanRelocations(...)` (`lib/Object/ObjectLinker.cpp`) is the “planning” pass. It typically:
- Invokes `Relocator::scanRelocation(...)` per relocation, which is where the backend decides whether it needs to reserve GOT/PLT entries, create or reserve dynamic relocations, create stubs/trampolines, etc. (target-specific logic is in `lib/Target/*/*Relocator.cpp`).
- Collects copy-relocation candidates per input, then creates copy relocations once per symbol (see `createCopyRelocation(...)` / `addCopyReloc(...)` in `lib/Object/ObjectLinker.cpp`).
- Merges per-file dynamic relocation vectors into a single “reloc input” (`getDynamicSectionHeadersInputFile()`) so later code can treat them consistently.
- Runs `ObjectLinker::finalizeScanRelocations()` which calls `GNULDBackend::finalizeScanRelocations()` for back-end-specific finalization.

In relocatable/partial links, ELD uses `partialScanRelocation(...)` instead.

6.12.3. Create output relocation sections (`--emit-relocs`)

If `--emit-relocs` is enabled, ELD creates output relocation sections during prelayout:
- `ObjectLinker::prelayout()` calls `createRelocationSections()`.
- `createRelocationSections()` counts relocations per output section and creates the corresponding output relocation sections (`.rel.<sec>` / `.rela.<sec>` style, based on target) sized to hold all entries.

6.12.4. Apply relocations (writes relocation results)

Relocation application happens in `ObjectLinker::relocation(...)`:
- Applies internal/linker-created relocations.
- Applies input relocations, skipping relocations that are known to be relaxed or that target discarded/ignored sections/symbols.

- Applies branch-island (relaxation) relocations after input relocations are applied.
- If `--emit-relocs` is enabled, emits external-form relocation records into the output relocation sections (via `EmitOneReloc`).

Finally, `syncRelocations(...)` writes relocation results into the output buffer, including extra ordering/barriers to avoid races when multi-threaded.

Debug tips:
- `--trace=reloc=<pattern>` pinpoints a single relocation kind.
- `--trace=symbol=<name>` helps tie relocations back to symbol resolution.
- If you see overflows/unencodable immediates, diagnostics originate from `Relocation::issueSignedOverflow(...)` / `issueUnencodableImmediate(...)` in `lib/Readers/Relocation.cpp`.

6.13. Dynamic relocations (what creates `.rel[a].dyn` / `.rel[a].plt`)

Dynamic relocation entries are typically created/reserved during the relocation scan phase inside the target relocator and backend:
- Target relocators decide whether a given relocation needs: * a static relocation only, * a dynamic relocation (REL/RELA), * a PLT/GOT entry (and an associated relocation), * a copy relocation (for executable data symbol imports).
- Backends provide the actual sections for dynamic relocations (for example `.rela.dyn` / `.rela.plt`) and may sort/finalize them. A common set of helper logic lives in `lib/Target/GNULDBackend.cpp`.

You can generally think of the relocation scan as “reserving and populating dynamic relocation vectors”, and layout/emission as “placing and writing those sections”.

6.14. Garbage collection (`--gc-sections`)

Garbage collection in ELD is graph reachability over sections, built from relocations and a chosen root set (“entry sections”).

6.14.1. Where it runs

The default GC pass is triggered during the resolve phase:
- `ObjectLinker::dataStrippingOpt()` checks `IRBuilder::shouldRunGarbageCollection()` and calls `ObjectLinker::runGarbageCollection(\"GC\")`.
- The implementation is `GarbageCollection` in `lib/GarbageCollection/GarbageCollection.cpp`.

6.14.2. How the graph is built

`GarbageCollection::setUpReachedSectionsAndSymbols()`:
- Traverses input relocations and records, per “apply section”, the set of reachable target sections and reachable symbols.
- Handles special cases:
  - Script-defined symbols: walks the assignment expression’s symbol references.
  - Magic `__start_*`/`__stop_*` symbols: forces sections with matching names into the reachable set.
  - Bitcode: defers some reachability until it can map referenced symbols back to bitcode “input sections”.
- Allows the backend to add extra reachability via `GNULDBackend::setUpReachedSectionsForGC(...)`.

6.14.3. How entry sections are chosen

`GarbageCollection::getEntrySections()` considers multiple root sources:
- The configured entry symbol (if it resolves to a fragment).
- Sections matched by `KEEP(...)` in the linker script.
- When producing dynamic outputs, exported/visible symbols (subject to version script scoping) contribute entry sections.
- Sections marked with `SHF_GNU_RETAIN` are treated as entry-like.

6.14.4. Mark-and-sweep

`GarbageCollection::findReferencedSectionsAndSymbols(...)` performs a BFS from entry sections, following the reachability map built earlier, producing a live set. `stripSections(...)` then marks sections not in the live set as ignored (and can optionally print what got collected).

Debug tips:
- `--print-gc-sections` shows what got collected.
- `--trace=garbage-collection` and `--trace=live-edges` are useful when a section is unexpectedly dead/alive.

- If GC keeps/drops a zero-sized section unexpectedly, check whether it is the target of a relocation or contains a symbol (see the FAQ discussion of zero-sized sections).

6.15. Fragment model (Fragment / FragmentRef)

ELD uses a fragment model internally where **fragments are the minimum linking unit**, not sections.

6.15.1. Fragments

`Fragment` (`include/eld/Fragment/Fragment.h`) represents a typed chunk of content that will appear in the output. Examples include:

- raw data regions (region fragments),
- stubs / trampolines / branch island content,
- GOT / PLT entries,
- mergeable strings,
- `.eh_frame`-related pieces (CIE/FDE fragments),
- build-id fragments, timing fragments, and others.

Each fragment belongs to an owning (input) section and has:

- an alignment requirement,
- an assigned (unaligned) offset, and derived padding size,
- an `emit(...)` implementation that writes bytes during output generation.

6.15.2. Offsets, padding, and “why is this input marked used?”

During output section construction, ELD assigns fragment offsets linearly. The final effective offset includes per-fragment padding computed by `Fragment::paddingSize()` (`lib/Fragment/Fragment.cpp`). When a fragment offset is assigned, `Fragment::setOffset(...)` also marks the owning input as “used” when it contributes allocatable content (this feeds into GC and diagnostics).

6.15.3. FragmentRef (symbol/relocation addressing)

`FragmentRef` (`include/eld/Fragment/FragmentRef.h`) is a pointer to:

- a fragment, plus
- an additional byte offset within that fragment.

This is the core indirection used by:

- output symbols (symbols carry a `FragmentRef` to their definition),
- relocations (relocation “place” and/or “target” is a `FragmentRef`).

Output offset computation is not always “fragment offset + ref offset”:

- `FragmentRef::getOutputOffset(...)` special-cases `.eh_frame` to map offsets through the split/piece layout.
- It also special-cases merged strings so references land on the deduplicated output string (including suffix merging).

Debug tips:

- If a relocation points somewhere surprising, inspect: * the relocation’s `targetRef` (place), and * the symbol’s `fragRef` (definition), and remember both can have special-cased output offset behavior.

6.16. Map files (layout printers)

Map emission is handled after the link attempt in `Linker::printLayout()` and also from the crash signal handler.

Key options:

- `-M` / `--print-map`: enable map generation
- `--Map=<filename>`: choose map output file
- `--MapStyle=<YAML|Text|Binary>`: choose format(s)
- `--MapDetail=<option>`: more detail in maps
- `--color-map`: colorize map output
- `--trampoline-map <filename>`: trampoline information (YAML)

6.17. Reproducing failures (tarball + mapping file)

ELD can capture a self-contained reproducer for link issues:

- `--reproduce <tarfilename>|default`: always produce a tarball; use `--reproduce=default` to name the tar after the output file (`<output>.tar`, where `<output>` is the `-o` filename, or `a.out.tar` if `-o` is not given)
- `--reproduce-compressed <tarfilename>`: compressed tarball
- `--reproduce-on-fail <tarfilename>|default`: only on failure; use `--reproduce-on-fail=default` to name the tar after the output file

- `ELD_REPRODUCE_CREATE_TAR`: environment variable that forces reproducer creation (uses a temporary tar file if no filename is provided)

Additional reproduce helpers:

- --mapping-file <INI-file>: reproduce link using a mapping file
- --dump-mapping-file <outputfilename>: dump mapping info
- --dump-response-file <outputfilename>: dump rewritten response file

The reproduce tarball logic is wired through:

- `GnuLdDriver::handleReproduce(...)` and `writeReproduceTar(...)` in `lib/LinkerWrapper/GnuLdDriver.cpp`
- `Module::createOutputTarWriter()` creation decision via `LinkerConfig::shouldCreateReproduceTar()` (`lib/Config/LinkerConfig.cpp`)

6.18. Crash/signal behavior (what gets written on a crash)

ELD installs a default signal handler in `GnuLdDriver::doLink(...)`:

- Flushes a text map file (if configured).
- Detects likely plugin crashes and reports them.
- Writes a temporary `.sh` script that appends `--reproduce=default` to the original command line; when the user reruns that script the tarball is named <output>.tar based on the `-o` output filename.

This is implemented in `GnuLdDriver::defaultSignalHandler(...)` in `lib/LinkerWrapper/GnuLdDriver.cpp`.

6.19. Where failures typically come from (symptoms -> pipeline stage)

This section is meant as a quick index: if you see a symptom, these are the stages/files to inspect first. Many of these topics are also discussed in more detail in `docs/userguide/documentation/linker_faq.md`.

6.19.1. Driver/target selection failures

Symptoms:

- “unsupported emulation” / “cannot find target”
- wrong backend chosen when using `-m` / `-march`

Start here:

- `Driver::getDriverFlavorFromLinkCommand(...)` in `lib/LinkerWrapper/Driver.cpp`
- `GnuLdDriver::processTargetOptions(...)` and target lookups in `GnuLdDriver::doLink(...)` (`lib/LinkerWrapper/GnuLdDriver.cpp`)

6.19.2. Input specification / archive/group issues

Symptoms:

- “mismatched group” / “mismatched lib”
- unexpected missing objects from an archive

Start here:

- `GnuLdDriver::createInputActions(...)` for `--start-group`/`--end-group` and `--start-lib`/`--end-lib` balancing and ordering
- Use `-t` / `--trace=files` to confirm what ELD actually processed

6.19.3. Linker script rule-matching errors

Symptoms:

- “no linker script rule for “.bss”” / “.data.bar” style errors
- sections landing in unexpected output sections

Start here:

- `ObjectLinker::assignOutputSections(...)` and friends in `lib/Object/ObjectLinker.cpp`
- Emit a map file and confirm whether the input section matched any rule; the FAQ has a guide for diagnosing these errors and for finding used/unused rules.
- If this is a *script parsing/syntax* problem (rather than rule matching), enable `--trace=linker-script` / `--trace-linker-script` and use `--reproduce[-on-fail]` to capture the exact script(s) and rewritten command line that ELD used.

Practical tips:

- Reduce the script: comment out most rules and add back until the behavior flips. For rule-matching bugs, keep only the relevant `SECTIONS` rules and a minimal `MEMORY` map.
- Prefer map files while iterating: they show which input section went to which output section and why that changed across experiments.

6.19.4. Linker script parsing and evaluation errors

Symptoms:
- parse errors (unexpected token, unexpected `)`, etc)
- script expression issues (undefined symbol in an expression, unexpected value)
- placement issues that are script-driven (for example: region overflow, PHDR mismatch, or a section being forced into an incompatible segment)

Start here:
- `--trace=linker-script` / `--trace-linker-script` to see script parsing, includes, and key evaluation decisions.
- A text map file to confirm what the script actually did: memory regions, output section addresses, and segment layout are usually visible there.
- `--reproduce[-on-fail]` so the exact script(s) used by the link are captured alongside the rewritten response file (this is critical when scripts are generated by the build system).

6.19.5. Undefined references and symbol resolution surprises

Symptoms:
- “undefined reference” failures
- symbol unexpectedly resolved from a different archive/object

Start here:
- Resolve phase in `Linker::resolve()` (`lib/Core/Linker.cpp`) and the diagnostic engine output
- Use `--trace=symbol=<name>` (or `--trace=all-symbols`) to see the resolution path

When the failure is a *runtime* crash (not a link error), symbol resolution can still be the root cause:
- Wrong interposition/visibility (a symbol resolves but to an unexpected definition at runtime).
- Lazy binding via PLT (a crash happens on the first call into a function that is resolved late by the dynamic loader).

Quick inspections:

```
# Symbol tables (static and dynamic), with type/binding/visibility:
llvm-readelf -s --demangle --extra-sym-info ./app | less
llvm-readelf -s --demangle --extra-sym-info ./libfoo.so | less

# Dynamic symbols only (what the runtime loader sees):
llvm-readelf --dyn-syms --demangle --extra-sym-info ./libfoo.so | less

# Undefined dynamic symbols (what must be provided by dependencies):
llvm-readelf --dyn-syms --demangle --extra-sym-info ./libfoo.so | awk '$7==\"UND\" {print}'

# Imported/Exported symbols (alternate views):
nm -D --defined-only ./libfoo.so | less
nm -D --undefined-only ./libfoo.so | less
```

6.19.6. Garbage collection removed something needed

Symptoms:
- function/data present in inputs but missing from output
- a section disappears only with `--gc-sections`

Start here:
- `GarbageCollection` implementation in `lib/GarbageCollection/GarbageCollection.cpp`
- `--print-gc-sections` plus `--trace=garbage-collection` / `--trace=live-edges`
- Ensure linker-script `KEEP(...)` is used for sections that must never be GC’d

6.19.7. Relocation overflows / unencodable immediates / target-specific relocation bugs

Symptoms:
- overflow/unencodable relocation diagnostics
- crashes during relocation scan/apply
- output runs but has wrong addresses at runtime

Start here:
- Scan phase: `ObjectLinker::scanRelocations(...)` and target relocators (`lib/Target/*/*Relocator.cpp`)
- Apply phase: `ObjectLinker::relocation(...)` and sync/writeback (`lib/Object/ObjectLinker.cpp`)
- Diagnostics: `lib/Readers/Relocation.cpp` (location printing, overflow, etc)

6.19.8. Trampolines / stubs / relaxation issues

Symptoms:
- failures mentioning trampolines, far calls, or branch islands
- layout changes causing new trampolines or changing trampoline reuse

Start here:
- `ObjectLinker::initStubs()` and target stub factories/backends
- Map/trampoline map options (`--trampoline-map`) plus FAQ sections on trampoline naming and reuse controls

6.19.9. LTO failures

Symptoms:
- failures only with `-flto` / ThinLTO / Full LTO
- “LTO merge error” / codegen diagnostics

Start here:
- `ObjectLinker::createLTOObject()` and LTO diagnostics handler in `lib/Object/ObjectLinker.cpp`
- `--trace=LTO` / `--trace-lto` and `--reproduce[-on-fail]` to capture inputs and generated objects
- `--save-temps` (or `--save-temps=<dir>`) to preserve intermediate LTO artifacts for inspection (files use the prefix `<output>.llvm-lto.*`)
- If you need stable, non-temporary LTO-generated objects: `--lto-obj-path=<prefix>` (also keeps the objects from being deleted after LTO)

6.19.10. Plugin-caused failures

Symptoms:
- crashes only when a plugin is configured
- non-deterministic behavior across runs with the same inputs

Start here:
- `--plugin-activity-file=<file>` to capture plugin activity
- `--no-default-plugins` to isolate
- Crash handler output from `GnuLdDriver::defaultSignalHandler(...)` can explicitly call out a plugin as the likely crash source

6.19.11. Output emission failures

Symptoms:
- “unwritable output” / commit errors
- output size verification failures

Start here:
- `Linker::emit()` in `lib/Core/Linker.cpp` (`llvm::FileOutputBuffer` creation/commit)

6.20. Practical debugging checklist

When a link fails and you need actionable data quickly, try (in order):

1. Add `--reproduce-on-fail=default` (or `--reproduce=default`) to get a `<output>.tar` tarball automatically; supply an explicit filename if needed.
2. Add `--verbose --trace=command-line --trace=files`.
3. Enable map output: `-M --Map=layout.map --MapStyle=Text` (or YAML).
4. If plugins are involved: `--plugin-activity-file=plugins.json` and try `--no-default-plugins` to isolate.
5. If time-sensitive or flaky: `--print-timing-stats` and consider `--emit-timing-stats=<file>` to capture timing consistently.

6.21. Debugging runtime crashes in ELD-built images

This section is for cases where the link *succeeds* but the produced ELF image (executable / shared library / firmware image) fails at runtime (crash, abort, unexpected exception, bad unwind/backtrace, etc.).

6.21.1. Preserve the right artifacts

For runtime debugging, the most common blocker is having only a stripped image with no symbols or line tables.

Keep (or be able to re-create) at least:
- The exact linked ELF that ran (same build-id if you use build-ids).
- An unstripped ELF (or a separate `.debug` file) that matches the runtime image.

- The ELD map file (`--Map=... --MapStyle=Text` or YAML) for fast address-to-section/symbol correlation.
- The crash report: PC/LR/SP, full backtrace if available, and a core dump when possible.

If your production image must be stripped, keep debug info out-of-band using `llvm-objcopy` (or GNU `objcopy`):

```
llvm-objcopy --only-keep-debug app app.debug
llvm-objcopy --strip-debug --strip-unneeded app app.stripped
llvm-objcopy --add-gnu-debuglink=app.debug app.stripped
```

6.21.2. Use linker map files for layout correlation

When debugging runtime failures that are layout-sensitive (for example: wrong PLT/GOT access, unexpected text/rodata placement, thunk/trampoline differences, RELRO placement), generate and keep a linker map file for the exact link:

```
eld ... -M --Map=layout.map --MapStyle=Text
```

The map file is often the fastest way to answer: “which output section/segment contains this address?” and “why did this archive member/section get pulled in?”.

6.21.3. Symbolize a crash address (PC) quickly

If you have an address from a crash report (for example `PC=0x...`) you can usually get <file:line> without opening a debugger:

```
# Pick one:
llvm-addr2line -f -C -e ./app 0xADDR
addr2line      -f -C -e ./app 0xADDR
```

For PIE executables and shared libraries under ASLR, `0xADDR` is typically a *runtime virtual address*. Convert it to an ELF-relative address first:

1. Find the module load base (`/proc/<pid>/maps` for a running process, or `image list -o -f` in lldb for a core).
2. Compute `REL = ADDR - BASE`.
3. Run `addr2line` on `REL` using the corresponding module file.

If you are using sanitizers, make sure symbolization is enabled and points at a working symbolizer:

- `LLVM_SYMBOLIZER_PATH=/path/to/llvm-symbolizer`
- `ASAN_OPTIONS=symbolize=1:abort_on_error=1` (plus your project defaults)
- `UBSAN_OPTIONS=print_stacktrace=1`

6.21.4. Debug with lldb (core dumps and live debugging)

For a core dump:

```
ulimit -c unlimited
./app   # reproduce crash
lldb -c core ./app
```

On systems using `systemd-coredump`, `coredumpctl` is often the easiest:

```
coredumpctl list ./app
coredumpctl dump <PID> --output=core
lldb -c core ./app
```

In lldb, start with:

- `bt` / `thread backtrace all`
- `register read` and `disassemble -m -p --start-address $pc`
- `image list -o -f` (verify loaded modules + load addresses)

If you cannot run the image locally (cross/embedded), use `lldb-server` on the target and attach from the host toolchain debugger.

6.21.5. Run musl builds under qemu (quick cross-runtime triage)

If you need a fast, reproducible runtime environment (especially for cross-target issues), a practical workflow is: build a small musl-based binary and run it under qemu (user-mode or system-mode).

6.21.5.1. User-mode qemu (recommended for simple tests)

For Linux user-mode emulation, run the target executable directly:

```
# Examples (pick your target qemu):
qemu-aarch64 ./app
qemu-arm     ./app
qemu-riscv64 ./app
qemu-riscv32 ./app
qemu-x86_64  ./app
```

If the binary is dynamically linked, point qemu at a sysroot that contains the target loader + shared libraries (musl or glibc as appropriate):

```
qemu-aarch64 -L /path/to/<triplet>/sysroot ./app
```

Common qemu-user tips:

- `-strace` prints syscalls (useful when a crash is actually an `ENOENT` / loader issue).
- `-E VAR=...` sets environment variables for the emulated program (for example `-E LD_LIBRARY_PATH=...`).
- `-d in_asm,cpu,exec -D qemu.log` logs executed instructions; it is noisy but can pinpoint the last instruction before a crash.
- `-g <port>` enables a gdbstub; you can attach with lldb using gdb-remote:

  ```
  qemu-aarch64 -g 1234 ./app
  lldb ./app
  (lldb) gdb-remote 1234
  ```

6.21.5.2. System-mode qemu (when you need a full OS image)

Use system-mode (`qemu-system-*`) when user-mode is insufficient (for example: kernel/driver interactions, missing syscalls, or you need a full rootfs).

In system-mode, typical debugging flags include:

- `-s -S` (open gdbstub and stop at reset)
- `-d in_asm,cpu,exec -D qemu.log` (instruction logging; very verbose)

6.21.6. Inspect exception handling and unwinding

Runtime failures that look like “crash while unwinding”, “terminate called after throwing”, or incorrect backtraces typically reduce to missing/mismatched unwind or exception tables.

At link time, ELD may merge/synthesize unwind-related sections (for example `.eh_frame` and `.eh_frame_hdr`) and can also process SFrame (`.sframe` with `--sframe-hdr`). For ARM EHABI you may also see `.ARM.exidx` / `.ARM.extab`.

Quick checks:

```
llvm-readelf -S ./app | grep -E \"eh_frame|eh_frame_hdr|gcc_except_table|ARM\\.exidx|ARM\\.extab|sframe\"
llvm-readelf --unwind ./app   # unwind info (includes .eh_frame when present)
```

Common causes of missing/insufficient unwind info:

- Built without unwind tables (toolchain flags such as `-fno-asynchronous-unwind-tables`, `-fno-unwind-tables`).
- Over-aggressive stripping (for example link-time `--strip-debug`) combined with not keeping a matching `.debug` file.
- Inconsistent binaries/libraries at runtime (debugging with one ELF but running a different one; mismatched build-ids).
- For C++ exceptions: missing runtime pieces (for example `__gxx_personality_v0` not resolved, or the wrong unwinder/libgcc_s on the target).

If the problem is "backtrace is garbage" rather than exceptions specifically, also consider building with frame pointers (for example `-fno-omit-frame-pointer`) and validating that your unwinder matches the format your toolchain emits (`.eh_frame` vs SFrame vs target-specific unwind tables).

6.21.7. Inspect loader and shared-library problems

Crashes very early in process startup (before `main`) are often due to dynamic loader configuration rather than "code bugs".

```
llvm-readelf -l ./app    # interpreter (PT_INTERP) and program headers
llvm-readelf -d ./app    # NEEDED, RPATH/RUNPATH
ldd ./app            # what the system thinks will load (when available)
```

For glibc-based systems, `LD_DEBUG` can explain loader decisions:

```
LD_DEBUG=libs,bindings ./app
```

6.21.8. Target ABI / relocations / GOT-PLT debugging notes

When debugging runtime crashes that involve dynamic linking, relocation application, or address materialization sequences, it helps to look at:

- Relocations: `llvm-readelf -r ./app` and `llvm-readelf --dyn-relocations ./app`.
- Dynamic table: `llvm-readelf -d ./app` (`NEEDED`, `RPATH/RUNPATH`, flags).
- Program headers: `llvm-readelf -l ./app` (`PT_LOAD` flags/alignment, `PT_GNU_RELRO`).
- GOT/PLT-related sections and their contents:

  ```
  llvm-readelf -S ./app | grep -E \"\\.plt|\\.got|\\.rela(\\.plt|\\.dyn)|\\.rel(\\.plt|\\.dyn)\"
  llvm-readelf -x .got ./app        2>/dev/null | less
  llvm-readelf -x .got.plt ./app    2>/dev/null | less
  llvm-readelf -x .plt ./app        2>/dev/null | less
  ```

  Correlate entries with disassembly + relocations:

  ```
  llvm-objdump -dr --no-show-raw-insn ./app | less
  llvm-readelf -r ./app | less
  ```

6.21.8.1. External ABI / ELF references (authoritative relocation tables)

When you need a definitive answer for relocation semantics, PLT/GOT conventions, TLS models, or calling convention/stack rules, prefer the architecture psABI/ABI documents (rather than reverse-engineering from a tool implementation).

Useful starting points:

- Generic ELF / System V ABI:
  - Linux Foundation reference index: ELF and ABI Standards
  - System V gABI Edition 4.1: gabi41.pdf
  - ELF Object File Format (modern publication of the ELF chapters): Xinuos ELF spec (HTML)
- x86_64 System V psABI:
  - psABI PDF: x86_64-SysV-psABI.pdf
  - Sources/repo: x86-64 psABI (GitLab)
- Arm AArch32 / AArch64:
  - Official Arm ABI repository (AAELF32/AAELF64, PCS, EHABI, etc): ARM-software/abi-aa
  - AAELF (Arm ELF, AArch32): IHI0044E_aaelf.pdf
  - AAELF64 (AArch64 ELF): IHI0056G_2020Q2_aaelf64.pdf
- RISC-V psABI:
  - Spec site: riscv-elf-psabi-doc
  - Sources/releases: riscv-elf-psabi-doc (GitHub)
- Qualcomm Hexagon:
  - ABI user guide (includes Hexagon-specific ELF/relocations): Qualcomm Hexagon ABI User Guide (PDF)

Architecture-specific relocation names are target-defined; a quick way to see what you are dealing with is the relocation inventory command in this guide. The following are *common* relocation families you may see during runtime triage:

- **x86_64 ABI**: look for `R_X86_64_*` (for example: `*_RELATIVE`, `*_JUMP_SLOT`, `*_GLOB_DAT`, `*_PC32`, `*_PLT32`, `*_GOTPCREL*`).

- **AArch64 ABI**: look for `R_AARCH64_*` (for example: `*_CALL26`, `*_JUMP26`, `*_ADR_PREL_PG_HI21`, `*_ADD_ABS_LO12_NC`, `*_RELATIVE`, `*_JUMP_SLOT`, `*_GLOB_DAT`).
- **ARM ABI (arm/thumb)**: look for `R_ARM_*` (for example: `*_CALL`, `*_JUMP24`, `*_THM_CALL`, `*_THM_JUMP24`) and, on EHABI platforms, unwind tables like `.ARM.exidx`/`.ARM.extab`.
- **RISC-V 32/64**: look for `R_RISCV_*` (for example: `*_PCREL_HI20`, `*_PCREL_LO12_*`, `*_CALL*`, `*_JAL`, plus dynamic relocations like `*_RELATIVE`, `*_JUMP_SLOT`, `*_GLOB_DAT`).
- **Hexagon ABI**: look for `R_HEX_*` relocations; use `llvm-readelf -r` and disassembly to connect the relocation type to the instruction sequence.

6.21.8.2. ARM-specific: verifying veneers/thunks

On ARM/Thumb, the linker may need to create veneers/thunks (branch islands) when branches cannot reach their targets or when interworking is required.

When a crash looks like a bad branch target or a call landing in the wrong mode:
- Enable and inspect trampoline/thunk diagnostics and maps: `--trampoline-map` (plus the usual text map file).
- Disassemble around the call site and look for a veneer sequence and its relocation(s): `llvm-objdump -dr --start-address=... --stop-address=...`.
- Confirm the callee symbol type and interworking expectations in the symbol table (`llvm-readelf -s --extra-sym-info`).

6.21.9. Minimize runtime failures with A/B experiments

When a runtime crash is hard to reason about, treat it like a minimization problem: change one knob at a time until the failure becomes deterministic, then pinpoint which library/object/relocation pattern is responsible.

Dynamic-linking knobs that often turn “mystery crashes” into actionable data:
- Force eager binding (converts lazy-PLT crashes into startup failures):

  ```
  LD_BIND_NOW=1 ./app
  ```

  Or make it a link-time property of the binary: `-Wl,-z,now`.
- Strengthen shared-library resolution rules (catch unresolved imports sooner): `-Wl,-z,defs` (or `-Wl,--no-undefined` depending on toolchain).
- Toggle dependency pruning and interposition-related behavior: `-Wl,--as-needed` / `-Wl,--no-as-needed` and (when applicable) `-Wl,-Bsymbolic` / `-Wl,-Bsymbolic-functions`.
- Toggle RELRO (affects which GOT/relocation targets become read-only at runtime): `-Wl,-z,relro` and `-Wl,-z,norelro`.

Use these knobs together with map files and relocation inspection (see below) to connect “crash address” -> “instruction” -> “relocation” -> “symbol” -> “DSO that provides it”.

6.21.10. Identify which shared library caused a crash (swap-and-isolate)

If the crash only reproduces with a particular shared library present, isolate it by swapping one dependency at a time:

1. Confirm what actually loads:

   ```
   ldd ./app
   llvm-readelf -d ./app | less
   ```

2. Swap the suspected DSO:
   - Use `LD_LIBRARY_PATH` to point at an alternate build directory.
   - Use `LD_PRELOAD` to force a specific `.so` (for interposition or to override a weak/indirect dependency).
   - Rename/move one dependency to force a load failure; if the crash disappears when the DSO is absent (or replaced), that narrows the search quickly.
3. Ensure you are swapping *compatible* binaries:
   - Same target triple/ABI, same libc/loader expectations.
   - Prefer swapping libraries built by the same toolchain revision first (compiler + assembler + linker + runtimes).
   - Verify build-ids match what you intend to test:

     ```
     llvm-readelf -n ./app | less
     llvm-readelf -n ./libfoo.so | less
     ```

If you suspect a linker-specific issue in a shared library, rebuild just that library with an alternate linker (GNU ld / gold / lld, depending on target support) and re-run the A/B test. If the crash tracks the linker choice with the same sources/flags, it is strong evidence of a link-time layout/relocation problem rather than a pure runtime logic bug.

6.21.11. Switching toolchains/compilers to bisect regressions

If the failure appeared after a toolchain update, a fast way to narrow root cause is to bisect *one dimension at a time*:

- Swap compiler only (keep assembler/linker constant) to see if codegen changes are responsible.
- Swap linker only (keep compiler constant) to see if layout/relocation handling is responsible.
- Swap runtime libraries (libc, libstdc++/libc++, libgcc_s/libunwind) to catch ABI or unwinder differences.

Use build-ids, map files, and the relocation inventory command to keep these experiments grounded in “what changed” rather than “what you think changed”.

6.21.12. Write small regression tests (symbols/relocs/dynamic flags)

When you can reproduce a runtime failure, try to turn it into a small *link-time observable* property so it can be tested without running on a target.

In this repo, most linker tests are lit tests (`*.test`) that:

- compile tiny C/asm inputs,
- run `%link` (eld) with specific flags, and
- verify ELF properties using `%readelf` (`llvm-readelf`), `llvm-readobj`, or `%objdump` + `FileCheck`.

Examples of properties that correlate strongly with runtime behavior:

- Symbol kind/binding/visibility (from `llvm-readelf -s --extra-sym-info` / `nm`).
- Relocation types and placement (from `llvm-readelf -r` / `llvm-readobj -r`).
- Dynamic linking flags and segments (`llvm-readelf -d` for `BIND_NOW` / `FLAGS_1`, and `llvm-readelf -l` for `PT_GNU_RELRO` and `p_align`).

If you need a starting point, see `test/Templates/ExampleOfMyLitTest.test` and existing tests under `test/lld/ELF` that check relocations and flags like `-z now` via `%readelf`/`%objdump`.

Illustrative lit pattern (dynamic flags + RELRO):

```
# RUN: %link %linkopts -shared -z now -z relro %t.o -o %t.so
# RUN: %readelf -d %t.so | FileCheck %s --check-prefix=DYN
# RUN: %readelf -l %t.so | FileCheck %s --check-prefix=PHDR
# DYN: BIND_NOW
# PHDR: GNU_RELRO
```

6.21.13. Other useful inspection tools

- Address/section correlation: `llvm-nm -n`, `nm -n`, `llvm-objdump -d`, `objdump -d`, and the ELD map file.
- ELF metadata: `llvm-readelf -h -l -S -s -n` (build-id in `llvm-readelf -n`).
- Relocation inventory across a build tree (quickly see which relocation types are present in objects/archives):

  ```
  find . -name \"*.o\" -o -name \"*.a\" | xargs llvm-readelf -r | awk '{print $3}' | sort -u | grep R_
  ```

  More robust (handles spaces in paths and avoids `find` precedence traps):

  ```
  find . \\( -name \"*.o\" -o -name \"*.a\" \\) -print0 \\
    | xargs -0 llvm-readelf -r \\
    | awk '{print $3}' \\
    | sort -u \\
    | grep R_
  ```

- Inspect relocations around a crash site (when investigating wrong codegen, bad PLT/GOT usage, or runtime loader fixups):

  ```
  # Disassemble with relocations shown (pick one):
  llvm-objdump -dr --no-show-raw-insn ./app | less
  objdump     -dr --no-show-raw-insn ./app | less

  # Narrow to a region around an address (adjust for PIE/ASLR first):
  llvm-objdump -dr --start-address=0xSTART --stop-address=0xSTOP ./app
  ```

- Segment properties / page alignment / RELRO (loader-relevant):

```
llvm-readelf -l ./app    # program headers: p_flags, p_align, PT_LOAD, PT_GNU_RELRO, PT_INTERP
llvm-readelf -d ./app    # dynamic tags: DT_BIND_NOW, (FLAGS / FLAGS_1), RPATH/RUNPATH
```

- System-call tracing: `strace -f` (and `ltrace` when applicable).
- Memory corruption: ASan/UBSan/TSan builds; `valgrind` when supported.

7. Shared Library Versioning

7.1. Overview

Shared library versioning is a mechanism used in operating systems to manage different versions of dynamic libraries (.so files on Unix-like systems). It ensures that applications link to the correct version of a library while maintaining compatibility.

7.2. Key Concepts

SONAME : - The SONAME (Shared Object Name) is an identifier embedded in a shared library that represents its ABI (Application Binary Interface) version.
- Applications link against the SONAME, not the actual filename, allowing upgrades without breaking compatibility.

Semantic Versioning : - Libraries often follow semantic versioning: `MAJOR.MINOR.PATCH`.
- **MAJOR**: Changes that break backward compatibility.
- **MINOR**: Backward-compatible feature additions.
- **PATCH**: Backward-compatible bug fixes.

7.3. ABI vs API

- **API (Application Programming Interface)**: The set of functions and symbols exposed by the library.
- **ABI (Application Binary Interface)**: The compiled interface, including calling conventions, data types, and memory layout.
- ABI stability is crucial for shared libraries because applications depend on binary compatibility.

7.4. Backward Compatibility

- Libraries should maintain backward compatibility within the same major version.
- Breaking ABI changes require incrementing the major version and updating the SONAME.

7.5. Best Practices

- Use SONAME to manage ABI versions.
- Avoid breaking ABI unless necessary.
- Provide symbolic links: : - `libexample.so` → `libexample.so.1` → `libexample.so.1.2.3`
- Document changes clearly.

7.6. Nuances

- Different distributions may package libraries differently.
- Some systems use `ldconfig` to manage symbolic links.
- Applications may fail if the expected SONAME is missing, even if the actual library file exists.

7.6.1. Examples

7.7. SONAME Usage Example

When building a shared library:

```
clang -shared -Wl,-soname,libexample.so.1 -o libexample.so.1.2.3 example.o
```

Resulting files:

```
libexample.so.1.2.3   # Actual library file
libexample.so.1       # SONAME symlink (ABI version)
libexample.so         # Generic symlink for development
```

Applications link against `libexample.so.1`, not the full filename.

7.8. Semantic Versioning Example

- libmath.so.1.0.0 → Initial release
- libmath.so.1.1.0 → Adds new functions (backward-compatible)
- libmath.so.2.0.0 → Changes function signatures (breaks ABI)

When moving from 1.x.x to 2.x.x, update SONAME to `libmath.so.2`.

7.9. ABI vs API Example

- **API Change (safe)**: Adding a new function `add_matrix()` does not break existing binaries.
- **ABI Break (unsafe)**: Changing `struct Matrix { int rows; int cols; }` to include a new field changes memory layout, breaking existing binaries.

7.10. Symbolic Link Structure

After installation:

```
/usr/lib/
    libexample.so -> libexample.so.1
    libexample.so.1 -> libexample.so.1.2.3
    libexample.so.1.2.3
```

ASCII Diagram:

```
libexample.so → libexample.so.1 → libexample.so.1.2.3
```

7.11. Nuances

- Different Linux distros may package libraries differently.
- If SONAME is missing, apps fail with:

```
error while loading shared libraries: libexample.so.1: cannot open shared object file
```

8. GNU ELF Symbol Versioning

✎ **Note**
Support for symbol versioning is planned for a future release. Stay tuned for updates

This document continues to show a minimal end-to-end demonstration of GNU ELF symbol versioning, then documents:

1. What the `.symver` / `__attribute__((symver))` mechanisms do.
2. What a GNU ld *version script* does.
3. Which ELF sections the linker/loader create/use for symbol versioning.

The example exports two versions of the same symbol and preserves backward compatibility across library releases.

8.1. A. Minimal Working Example (C + GNU ld)

8.2. Goal

Export two versions of an API symbol (`foo`), keeping ABI compatibility for old applications while making a new implementation the default for new applications.

8.3. Example files

`demo_v1.c` — first release of the library:

```c
// v1 of the library: exports foo()
#include <stdio.h>

void foo(void) { puts("foo v1"); }
```

`demo_v1.map` — version script for first release:

```
DEMO_1 {
  global: foo;
  local:  *;
};
```

This assigns `foo` to version node `DEMO_1` and hides everything else.

`demo_v2.c` — second release (keep old `foo` and add a new default `foo`):

```
#include <stdio.h>

/* Approach A: classic assembler directive (works with GCC & Clang) */
__asm__(".symver foo_v1,foo@DEMO_1");    // non-default (old) foo
__asm__(".symver foo_v2,foo@@DEMO_2");   // default (new) foo

void foo_v1(void) { puts("foo v1"); }
void foo_v2(void) { puts("foo v2 (default)"); }

/* New API symbol in v2 */
void bar(void) { puts("bar v2"); }
```

demo_v2.map — version script for second release:

```
DEMO_1 {
  global: foo;
  local:  *;
};

DEMO_2 {
  global: foo; bar;
} DEMO_1;  /* DEMO_2 inherits from DEMO_1 */
```

`main_old.c` — an “old” app built against v1:

```
void foo(void);
int main(void) { foo(); }
```

`main_new.c` — a “new” app built against v2:

```
void foo(void);
void bar(void);
int main(void) { foo(); bar(); }
```

`Makefile`:

```
CC?=clang
CFLAGS=-fPIC -O2 -Wall -Wextra

all: stage1 stage2 inspect

stage1: libdemo.so.1.0 p_old

libdemo.so.1.0: demo_v1.c demo_v1.map
    $(CC) $(CFLAGS) -shared -Wl,-soname,libdemo.so.1 \
        -Wl,--version-script=demo_v1.map -o $@ demo_v1.c
    ln -sf $@ libdemo.so

p_old: main_old.c libdemo.so
    $(CC) -Wl,-rpath,'$$ORIGIN' -L. -o $@ main_old.c -ldemo

stage2: libdemo.so.2.0 p_new

libdemo.so.2.0: demo_v2.c demo_v2.map
    $(CC) $(CFLAGS) -shared -Wl,-soname,libdemo.so.1 \
        -Wl,--version-script=demo_v2.map -o $@ demo_v2.c
    ln -sf $@ libdemo.so
```

```
p_new: main_new.c libdemo.so
	$(CC) -Wl,-rpath,'$$ORIGIN' -L. -o $@ main_new.c -ldemo

inspect:
	@echo "==> Exported (with versions) in libdemo.so"
	@nm -D --with-symbol-versions libdemo.so | egrep ' foo| bar' || true
	@echo; echo "==> Version sections in libdemo.so"
	@readelf -W --version-info libdemo.so | sed -n '1,120p'

clean:
	rm -f p_old p_new libdemo.so libdemo.so.* *.o
```

8.4. Build & Run

```
# Stage 1: build first library + "old" app
make stage1
LD_LIBRARY_PATH=. ./p_old
# -> prints: foo v1
```

```
# Stage 2: replace lib with v2 (keeps SONAME), rebuild "new" app
make stage2
LD_LIBRARY_PATH=. ./p_old
# -> still prints: foo v1  (old app bound to DEMO_1)
LD_LIBRARY_PATH=. ./p_new
# -> prints: "foo v2 (default)" and "bar v2"
```

8.5. Inspecting the Result

See exported symbols and their versions:

```
nm -D --with-symbol-versions libdemo.so | egrep ' foo| bar'
# ... foo@DEMO_1
# ... foo@@DEMO_2
# ... bar@@DEMO_2
```

`@@` marks the *default* version; `@` marks a non-default (older) version.

See version metadata sections:

```
readelf -W --version-info libdemo.so
```

You should see:
- Version definitions (from `.gnu.version_d`).
- Per-symbol version indices in `.gnu.version`.
- (In executables) Version needs in `.gnu.version_r`.

Optional runtime trace from the dynamic linker:

```
LD_DEBUG=versions LD_LIBRARY_PATH=. ./p_new 2>&1 | grep -E 'checking for version|needed'
```

8.5.1. B. What `.symver` / `__attribute__((symver))` Do

- `.symver` (assembler directive)
  - Binds a specific implementation symbol to a public name *and* a version node.
  - Syntax: `.symver <impl>, <public>@<NODE>` or `.symver <impl>, <public>@@<NODE>`.
  - `@@` marks the default implementation selected by new linkers.
  - The version node must exist in the version script when you build the DSO.
- `__attribute__((symver("...")))`
  - Front-end attribute that makes compiler emit the corresponding `.symver`.
  - Handy with LTO because it attaches at the language level.

Exactly one default definition should exist for a symbol (the one with `@@`).

8.5.2. C. What a Version Script Does

A GNU ld *version script* (use with `-Wl,--version-script=...`):

1. Declares **version nodes** and binds **symbols** to those nodes.
2. Controls visibility (`global:` exported; `local:` hidden).
3. Supports **inheritance** between nodes to evolve the ABI without bumping the SONAME.
4. For C++, you can match demangled names with an `extern "C++" { ... }` block.

If you want to tag all currently **exported** symbols with one version without changing visibility, this works:

```
V1 { *; };
```

This does not force hidden symbols to become exported; it only tags whatever is already exported.

8.5.3. D. ELF Sections Used by Symbol Versioning

When you link with versioned symbols, the following GNU extension sections (referenced by dynamic tags) are produced/used:

- `.gnu.version` (*SHT_GNU_versym*, referenced by `DT_VERSYM`)
  - A table parallel to `.dynsym` that stores a 16-bit version index per symbol.
- `.gnu.version_d` (*SHT_GNU_verdef*, referenced by `DT_VERDEF`/`DT_VERDEFNUM`)
  - Version **definitions** provided by this DSO (node names, indices, hashes).
- `.gnu.version_r` (*SHT_GNU_verneed*, referenced by `DT_VERNEED`/`DT_VERNEEDNUM`)
  - Version **requirements** (what this module needs from its dependencies).

`readelf -W --version-info` summarizes these sections so you can verify bindings and defaults.

8.5.4. E. Practical Notes & Common Pitfalls

- C++ name mangling:
  - If you version C++ functions with `.symver`, use the **mangled** name.
  - In version scripts you can use demangled names inside `extern "C++" { ... }`.
- Default vs. non-default:
  - Default shows as `@@` older variants show as `@`.
  - Newly linked apps bind to the default; previously linked apps keep using their recorded version.
- Diagnostics:
  - `nm -D --with-symbol-versions`: quick view of `foo@@V2` / `foo@V1`.
  - `readelf -W --version-info`: detailed view of `.gnu.version*` sections.
  - `LD_DEBUG=versions`: watch the dynamic linker's version checks at runtime.

8.5.5. F. One-Page Cheat Sheet

- `.symver` / `__attribute__((symver))`:
  - Attach a version to a symbol definition; exactly one default (`@@`) per symbol.
  - Ensure the version node exists in the link-time version script.
- Version script:
  - Define version nodes, bind symbols, control visibility, and express inheritance.
  - Can version all currently exported symbols via `V1 { *; };`.
- ELF sections:
  - `.gnu.version` — per-dynamic-symbol version indices (`DT_VERSYM`).
  - `.gnu.version_d` — version definitions (`DT_VERDEF*`).
  - `.gnu.version_r` — version requirements (`DT_VERNEED*`).

8.5.6. Appendix: Quick Commands

```
# Build first version and old app
make stage1
LD_LIBRARY_PATH=. ./p_old

# Upgrade library, build new app
make stage2
LD_LIBRARY_PATH=. ./p_old
LD_LIBRARY_PATH=. ./p_new

# Inspect exports and versions
nm -D --with-symbol-versions libdemo.so | egrep ' foo| bar'
readelf -W --version-info libdemo.so
```

```
# Trace loader checks
```

9. Image Structure and Generation

9.1. Processor-specific flags

Processor-specific flags are stored in the `e_flags` field of the ELF header. `eld` will set these flags according to the emulation specified or inferred from its inputs. The following are special cases for the `e_flags` field:

1. A lone empty object file:

   ```
   touch empty.o
   ld.eld empty.o -m<emulation>
   llvm-readelf -h a.out
   ```

   The output file will have flags set according to the emulation specified.
2. A symbol definition file:

   ```
   ld.eld -m<emulation> symbols.def
   ```

   The output file will have flags set according to the emulation specified.
3. A lone binary file:

   ```
   ld.eld -m<emulation> --format=binary foo.bin
   llvm-readelf -h a.out
   ```

   Flags are always 0 regardless of the emulation specified.

10. Image layout

At its core, the linker is responsible for transforming compiled object files into a final image that is ready to run on the device. This process includes resolving symbols, relocating sections, and most importantly, laying out the image in memory.

Image layout refers to the linker's assignment of addresses to code, data and metadata determining their arrangement in memory when the program runs, and to the placement of contents within the final output image.

In this document, we will take a deep dive into the linker's image layout process, examining the factors that influence the layout and the reasoning behind the specific choices the linker makes. Understanding why the image layout looks the way it does is challenging, but it's invaluable for diagnosing and fixing subtle, hard-to-decode layout issues.

But why is the image layout important?

The image layout is critical for correctness, performance and security. Among other things, it ensures alignment compliance, enables cache-friendly placement of code and data, and enhances security by enforcing memory protection policies such as preventing any executable page from having write permissions. In embedded systems, image layout is especially critical due to tight memory constraints, real-time performance needs, and hardware-specific placement requirements.

Before we dive deep into defining and understanding how the linker does image layout, it would be useful to understand the distinction between some of the terminology used in the linker.

10.1. Linker terminology

10.1.1. Input Sections

A section holds content that can be code, data and metadata (Example: `.text` , `.data`, `.bss`, `.comment`). The linker may reorder, merge or discard whole sections but it treats each section as an indivisible unit. The section header table describes each section's name, type, virtual address, file offset, alignment and more. **The section header table and the sections are used by the linker during linking.** These come from object files (.o) and libraries (.a) compiled from source code.

Examples:
- .text — code
- .data — initialized data
- .bss — uninitialized data
- .rodata — read-only data

10.1.2. Output sections

These are the merged and organized sections in the final binary, created by the linker.

The linker collects input sections of the same type and merges them into output sections.

You can rely on the default rules and built-in heuristics provided by the linker.

You can also control this mapping by using a linker script.

10.1.3. Segments

A segment is composed of one or more output sections and describes how the program should be loaded into memory at runtime. In ELF, The program header table describes each segment’s type, virtual and physical address, file size, memory size, permissions, and alignment requirements. **A loader only looks at the program header table to load an executable into memory.** It does not care about the section header table.

Key Points:
- Segments are containers for output sections
- They define memory permissions (read, write, execute)
- Used by the runtime loader

10.1.3.1. Segment alignment

10.1.3.1.1. Empty segments

- Loadable segments all have segment align set to the page size set at link time
- Non loadable segments have the segment alignment set to the minimum word size of the output ELF file

10.1.3.1.2. Non Empty segments

- Loadable non empty segments have the segment alignment set to the page size
- Non loadable segments are set to the maximum section alignment of the containing sections in the segment

Now let’s get started with understanding the image layout created by eld.

10.2. Image layout

There are two main approaches of defining the image layout:
- Using default behavior
- Using custom linker scripts

10.2.1. Using default behavior

When a linker performs layout without an explicit linker script, it relies on default rules and built-in heuristics provided by the linker implementation.

The linker has a default memory layout that defines:
- The order of sections (e.g., .text, .data, .bss)
- Default starting addresses (e.g., code starts at 0x08000000 for embedded systems or 0x400000 for ELF binaries on Linux)
- Alignment requirements for each section
- Default page alignment
- Whether program headers are loaded or not loaded

10.2.2. linker scripts

✎ **Note**

When using eld with a custom linker script, all default assumptions and behaviors built into the linker are overridden. The script provides complete control over the memory layout and section placement, effectively replacing the linker's internal defaults.

The linker script primary job is to define the image layout requirements. It achieves this with the `SECTIONS`, `PHDRS`, and the `MEMORY` commands.

- `SECTIONS` command specifies the output section properties and the mapping of input sections to the output sections.
- `PHDRS` specifies which program headers (also known as segments) should be created by the linker. If `PHDRS` is not specified, then the linker creates sensible PHDRs based on some default rules.
- `MEMORY` command specifies the available memory regions. The output sections can then be assigned to particular memory regions. It provides a convenient way of arranging the output sections into memory. See MEMORY for syntax, region attribute matching rules, and LMA/VMA placement details.

Linker Script describes these commands in more detail.

Here we will only highlight, or in some cases reiterate, some of the important and/or subtle layout-related features and behavior.

10.2.2.1. Output section contents

An output section is composed of input sections. If no `SECTIONS` command is provided, the linker applies sensible default rules to match the input sections to the output sections.

When a `SECTIONS` command is present, the linker script input section description, commonly referred to as linker script rule or just rule, control the input-output section mapping. Input sections that match no rule are called orphans sections. Each orphan section is placed into an output section with the same name; if no such output section exists, then the linker creates a new one.

10.2.2.2. Image base address (starting address)

The default image base address depend upon the target, the linking type (whether the link is creating an executable or a shared library), and whether the link contains a linker script. The address 0 is used as the starting address if a linker script is present or if the link is creating a shared library.

The default image base value can be overridden by using `--image-base` command-line option.

10.2.2.3. Output section virtual memory address (VMA)

Multiple mechanisms can assign or influence the addresses of output sections. A linker script can specify addresses explicitly; command-line options can set section start addresses; and script directives such as `MEMORY` allows to conveniently arrange sections without hard-coding absolute addresses. Let's examine all these methods, detail their behavior, and clarify their precedence.

10.2.2.3.1. Brief review of the methods

10.2.2.3.1.1. 1) No address/memory-region specified

If no explicit address or MEMORY region is specified for an output section, it is placed at the current value of the location counter (`.`). The location counter is initialized to the image base, and is automatically incremented whenever a content is added. For example, if the value of the location counter is 0x1000 before assigning address to `.foo` output section (size 0x200), then `foo` is placed at `0x1000` and the location counter advances to 0x1200.

The location counter can be explicitly incremented with a linker script assignment:

```
. = . + ALIGN(0x8);
```

If an output section has an explicit address (or a memory region), then the location counter is set to the address (or the valid address in the memory region) before placing the output section.

There is an exception to this, orphan sections whose name ends with `@<address>` are placed exactly at the specified address.

10.2.2.3.1.2. 2) Explicit linker script VMA address

Linker script can specify an explicit address for an output section:

```
section [address] :
{
  output-section-command
  ...
}
```

Let's see this in action with the help of an example:

```
// 1.c
int foo() { return 1; }

int bar() { return 3; }
```

```
// script.t
SECTIONS {
  .foo (0x1000) : { *(.text.foo) }
  .bar (0x2000) { *(.text.bar) }
}
```

The linker assigns the exact addresses as specified in the linker script. `.foo` is assigned the VMA `0x1000` and `bar` is assigned the VMA `0x2000`.

The linker assigns the exact addresses even if the addresses are not *exactly* aligned. For the below example, `.foo` will get VMA assigned to `0x1001` and `.bar` will get VMA assigned to `0x2002` even though they do not satisfy the alignment requirements. In this case, the output section contains alignment padding at the beginning such that the alignment requirements for the actual content (input sections) is satisfied.

```
// 1.c
int foo() { return 1; }

int bar() { return 3; }
```

```
// script.t
SECTIONS {
  .foo (0x1001) : { *(.text.foo) }
  .bar : AT(0x2002) { *(.text.bar) }
}
```

However, if `ALIGN` is also specified, then the explicit VMA is aligned to the alignment specified in `ALIGN`.

Add an example showing explicit VMA + `ALIGN(...)` interaction.

10.2.2.3.1.3. 3) Command-line options for setting section addresses

There are various linker command-line options for setting output section VMA: `-Tbss`, `-Tdata`, `-Ttext` and `--section-start`.

When both the linker script and the command line specify an output-section address, the command-line option takes precedence and overrides the script's explicit address.

10.2.2.3.1.4. 4) Memory region

Assigning an output section to a MEMORY region places it at the region's next available address, subject to alignment and size constraints.

Let's understand this with the help of an example:

```
// 1.c
int a;
int u = 11;
int v = 13;
int foo() { return 1; }
int bar() { return 3; }
```

```
// script.t
MEMORY {
  RAM : ORIGIN = 0x1000, LENGTH = 0x1000
}

SECTIONS {
  .data : { *(.data*) } >RAM
  .foo : { *(.text.foo) } >RAM
  .bss : {
    *(.bss*)
  } >RAM
  .bar : { *(.text.bar) } >RAM
}
```

For the example above, the virtual addresses of the output section are shown below, along with the assumed size and alignment.

- `.data` (size: 0x8; alignment: 0x8): 0x1000
- `.foo` (size: 0x8; alignment: 0x8): 0x1008
- `.bss` (size: 0x4; alignment: 0x4): 0x1010
- `.bar` (size: 0x8; alignment: 0x8): 0x1014

10.2.2.3.2. Precedence of virtual address assignment methods

The precedence is:

1. Command-line options for setting section addresses
2. Explicit linker script VMA address
3. Memory region
4. No address/memory-region specified

10.2.2.4. Segment creation when `PHDRS` is not specified

eld uses a set of heuristics to decide when to start a new segment. Before detailing those heuristics, we need to note how eld walks the layout:

eld processes output sections in the order specified by the linker script; if no script is provided, then it uses the order in which the linker has created default output sections.

We use the below terms to describe when eld creates a segment:

- Previous output section to refer to the output section that immediately precedes the current one in the traversal order.
- Previous allocatable output section to refer to the nearest preceding allocatable output section.
- Previous LOAD segment to refer to the segment of the nearest preceding allocatable output section.

With this traversal model and the terms in mind, we can now describe the segment-creation heuristics.

A new load segment is created when:

- There is no previous LOAD segment.
- Explicit output section address has been set using linker command-line options such as: `-Ttext` or `--section-start`.
- The segment flags required for the current output section is incompatible with the previous LOAD segment. By default, the segment flags `R` and `RE` are compatible, whereas `RW` is incompatible with `R` and `RW`. `--rosegment` and `--omagic` influence which sections can be part of the same segment.
- The memory region of the current output section is different than the memory region of the previous allocatable output section.
- The previous output section type is NOBITS and the current output section type is PROGBITS.
- The previous output section virtual memory address is greater than the current output section virtual memory address.
- The VMA difference between the previous allocatable output section is greater than the segment alignment.

Other segment types:

- A `PT_TLS` segment is created when an input file contains `.tdata`/`.tbss`.
- A `PT_DYNAMIC` segment is created when a shared library or a dynamic executable is getting built.
- A `PT_GNU_EH_FRAME` segment is created when the output contains `eh_frame_hdr` section.

10.2.2.5. Segment creation when `PHDRS` is specified

When the `PHDRS` is specified in the linker script, then the linker only creates the section specified in the `PHDRS` command. No additional segments are created.

10.2.2.6. Linker script assignment evaluation

Linker script assignment evaluation can influence the image layout as the location counter value can be modified using a linker script assignment and the location counter controls where the next content would be placed.

eld does not support lazy expression evaluation and forward references in expressions.

In eld, the linker script assignments outside the `SECTIONS` command are evaluated before the linker script assignments inside the `SECTIONS` command. The linker script assignments order is:

- First, all the linker script assignments outside the `SECTIONS` command are evaluated in the specified order.
- Then all the linker script assignments inside the `SECTIONS` command are evaluated in the specified order.

The linker script assignments specified using `--defsym` are considered as outside-`SECTIONS` linker script assignments.

10.2.3. Linker options that affect layout

10.2.3.1. `--rosegment`

By default, readonly non-executable (`R`) sections such as `.rodata` sections and executable sections (`RX`) such as `.text` can be part of the same segment. When `--rosegment` is specified, a different segment is created for readonly non-executable segments.

10.2.3.2. `--omagic`

If `--omagic` is specified, then readonly non-executable (`R`), executable (`RX`), and read-write (`RW`) sections can be part of the same segment. Moreover, the segment alignment is set to the maximum section alignment instead of the page alignment.

10.2.3.3. `--align-segments`

This option cannot be used with linker scripts. When used, the addresses of segments (both virtual and physical addresses) are aligned to the page boundaries.

10.2.3.4. `--enable-bss-mixing` and `--disable-bss-conversion`

These options control how the linker treats BSS sections relative to non-BSS sections within the same segment.

`--disable-bss-conversion` controls whether the linker converts BSS to non-BSS when BSS/non-BSS sections are mixed. When this option is passed, BSS remains NOBITS when BSS and Non-BSS are mixed in a segment. This option must be combined with `--enable-bss-mixing` if the BSS section will be placed before a non-BSS section because otherwise BSS before non-BSS is an error.

This combination of options is useful for reducing file size of a program. For example, if a program has a layout requirement that a BSS section (let's say `.bss`) must be placed before a non-BSS section (let's say `.data`) in the same segment, and the addresses of `.bss` and `.data` are `0x1000` and `0x2000` respectively, then with the default behavior `.bss` will be converted to PROGBITS and 0s would need to be filled explicitly in the file for this section. This can significantly increase the file size. On the contrary, when `--disable-bss-conversion --enable-bss-mixing`, `.bss` will remain as NOBITS and thus no file size will be consumed by this section.

Please note that with such a layout you may need a custom loader because most standard loaders would not accept a layout where a BSS section is followed by a non-BSS section in the same segment.

Note
These options are currently supported only for ARM, AArch64, and RISCV targets.

Let's understand these options in more detail with the help of an example:

Below is a minimal example showing how these options affect layout when placing both `.bss` and `.data` in the same segment.

bssmix.c:

```
int bssvar;            // goes to .bss (NOBITS)
int data = 1;          // goes to .data (PROGBITS)
int main() { return data + bssvar; }
```

Linker script (script.t):

```
PHDRS {
  A PT_LOAD;
  B PT_LOAD;
}

SECTIONS {
  .text (0x1000) : { *(.text*) } :A
  .data (0x3000) : { *(.data*) } :B
  .bss  (0x2000) : { *(.bss*)  } :B
}
```

Build and link:

```
$ clang -o bssmix.o --target=aarch64-unknown-elf -c bssmix.c -ffunction-sections -fdata-sections

# .bss section is promoted to PROGBITS because .bss is placed before .data in the segment B.
$ ld.eld -o bssmix.out bssmix.o -T script.t

# Error: Mixing BSS and non-BSS sections in segment. non-BSS '.data' is after
# BSS '.bss' in linker script
$ ld.eld -o bssmix.disable_conversion.out bssmix.o -T script.t --disable-bss-conversion

# .bss section is placed before .data in the segment B and BSSness of .bss is preserved,
# that is, it is not promoted to PROGBITS.
$ ld.eld -o bssmix.disable_conversion.out bssmix.o -T script.t --disable-bss-conversion \
    --enable-bss-mixing
```

10.2.4. Symbol table order

The linker writes symbols to the output `.symtab` and `.dynsym` in a fixed, reproducible order. Two links of the same inputs always produce byte-identical symbol tables, regardless of how many threads the linker uses.

10.2.4.1. `.symtab`

Symbols are ordered as:

1. Section symbols.
2. Local symbols (ELF requires all locals before all globals).
3. Global symbols.

Within each group, symbols are ordered by the input file they came from (in command-line order), then by their index in that input's own symbol table. This preserves the original source order of symbols from each object file.

10.2.4.2. `.dynsym`

The dynamic symbol table holds only the exported (dynamic) symbols. They are ordered as:

1. Undefined symbols (including symbols defined by shared objects).
2. Defined symbols.

Within each group, symbols are ordered by input file (command-line order), then by their index in that input's symbol table, matching `.symtab`.

10.2.5. Advanced image layout control using linker plugins

More finer-grained control over the image layout can be achieved using linker plugins.

- Custom Layout Logic: Plugins can override default section placement logic to optimize for cache locality or hardware-specific constraints.

- Dynamic Behavior: Unlike static scripts, plugins can make layout decisions based on runtime metadata or build-time heuristics.
- Programmatic Layout: Developers can write plugins (e.g., LayoutOptimizer) that programmatically define how sections are arranged, enabling more flexible and performance-tuned binaries

This approach is especially useful in embedded systems where:

- Memory is constrained and fragmented.
- Performance depends on precise placement of code/data.
- Debugging requires reproducible and traceable layouts.

11. Linker Plugins

Linker plugin framework is a user-friendly solution to add custom link behavior. This allows non-linker developers to tweak and hack linker for the specialized use-cases. It is similar to how clang plugins let you tweak frontend compliation and LLVM passes let you transform the LLVM IR. [[The core idea is to make the tool extendable and hackable by providing the framework]].

But why would you ever need to customize link? Short answer: Embedded development can get really fun at times. Long and more useful answer will be discussed as we move on.

This guide contains all that you need to know about linker plugins, and how to effectively write one yourself.

Overview

- Implemented as a C++ class.
- Facilitates user-defined behavior to crucial parts of the link process.
- Provides finer control over the output image layout than a linker script.
- Allows inspecting symbols, chunks, relocation, section mapping, input and output sections.
- Allows adding and modifying section mapping rules, relocations, chunk attributes, and symbols.
- Plugins can communicate with the linker as well as with each other.
- Easy traceability of plugins using `--trace=plugin` option.
- `PluginADT` provides wrappers for many common linker data types.

11.1. Linker Plugins

11.1.1. What and why of linker plugins

Linker plugins make it possible to run user-defined code during the link process. Implemented as a C++ class, they are a programmatical way to add new functionalities and modify default linker behavior.

But why modify default linker behavior? We will now discuss the long answer we promised at the beginning of the linker plugins documentation.

Plugins provide more control and flexibility over the output image layout as compared to the linker scripts. For complicated rules, linker scripts are often very cumbersome to write, sometimes desired rule matching behavior might even be impossible from just linker script. For example, a plugin can move around chunks from one output section to another. Linker plugins also allow the user to alter program layout dynamically which is not possible using linker script.

Plugins add custom behavior to the linker by defining behavior for the hooks that the linker runs in various stages of the link process. These hooks allows plugins to modify the default linker behavior, provide a fine-grained control over the output image layout, add new functionalities to the linker.

This guide will describe all that you need to know about linker plugins, and how to effectively write one yourself. This guide assumes that the reader has a basic understanding of linkers, linker relocations and linker scripts.

11.1.2. Elements of the link process

Before we move further, let's first quickly go over some important elements of the link process. A solid understanding of these elements will help to better understand the link process, and linker's capabilities and limitations. This in turn will help to effectively design and write plugins.

11.1.2.1. Symbols

Formally, a symbol is a name associated with a value. This value can be an offset within a section, a virtual memory address, or simply an absolute value. Symbols are an integral component of an object file. Among other things, they are used to refer to global variables and functions in a program.

Note
Global variables were specifically mentioned because local variables are created and managed on the stack at run-time and linker is blissfully unaware of them. Beware: local symbols are not the same as local variables!

`eld::plugin::Symbol` represents a linker symbol.

11.1.2.1.1. Symbol resolution

For each group of non-local symbols with the same name, symbol resolution selects one of these symbols using well-defined rules. The selected symbol is part of the output image symbol table and is used to resolve symbol references to that name.

11.1.2.2. Input Section

Each relocatable object module, *M*, has input sections. Input sections contains most of the object file information for the linking view: instructions, data, symbol table, relocation information and so on.

`eld::plugin::Section` represents an input section.

11.1.2.3. Output Section

Output section contains one or more input sections. The output sections are in turn mapped to a segment. A segment contains one or more output sections, and is used for finally loading the program. Typically, output sections with same *Alloc*, *Exec*, and *Write* permissions are mapped to the same segment.

`eld::plugin::OutputSection` represents an output section.

11.1.2.4. Chunk

In ELD's terminology, a section is made up of many sub-parts called as chunk or fragment. Plugins can move chunks from one output section to another. Linker scripts do not have this level of control over the output image layout.

`eld::plugin::Chunk` is a handler for a chunk. It can be used to inspect the properties, symbols and content of a chunk.

11.1.2.5. Relocation

Relocation is the process of connecting symbolic references with symbolic definitions. All symbolic references of the same symbol should be connected to the same definition. Object files contain relocation entries that describes how the relocations should be processed.

`eld::plugin::Use` represent relocations.

11.1.2.6. Section Mapping

Each input section is either discarded or is mapped to some output section. Linker has some default mapping rules that defines which input section is to be mapped to which output section. Traditionally, linkers allowed users to add their own custom mapping rules using linker script `SECTIONS` command.

Now, we have covered the basic elements of the link process. Let's dive into the linker plugin functionality.

11.1.3. Plugin Types

ELD has different plugin types. Plugin types differ in which hooks they provide. Hooks are pre-defined spots in the link pipeline where a plugin can tap into the linker to customize the link. Plugin taps into the linker by defining the callback function for the hooks. The linker calls these callback functions at the hook sites. For brevity, we will refer callback function for the hook as hook callback function.

There two distinct kinds of plugins: LinkerPlugin plugin and layout plugins. LinkerPlugin plugin is more general and provides hook that covers the entire link process. On the other hand, the layout plugins have specialized hooks for iterating over certain link components (input sections, output sections, …) instead of having general hooks. Different layout plugin types iterate over different link components. For example, `SectionMatcher` plugin type provide hooks to process input sections and `OutputSectionIterator` plugin type provides hooks to process output sections.

There are a total of 6 plugin types. One LinkerPlugin plugin type and 5 layout plugin types. For each plugin type, there is a corresponding plugin class. Plugin authors create a new plugin by inheriting from the corresponding plugin class and implementing the hook callback functions.

The 6 plugin interfaces along with their corresponding C++ class and header file are listed below.

Plugin interface	Plugin interface class	Header file
LinkerPlugin	LinkerPlugin	ELD/PluginAPI/LinkerPlugin.h
SectionMatcher	SectionMatcherPlugin	ELD/PluginAPI/SectionMatcherPlugin.h
SectionIterator	SectionIteratorPlugin	ELD/PluginAPI/SectionIteratorPlugin.h
ControlFileSize	ControlFileSizePlugin	ELD/PluginAPI/ControlFileSizePlugin.h
ControlMemorySize	ControlMemorySizePlugin	ELD/PluginAPI/ControlMemorySizePlugin.h
OutputSectionIterator	OutputSectionIteratorPlugin.h	ELD/PluginAPI/OutputSectionIteratorPlugin.h

11.1.4. Linker Wrapper

11.1.5. Running a linker plugin

There are two methods to run linker plugins:

- Adding a plugin invocation command in a linker script. *(Not supported for :code:`LinkerPlugin` plugins)*
- Specifying plugin configuration file using `--plugin-config` option. *(Recommended way)*

We call these methods as *plugin load specification* because they specify how the linker should load a plugin.

11.1.5.1. Adding a plugin invocation command in a linker script.

```
<PluginTypeKeyword>("LibraryName", "PluginName" [, "PluginOption"])
```

For output section plugin types, `ControlMemorySizePlugin` and `ControlFileSizePlugin`, users should add the plugin invocation command to the output section description of the output section on which these plugins should run. For example:

```
SECTIONS {
  output_section <PluginTypeKeyword>("LibraryName", "PluginName" [, "PluginOption"]) : {
    *(.text)
  }
}
```

> **Note**
> Only one output section plugin can be attached to an output section.

- **PluginTypeKeyword**

  Each plugin type has a corresponding linker script plugin keyword.

Plugin interface type	Linker script keyword
SectionMatcher	PLUGIN_SECTION_MATCHER
SectionIterator	PLUGIN_ITER_SECTIONS
ControlFileSize	PLUGIN_CONTROL_FILESZ
ControlMemorySize	PLUGIN_CONTROL_MEMSZ
OutputSectionIterator	PLUGIN_OUTPUT_SECTION_ITER

- **LibraryName**

  - Name of the dynamic library that contains the plugin for the linker to load.
  - Finds the library in the same search paths as if the library was passed as an input to the linker.
  - Uses the name of the library without the lib prefix on Linux/macOS and without the .so/.dll/.dylib suffix on Linux/Windows/macOS, respectively

- **PluginName**

  Name of the plugin. The linker queries the dynamic library to provide an implementation of the plugin with the name *PluginName* for the specified interface type.

- **PluginOption**

  It can be used to pass an option to the plugin.

11.1.5.2. Specifying plugin configuration file using `--plugin-config` option

`--plugin-config` option takes a plugin configuration yaml file. Plugin configuration file should contain a `GlobalPlugins` list. Elements of the `GlobalPlugins` list defines which plugins should be loaded. Each value of the list is an object containing all the information required to load and initialize a specific plugin.

Plugin configuration file format should be as follows::

```
---
GlobalPlugins:
  - Type: PluginInterfaceType
    Name: PluginName
    Library: LibraryName
    Options: PluginOption

OutputSectionPlugins:
  - OutputSection : OutputSectionName
    Type : PluginInterfaceType
    Name : PluginName
    Library : LibraryName
    Options : PluginOption
```

`GlobalPlugins` list can specify any number of elements. `Options` member is optional.

`Library` name should be specified without the lib prefix on Linux/macOS and without the .so/.dll/.dylib suffix on Linux/Windows/macOS

`ControlMemorySizePlugin` and `ControlFileSizePlugin` are output section plugins. Therefore, in the plugin configuration file, they need to be added to `OutputSectionPlugins` list.

Note
Only one output section plugin can be attached to an output section.

11.1.6. User Plugin Workflow

The following steps describe how to develop a plugin:

1. Determine the appropriate plugin interfaces for your plugin.

   A plugin type can be one of `LinkerPlugin`, `SectionIterator`, `SectionMatcher`, `OutputSectionIterator`, `ControlMemorySize`, and `ControlFileSize`.

   A plugin should ideally follow the unix-philosophy of doing one thing well. You should generally try to avoid inheriting from multiple plugin interfaces if possible. It generally leads to more complicated code, and a plugin that does more than one thing. But that being said, if you do think functionalities of multiple plugin interfaces will improve your plugin's design and readability, then you should go ahead with inheriting from multiple plugins.

   You can also create and chain multiple plugins for your task, similar in ideology to how unix utilities operate, where action of one plugin depends on the action and result of the previous one.

1. Create a C++ source file that will contain your plugin(s).

   Plugins source file needs to include the headers of the plugin interface(s) that the plugin(s) inherits from. Plugin interfaces header files are contained in, `${HEXAGON_TOOLCHAIN}/Tools/include/ELD/PluginAPI`.

1. Create a C++ class that inherits from the appropriate plugin interface(s) for each plugin.
2. Override virtual functions.

Virtual functions are the means the linker works with the plugins. Each plugin interface provides hooks at various place in the linker codebase, the hooks functionalities are implemented by overriding virtual functions. There are also other virtual functions that do not implement functionalities for the hooks, but are required to work with the plugin infrastructure. Example of these virtual functions are: `Plugin::GetName`, `Plugin::GetLastError`, `Plugin::GetLastErrorAsString` etc.

1. Associate the plugins with unique names.
2. Build a shared library for the source file.

In a unix environment, shared library for the plugins can be created as:

```
# Compile the plugins source file
clang++ -c -I${HEXAGON_TOOLCHAIN_ROOT}/Tools/include ${SOURCE_BASENAME}.cpp -fPIC -stdlib=libc++

# Link the plugin library with linker wrapper library, LW.
# Linux: lib${SOURCE_BASENAME}.so   macOS: lib${SOURCE_BASENAME}.dylib
clang++  -shared  ./${SOURCE_BASENAME}.o  -L${HEXAGON_TOOLCHAIN_ROOT}/Tools/lib  -lLW  -stdlib=libc++  -o
lib${SOURCE_BASENAME}.so
```

1. Define `RegisterAll` function in C linkage.

`RegisterAll` function goal is to register the plugins contained in the file. Registering a plugin here simply means to initialize a plugin object.

Linker calls this function for each plugin library. This function should creates a plugin object for each plugin that the library contains.

1. Define a `getPlugin(const char *pluginName)` function in C linkage.

`getPlugin(const char* plugin)` function goal is to return the requested plugin. Each plugin in the plugin source file should be uniquely identifiable by a name.

Linker calls this function at the time of loading plugins to get a plugin object of plugin named `pluginName`.

1. Make the plugin report its API version

Linker verifies linker plugin compatibility. Linker will report an error and refuse to load a plugin if it determines that it is not compatible.

Linker plugin compatibility is determined using Linker API version. Plugin API version consists of major and minor version numbers. A plugin is compatible with the linker if plugin’s major API version is equal to the linker major API version and plugin’s minor API version is equal or lower to the linker’s minor API version. The major API version is incremented when a existing functionality in the API is changed in an incompatible way. Such changes are expected to be rare. The minor API version is incremented when new functionality is added to the Plugin API. Note that Plugin API versions are distinct from linker release version numbers. The current linker Plugin API version is reported by the `--version` command line option.

For the versioning mechanism to work, each plugin must report its API version by exporting the `getPluginAPIVersion(unsigned int *Major, unsigned int *Minor)` function (with C linkage). For convenience, this function is defined in the header file `PluginVersion.h` and including this file in one of the plugin’s C++ source files will automatically make the plugin report its correct API version.

Starting with version 19.0, each plugin must report its API version. Linker will refuse to load a plugin that does not report its version.

1. (Optionally) Define a `getPluginConfig(const char *pluginName)` in C linkage to return the configuration object of the requested plugin.
2. (Optionally) Define a `Cleanup` function in C linkage.

This function is called at the time unloading plugins. This function generally deletes any allocated memory that was required for the lifetime of the plugin library.

11.1.7. How the linker runs plugin

Linker performs the following operations to load, run and unload plugins.

1. Finds all the plugins, from linker script and plugins configuration file, that needs to be attached to the link process.
2. Loads all the specified plugin libraries.
    1. To find plugin libraries, `LD_LIBRARY_PATH` is used on Linux. On macOS, `DYLD_LIBRARY_PATH` is searched first, then `LD_LIBRARY_PATH`.
    2. Standard method for searching dynamic libraries is used in Windows.
3. Calls `RegisterAll` function from each plugin library. This function should ideally create the plugin objects for each plugin defined in the library.
4. Calls `getPlugin(const char *pluginName)` function for each plugin. This function returns an appropriate plugin object for *pluginName* plugin. This object is used to run the plugin.
5. Calls `getPluginConfig(const char *pluginName)` function, if available, for each plugin. This function returns an appropriate plugin configuration object for *pluginName* plugin.
6. Inspects the plugin to verify that the plugin load specification and plugin type have same plugin interface type.
7. Initializes each plugin at their *Init* hook point by calling `Plugin::Init(std::string Option)` virtual function. *Option* argument is optionally specified by the plugin user at the plugin load specification.
8. Calls `Cleanup` function, if available, for each plugin library. This function should ideally delete any allocated memory that was required for the lifetime of the plugin library.
9. Unload the plugins and the plugin libraries.

11.1.8. Tracing plugins

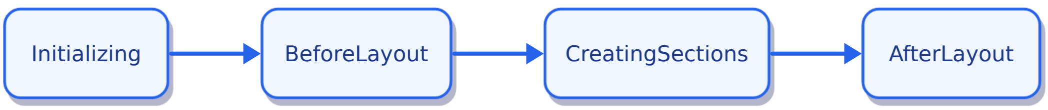
Plugin workflow can be traced using the option `--trace=plugin`.

This option informs the linker to emit detailed diagnostics for all the plugins.

A sample trace output is shown below:

```
...
...
Note: Registration function found RegisterAll in Library libDiagOpt.so
Note: Plugin handler getPlugin found in Library libDiagOpt.so
Note: Cleanup function found Cleanup in Library libDiagOpt.so
Note: Plugin Config function getPluginConfig found in library libDiagOpt.so
Note: Registering all plugin handlers for plugin types
Note: Found plugin handler for plugin type DIAGRELOCATION in Library libDiagOpt.so
Note: Initializing Plugin libDiagOpt.so, requested by Plugin DIAGRELOCATION having Name DIAGOPT
...
...
Note: Plugin DIAGRELOCATION Destroyed

<------------------------> DiagOpt::Destroy() AfterLayout<------------------------>
Note: Unloaded Library libDiagOpt.so.16
```

11.1.9. Link States



The link process has different run states, also known as link states. The actions that a plugin can perform at any given time depend on the current link state. Many actions are only meaningful for specific link states.

Understanding what happens in each link state will help determine which actions can be performed in each state. Therefore, a thorough understanding of linker and link states is crucial for writing a linker plugin. Different link states are described below:

11.1.9.1. Initializing

The linker begins with the *Initializing* link state. In this state, the linker reads input files, initializes standard sections and configurations, and read relocations. Not much of interest happens during this state.

Let's move on to the more happening states.

11.1.9.2. BeforeLayout

After the *Initializing* state, the linker enters the *BeforeLayout* link state. Here, it performs section rule matching, garbage collection, updates input sections with attributes (such as KEEP) present in the linker script, and updates the linker cache-file, among other things.

11.1.9.3. CreatingSections

After the *BeforeLayout* state, the linker enters the *CreatingSections* link state. Here, it transfers the contents (chunks) of input sections into output sections and finalizes the layout and symbols.

Note that the section rule-matching is already complete before the *CreatingSections* state, therefore, plugins cannot alter the section rule-matching in *CreatingSections* state or any subsequent state.

11.1.9.4. AfterLayout

After the *BeforeLayout* state, the linker enters *Afterlayout* link state. Here, it finalizes the relocations and writes the output object file.

In this link state, a plugin can compute output image layout checksum using `eld::plugin::LinkerWrapper::getImageLayoutChecksum`. A plugin cannot perform any action that tries to modify the image layout now.

11.1.10. Deep Dive into the Plugin Types

There are two distinct kinds of linker plugins: LinkerPlugin and layout plugins. LayoutPlugins focus on simplifying the modification of specific layout components, such as input and output sections. In contrast LinkerPlugins are more general, uniform and offer more functionalitier than the layout plugins.

11.1.10.1. PluginBase class

This is the base plugin class for all the plugin interface classes. It provides common functionalities to all the plugin interfaces. It is a pure virtual class and thus cannot be instantiated / directly used.

class PluginBase

Subclassed by eld::plugin::LinkerPlugin, eld::plugin::Plugin

Public Types

enum Type

Plugin interface types.

Values:

enumerator ControlFileSize

enumerator ControlMemorySize

enumerator SectionIterator

enumerator SectionMatcher

enumerator OutputSectionIterator

enumerator LinkerPlugin

Public Functions

uint32_t getPluginID() const

Returns the unique ID associated with the plugin.

inline Type getType() const

Returns the type of the plugin.

virtual std::string GetName() = 0

Returns the name of the plugin.

inline virtual std::string GetDescription()

Returns plugin description.

Friends

***friend class* eld::LinkerScript**

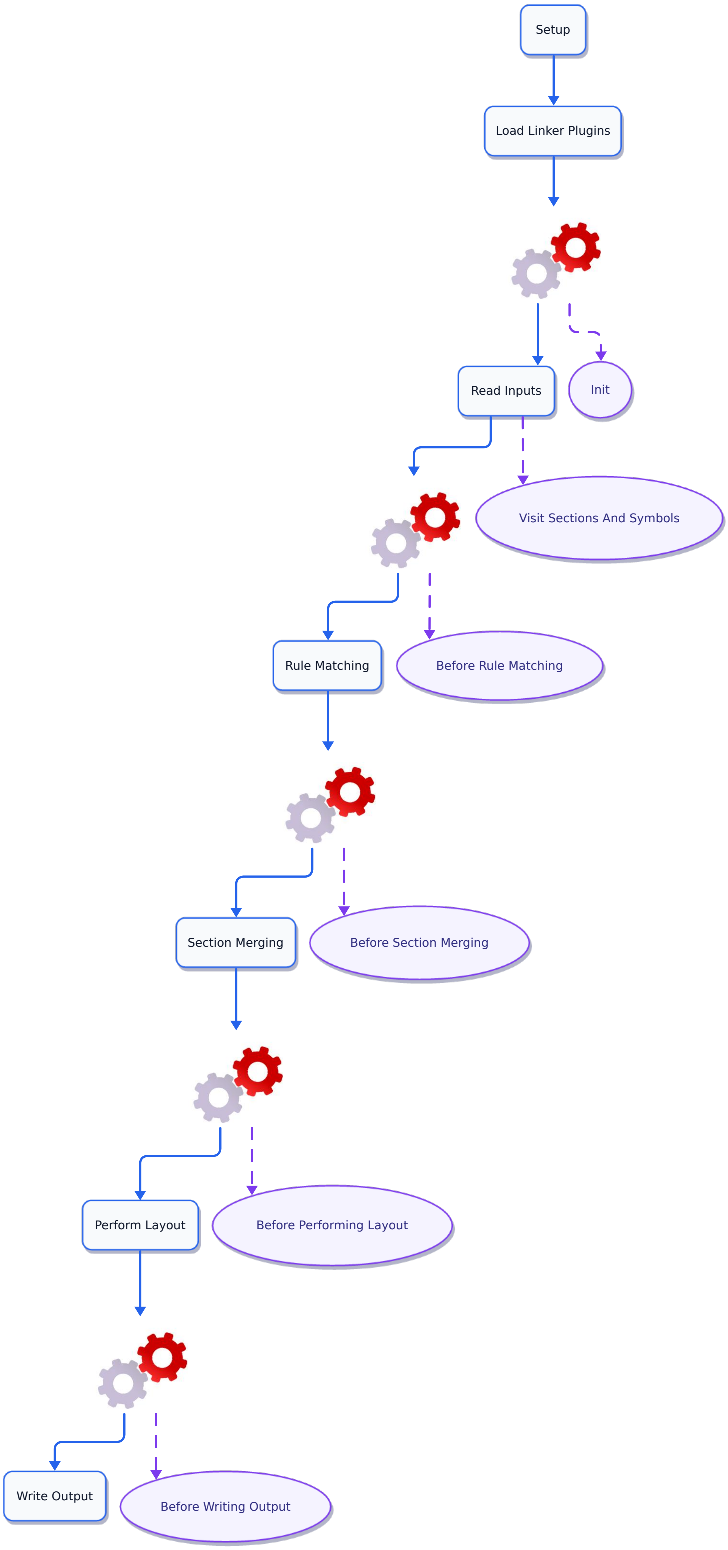
Please note, that even though the virtual functions – `Plugin::Init`, `Plugin::Run` and `Plugin::Destroy` are available in all the plugin interfaces, they map to different hooks in the linker for different plugin interfaces. For example, `Run` hook of `SectionIteratorPlugin` is not equivalent to the `Run` hook of `OutputSectionIteratorPlugin` even though in both cases, the hooks functionality is defined by overriding the `Run` virtual function. For the base `Plugin`, these functions are not mapped to any hook in the linker.

11.1.10.2. LinkerPlugin Type

`LinkerPlugin` is the most versatile and the recommended plugin type for creating new plugins. It has hooks that covers the entire linker flow and offers more functionality than the layout plugins. `LinkerPlugin` has hooks of the two forms:

- `ActBefore<LinkState>`
- `Visit<LinkComponent>`

The `ActBefore<LinkState>` hooks are called *just* before the linker enters the link state `LinkState`. The `Visit<LinkComponent>` hooks are called *immediately* after the linker creates a link component (input section, symbol, …). For example, `VisitSymbol(eld::plugin::Symbol S)` is called immediately after the symbol is created.

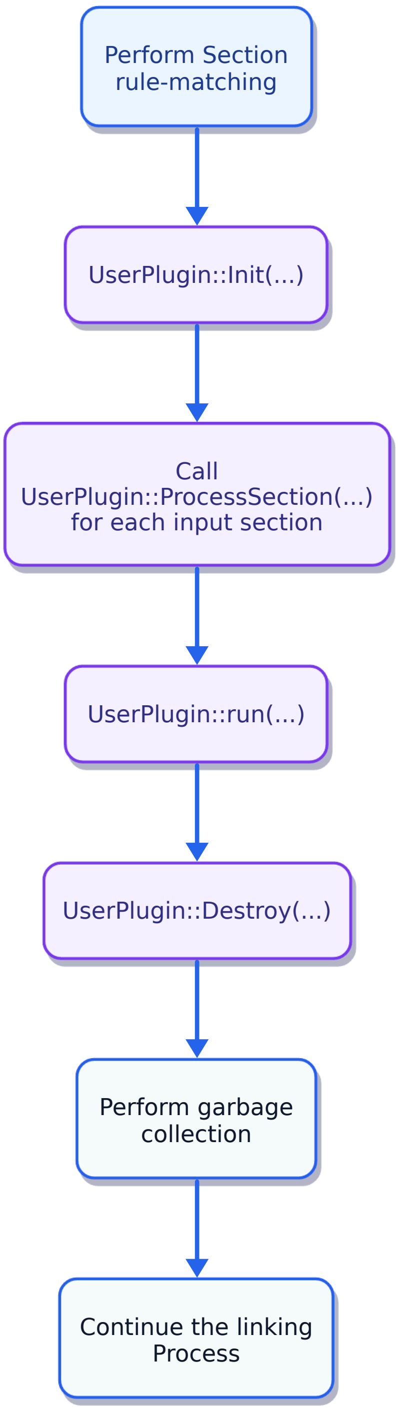


11.1.10.3. Section Matcher Interface

`SectionMatcherPlugin` interface iterates input sections. It provides four hooks: `Init`, `ProcessSection`, `Run` and `Destroy`. To define the behavior of these hooks, the functions `Plugin::Init`, `SectionMatcher::ProcessSection` and `Plugin::Destroy` must be overridden.

`ProcessSection(Section S)` callback hook function is called for each input section.

SectionMatcherPlugin is called into action after the linker reads input files and performs section rule-matching, but before garbage collection. As a consequence, garbage-collection information cannot be accessed during SectionMatcherPlugin run. Link state throughout the plugin run is eld::plugin::LinkerWrapper::BeforeLayout.



UserPlugin::Init is called before UserPlugin::ProcessSection, and UserPlugin::Run and UserPlugin::Destroy are called subsequently after processing input sections, as described in the figure above.

Linker script keyword for `SectionMatcherPlugin` interface is `PLUGIN_SECTION_MATCHER`. Therefore, to run a `SectionMatcherPlugin` plugin using a linker script, add the following line to the linker script:

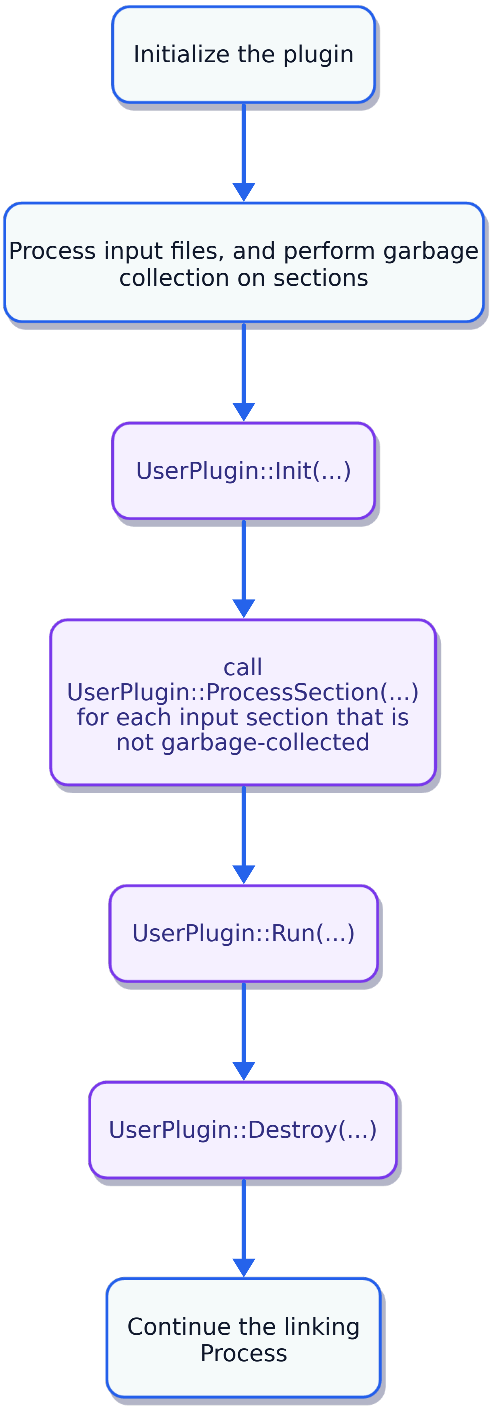
```
PLUGIN_SECTION_MATCHER("LibraryName", "PluginName" [, "PluginOption"])
```

11.1.10.4. Section Iterator Interface

`SectionIteratorPlugin` interface iterates over input sections. It provides four hooks: `Init`, `ProcessSection`, `Run` and `Destroy`. To define the behavior of these hooks, the functions `Plugin::Init`, `Plugin::ProcessSection`, `SectionIterator::ProcessSection` and `Plugin::Destroy` must be overridden.

`ProcessSection(Section S)` callback hook function is not called for garbage collected input sections. However, it is called for discarded input sections.

`SectionIteratorPlugin` is called into action after the linker performs section rule-matching and garbage collection. As a consequence, section rule matching and garbage collection information is accessible during `SectionIteratorPlugin` run. Link state throughout the plugin run is `eld::plugin::LinkerWrapper::BeforeLayout`.



`UserPlugin::Init` is called before `ProcessSection`, and `UserPlugin::Run` and `UserPlugin::Destroy` are called after `ProcessSection`, as described in the figure above.

Linker script keyword for `SectionIteratorPlugin` interface is `PLUGIN_ITER_SECTIONS`. Therefore, to run a `SectionIteratorPlugin` plugin using a linker script, add the following line to the linker script:

```
PLUGIN_ITER_SECTIONS("LibraryName", "PluginName" [, "PluginOption"])
```

11.1.10.5. Output Section Iterator Interface

`outputSectionIteratorPlugin` interface iterates over all the output sections. It provides four hooks: `Init`, `ProcessOutputSection`, `Run` and `Destroy`. To define the behavior of these hooks, the functions `Plugin::Init`, `Plugin::ProcessSection`, `OutputSectionIteratorPlugin::ProcessOutputSection` and `Plugin::Destroy` must be overridden. `OutputSectionIteratorPlugin` is called at different link states:

- BeforeLayout
- CreatingSections
- AfterLayout

All the plugin hooks are called for each of the link states. That means, each of `Init`, `Run` and `Destory` are called 3 times over the complete plugin run. And `ProcessOutputSection` is called three times for each output section over the complete plugin run.

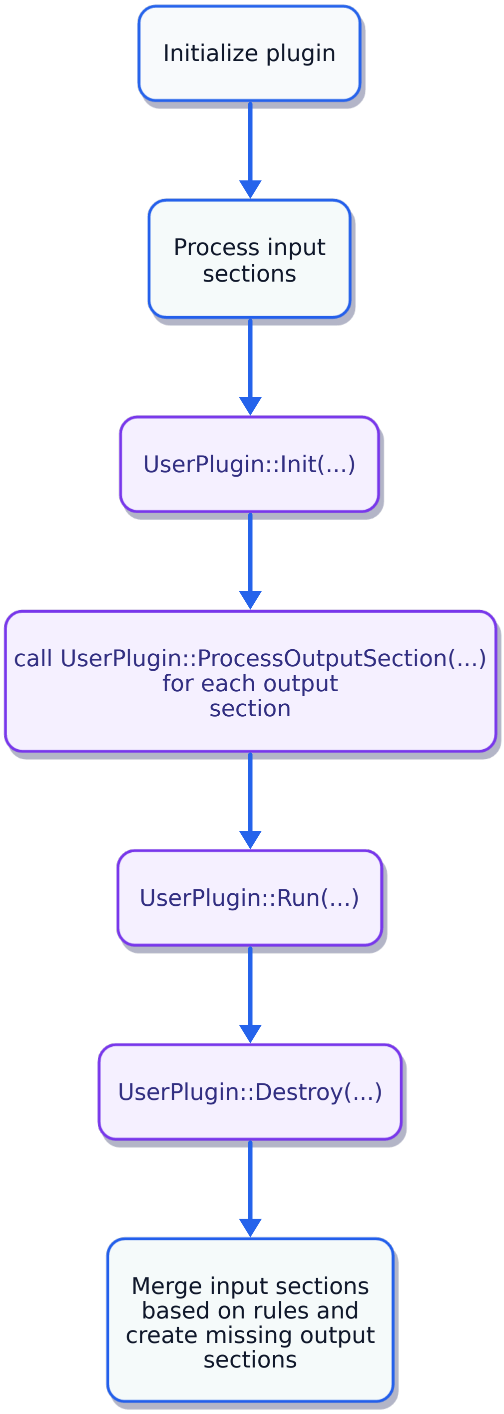
The link state can be queried via `LinkerWrapper::getState` member function.

Linker script keyword for `OutputSectionIteratorPlugin` interface is `PLUGIN_OUTPUT_SECTION_ITER`. Therefore, to run a `SectionIteratorPlugin` using a linker script, add the following line to the linker script:

```
PLUGIN_OUTPUT_SECTION_ITER("LibraryName", "PluginName" [, "PluginOption"])
```

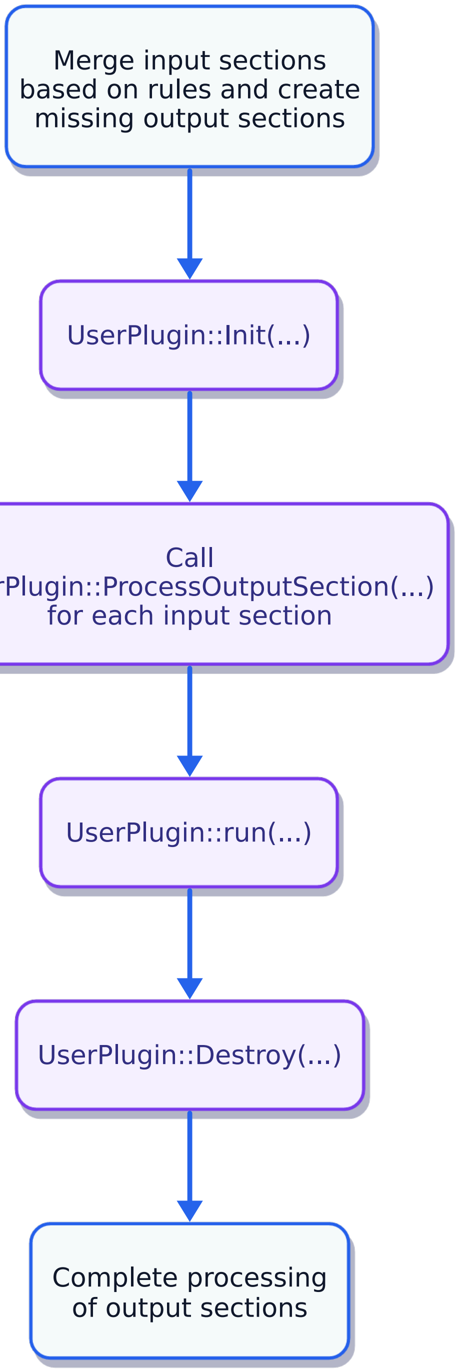
Different phases of `OutputSectionIteratorPlugin` are described below:

11.1.10.5.1. BeforeLayout – Phase 1



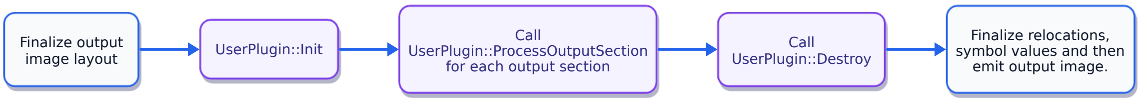
This phase is called into action after the linker has performed section rule-matching and before the linker has performed section merging.

11.1.10.5.2. CreatingSections – Phase 2



This phase is called into action after the linker has performed section merging.

11.1.10.5.3. AfterLayout – Phase 3

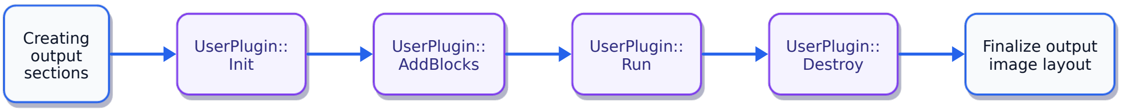


This phase is called into action after the linker has finalized the image layout and before the linker has written the output object file.

11.1.10.6. ControlMemorySizePlugin

This interface allows the plugin to add input sections to the output image. These newly added input sections are generally, but not necessary, based on some already existing output section.

`ControlMemorySizePlugin` is an output section plugin. Plugins of this type has to explicitly mark each output section that it needs to run on. Since this is an output section plugin, `init` and `destroy` functions are also called for each output section the plugin runs on. This plugin is called in the `LinkerWrapper::CreatingSections` step of the linker.



`init`, `AddBlocks`, `Run`, `GetBlocks` and `Destroy` functions are called for each output section.

Brief description of `ControlMemorySizePlugin` hooks:

11.1.10.6.1. `init`

This hook initializes the plugin for the output section.

11.1.10.6.2. `AddBlocks`

This hook gives access to the current output section to the plugin. `Block` parameter of the hook function contains the current output section. It is generally used in the creation of the new block which linker can use as an additional input section.

11.1.10.6.3. `Run`

This hook is generally used to process the current output section, and create any new sections that linker should use as additional input sections.

11.1.10.6.4. `GetBlocks`

This hook returns a vector of `Block`. These blocks are used by the linker as additional input sections.

11.1.10.6.5. `Destroy`

This hook is used to perform any required clean-up.

Linker script keyword for ControlMemorySize interface is `PLUGIN_CONTROL_MEMSZ`. Therefore, to run a SectionIterator plugin on an output section, `.out` using a linker script, add the following line to the output section description of `.out` in the linker script:

```
SECTIONS {
   .out PLUGIN_CONTROL_MEMSZ("LibraryName", "PluginName" [, "PluginOption"]) : { *(.out) }
 }
```

11.2. Linker Plugin Examples

This section introduces and explains plugin framework APIs in a hands-on manner. The goal of these examples is to demonstrate how to use plugin framework APIs, rather than to present practical plugin use cases. Each plugin example introduces some new plugin framework APIs while keeping the example short and simple.

Each example will include the plugin, instructions on how to build and run it, and an analysis of the plugin's run. To build and run the plugin, you will need access to the Hexagon toolchain. In the snippets below, the substitution `${HEXAGON}` refers to the Hexagon toolchain installation path. Let's get started.

11.2.1. Exclude symbols from the output image symbol table

This example plugin describes how to exclude symbols from the output image symbol table. Note that the plugin does not completely remove symbols from the link process, it only excludes symbols from the output image symbol table.

ExcludeSymbols plugin with self-contained documentation.

```
#include "ELD/PluginAPI/PluginVersion.h"
#include "ELD/PluginAPI/SectionIteratorPlugin.h"
#include <string>
#include <vector>

class ExcludeSymbols : public eld::plugin::SectionIteratorPlugin {
public:
  ExcludeSymbols() : eld::plugin::SectionIteratorPlugin("ExcludeSymbols") {}

  // 'Init' callback hook can be used for initialization and preparations.
  void Init(std::string Options) override {
    m_SymbolsToRemove = {"foo", "fooagain", "bar", "baragain"};
  }

  // 'processSection' callback hook is called for each input section that is not
  // garbage-collected.
  void processSection(eld::plugin::Section O) override {}

  // 'Run' callback hook is called after 'processSection' callback hook calls.
  // It is called once for each section iterator plugin run.
  eld::plugin::Plugin::Status Run(bool trace) override {
    for (auto symName : m_SymbolsToRemove) {
      eld::plugin::Symbol S = Linker->getSymbol(symName);
      if (S)
        Linker->removeSymbolTableEntry(S);
    }
    return eld::plugin::Plugin::Status::SUCCESS;
  }

  // 'Destroy' callback hook can be used for finalization and clean-up tasks.
  // It is called once for each section iterator plugin run.
  void Destroy() override {}

  uint32_t GetLastError() override { return 0; }

  std::string GetLastErrorAsString() override { return "Success"; }

  std::string GetName() override { return "ExcludeSymbols"; }

private:
  std::vector<std::string> m_SymbolsToRemove;
};

eld::plugin::Plugin *ThisPlugin = nullptr;
```

```
extern "C" {
bool RegisterAll() {
  ThisPlugin = new ExcludeSymbols{};
  return true;
}

eld::plugin::Plugin *getPlugin(const char *pluginName) { return ThisPlugin; }

void Cleanup() {
  if (!ThisPlugin)
    return;
  delete ThisPlugin;
  ThisPlugin = nullptr;
}
}
```

ExcludeSymbols plugin removes the symbols ‘foo’, ‘fooagain’, ‘bar’, and ‘baragain’, if they exist, from the output image symbol table.

To build the plugin, run the following command:

```
clang++-14 -o libExcludeSymbols.so ExcludeSymbols.cpp -std=c++17 -stdlib=libc++ -fPIC -shared \
-I${HEXAGON}/include -L${HEXAGON}/lib -lLW
```

Now, let’s see the effect of this plugin on a sample program.

1.c

```
int foo() { return 1; }
int fooagain() { return 2; }
int abc() { return 3; }

int bar = 3;
int baragain = 4;
int def = 5;

int main() { return 0; }
```

1.linker.script

```
PLUGIN_ITER_SECTIONS("ExcludeSymbols", "ExcludeSymbols");
```

Now, let’s build **1.c** with the *ExcludeSymbols* plugin enabled.

```
export LD_LIBRARY_PATH="${HEXAGON}/lib:${LD_LIBRARY_PATH}"
hexagon-clang -o 1.o 1.c -c -ffunction-sections -fdata-sections
hexagon-link -o 1.elf 1.o -T 1.linker.script
```

We can list the symbols present in an object file using hexagon-readelf.

```
hexagon-readelf -s 1.elf
```

You can observe in the symbol table output that ‘foo’, ‘fooagain’, ‘bar’, and ‘baragain’ symbols has been removed from the symbol table as directed by the plugin.

11.2.2. Add Linker Script Rules

This example plugin describes how to add and modify linker script rules. By doing so, you can perform section rule-matching at a more granular level than what is possible through linker scripts.

`LinkerScriptRule` is similar to an input section description in that it has a corresponding output section. However, unlike an input section description, it does not match input sections by pattern matching. Instead, a plugin must manually match chunks to a `LinkerScriptRule` object. This generally involves moving chunks from one `LinkerScriptRule` object to another. It is important to remove a chunk from the old `LinkerScriptRule` object once it has been added to a new one. It is an undefined behavior for a chunk to be part of multiple `LinkerScriptRule` objects.

Also, you must only move chunks from one `LinkerScriptRule` object to another in the *CreatingSections* link state. It is an undefined behavior to move chunks in other link states.

The *AddRule* plugin moves chunks from `foo` and `bar` output sections to the `var` output section. Let's see how to create this plugin.

AddRule plugin with self-contained documentation.

```
#include "ELD/PluginAPI/OutputSectionIteratorPlugin.h"
#include "ELD/PluginAPI/PluginVersion.h"

class AddRule : public eld::plugin::OutputSectionIteratorPlugin {
public:
  AddRule() : eld::plugin::OutputSectionIteratorPlugin("AddRule") {}

  // 'Init' callback hook can be used for initialization and preparations.
  // This plugin does not need any initialization or preparation.
  void Init(std::string cfg) override {}

  // 'processOutputSection' callback hook is called once for each output
  // section. In this function, the plugin stores 'var, 'foo' and 'bar' output
  // sections in member variables.
  void processOutputSection(eld::plugin::OutputSection O) override {
    // OutputSectionIterator plugin essentially runs three times.
    // It is run once for each of the following three link states: BeforeLayout,
    // CreatingSections and AfterLayout.
    // We are only interested in one link state, CreatingSections, as chunks can
    // only be moved from one LinkerScriptRule to another in the
    // CreatingSections link state. Thus, we simply return for the other link
    // states. We will do this for each callback hook function.
    if (Linker->getState() != eld::plugin::LinkerWrapper::State::CreatingSections)
      return;
    if (O.getName() == "var") {
      m_Var = O;
    } else if (O.getName() == "foo") {
      m_Foo = O;
    } else if (O.getName() == "bar") {
      m_Bar = O;
    }
  }

  // 'Run' callback hook is called after all the 'processSection' callback hook
  // calls.
  eld::plugin::Plugin::Status Run(bool trace) override {
    if (Linker->getState() != eld::plugin::LinkerWrapper::State::CreatingSections)
      return eld::plugin::Plugin::Status::SUCCESS;

    auto lastRule = m_Var.getLinkerScriptRules().back();
    // Create a new rule for m_Var output section.
    // Annotation is used to name the linker sript rule, and is useful
    // for diagnostic purposes.
    eld::plugin::LinkerScriptRule newRule = Linker->createLinkerScriptRule(
        m_Var, /*Annotation=*/"Move foo and bar chunks to var");
    // Insert the newly created linker script rule in the m_Var output section.
    // We can also insert the newly created rule before some already existing
    // rule using LinkerWrapper::insertBeforeRule API.
    Linker->insertAfterRule(m_Var, lastRule, newRule);
    moveChunks(m_Foo, newRule);
    moveChunks(m_Bar, newRule);

    return eld::plugin::Plugin::Status::SUCCESS;
  }

  // 'Destroy' callback hook can be used for finalization and clean-up tasks.
  // It is called once for each section iterator plugin run.
  void Destroy() override {}
```

```
  uint32_t GetLastError() override { return 0; }

  std::string GetLastErrorAsString() override { return "Success"; }

  std::string GetName() override { return "AddRule"; }

private:
  void moveChunks(eld::plugin::LinkerScriptRule oldRule,
                  eld::plugin::LinkerScriptRule newRule) {
    for (eld::plugin::Chunk C : oldRule.getChunks()) {
      // It is crucial to maintain that no two LinkerScriptRule objects contain
      // the same chunk. It is an undefined behavior for a chunk to be contained
      // by multiple linker script rules.
      Linker->addChunk(newRule, C);
      Linker->removeChunk(oldRule, C);
    }
  }

  void moveChunks(eld::plugin::OutputSection oldSection,
                  eld::plugin::LinkerScriptRule newRule) {
    for (eld::plugin::LinkerScriptRule rule : oldSection.getLinkerScriptRules())
      moveChunks(rule, newRule);
  }

private:
  eld::plugin::OutputSection m_Var = eld::plugin::OutputSection(nullptr);
  eld::plugin::OutputSection m_Foo = eld::plugin::OutputSection(nullptr);
  eld::plugin::OutputSection m_Bar = eld::plugin::OutputSection(nullptr);
};

eld::plugin::Plugin *ThisPlugin = nullptr;

extern "C" {
// RegisterAll should initialize all the plugins that a plugin library aims
// to provide. Linker calls this function before running any plugins provided
// by the library.
bool RegisterAll() {
  ThisPlugin = new AddRule{};
  return true;
}

// Linker calls this function to request an instance of the plugin
// with the plugin name pluginName. pluginName is provided in the plugin
// invocation command.
eld::plugin::Plugin *getPlugin(const char *pluginName) { return ThisPlugin; }

// Cleanup should free all the resources owned by a plugin library.
// Linker calls this function after all runs of the plugins provided
// by the library have completed.
void Cleanup() {
  if (!ThisPlugin)
    return;
  delete ThisPlugin;
  ThisPlugin = nullptr;
}
}
```

To build the plugin, run the following command:

```
clang++-14 -o libAddRule.so AddRule.cpp -std=c++17 -stdlib=libc++ -fPIC -shared \
-I${HEXAGON}/include -L${HEXAGON}/lib -lLW
```

Now, let’s see the effect of this plugin on a sample program.

1.c

```
int foo() { return 1; }
int bar() { return 2; }
int baz() { return 3; }

int int_var = 1;
long long_var = 2;
double double_var = 3;

int main() {
  return 0;
}
```

1.linker.script

```
SECTIONS {
  foo : { *(.text.foo) }
  bar : { *(.text.bar) }
  baz : { *(.text.baz) }
  var : { *(*var) }
}

PLUGIN_OUTPUT_SECTION_ITER("AddRule", "AddRule");
```

Now, let’s build 1.c with the *AddRule* plugin enabled.

```
export LD_LIBRARY_PATH="${HEXAGON}/lib:${LD_LIBRARY_PATH}"
hexagon-clang -o 1.o 1.c -c -ffunction-sections -fdata-sections
hexagon-link -o 1.elf 1.o -T 1.linker.script -Map 1.map.txt
```

We can see the symbol to section mapping using hexagon-readelf.

```
hexagon-readelf -Ss 1.elf
```

readelf output:

```
There are 8 section headers, starting at offset 0x12a0:

Section Headers:
  [Nr] Name              Type            Address  Off    Size   ES Flg Lk Inf Al
  [ 0]                   NULL            00000000 000000 000000 00      0   0  0
  [ 1] baz               PROGBITS        00000000 001000 00000c 00  AX  0   0 16
  [ 2] .text.main        PROGBITS        00000010 001010 000014 00  AX  0   0 16
  [ 3] var               PROGBITS        00000030 001030 00002c 00 WAXp  0   0 16
  [ 4] .comment          PROGBITS        00000000 00105c 0000d2 01  MS  0   0  1
  [ 5] .shstrtab         STRTAB          00000000 00112e 000037 00      0   0  1
  [ 6] .symtab           SYMTAB          00000000 001168 0000e0 10      7   6  4
  [ 7] .strtab           STRTAB          00000000 001248 000054 00      0   0  1
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
  L (link order), O (extra OS processing required), G (group), T (TLS),
  C (compressed), x (unknown), o (OS specific), E (exclude),
  R (retain), p (processor specific)

Symbol table '.symtab' contains 14 entries:
   Num:    Value  Size Type    Bind   Vis       Ndx Name
     0: 00000000     0 NOTYPE  LOCAL  DEFAULT   UND
     1: 00000000     0 SECTION LOCAL  DEFAULT     1 baz
     2: 00000000     0 SECTION LOCAL  DEFAULT     4 .comment
     3: 00000010     0 SECTION LOCAL  DEFAULT     2 .text.main
     4: 00000030     0 SECTION LOCAL  DEFAULT     3 var
     5: 00000000     0 FILE    LOCAL  DEFAULT   ABS 1.c
     6: 00000000    12 FUNC    GLOBAL DEFAULT     1 baz
     7: 00000010    20 FUNC    GLOBAL DEFAULT     2 main
     8: 00000030     4 OBJECT  GLOBAL DEFAULT     3 int_var
     9: 00000034     4 OBJECT  GLOBAL DEFAULT     3 long_var
```

```
    10: 00000038     8 OBJECT  GLOBAL DEFAULT     3 double_var
    11: 00000040    12 FUNC    GLOBAL DEFAULT     3 foo
    12: 00000050    12 FUNC    GLOBAL DEFAULT     3 bar
    13: 00000061     0 NOTYPE  GLOBAL DEFAULT   ABS __end
```

You can observe in the readelf output that the section `var` contains the `foo` and the `bar` symbols, which is exactly what we expected from the plugin.

11.2.3. Reading INI Files Functionality

The plugin framework provides built-in support for the input/output operations of INI files. Linker plugins, like any other tool, may require external options and configurations. Traditionally, linker plugins use INI files for this purpose. This plugin demo demonstrates how to use INI files to provide options and configurations to a plugin. You can also use JSON, YAML, plain text, or any other format you prefer. However, for consistency with other linker plugins, I suggest using INI files for the plugin options and configurations.

The demo will feature the *PrintSectionsInfo* plugin. This plugin will print section information for all input sections that match any of the section name patterns specified in the plugin configuration file. The plugin will expect the configuration file to in INI format.

PrintSectionsInfo plugin with self-contained documentation.

```
#include "ELD/PluginAPI/PluginVersion.h"
#include "ELD/PluginAPI/SectionMatcherPlugin.h"
#include <iostream>
#include <set>
#include <string>

class PrintSectionsInfo : public eld::plugin::SectionMatcherPlugin {
public:
  PrintSectionsInfo() : eld::plugin::SectionMatcherPlugin("PrintSectionsInfo") {}

  // 'Init' callback hook can be used for initialization and preparations.
  // We will read the configuration file here.
  void Init(std::string cfg) override {
    // 'Linker' is an inherited LinkerWrapper member variable. LinkerWrapper
    // is to be used to make any calls to the Linker.
    // LinkerWrapper::findConfigFile searches the file in standard paths and
    // returns a resolved path if the file is found.
    std::string configPath = Linker->findConfigFile(cfg);
    auto expINIFile = Linker->readINIFile(configPath);
    // If plugin configuration file cannot be read, then report the error,
    // set linker to fatal error and return.
    if (!expINIFile) {
      Linker->reportDiagEntry(std::move(expINIFile.error()));
      Linker->setLinkerFatalError();
      return;
    }
    eld::plugin::INIFile I = std::move(expINIFile.value());

    // Read patterns from the plugin configuration file and store
    // them in a member variable. These patterns will be used later
    // to decide which sections information should be printed.
    auto sections = I.getSection("sections");
    for (auto item : sections) {
      if (item.second == "1")
        m_SectionPatterns.insert(item.first);
    }
  }

  // 'processSection' callback hook is called for each input section.
  void processSection(eld::plugin::Section S) override {
    if (shouldPrintSectionInfo(S)) {
      std::cout << S.getName() << "\n";
      std::cout << "Input file: " << S.getInputFile().getFileName() << "\n";
      std::cout << "Section index: " << S.getIndex() << "\n";
      std::cout << "Section alignment: " << S.getAlignment() << "\n";
```

```
      std::cout << "\n";
    }
  }

  // 'Run' callback hook is called after 'processSection' callback hook calls.
  // It is called once for each section iterator plugin run.
  eld::plugin::Plugin::Status Run(bool trace) override {
    return eld::plugin::Plugin::Status::SUCCESS;
  }

  // 'Destroy' callback hook can be used for finalization and clean-up tasks.
  // It is called once for each section iterator plugin run.
  void Destroy() override {}

  uint32_t GetLastError() override { return 0; }

  std::string GetLastErrorAsString() override { return "Success"; }

  std::string GetName() override { return "PrintSectionsInfo"; }

private:
  // Returns true if the section information should be printed.
  bool shouldPrintSectionInfo(eld::plugin::Section S) {
    // If the section name matches the pattern specified in the plugin
    // configuration file then return true.
    for (auto pattern : m_SectionPatterns) {
      if (S.matchPattern(pattern))
        return true;
    }
    return false;
  }

  // Stores section name patterns.
  std::set<std::string> m_SectionPatterns;
};

eld::plugin::Plugin *ThisPlugin = nullptr;

extern "C" {
// RegisterAll should initialize all the plugins that the plugin library aims
// to provide. Linker calls this function before running any plugins provided
// by the library.
bool RegisterAll() {
  ThisPlugin = new PrintSectionsInfo{};
  return true;
}

// Linker calls this function to request an instance of the plugin
// with the plugin name pluginName. pluginName is provided in the plugin
// invocation command.
eld::plugin::Plugin *getPlugin(const char *pluginName) { return ThisPlugin; }

// Cleanup should free all the resources owned by the plugin library.
// Linker calls this function after all runs of the plugins provided
// by the library have completed.
void Cleanup() {
  if (!ThisPlugin)
    return;
  delete ThisPlugin;
  ThisPlugin = nullptr;
}
}
```

To build the plugin, run:

```
clang++-14 -o libPrintSectionsInfo.so PrintSectionsInfo.cpp -std=c++17 -stdlib=libc++ -fPIC -shared \
-I${HEXAGON}/include \
-L${HEXAGON}/lib -lLW
```

Now, let's see the effect of this plugin on a sample program.

1.c

```
int i[5] = {1, 2, 3, 4, 5};
int arr[100];

int foo() { return 1; }
int bar() { return 2; }

int main() { return 0; }
```

1.linker.script

```
# PluginConf.ini is the plugin configuration file name.
PLUGIN_SECTION_MATCHER("PrintSectionsInfo", "PrintSectionsInfo", "PluginConf.ini");
```

PluginConf.ini

```
[sections]
*i=1
*arr=1
*foo=0
*bar=1
```

Now, let's build 1.c with *PrintSectionsInfo* plugin enabled.

```
export LD_LIBRARY_PATH="${HEXAGON}/lib:${LD_LIBRARY_PATH}"
hexagon-clang -o 1.o 1.c -c -ffunction-sections -fdata-sections
hexagon-link -o 1.elf 1.o -T 1.linker.script
```

The plugin gives the below output:

```
.text.bar
Input file: 1.o
Section index: 4
Section alignment: 16

.data.i
Input file: 1.o
Section index: 6
Section alignment: 8

COMMON.arr
Input file: CommonSymbols
Section index: 0
Section alignment: 8
```

As expected, the plugin prints the section information of the sections whose names match a pattern specified in the plugin configuration file, *PluginConf.ini*.

11.2.4. Modify Relocations

The *ModifyRelocations* example plugin demonstrates how to inspect and modify relocations. This plugin modifies the relocation symbol from `HelloWorld` to `HelloQualcomm`. Additionally, it prints the relocation source section and symbol for each relocation it iterates over.

To inspect and modify relocations, *ModifyRelocations* uses `LinkerPluginConfig`. `LinkerPluginConfig` provides a callback hook for relocations. This callback hook function is called for each relocation that is of a registered relocation type. Relocation types can be registered using `LinkerWrapper::registerReloc`.

ModifyRelocations plugin with self-contained documentation.

```
#include "ELD/PluginAPI/PluginVersion.h"
#include "ELD/PluginAPI/SectionIteratorPlugin.h"
```

```cpp
#include <iostream>

class ModifyRelocations : public eld::plugin::SectionIteratorPlugin {
public:
  ModifyRelocations() : SectionIteratorPlugin("ModifyRelocations") {}

  // 'Init' callback hook can be used for initialization and preparations.
  // This plugin does not need any initialization or preparation.
  void Init(std::string cfg) override {}

  // 'processSection' callback hook of SectionIteratorPlugin is called for
  // each non-garbage collected section.
  void processSection(eld::plugin::Section S) override {}

  // 'Run' callback hook is called after all the 'processSection' callback hook
  // calls. It is called once for each section iterator plugin run.
  // This plugin does not need to run anything.
  eld::plugin::Plugin::Status Run(bool trace) override {
    return eld::plugin::Plugin::Status::SUCCESS;
  }

  // 'Destroy' callback hook can be used for finalization and clean-up tasks.
  // It is called once for each section iterator plugin run.
  // This plugin does not need any finalization and clean-up.
  void Destroy() override {}

  uint32_t GetLastError() override { return 0; }

  std::string GetLastErrorAsString() override { return "Success"; }

  std::string GetName() override { return "ModifyRelocations"; }

  eld::plugin::LinkerWrapper *getLinker() { return Linker; }
};

// LinkerPluginConfig allows to inspect and modify relocations.
class ModifyRelocationsPluginConfig : public eld::plugin::LinkerPluginConfig {
public:
  ModifyRelocationsPluginConfig(ModifyRelocations *P)
      : LinkerPluginConfig(P), P(P) {}

  void Init() override {
    // Register R_HEX_B22_PCREL relocation type.
    // Linker will call RelocCallBack callback hook function on each relocation
    // that is of a registered relocation type.
    std::string b22pcrel = "R_HEX_B22_PCREL";
    uint32_t relocationType =
        P->getLinker()->getRelocationHandler().getRelocationType(b22pcrel);
    P->getLinker()->registerReloc(relocationType);
  }

  // Relocation callback hook function.
  void RelocCallBack(eld::plugin::Use U) override {
    // Print relocation source section name and symbol names.
    std::string sourceSectionName = U.getSourceChunk().getName();
    std::cout << "Relocation callback. Source section: " << sourceSectionName
              << ", symbol: " << U.getName() << "\n";
    // Change relocation symbol from HelloWorld to HelloQualcomm.
    if (U.getSymbol().getName() == "HelloWorld") {
      eld::Expected<eld::plugin::Symbol> expHelloQualcommSymbol =
          P->getLinker()->getSymbol("HelloQualcomm");
      ELDEXP_REPORT_AND_RETURN_VOID_IF_ERROR(
          P->getLinker(), expHelloQualcommSymbol);
      eld::plugin::Symbol helloQualcommSymbol =
          std::move(expHelloQualcommSymbol.value());
      U.resetSymbol(helloQualcommSymbol);
    }
  }
```

```
private:
  ModifyRelocations *P;
};

eld::plugin::Plugin *ThisPlugin = nullptr;
eld::plugin::LinkerPluginConfig *ThisPluginConfig = nullptr;

extern "C" {
// RegisterAll should initialize all the plugins and plugin configs that a
// plugin library aims to provide. Linker calls this function before running
// any plugins provided by the library.
bool DLL_A_EXPORT RegisterAll() {
  ThisPlugin = new ModifyRelocations();
  ThisPluginConfig = new ModifyRelocationsPluginConfig(
      dynamic_cast<ModifyRelocations *>(ThisPlugin));
  return true;
}

// Linker calls this function to request an instance of the plugin
// with the plugin name pluginName. pluginName is provided in the plugin
// invocation command.
eld::plugin::Plugin *getPlugin(const char *pluginName) { return ThisPlugin; }

// Linker calls this function to request an instance of the plugin
// configuration for the plugin with the plugin name pluginName.
// pluginName is provided in the plugin invocation command.
eld::plugin::LinkerPluginConfig DLL_A_EXPORT *getPluginConfig(const char *pluginName) {
  return ThisPluginConfig;
}

// Cleanup should free all the resources owned by a plugin library.
// Linker calls this function after all runs of the plugins provided
// by the library have completed.
void DLL_A_EXPORT Cleanup() {
  if (ThisPlugin)
    delete ThisPlugin;
  if (ThisPluginConfig)
    delete ThisPluginConfig;
}
}
```

To build the plugin, run:

```
clang++-14 -o libModifyRelocations.so ModifyRelocations.cpp -std=c++17 -stdlib=libc++ -fPIC -shared \
-I${HEXAGON}/include -L${HEXAGON}/lib -lLW
```

Now, let’s see the effect of this plugin on a sample program.

1.c

```
#include <stdio.h>

const char *HelloWorld() {
  return "Hello World!";
}

const char *HelloQualcomm() {
  return "Hello Qualcomm!";
}

int main() {
  printf("%s\n", HelloWorld());
}
```

1.linker.script

```
PLUGIN_ITER_SECTIONS("ModifyRelocations", "ModifyRelocations");
```

Now, let's build 1.c with *ModifyRelocations* plugin enabled.

```
export LD_LIBRARY_PATH="/local/mnt/workspace/partaror/llvm-project-formal/obj/bin:${LD_LIBRARY_PATH}"
hexagon-clang -o 1.elf 1.c -ffunction-sections -fdata-sections -Wl,-T,1.linker.script
```

Running the above commands gives the below output:

```
Relocation callback. Source section: .text, symbol: .start
...
...
Relocation callback. Source section: .text.main, symbol: HelloWorld
Relocation callback. Source section: .text.main, symbol: printf
...
```

This output is emitted by the plugin when it's iterating over relocations of registered types.

Now, let's try running the generated binary image, *1.elf*:

```
$ hexagon-sim ./1.elf

hexagon-sim INFO: The rev_id used in the simulation is 0x00008d68 (v68n_1024)
Hello Qualcomm!

Done!
        T0: Insns=5479 Packets=2946
        T1: Insns=0 Packets=0
        T2: Insns=0 Packets=0
        T3: Insns=0 Packets=0
        T4: Insns=0 Packets=0
        T5: Insns=0 Packets=0
        Total: Insns=5479 Pcycles=8841
```

Hello World! has been replaced by *Hello Qualcomm!*. It's the result of plugin changing the relocation symbol from `HelloWorld` to `HelloQualcomm` for all `R_HEX_B22_PCREL` relocations.

11.2.5. Section Rule-Matching

This example plugin describes how to modify section rule-matching using a plugin. A plugin can create section overrides using the `LinkerWrapper::setOutputSection` API. Section overrides created by a plugin override linker script section rule-matching. Plugins must call `LinkerWrapper::finishAssignOutputSections` after all section overrides have been created. `LinkerWrapper::finishAssignOutputSections` brings the section override change into effect.

Section overrides must only be used in the *BeforeLayout* link state. After the *BeforeLayout* state, chunks from input sections get merged into output sections, making section overrides meaningless.

The *ChangeOutputSection* plugin sets the output section of `.text.foo` to `bar`. That's it. Let's see how to create this plugin.

ChangeOutputSection plugin with self-contained documentation.

```
#include "ELD/PluginAPI/PluginVersion.h"
#include "ELD/PluginAPI/SectionIteratorPlugin.h"
#include <iostream>

class ChangeOutputSection : public eld::plugin::SectionIteratorPlugin {
public:
  ChangeOutputSection() : eld::plugin::SectionIteratorPlugin("ChangeOutputSection") {}

  // 'Init' callback hook can be used for initialization and preparations.
  // This plugin does not need any initialization or preparation.
  void Init(std::string cfg) override {}

  // 'processSection' callback hook of SectionIteratorPlugin is called for
  // each non-garbage collected section.
```

```
  void processSection(eld::plugin::Section S) override {
    if (S.matchPattern("*foo")) {
      // Changes the output section of the section S to
      // bar. LinkerWrapper::setOutputSection must only be
      // called in BeforeLayout link state. Section overrides created
      // after BeforeLayout link state do not work and can result in
      // undefined behavior.
      //
      // Annotation is useful for diagnostic purposes. Later, we will see where
      // to find these annotations.
      Linker->setOutputSection(
          S, "bar",
          /*Annotation=*/"Setting output section of '.text.foo' to 'bar'");
    }
  }

  // 'Run' callback hook is called after all the 'processSection' callback hook
  // calls. It is called once for each section iterator plugin run.
  eld::plugin::Plugin::Status Run(bool trace) override {
    return eld::plugin::Plugin::Status::SUCCESS;
  }

  // 'Destroy' callback hook can be used for finalization and clean-up tasks.
  // It is called once for each section iterator plugin run.
  void Destroy() override {
    // LinkerWrapper::finishAssignOutputSections must be called
    // after all section overrides have been created by the plugin.
    // It brings the created section overrides into effect.
    Linker->finishAssignOutputSections();
  }

  uint32_t GetLastError() override { return 0; }

  std::string GetLastErrorAsString() override { return "Success"; }

  std::string GetName() override { return "ChangeOutputSection"; }
};

eld::plugin::Plugin *ThisPlugin = nullptr;

extern "C" {
// RegisterAll should initialize all the plugins that a plugin library aims
// to provide. Linker calls this function before running any plugins provided
// by the library.
bool RegisterAll() {
  ThisPlugin = new ChangeOutputSection{};
  return true;
}

// Linker calls this function to request an instance of the plugin
// with the plugin name pluginName. pluginName is provided in the plugin
// invocation command.
eld::plugin::Plugin *getPlugin(const char *pluginName) { return ThisPlugin; }

// Cleanup should free all the resources owned by a plugin library.
// Linker calls this function after all runs of the plugins provided
// by the library have completed.
void Cleanup() {
  if (!ThisPlugin)
    return;
  delete ThisPlugin;
  ThisPlugin = nullptr;
}
}
```

To build the plugin, run the following command:

```
clang++-14 -o libChangeOutputSection.so ChangeOutputSection.cpp -std=c++17 -stdlib=libc++ -fPIC -shared \
-I${HEXAGON}/include -L${HEXAGON}/lib -lLW
```

1.c

```
int foo() { return 1; }
int bar() { return 2; }

int main() { return 0; }
```

1.linker.script

```
PLUGIN_ITER_SECTIONS("ChangeOutputSection", "ChangeOutputSection");

SECTIONS {
  foo : { *(.text.foo) }
  bar : { *(.text.bar) }
  text : { *(.text*) }
}
```

Now, let’s build 1.c with the *ChangeOutputSection* plugin enabled.

```
export LD_LIBRARY_PATH="${HEXAGON}/lib:${LD_LIBRARY_PATH}"
hexagon-clang -o 1.o 1.c -c -ffunction-sections -fdata-sections
hexagon-link -o 1.elf 1.o -T 1.linker.script -Map 1.map.txt
```

Now, let’s see the output section of `foo`.

```
hexagon-readelf -Ss 1.elf
```

hexagon-readelf output:

```
There are 7 section headers, starting at offset 0x1200:

Section Headers:
  [Nr] Name              Type            Address  Off    Size   ES Flg Lk Inf Al
  [ 0]                   NULL            00000000 000000 000000 00      0   0  0
  [ 1] bar               PROGBITS        00000000 001000 00001c 00  AX  0   0 16
  [ 2] text              PROGBITS        00000020 001020 000014 00  AX  0   0 16
  [ 3] .comment          PROGBITS        00000000 001034 0000d2 01  MS  0   0  1
  [ 4] .shstrtab         STRTAB          00000000 001106 00002d 00      0   0  1
  [ 5] .symtab           SYMTAB          00000000 001134 000090 10      6   5  4
  [ 6] .strtab           STRTAB          00000000 0011c4 00002a 00      0   0  1
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
  L (link order), O (extra OS processing required), G (group), T (TLS),
  C (compressed), x (unknown), o (OS specific), E (exclude),
  R (retain), p (processor specific)

Symbol table '.symtab' contains 9 entries:
   Num:    Value  Size Type    Bind   Vis       Ndx Name
     0: 00000000     0 NOTYPE  LOCAL  DEFAULT   UND
     1: 00000000     0 SECTION LOCAL  DEFAULT     1 bar
     2: 00000000     0 SECTION LOCAL  DEFAULT     3 .comment
     3: 00000020     0 SECTION LOCAL  DEFAULT     2 text
     4: 00000000     0 FILE    LOCAL  DEFAULT   ABS 1.c
     5: 00000000    12 FUNC    GLOBAL DEFAULT     1 bar
     6: 00000010    12 FUNC    GLOBAL DEFAULT     1 foo
     7: 00000020    20 FUNC    GLOBAL DEFAULT     2 main
     8: 00000039     0 NOTYPE  GLOBAL DEFAULT   ABS __end
```

You can observe from the readelf output that the `bar` section contains the `foo` symbol. This means the output section of `.text.foo` was indeed changed to `bar`.

That’s all well and good, but how do we see the nice annotation we added when calling `LinkerWrapper::setOutputSection`? All plugin actions, along with their annotations, are recorded in the map file. The text map file can be used to quickly view plugin actions and the annotations.

To see all the plugin action information, you can either view the text map file and find the “Detailed Plugin information” component, or run the command below to directly view the information from the text map file.

```
less +/"Detailed Plugin information" 1.map.txt
```

Output

```
# Detailed Plugin information
# Plugin #0     ChangeOutputSection
#       Modification #1 {C, Comment : Setting output section of '.text.foo' to 'bar'}
#       Section :.text.foo      1.o
#       Original Rule : *(.text.foo) #Rule 1, 1.linker.script
#       Modified Rule : *(bar) #Rule 4, Internal-LinkerScript (Implicit rule inserted by Linker)
```

11.2.6. Change Symbol Value to Another Symbol

The *ChangeSymbolValue* plugin demonstrates how to change the value of a symbol to another symbol. Specifically, it changes the value of the `HelloWorld` symbol to the value of the `HelloQualcomm` symbol. It also attempts (unsuccessfully) to change the value of the `HelloWorldAgain` symbol to the value of the `HelloQualcommAgain` symbol. We will see why this happens.

Now, let’s see how to create the *ChangeSymbolValue* plugin.

The *ChangeSymbolValue* plugin with self-contained documentation.

```
#include "ELD/PluginAPI/OutputSectionIteratorPlugin.h"
#include "ELD/PluginAPI/PluginVersion.h"
#include <iostream>

class ChangeSymbolValue : public eld::plugin::OutputSectionIteratorPlugin {
public:
  ChangeSymbolValue()
      : eld::plugin::OutputSectionIteratorPlugin("ChangeSymbolValue") {}

  // 'Init' callback hook can be used for initialization and preparations.
  // This plugin does not need any initialization or preparation.
  void Init(std::string cfg) override {}

  // 'processOutputSection' callback hook is called once for each output
  // section.
  void processOutputSection(eld::plugin::OutputSection O) override {}

  // 'Run' callback hook is called after all the 'processSection' callback hook
  // calls.
  eld::plugin::Plugin::Status Run(bool trace) override {
    if (Linker->getState() != eld::plugin::LinkerWrapper::State::AfterLayout)
      return eld::plugin::Plugin::Status::SUCCESS;
    // Try to reset the 'HelloWorld' symbol value to the value of
    // 'HelloQualcomm' symbol.
    eld::plugin::Symbol helloWorldSymbol = Linker->getSymbol("HelloWorld");
    eld::plugin::Symbol helloQualcommSymbol = Linker->getSymbol("HelloQualcomm");
    bool resetSym =
        Linker->resetSymbol(helloWorldSymbol, helloQualcommSymbol.getChunk());
    if (resetSym)
      std::cout
          << "'HelloWorld' symbol value has been successfully reset to the "
          << "value of 'HelloQualcomm' symbol.\n";
    else
      std::cout << "Symbol value resetting failed for 'HelloWorld'.\n";

    // Try to reset the 'HelloWorldAgain' symbol value to the value of
    // 'HelloQualcommAgain' symbol.
    eld::plugin::Symbol helloWorldAgainSymbol = Linker->getSymbol("HelloWorldAgain");
```

```
    eld::plugin::Symbol helloQualcommAgainSymbol =
        Linker->getSymbol("HelloQualcommAgain");
    resetSym = Linker->resetSymbol(helloWorldAgainSymbol,
                                   helloQualcommAgainSymbol.getChunk());
    if (resetSym)
      std::cout << "'HelloWorldAgain' symbol value has been successfully reset "
                << "to the "
                << "value of 'HelloQualcommAgain' symbol.\n";
    else
      std::cout << "Symbol value resetting failed for 'HelloWorldAgain'.\n";
    return eld::plugin::Plugin::Status::SUCCESS;
  }

  void Destroy() override {}

  uint32_t GetLastError() override { return 0; }

  std::string GetLastErrorAsString() override { return "Success"; }

  std::string GetName() override { return "ChangeSymbol"; }
};

eld::plugin::Plugin *ThisPlugin = nullptr;

extern "C" {
// RegisterAll should initialize all the plugins that a plugin library aims
// to provide. Linker calls this function before running any plugins provided
// by the library.
bool RegisterAll() {
  ThisPlugin = new ChangeSymbolValue{};
  return true;
}

// Linker calls this function to request an instance of the plugin
// with the plugin name pluginName. pluginName is provided in the plugin
// invocation command.
eld::plugin::Plugin *getPlugin(const char *pluginName) { return ThisPlugin; }

// Cleanup should free all the resources owned by a plugin library.
// Linker calls this function after all runs of the plugins provided
// by the library have completed.
void Cleanup() {
  if (!ThisPlugin)
    return;
  delete ThisPlugin;
  ThisPlugin = nullptr;
}
}
```

To build the plugin, run:

```
clang++-14 -o libChangeSymbolValue.so ChangeSymbolValue.cpp -std=c++17 -stdlib=libc++ -fPIC -shared \
-I${HEXAGON}/include -L${HEXAGON}/lib -lLW
```

Now, let's see the effect of this plugin on a sample program.

1.c

```
#include <stdio.h>

const char *HelloWorld;
const char *HelloQualcomm = "Hello Qualcomm!";

const char *HelloWorldAgain = "Hello again World!";
const char *HelloQualcommAgain = "Hello again Qualcomm!";

int main() {
```

```
  printf("%s\n", HelloWorld);
  printf("%s\n", HelloWorldAgain);
}
```

1.linker.script

```
PLUGIN_OUTPUT_SECTION_ITER("ChangeSymbolValue", "ChangeSymbolValue");
```

Now, let’s build 1.c with *ChangeSymbolValue* plugin enabled.

```
export LD_LIBRARY_PATH="${HEXAGON}/lib:${LD_LIBRARY_PATH}"
hexagon-clang -o 1.elf 1.c -ffunction-sections -fdata-sections -Wl,-T,1.linker.script
```

Running the above commands produces the following output:

```
'HelloWorld' symbol value has been successfully reset to the value of 'HelloQualcomm' symbol.
Symbol value resetting failed for 'HelloWorldAgain'.
```

The above output is printed by the plugin. It indicates whether the resetting of symbol values was successful or not. So, why was the resetting of ‘HelloWorldAgain’ not successful? `LinkerWrapper::resetSymbol` can only resets the value of the symbols that do not already have a value. `LinkerWrapper::resetSymbol` silently fails and returns false if the symbol requested to be reset already has a value.

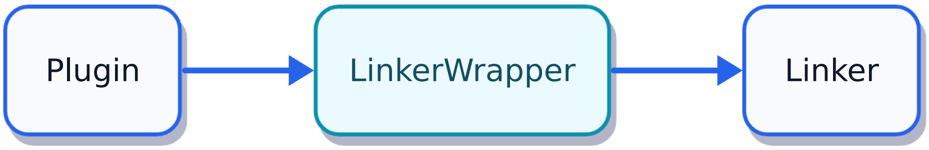
Now, let’s run the generated **1.elf** binary:

```
$ hexagon-sim ./1.elf

hexagon-sim INFO: The rev_id used in the simulation is 0x00008d68 (v68n_1024)
Hello Qualcomm!
Hello again World!

Done!
        T0: Insns=6810 Packets=3558
        T1: Insns=0 Packets=0
        T2: Insns=0 Packets=0
        T3: Insns=0 Packets=0
        T4: Insns=0 Packets=0
        T5: Insns=0 Packets=0
        Total: Insns=6810 Pcycles=10677
```

As expected, the output first prints, ‘Hello Qualcomm!’, as the value of the `HelloWorld` symbol successfully got reset to the value of `HelloQualcomm`. The output then prints, ‘Hello again World!’, as the resetting of `HelloWorldAgain` to `HelloQualcommAgain` was unsuccessful.

11.3. Linker Plugin API Usage and Reference

11.3.1. Linker Wrapper



Plugins interacts with the linker using LinkerWrapper.

class LinkerWrapper

LinkerWrapper provides a bridge between the linker plugin and the linker. The plugins uses a LinkerWrapper object to interact and communicate with the linker.

Note
Even though, all LinkerWrapper APIs are available to all plugin interface types at all link stages, some APIs are only meaningful for certain plugin interface types and link stages.

Public Types

enum LinkMode

Link modes.

Values:

enumerator UnknownLinkMode

enumerator StaticExecutable

enumerator DynamicExecutable

enumerator SharedLibrary

enumerator PIE

enumerator PartialLink

using InputSectionComparator = std::function<bool(const plugin::Section&, const plugin::Section&)>

Comparator used by sortInputSectionsForSectionMerging to order the input section vector that is used by the section-merging step.

Public Functions

explicit LinkerWrapper(eld::Plugin*, eld::Module&)

Construct LinkerWrapper object.

void RequestGarbageCollection()

Tell the linker that garbage collection should run. This method must be called early, at least no later than when symbols are created, for example from Init(). This call will set the initial state of each symbol before garbage collection and force the GC pass. This is not really needed, but avoiding this will require a bit of refactoring.

eld::Expected<std::vector<Use>> getUses(Chunk &C)

Returns a vector of Uses of symbols defined in chunk C.

Note
This function's result is valid for the link states *BeforeLayout* and beyond.

Parameters

C – Chunk containing the symbol definition.

Returns

vector of Uses that are referred from the Chunk C.

std::string getLinkerVersion() const

Returns eld version.

inline eld::Module *getModule() const

Returns the linker module.

eld::Expected<std::vector<Use>> getUses(Section &S)

Returns a vector of Uses that are referred from the Section S.

Note
This function's result is valid for the link states *BeforeLayout* and beyond.

Parameters

S – Section

Returns

vector of uses that are referred from the Section S.

eld::Expected<Symbol> getSymbol(const std::string &Sym) const

Returns the linker symbol Sym.

Note
This function's result is valid for the link states *BeforeLayout* and beyond.

Parameters

Sym – Symbol name

Returns

if successfully found, then Symbol object which represents the specified symbol; Otherwise an empty Symbol object.

eld::Expected<void> addFileToReproduceTar(std::string &FileName)

Adds a plugin generated file to the reproduce tar ball.

Note
This utility works only when the —reproduce flag is being used.

Parameters

FileName – The file path of the file

eld::Expected<Symbol> addSymbol(InputFile InputFile, const std::string &Name, plugin::Symbol::Binding Binding, Section InputSection, plugin::Symbol::Kind Kind, plugin::Symbol::Visibility Visibility = plugin::Symbol::Default, unsigned Type = 0, uint64_t Size = 0)

Create a symbol but do not provide symbol resolution information (chunk or value). Used to define bitcode symbols. Note that InputSection is optional. IMPORTANT: The symbol is resolved right away and the returned symbol will reference the resolved symbol. This means that the returned symbol may not be the bitcode symbol.

eld::Expected<void> setOutputSection(Section &S, const std::string &OutputSection, const std::string &Annotation = "")

Override the output section to OutputSection for Section S.

> **Note**
> This function must only be used in *BeforeLayout* link state.

Parameters

- **S** – Section for which the output section needs to be overriden.
- **OutputSection** – Name of the output section.
- **Annotation** – Annotations are used for diagnostic purposes.

eld::Expected<void> setPreserveSymbol(Symbol Symbol)

Mark the symbol as preserved for garbage collection.

void setSectionName(plugin::Section, const std::string&)

Rename a section.

void setRuleMatchingInput(plugin::Section, plugin::InputFile)

For a given section, set its input used for rule matching. Matching input file must be set for post-LTO sections.

eld::Expected<void> addReferencedSymbol(plugin::Section ReferencingSection, plugin::Symbol ReferencedSymbol)

Add a reference from ReferencingSection to ReferencedSymbol.

eld::Expected<OutputSection> getOutputSection(Section &S) const

Return the corresponding output section for the section S.

> **Note**
> This function's result is valid for link states *BeforeLayout* and beyond.

> **Note**
> This function's result is invalid for *SectionIteratorPlugin* run even though the link state is *Before-layout*.

Parameters

S – Input section

Returns

corresponding output section for the section S.

eld::Expected<OutputSection> getOutputSection(const std::string &OutputSection)

Finds an output section with the name OutputSection.

> **Note**
> This function's result is valid for link states *BeforeLayout* and beyond.

> **Note**
> This function's result is invalid for *SectionIteratorPlugin* run even though the link state is *Before-layout*.

Parameters

OutputSection – Output section name

Returns

If successfully found, an output section representing the specified output section; Otherwise an empty plugin::OutputSection object.

eld::Expected<std::unique_ptr<const uint8_t[]>> getOutputSectionContents(OutputSection &O) const

Returns

contents of this output section

eld::Expected<void> finishAssignOutputSections()

Finish assigning output sections.

This function should be called by the plugin after all the section rule-matching modifications has been done by the plugin.

> **Note**
> If the plugin has not changed any section rule-matching, then there is no need to call this function.

> **Note**
> This function must only be used in *BeforeLayout* link state.

eld::Expected<std::vector<plugin::Section>> getInputSectionsForSectionMerging() const

Returns the input sections vector that the linker will consume for the section-merging step.

For each linker script rule, the section-merging step merges the matched input sections and place them into the rule. The order of the input sections in the rule, and consequently, the output image, depends upon this input sections vector.

By default, the order of input sections in this vector is the input order, that is, [Input[0].sections…, Input[1].sections…, Input[2].sections…, …].

> **Note**
> This function must only be used in the *ActBeforeSectionMerging* link state.

eld::Expected<void> sortInputSectionsForSectionMerging(InputSectionComparator cmp, std::string_view annotation = “”)

Stable sort the input sections vector that will be used for the section-merging step. The order of equivalent elements as per the comparator is guaranteed to be preserved.

By default, the order of input sections in this vector is the input order, that is, [Input[0].sections…, Input[1].sections…, Input[2].sections…, …].

The sort is performed with `std::stable_sort`, so input sections that compare equivalent under `Cmp` retain their relative order from the pre-sort list. `Cmp` must define a strict weak ordering.

> **Note**
> This function must only be used in the *ActBeforeSectionMerging* link state.

Parameters

- **cmp** – A comparator returning `true` when the first argument is less than (is ordered before) the second argument.
- **annotation** – Optional human-readable note recorded in the plugin activity log alongside this call.

eld::Expected<void> reassignVirtualAddresses()

This function may be called to reassign section addresses to reflect newly added output sections that have not yet been assigned an address.

> **Note**
> This function may only be used in *CreatingSegments* state.

void setLinkerFatalError()

Set the linker to fatally error. This function will cause link process to terminate as soon as it can. It must be used if a plugin has encountered an unrecoverable error.

inline eld::Plugin *getPlugin() const

Returns the underlying linker plugin associated with the linker wrapper.

bool matchPattern(const std::string &Pattern, const std::string &Name) const

Returns true if the string Name matches the pattern specified by Pattern; Otherwise returns false.

Pattern matching here follows unix glob style pattern matching.

Parameters
- **Pattern** – glob pattern
- **name** – string to be matched against the pattern

Returns

true if Name matches the pattern specified by Pattern; Otherwise returns false.

~LinkerWrapper()

Destroy the Linker Wrapper.

bool isPostLTOPhase() const

Return true if the linker is processing post LTO objects.

eld::Expected<void> addSymbolToChunk(Chunk &C, const std::string &Symbol, uint64_t Val)

Adds a local symbol Symbol to the chunk C.

Parameters
- **C** – Chunk in which the symbol should be added.
- **symbol** – Name of the symbol
- **Val** – Value of the symbol

eld::Expected<Chunk> createPaddingChunk(uint32_t Alignment, size_t PaddingSize)

Creates a Padding Chunk in the section, '.bss.paddingchunk.${PluginName}', having contents as 0x0.

Parameters
- **Alignment** – Required alignment for the padding chunk
- **PaddingSize** – Required size for the padding chunk

Returns

The created padding chunk

eld::Expected<Chunk> createCodeChunk(const std::string &Name, uint32_t Alignment, const char *Buf, size_t Sz)

Creates a code chunk in the section, '.text.codechunk.${Name}.${PluginName}', having contents specified by the buffer Buf of size Sz. The created Code chunk will have all the usual properties of the code section such as PROGBITS section type, and allocation and execution permissions.

Parameters
- **Name** – It is used to determined the section name of the chunk.
- **Alignment** – Alignment to be used for the chunk
- **Buf** – Buffer containing the data that the the chunk will store.
- **Sz** – Size of the buffer Buf

Returns

The created code chunk

eld::Expected<Chunk> createDataChunk(const std::string &Name, uint32_t Alignment, const char *Buf, size_t Sz)

Creates a data chunk in the section, '.data.codechunk.${Name}.${PluginName}', having contents specified by the buffer Buf of size Sz. Data chunk will have all the usual properties of the data section such as PROGBITS section type, and allocation and write permissions.

Parameters
- **Name** – It is used to determine the section name of the chunk
- **Alignment** – Alignment to be used for the chunk
- **Buf** – Buffer containing the data that the chunk will store.
- **Sz** – Size of the buffer Buf

Returns

The created data chunk.

eld::Expected<Chunk> createDataChunkWithCustomName(const std::string &Name, uint32_t Alignment, const char *Buf, size_t Sz)

Creates a data chunk in the section, ‘Name’, having contents specified by the buffer Buf of size Sz. Data chunk will have all the usual properties of the data section such as PROGBITS section type, and allocation and write permissions.

Parameters
- **Name** – It is used to determine the section name of the chunk
- **Alignment** – Alignment to be used for the chunk
- **Buf** – Buffer containing the data that the chunk will store.
- **Sz** – Size of the buffer Buf

Returns

The created data chunk.

eld::Expected<Chunk> createBSSChunk(const std::string &Name, uint32_t Alignment, size_t Sz)

Create a Data Chunk in .bss section.

eld::Expected<void> replaceSymbolContent(plugin::Symbol S, const uint8_t *Buf, size_t Sz)

Replace symbol S content with the buffer Buf of size Sz.

> **Note**
> This function can be used in any link state upto and including *AfterLayout*.

Parameters
- **S** – Symbol whose content has to be replaced.
- **Buf** – Buffer containing the new content.
- **Sz** – Buffer size

Returns

void if the symbol’s content is successfully replaced; Otherwise returns a DiagnosticEntry object (as std::unique_ptr<DiagnosticEntry>) describing the error.

eld::Expected<void> addChunk(const LinkerScriptRule &Rule, const Chunk &C, const std::string &A = “”)

Adds the chunk C to the LinkerScriptRule rule.

> **Note**
> This function must only be used in *CreatingSections* link state.

Parameters
- **Rule** – Linker script rule to which the chunk C must be added.
- **C** – Chunk that is to be added.
- **A** – Annotation to be used for diagnostics purpose.

eld::Expected<void> removeChunk(const LinkerScriptRule &Rule, const Chunk &C, const std::string &A = “”)

Removes the chunk C from the output section of the rule.

> **Note**
> This function must only be used in *CreatingSections* link state.

Parameters
- **Rule** – Linker script rule from which chunk has to be removed
- **C** – Chunk that is to be removed.
- **A** – Annotation to be used for diagnostics purpose.

eld::Expected<void> updateChunks(const LinkerScriptRule &Rule, const std::vector<Chunk> &V, const std::string &A = “”)

Reset the chunks of the output section of the rule to the sequence of chunks specified by V. More precisely, this function removes all the existing chunks from the output sections, and adds the sequence of chunks , in order, as specified by V.

> **Note**
> This function must only be used in *CreatingSections* link state.

Parameters
- **Rule** – Linker script rule whose output section's chunks have to be updated.
- **V** – Sequence of chunks that are to be added to the output section of the rule.
- **A** – Annotation to be used for diagnostics purposes.

std::string getRepositoryVersion() const

Returns a string containing Linker and LLVM repository version informations.

void recordPluginData(uint32_t Key, void *Data, const std::string &Annotation)

Record Plugin Data from a plugin.

std::vector<PluginData> getPluginData(const std::string &PluginName)

Return the recorded plugin data.

llvm::Timer *CreateTimer(const std::string &Name, const std::string &Description, bool IsEnabled)

Profiling Plugin support.

eld::Expected<void> registerReloc(uint32_t RelocType, const std::string &Name = "")

Register a relocation for the plugin to call back.

RelocationHandler getRelocationHandler() const

Returns the relocation handler.

bool isMultiThreaded() const

Returns true if the linker is using multiple threads.

size_t getPluginThreadCount() const

Returns number of threads used by the linker.

LinkerScript getLinkerScript()

Return the associated LinkerScript.

std::string getFileContents(std::string FileName)

Get FileName file contents.

If FileName doesn't exist, then returns an empty string.

For correct functionality with the reproducer, the `FileName` must be the file path returned by the `LinkerWrapper::findConfigFile`.

eld::Expected<std::string> findConfigFile(const std::string &FileName) const

Returns

path to file if it is found. If not found, then an Error DiagnosticEntry is returned.

eld::Expected<INIFile> readINIFile(std::string FileName)

Read a config file in ini format.

Parameters

FileName – ini file to read

Returns

eld::Expected<INIFile> object containing either the successfully read INI file, or a diagnostic object describing the failure..

eld::Expected<void> writeINIFile(INIFile &INI, const std::string &OutputPath) const

Write an ini file.

Parameters
- **ini** – INIFile
- **OutputPath** – path of the file to write

Returns

std::unique_ptr<DiagnosticEntry> object that describes the error; if no error occurs, then returns void.

eld::Expected<TarWriter> getTarWriter(const std::string &Name) const

Creates and returns eld::Expected<TarWriter> to support creating a tar archive with the name passed and add files to it.

Parameters

std::string – &Name for tar file name.

Returns

eld::Expected<TarWriter>

bool isTimingEnabled() const

Returns

true if Timing is enabled for this plugin; Otherwise returns false.

std::vector<plugin::InputFile> getInputFiles() const

> **Note**
> This function do not returns the input linker scripts.

Returns

the input object files visited by the Linker.

LinkMode getLinkMode() const

Returns the link mode.

bool is32Bits() const

Return true if 32 bit target.

char *getUninitBuffer(size_t S) const

Allocate an uninitialized buffer that will live throughout the link process.

Parameters

S – Size of the uninitialized buffer

Returns

base address of the created uninitialized buffer.

eld::Expected<void> resetSymbol(plugin::Symbol S, Chunk C)

Reset an undefined symbol to another chunk. Reset here means, change the source chunk of the symbol. This functions marks that symbol S is defined in the chunk C.

> **Note**
> This function can be used in any link state including and upto *Afterlayout*.

> **Note**
> This function only works for symbols that do not have a value. The function returns false, if the symbol requested to be reset is a defined symbol with an initial value.

Parameters

- **S** – Symbol whose source chunk is to be changed.
- **C** – New source chunk for the symbol.

Returns

true if resetting is successful; Otherwise false.

eld::Expected<void> setSymbolAddress(plugin::Symbol S, uint64_t Addr)

Detach symbol S from its current fragment (if any) and make it an absolute symbol with value Addr. The symbol's output section index becomes SHN_ABS, and its value is no longer derived from any chunk's placement.

Unlike resetSymbol(), this works on already-defined symbols.

Note
This function must only be used in the *AfterLayout* link state, i.e. after layout, garbage-collection, and relaxation have converged, and before the symbol table and relocations are finalized and emitted.

Note
Reassigning a symbol's address this way does not change the address range that the linker's virtual-address overlap check associates with the symbol's original output section. If the new address could fall inside another section's address range, declare the symbol's output section with the `INFO`, `COPY`, `DSECT`, or `OVERLAY` linker script section type. These types are equivalent in ELD: each removes the section from PT_LOAD segment placement by clearing `SHF_ALLOC` on the section, which is recommended whenever the section content addresses are meant to be referenced by the code at runtime (e.g. debug strings) but not actually dereferenced or loaded into memory at runtime.

Parameters
- **S** – Symbol whose address is to be set.
- **Addr** – Absolute value to assign to the symbol.

eld::Expected<Use> createAndAddUse(Chunk C, off_t Offset, uint32_t RelocationType, plugin::Symbol S, int64_t Addend)

Create and return a Use for a Chunk, and add it to the Chunk.

eld::Expected<void> getTargetDataForUse(Use &U, uint64_t &Data) const

Return the value of the relocation target.

eld::Expected<void> setTargetDataForUse(Use &U, uint64_t Data)

Set the value of the relocation target.

eld::Expected<uint32_t> getImageLayoutChecksum() const

Returns the 32-bit checksum of the image layout if this function is used when the linker state is *AfterLayout* Otherwise returns 0.

Note
This function's result is only valid when the link state is *AfterLayout*.

std::string getOutputFileName() const

Returns the output file name of the current link process.

eld::Expected<LinkerScriptRule> createLinkerScriptRule(OutputSection S, const std::string &Annotation)

Create a Linker script rule.

Note
Annotation must not be empty. If annotation is empty, then this function returns an empty LinkerScriptRule object.

Parameters
- **S** – Corresponding output section of the linker script rule.
- **Annotation** – It is used to create the rule name. It's also helpful for debugging purposes.

eld::Expected<void> insertAfterRule(plugin::OutputSection O, LinkerScriptRule Rule, LinkerScriptRule RuleToAdd)

Add a linker script rule after an existing rule. A rule describes an input section description.

Note
This function must only be used when the link state is *CreatingSections*. Behavior of using this function in other linker states is undefined.

Parameters
- **O** – Output section of the rule.
- **Rule** – Rule after which RuleToAdd have to be inserted.

- **RuleToAdd** – Rule that is to be added.

Returns

true if rule is successfully inserted; Otherwise false.

eld::Expected<void> insertBeforeRule(plugin::OutputSection O, LinkerScriptRule Rule, LinkerScriptRule RuleToAdd)

Add a linker script rule before an existing rule.

> **Note**
> This function should only be used when the linker state is *CreatingSections*. Behavior of using this function in other linker states is undefined.

Parameters

- **O** – Output section of the rule.
- **Rule** – Rule before which RuleToAdd have to be inserted.
- **RuleToAdd** – Rule that is to be added.

Returns

true if rule is successfully inserted; Otherwise false.

void removeSymbolTableEntry(plugin::Symbol S)

Exclude the symbol S from the symbol table.

This function only removes the symbol from the symbol table. It does not remove the symbol content from the output image.

> **Note**
> This function can be used in any link state including and upto *AfterLayout*.

Parameters

S – Symbol to remove

eld::Expected<std::vector<plugin::Symbol>> getAllSymbols() const

Returns all symbols.

> **Note**
> Currently, the function returns an empty vector for SectionIteratorPlugin and SectionMatcherPlugin.

> **Note**
> The function returns all non-section symbol names for OutputSectionIteratorPlugin when linker state is including and upto *CreatingSections*. The function returns all symbols names, including section symbol names, when the linker state is *AfterLayout*.

DiagnosticEntry::DiagIDType getFatalDiagID(const std::string &formatStr)

Returns an ID for a fatal error diagnostic with the specified format string.

DiagnosticEntry::DiagIDType getErrorDiagID(const std::string &formatStr)

Returns an ID for an error diagnostic with the specified format string.

DiagnosticEntry::DiagIDType getWarningDiagID(const std::string &formatStr)

Returns an ID for a warning diagnostic with the specified format string.

DiagnosticEntry::DiagIDType getNoteDiagID(const std::string &formatStr) const

Returns an ID for a note diagnostic with the specified format string.

DiagnosticEntry::DiagIDType getVerboseDiagID(const std::string &formatStr)

Returns an ID for a verbose diagnostic with the specified format string.

bool reportDiagEntry(std::unique_ptr<DiagnosticEntry> de)

Issue a diagnostic using DiagnosticEntry.

Returns

true if diagnostic is successfully emitted; Otherwise returns false.

template<class ...Args>
inline bool reportDiag(DiagnosticEntry::DiagIDType id, Args&&... args) const

Report a diagnostic.

DiagnosticBuilder getDiagnosticBuilder(DiagnosticEntry::DiagIDType id) const

Create and return a DiagnosticBuilder for the diagnostic ID 'id'.

For all cases, reporting diagnostics using 'LinkerWrapper::report' should be preferred. DiagnosticBuilder can be beneficial in some cases. One such case is to reduce code-duplication when emitting same family of diagnostics.

DiagnosticBuilder should be used carefully as incorrect usage can cause deadlock in a program. In particular, no two DiagnosticBuilder objects should be in scope at the same time in the same thread.

bool isChunkMovableFromOutputSection(const Chunk &C) const

Allow plugins to determine if a chunk can be moved out of an output section and placed in a different output section.

> **Note**
> This function needs to be used by plugins before moving any Chunk as a safety net

Parameters

C – Chunk

Returns

true if Chunk can be moved

std::string_view getCurrentLinkStateAsStr() const

Returns the string representation of the current link state.

bool isVerbose() const

Returns true if user has requested verbose diagnostics.

bool isTraced() const

Returns true if this plugin is being traced, i.e. —trace=plugin was given without a scope, or with a scope (—trace=plugin=<name>) that matches this plugin's name.

eld::Expected<std::vector<Segment>> getSegmentTable() const

Returns

the segment table

eld::Expected<void> applyRelocations(uint8_t *Buf)

Applies relocations to an unrelocated buffer `Buf`.

eld::Expected<void> doRelocation()

Apply all relocations or return an error.

eld::Expected<DynamicLibrary> loadLibrary(const std::string &LibraryName)

Return a handle to the library `LibraryName` or an error. `LibraryName` will be searched using the linker search path including -L directories. First the file called `LibraryName` will be searched. If it is not found, `libLibraryName.so` (Linux), `libLibraryName.dylib` (macOS), or `libraryName.dll` on windows will be searched. If neither are found an error is returned.

eld::Expected<void*> getFunction(void *LibraryHandle, const std::string &FunctionName) const

Returns

a handle to the function `FunctionName` or an error.

std::vector<UnbalancedChunkMove> getUnbalancedChunkRemoves() const

Returns a vector of (Chunk, LinkerScriptRule) pairs for chunks that were removed by the plugin but have not been inserted back to the output image.

std::vector<UnbalancedChunkMove> getUnbalancedChunkAdds() const

Returns a vector of (Chunk, LinkerScriptRule) pairs for chunks that are inserted to a rule but are not removed from their original rule. This typically happens when a plugin inserts a chunk from rule A to rule B but forgets to remove it from rule A.

eld::Expected<void> addChunkToOutput(Chunk C)

Add chunk C to the output. The output section where C lives will be deterimned according to the linker script.

eld::Expected<void> resetOffset(OutputSection O)

Clear the offset for this section to be reassigned later by the linker.

eld::Expected<std::pair<OutputSection, LinkerScriptRule>> getOutputSectionAndRule(Section S)

Return the OutputSection and LinkerScriptRule for a given Input section according to the linker script.

std::optional<std::string> getEnv(const std::string &envVar) const

Returns the value of the environment variable envVar.

eld::Expected<void> linkSections(OutputSection A, OutputSection B) const

Link Section A to B using the sh_link field in the output ELF.

eld::Expected<void> registerCommandLineOption(const std::string &opt, bool hasValue, const CommandLineOptionHandlerType &optionHandler)

Registers the command-line option `opt`.

`optionHandler` callback function is invoked for each registered option present in the link command-line.

`opt` must always start with `--`. If `hasValue` is true, then options of the form `--<OptionName>=<value>` are matched; Otherwise options of the form `--<OptionName>` are matched.

Note
`opt` must not match any in-built linker command-line option.

eld::Expected<void> enableVisitSymbol()

Enables VisitSymbol hook. VisitSymbol hook is disabled by default.

eld::Expected<void> setRuleMatchingSectionNameMap(InputFile IF, std::unordered_map<uint64_t, std::string> sectionMap)

Sets rule-matching section name map for the input file IF. Rule matching section name map contains the mapping of section index to the rule-matching section name.

If a section has rule-matching section name, then this name is used, instead of the actual section name, for linker script rule-matching. For example, if for an input file IF, the rule-matching section name map is as shown below, and section with index 3 is '.text.foo', then for linker script rule-matching, then linker will use the name '.code.foo' instead of '.text.foo'.

{ 3 -> .code.foo … … }

Note
Rule-matching section map can be set only once per input-file. Setting a rule-matching section name map for an input file is an error if the file already has one.

const std::optional<std::unordered_map<uint64_t, std::string>> &getRuleMatchingSectionNameMap(const InputFile &IF)

Returns the rule-matching section name map for the input file IF.

void showRuleMatchingSectionNameInDiagnostics()

Instruct linker to show rule-matching section name in diagnostics and map-files.

eld::Expected<void> setAuxiliarySymbolNameMap(InputFile IF, const AuxiliarySymbol-NameMap &symbolNameMap)

Sets the auxiliary symbol name map for the input file IF. Auxiliary symbol name map contains the mapping of symbol index to the auxiliary symbol name.

Auxiliary symbol names are printed along with the original symbol names in the diagnostics and map-files. The auxiliary symbol names are helpful for providing more meaningful diagnostics.

Auxiliary symbol names do not affect the link behavior.

> **Note**
> Auxiliary symbol name map can be set only once per input-file. Setting an auxiliary symbol name map for an input file is an error if the file already has one.

eld::Expected<uint64_t> getOutputSymbolIndex(plugin::Symbol S)

Returns the index of the symbol S in the output image symbol table.

eld::Expected<bool> doesRuleMatchWithSection(const LinkerScriptRule &R, const Section &S, bool doNotUseRMName = false)

Returns true if the rule R matches the section S.

If doNotUseRMName is true, then the section actual name is used for rule-matching instead of the rule-matching name. By default, rule-matching name is preferred over the section name.

This function is thread-safe.

struct UnbalancedChunkMove

Represents an unbalanced chunk move. For unbalanced chunk remove: chunk: The removed chunk rule: Rule from which it was removed.

For unbalanced chunk add: chunk: The added chunk rule: Rule to which it was added.

11.3.2. Plugin Abstract Data Types

11.3.2.1. Chunk

struct Chunk

A section is formed of a set of chunks. Chunk is a handler for one such section chunk. A Chunk object can be used to inspect the properties, symbols and content of a chunk.

Chunk is a wrapper for an internal fragment class. If two Chunk objects both refer to the same underlying fragment, then they represent the same chunk.

Subclassed by eld::plugin::MergeStringChunk

Public Functions

std::string getName() const

Returns the name of the section that contains the chunk. If the object is an empty handler, then returns an empty string.

plugin::InputFile getInputFile() const

Returns the input file handler of the file from which the chunk has originated. If the object is an empty handler, then returns an empty input file handler.

uint32_t getSize() const

Returns the size of the chunk.

uint32_t getAlignment() const

Returns the alignment of the chunk.

plugin::Section getSection() const

Returns the section from which the chunk has originated.

inline eld::Fragment *getFragment() const

Returns the pointer to the underlying fragment object of the Chunk.

const char *getRawData() const

Returns the raw data that the Chunk contains.

std::vector<plugin::Symbol> getSymbols() const

Returns the list of symbols that are defined in this chunk.

inline bool operator<(const Chunk &RHS) const

Pointer comparison between the underlying fragment pointers of LHS and RHS.

inline bool operator>(const Chunk &RHS) const

Pointer comparison between the underlying fragment pointers of LHS and RHS.

inline bool operator==(const Chunk &RHS) const

Returns true if both the LHS and RHS represents the same chunk.

inline operator bool() const

Returns true if the underlying fragment pointer is non-null.

bool hasContent() const

Returns true if the underlying fragment pointer is non-null.

bool isProgBits() const

Returns true if the chunk is stored in the image. In ELF, such chunks are from sections of the PROGBITS type.

bool isNoBits() const

Returns true if the chunk is not stored in the image. In ELF, such chunks are from sections of the NOBITS type.

bool isCode() const

Returns true if the chunk contains code. More concretely, return true if the section from which the chunk originates has SHF_EXECINSTR flag set.

bool isAlloc() const

Returns true if the chunk is allocated in memory. In ELF, such chunks are from sections that have the ALLOC attribute.

bool isWritable() const

Returns true if the chunk is writable. In ELF, such chunks are from sections that have the WRITE attribute.

bool isMergeableString() const

Does this chunk contain mergeable strings ?

size_t getAddress() const

Retrieve the address of the Chunk.

std::vector<plugin::Section> getDependentSections() const

Returns a sequence of all the sections that are dependent on the section from which the chunk has originated.

struct Compare

Returns true if the chunks A and B originates from different input sections; Otherwise returns false.

11.3.2.2. LinkerScriptRule

struct LinkerScriptRule

LinkerScriptRule represents an output section command in a linker script. For example:

```
section {
  .text : {
    *(.text)
    LONG(1)
    _etext = .;
  }
}
```

In this example,

*

(.text),

LONG(1)

, and

_etext=1

, all are output section commands. They are all represented as LinkerScriptRule objects. Here

.text is the corresponding output section of the rule.

A LinkerScriptRule object can be used to inspect, and modify linker rules. Rules determine the output image layout and it's contents.

LinkerScriptRule is a wrapper for an internal RuleContainer class. If two LinkerScriptRule objects have the same underlying RuleContainer object, then both the objects represent the same rule.

Public Functions

bool canMoveSections() const

Returns true if the sections matched by the linker script rule can be moved to another output section; Otherwise returns false.

bool isLinkerInsertedRule() const

Returns true if this rule is inserted by the linker; Otherwise returns false.

bool isKeep() const

Returns true if the rule corresponds to KEEP sections.

std::optional<uint64_t> getHash() const

Returns the hash of the linkerscript rule.

plugin::Script::InputSectionSpec getInputSectionSpec() const

Returns the input section specification of the linker script rule.

> **Note**
> An Input Section description is an internal representation of a linker script rule.

plugin::OutputSection getOutputSection() const

Returns the output section corresponding to the rule.

bool hasExpressions() const

Returns true if the rule has any expression; Otherwise returns false.

Expressions are symbol assignments.

bool doesExpressionModifyDot() const

Returns true if the expression modifies the location counter ('.'); Otherwise returns false.

std::string asString() const

Return the rule as a string for diagnostics.

std::vector<plugin::Chunk> getChunks() const

Return chunks associated with this linker script rule.

std::vector<plugin::Section> getSections()

Return sections associated with the linker script rule.

std::vector<plugin::Section> getMatchedSections() const

Return all sections matched by this rule including garbage collected sections.

std::vector<plugin::Chunk> getChunks(plugin::Section S)

Get all the Chunks in the rule that correspond to Section S.

inline eld::RuleContainer *getRuleContainer() const

Return the corresponding RuleContainer of the linker script rule.

inline operator bool() const

Returns true if the object has a corresponding non-null RuleContainer object; Otherwise returns false.

inline bool operator()(const LinkerScriptRule &A, const LinkerScriptRule &B) const

Returns true if A and B have different underlying RuleContainer objects.

inline bool operator<(const LinkerScriptRule &RHS) const

Pointer comparision between the underlying RuleContainer handle of LHS and RHS.

inline bool operator>(const LinkerScriptRule &RHS) const

Pointer comparision between the underlying RuleContainer handle of LHS and RHS.

inline bool operator==(const LinkerScriptRule &RHS) const

Returns true if LHS and RHS have the same RuleContainer objects.

11.3.2.3. OutputSection

struct OutputSection

OutputSection is a handler for an output section. An OutputSection object can be used to inspect an output section, and get it's associated linker script rules.

It is a wrapper for an internal OutputSectionEntry class. If two OutputSection objects points to the same underlying OutputSectionEntry object, then they both represent the same output section.

Public Functions

std::string getName() const

Returns the name of the output section.

If the object is an empty handler, then returns an empty string.

uint64_t getAlignment() const

Returns the alignment of the output section.

uint64_t getHash() const

Returns the output section hash.

std::vector<eld::plugin::LinkerScriptRule> getRules()

Returns the linkerscript rules corrsponding to the output section.

uint64_t getFlags() const

Returns the section attribute flags. Each bit of flag defines some attribute. If a flag bit is set, then the attribute which corresponds to that bit is on for the section. Otherwise, the attribute is off and does not apply. Examples of section attributes: *SHF_WRITE*, *SHF_ALLOC*, *SHF_EXECINSTR*, and *SHF_MASKPROC* etc.

uint64_t getType() const

Returns the section semantics type. Section type specifies the section semantics e.g. SHT_PROGBITS, SHT_NOBITS, SHT_NOTE, etc.

uint64_t getIndex() const

Returns the Index of the output section. If the object is an empty handler, then returns 0.

uint64_t getSize() const

Returns the number of bytes that the section occupies in the file.

std::vector<plugin::LinkerScriptRule> getLinkerScriptRules()

Returns all linker script rules for the output section.

inline const eld::OutputSectionEntry *getOutputSection() const

Returns a pointer to the underlying OuputSectionEntry object. This can be used as a unique identifier for the output section.

eld::Expected<std::optional<Segment>> getLoadSegment(const LinkerWrapper &LW) const

Returns

The load segment for this output section, if one exists. This API should only be called in CreatingSections or AfterLayout linker states.

eld::Expected<std::vector<Segment>> getSegments(const LinkerWrapper &LW) const

Returns

All segments this OutputSection belongs to. This API should only be called in CreatingSections or AfterLayout linker states.

inline eld::OutputSectionEntry *getOutputSection()

Returns a pointer to the underlying OutputSectionEntry object. This can be used as a unique identifier for the output section.

eld::Expected<uint64_t> getVirtualAddress(const LinkerWrapper&) const

Returns the virtual address of the output section.

eld::Expected<uint64_t> getPhysicalAddress(const LinkerWrapper&) const

Returns the physical address of the output section.

inline operator bool() const

Returns true if the object represents a non-null output section.

inline bool operator()(const OutputSection &A, const OutputSection &B) const

Returns true if A and B represents different output sections.

inline bool operator<(const OutputSection &RHS) const

Pointer comparison between the underlying OutputSectionEntry handles of LHS and RHS.

inline bool operator>(const OutputSection &RHS) const

Pointer comparison between the underlying OutputSectionEntry handles of LHS and RHS.

inline bool operator==(const OutputSection &RHS) const

Returns true if both the LHS and RHS objects represents the same output sections.

eld::Expected<uint64_t> getOffset() const

Return the offset of this OutputSection, or an error if it has not been assigned one.

eld::Expected<void> setOffset(uint64_t Offset, const LinkerWrapper &LW)

Set the output offset of this section, or error.

bool isNoBits() const

Returns true if this section is of type SHT_NOBITS (the section occupies no file space).

bool isDiscard() const

Returns true if the section is being discarded.

bool isCode() const

Returns true if the section contains code with SHF_EXECINSTR flag set.

bool isAlloc() const

does this section have the SHF_ALLOC flag

std::vector<Stub> getStubs() const

Returns a list of all stubs created for this output section.

Expected<void> setPluginOverride(Plugin *P, const LinkerWrapper &LW)

Plugin

P assumes repsonsibility for this section. Some layout warnings/errors will be ignored for such sections (e.g. “loadable section

`foo` not in any load segment”). This API may only be used in

CreatingSegments

state.

11.3.2.4. Section

struct Section

Section is a handler for an ELF input section. A Section object can be used to inspect section properties, chunks and symbols. It can also be used to discard a section.

Section is a wrapper for an internal Section class. If two objects have the same underlying Section object, then they both represents the same section.

Public Functions

std::string getName() const

Returns the name of the section. If the object is an empty handler, then returns an empty string.

inline SectionType getType() const

Returns the type of the input section. If the object is an empty handler, then returns SectionType::Default.

plugin::InputFile getInputFile() const

Returns the InputFile of the input section. If the object is an empty handler, then returns an empty InputFile object.

uint32_t getIndex() const

Returns index of the section in the input file. If the object is an empty handler, then returns 0xFFFFFFFF.

uint32_t getSize() const

Returns the size of the section.

bool isDiscarded() const

Returns true if the section is being discarded.

bool isGarbageCollected() const

Returns true if the section is garbage collected.

uint32_t getAlignment() const

Returns the alignment of the section if the object.

bool isELFSection() const

Retuns true if the section is an ELF section; returns false otherwise.

bool isIgnore() const

Returns true if the ignore property is set for the input section; returns false otherwise.

Note
The linker intentionally skips these sections during the linking process. These sections are present in the input files but are not a part of the final executable.

bool isNull() const

Returns true if the input section is null false otherwise

Note
A null input section typically refers to a section in an object or executable file that has a type of SHT_NULL (in ELF format) or is otherwise empty or unused.

bool isStackNote() const

Returns true if the input section is a GNU stack note; returns false otherwise

Note
The GNU stack note refers to a special note section in ELF binaries that specifies whether the program stack should be executable or not.

bool isNamePool() const

Returns true if the input section has namepool kind set; returns false otherwise.

Note
A namepool input section is used to store names of symbols which makes it easier to search for symbols thereby assisting in symbol resolution.

bool isRelocation() const

Returns true if the input section has relocation kind set; returns false otherwise.

Note
A relocation type of input section refers to a section in an object file that contains relocation entries. These entries are instructions for the linker to adjust addresses or symbol references when combining object files into a final executable or shared library.

bool isGroup() const

Returns true if the input section is of Group kind; returns false otherwise.

Note
A group input section refers to a section that belongs to a group of related sections, this is often used to manage duplicate code or data across multiple files.

bool hasOldInputFile() const

Returns true if an old input file has been set for the input section; returns false otherwise.

plugin::InputFile getRuleMatchingInput() const

Returns the input file used for rule matching for this section. If a rule-matching input was explicitly set via LinkerWrapper::setRuleMatchingInput, that input is returned; otherwise the section's current input file is returned. Returns a null InputFile if the object is an empty handler.

uint64_t getSectionHash() const

Returns the hash of the input section.

eld::Expected<void> overrideLinkerScriptRule(LinkerWrapper &LW, plugin::LinkerScriptRule R, const std::string &Annotataion)

Sets the linker script rule for the input section.

Parameters

R – the Linkerscript rule to be set.

bool matchPattern(const std::string &Pattern) const

Returns true if the section name matches the pattern Pattern.

Note
Here, pattern matching follows unix glob style pattern matching.

Parameters

Pattern – glob pattern

Returns

true if the section name matches the pattern Pattern; Otherwise false.

std::vector<plugin::Symbol> getSymbols() const

Returns a sequence of symbols contained by this section.

std::vector<plugin::Chunk> getChunks() const

Returns a sequence of chunks contained by this section.

void markAsDiscarded()

Discard the section from the output object file.

Warning
This is a dangerous function, if not used carefully, it can discard important information, that can result in invalid output object file. TODO: What exactly discarding a section means?

bool isProgBits() const

Returns true if the section is stored in the image. In ELF, such sections are of the PROGBITS type.

bool isNoBits() const

Returns true if the section is not stored in the image. In ELF, such sections are of the NOBITS type.

bool isCode() const

Returns true if the section contains code. More precisely, it returns true if the section has SHF_EXECINSTR flag set.

bool isAlloc() const

Returns true if the section is allocated in memory. In ELF, such sections have the ALLOC attribute.

bool isWritable() const

Returns true if the section is writable. In ELF, such sections have the WRITE attribute.

inline eld::Section *getSection() const

Return a pointer to the underlying section handle.

This can be used as a unique identifier for the section.

plugin::OutputSection getOutputSection() const

Returns the output section for this section.

LinkerScriptRule getLinkerScriptRule() const

Return the linker script rule that is matched for this section.

std::vector<plugin::Section> getDependentSections() const

Returns a sequence of sections that are dependent on this section.

inline operator bool() const

Returns true if the object represents a non-null section.

inline bool operator()(const Section &A, const Section &B) const

Returns true if A and B represents different sections.

inline bool operator<(const Section &RHS) const

Pointer comparision between the underlying section handlers.

inline bool operator>(const Section &RHS) const

Pointer comparision between the underlying section handlers.

inline bool operator==(const Section &RHS) const

Return true if both the lhs and rhs represents the same section.

11.3.2.5. Use

struct Use

Use represents references from a chunk. In ELD terminology, source address and source chunk of the Use are the places where relocation has to be performed. And target address and target chunk are the places which contain the relocation symbol definition. Use can be used to inspect symbol references.

Use is a wrapper class for an internal relocation class. If two Use objects have the same underlying relocation objects, then the two Use objects represents the same Use.

Public Types

enum Status

Symbol status.

Values:

enumerator Ok

enumerator SymbolDoesNotExist

enumerator Error

Public Functions

std::string getName() const

If the Use represents a valid relocation, then return the symbol name described by this Use; Otherwise return an empty string. The symbol name is returned as how it is recorded in the input ELF file.

Chunk getChunk() const

If the Use represents a valid relocation, then returns the chunk that defines the symbol represented by this Use; Otherwise returns an empty Chunk object. DEPRECATED in favor of Use::getTargetChunk

Chunk getSourceChunk() const

If the Use represents a valid relocation, then returns the chunk that contain the symbol reference represented by this Use; Otherwise returns an empty Chunk object.

plugin::Symbol getSymbol() const

If the Use represents a valid relocation, then returns the corresponding symbol info of the relocation; Otherwise returns a null Symbol object.

inline eld::Relocation *getRelocation() const

Returns a pointer to the underlying relocation object.

Use::Status resetSymbol(plugin::Symbol S)

Reset the symbol info to a different symbol info object.

Parameters

S – Symbol that should replace the already existing symbol.

Returns

Status of the replacement.

uint32_t getType() const

If the Use has a non-null relocation handle, then returns the relocation type; Otherwise returns UINT32_MAX.

off_t getOffsetInChunk() const

If the Use represents a valid relocation, then returns the source chunk offset in it’s corresponding section; Otherwise returns −1.

size_t getSourceAddress() const

If the Use represents a valid relocation; then return the address of the Use — target address of relocation; Otherwise returns UINT64_MAX.

plugin::Chunk getTargetChunk() const

If the Use represents a valid relocation; then returns the chunk that defines the symbol represented by this Use; Otherwise returns an empty Chunk object.

off_t getTargetChunkOffset() const

If the Use has a non-null relocation handle; then returns the chunk that defines the symbol represented by this Use;

inline operator bool() const

Returns true if the Use has a non-null relocation handle.

inline bool operator()(const Use &A, const Use &B) const

Returns true if the Use A and B represent different Uses.

inline bool operator<(const Use &RHS) const

Pointer comparison between relocation handles of LHS and RHS.

inline bool operator>(const Use &RHS) const

Pointer comparison between relocation handles of LHS and RHS.

inline bool operator==(const Use &RHS) const

Returns true if the Use LHS and RHS represents the same use.

11.3.2.6. Symbol

struct Symbol

Symbol is a handler for a linker symbol. Symbol can be used to inspect a symbol.

Symbol is a wrapper for the internal ResolveInfo class. If two Symbol objects have the same underlying ResolveInfo object, then they both represent the same symbol.

Public Functions

Chunk getChunk() const

Returns the chunk that defines the symbol.

If the object is an empty handler, then returns an empty Chunk object.

std::string getName() const

Returns the symbol name.

If the object is an empty handler, then returns an empty string.

bool isLocal() const

Returns true if the symbol is local.

bool isWeak() const

Returns true if the symbol is weak.

bool isGlobal() const

Returns true if the symbol is global.

bool isUndef() const

Returns true if the symbol is undefined.

bool isCommon() const

Returns true if the symbol is common.

bool isSection() const

Returns true if the symbol is section.

bool isFunction() const

Returns true if the symbol is function.

bool isObject() const

Returns true if the symbol is object.

bool isFile() const

Returns true if the symbol is File.

bool isNoType() const

Returns true if the symbol is NoType.

bool isGarbageCollected() const

Returns true if the symbol got garbage collected.

std::string getResolvedPath() const

Returns the resolved path of a Symbol.

uint32_t getSize() const

Returns the symbol size.

A symbol size is the number of bytes contained in the object represented by the symbol.

size_t getValue() const

Returns the symbol value.

Please note that:

size_t getAddress() const

Get the Symbol Address. Most times the symbol value will be equal to the address of the symbol. If there is a need to access the address of the symbol before a symbol gets finalized, getAddress can be utilized.

off_t getOffsetInChunk() const

Returns the symbol's offset in chunk.

inline eld::ResolveInfo *getSymbol() const

Return a pointer to the underlying ResolveInfo object. This can be used as a unique identifier for the symbol.

inline operator bool() const

Conversion Operator.

inline bool operator()(const Symbol &A, const Symbol &B) const

Returns true if A and B represents different symbols.

inline bool operator<(const Symbol &RHS) const

Pointer comparison between the underlying ResolveInfo handles of LHS and RHS.

inline bool operator>(const Symbol &RHS) const

Pointer comparison between the underlying ResolveInfo handles of LHS and RHS.

inline bool operator==(const Symbol &RHS) const

Returns true if LHS and RHS represents the same symbol.

11.3.2.7. INIFile

struct INIFile

INIFile represents an ini configuration file. It provides APIs to easily inspect and modify the INI configuration.

Public Functions

explicit INIFile(const std::string FileName)

Initialize an INIFile.

void addSection(const std::string &section)

Add an empty section to this file if does not currently exist.

void insert(const std::string &section, const std::string &K, const std::string &V)

Insert a key-value pair into a given section If the section does not exist, it is automatically created.

Parameters
- **section** – Name of the section in which key-value pair has to be inserted
- **K** – Key
- **V** – Value

bool containsSection(const std::string &sectionName) const

Returns

true if a section with name sectionName is found in this file; Otherwise false.

bool containsItem(const std::string &sectionName, const std::string &key) const

Returns

true if the section `sectionName` contains the key named key; Otherwise false.

std::vector<std::string> getSections() const

Returns

vector of all section names in this file

std::vector<std::pair<std::string, std::string>> getSection(const std::string Section) const

Parameters

Section – Name of the section of the file, minus any brackets

Returns

A vector of all key/value pairs in a given section of this file If the provided section does not exist or is empty, an empty vector is returned

std::string getValue(const std::string Section, const std::string Item)

Get the value of an item in a given section of this file

Parameters
- **Section** – Name of the section, without brackets
- **Item** – Name of an item within the section

Returns

Value associated with Item in Section, if it exists If provided Section or Item does not exist, an empty string is returned

inline ErrorCode getErrorCode()

Returns

INIFile::ErrorCode depending on the last error

inline void setLastError(ErrorCode e)

Set last error.

std::string getLastErrorAsString()

Return last error as string.

operator bool() const

return true if file is non-empty.

inline eld::INIReader *getReader() const

Return the underlying INI Reader pointer for the file.

Public Static Functions

static eld::Expected<INIFile> Create(const std::string &filename)

Creates a INIFile object for filename. INIFile object can be used to easily perform suitable read and write operations to the INI file.

Returns

eld::Expected<INIFile> contains INIFile if initialisation of INIFile object was successful; Otherwise it contains a diagnostic object describing the error.

11.3.2.8. InputFile

struct InputFile

InputFile represents an input file. Input file can be an object file, a linker script, or anything else that the the linker can work with. An InputFile object can be used to inspect input file properties, and contents.

Subclassed by eld::plugin::BitcodeFile

Public Functions

std::string getFileName() const

Returns the file name of the input file if the object has an input file; Otherwise returns an empty string.

bool hasInputFile() const

Returns true if the object has an input file; Otherwise returns false.

bool isArchive() const

Returns true if the input is an archive member; Otherwise returns false.

bool isBitcode() const

Returns true if the input is LLVM bitcode; Otherwise returns false.

bool isLTOGeneratedObject() const

Returns true if the input file is an ELF object file generated by LTO; Otherwise returns false.

bool isObjectFile()

Returns true if the inputFile is an objectFile; Otherwise retruns false.

uint64_t getResolvedPathHash() const

Returns the hash of the resolved path.

uint64_t getArchiveMemberNameHash() const

Returns the hash of the Archive member.

std::string getMemberName() const

If the input file is an archive member, return the member name; Otherwise return an empty string.

std::vector<plugin::Symbol> getSymbols() const

Returns a sequence of all the symbols that are defined in this file.

std::vector<plugin::Section> getSections() const

Returns a sequence of all the sections in the input file.

std::optional<plugin::Section> getSection(uint64_t i) const

Get section at index i.

inline operator bool() const

Returns true if the object has an input file.

const char *getMemoryBuffer() const

Returns a pointer to the data of the input file.

size_t getSize() const

Returns the input file size.

bool isInternal() const

Returns true if the input file is an internal input file.

bool isArchiveMember() const

Returns ture if the input file is an archive member.

std::optional<plugin::InputFile> getArchiveFile() const

Returns the archive file that contains the input if the input is an archive member; Otherwise returns std::nullopt.

inline bool operator()(const InputFile &A, const InputFile &B) const

Returns true if A and B represents different input files.

inline bool operator<(const InputFile &RHS) const

Pointer comparison between the underlying input file handles of LHS and RHS.

inline bool operator>(const InputFile &RHS) const

Pointer comparison between the underlying input file handles of LHS and RHS.

inline bool operator==(const InputFile &RHS) const

Returns true if both LHS and RHS represents the same input file.

inline eld::InputFile *getInputFile() const

Returns the underlying input file handler. This can be used as a unique identifier for the input file.

uint32_t getOrdinal() const

Return the input ordinal which is the order in which the linker visited this input file

std::string decoratedPath() const

TODO.

11.3.2.9. PluginData

struct PluginData

PluginData provide a way for storing data and communicating between plugins.

Public Functions

inline operator bool() const

Conversion Operator.

inline bool operator()(const PluginData &A, const PluginData &B) const

Operators

11.3.2.10. AutoTimer

struct AutoTimer

Plugin Profiling support.

Public Functions

void start()

Start Timer.

void end()

End Timer.

inline operator bool() const

Conversion Operator.

inline bool operator()(const AutoTimer &A, const AutoTimer &B) const

Operators

11.3.2.11. Timer

struct Timer

Public Functions

void start()

Start Timer.

void end()

End Timer.

inline operator bool() const

Conversion Operator.

inline bool operator()(const Timer &A, const Timer &B) const

Operators

11.3.2.12. RelocationHandler

struct RelocationHandler

RelocationHandler handles relocations. It can be to inspect and compare relocations.

Public Functions

uint32_t getRelocationType(std::string Name) const

Get Relocation Type for a relocation.

std::string getRelocationName(uint32_t Type) const

Get the Relocation name from its type.

inline operator bool() const

Conversion Operator.

inline bool operator()(const RelocationHandler &A, const RelocationHandler &B) const

Operators

11.3.3. LinkerPluginConfig

class LinkerPluginConfig

LinkerPluginConfig allows to inspect and modify relocations. Each LinkerPluginConfig object has a corresponding plugin::Plugin object. LinkerPluginConfig provides callback hooks that are called for each registered relocation type. Relocation types must be registered via LinkerWrapper::RegisterReloc.

Public Functions

virtual void Init() = 0

The Init callback hook is called before any relocation callback hook call. It is used for initialization purposes. Typically, plugins register relocation types in the init callback function.

virtual void RelocCallBack(plugin::Use U) = 0

RelocCallBack is the callback hook function that the linker calls for each registered relocation.

This function must be thread-safe as the linker may handle relocations parallelly and thus may make calls to this function parallelly as well.

Note
There can be at most one registered relocation handler for each relocation type per plugin.

inline plugin::Plugin *getPlugin() const

Return the corresponding plugin.

Protected Attributes

plugin::Plugin *Plugin

Pointer to the corresponding plugin object.

Warning
Please do not modify the plugin object directly.

11.3.4. Utility Data Structures

11.3.4.1. JSON Objects

struct SmallJSONObject

Represents a JSON Object consisting of key value pairs. This class is append only. Items will appear in the order in which they were inserted

Public Functions

SmallJSONObject(SmallJSONObject &&Other)

Move constructor.

SmallJSONObject(const uint32_t InitialSize = 0)

Create a JSONObject and reserve InitialSize bytes for its string representation

void finish()

Insert the closing brace ('}') and complete the string representation of this object. Every SmallJSONObject must be finish()'ed in order to produce valid JSON.

bool isFinished() const

Has this Object been finish()'ed?

bool empty() const

Is this object empty.

struct SmallJSONArray

An append-only class represeting a heterogenous array of JSON values. Items are stored in insertion order.

Public Functions

SmallJSONArray(SmallJSONArray &&Other)

Move constructor.

SmallJSONArray(const uint32_t InitialSize = 0)

Create a SmallJSONArray and reserve InitialSize bytes for its string representation

void push_back(SmallJSONValue &&V)

Add a value to the string representing this JSON Array.

void finish()

Insert the closing bracket (']') and complete the string representation of this Array. Every SmallJSONArray must be finish()'ed in order to produce valid JSON.

bool isFinished() const

Has this Array been finish()'ed?

bool empty() const

Is this object empty?

struct SmallJSONValue

A Class representing a value representible in JSON. Includes Objects, Arrays, boolean, floating point, integral, string, and null types

Public Functions

SmallJSONValue(SmallJSONObject &&Obj)

Create a SmallJSONValue from a SmallJSONObject.

SmallJSONValue(SmallJSONArray &&Arr)

Create a SmallJSONValue from SmallJSONArray.

SmallJSONValue(bool b)

Create a SmallJSONValue representing true or false.

SmallJSONValue(const std::string S)

Create a SmallJSONValue representing a JSON string.

SmallJSONValue(std::nullptr_t null)

Create a SmallJSONValue representing a JSON null.

template<typename T, typename = std::enable_if_t<std::is_integral<T>::value>, typename = std::enable_if_t<!std::is_same<T, bool>::value>>
inline SmallJSONValue(T I)

Create a SmallJSONValue representing an integer type. Must be non-narrowing convertible to int64_t

template<typename T, typename = std::enable_if_t<std::is_floating_point<T>::value>, double* = nullptr>
inline SmallJSONValue(T D)

Create a SmallJSONValue representing a floating point type. Must be non-narrowing convertible to double

const std::string &str()

Get this JSONValue as a string in JSON format The string will be as compact as possible without any formatting or indentation

const uint8_t *getData() const

Get an unsigned integer pointer to raw data of the JSON object.

eld::Expected<plugin::MemoryBuffer> getMemoryBuffer(const std::string &BufferName)

Get a memory buffer for containing data.

11.3.4.2. TarWriter

class TarWriter

A utility class for creating tar archives.

Public Functions

eld::Expected<void> addBufferToTar(plugin::MemoryBuffer &buffer) const

Adds MemoryBuffer contents to tar. After tar is updated, the buffer is destroyed. NOTE: This function is not thread safe.

11.3.4.3. ThreadPool

class ThreadPool

A ThreadPool for asynchronous parallel execution on a defined number of threads.

The pool keeps a vector of threads alive, waiting on a condition variable for some work to become available.

Public Functions

template<typename Function, typename ...Args>
inline std::shared_future<void> run(Function &&F, Args&&... ArgList)

Run a function F in a thread pool.

template<typename Function>
inline std::shared_future<void> run(Function &&F)

Run a function F in a thread pool.

void wait()

Blocking wait for all the threads to complete and the queue to be empty.

11.3.5. Plugin Diagnostic Framework

Plugin diagnostic framework provides a uniform and consistent solution for reporting diagnostics from plugins. It is recommended to avoid custom solutions for printing diagnostics from a plugin.

First, let's go over some of the key benefits of using the Plugin Diagnostic Framework. After that, we'll see how to effectively use the framework.

The framework allows for creating and reporting diagnositcs on the fly. Diagnostics can be reported with different severities such as *Note*, *Warning*, *Error* and more. Each severity has a unique color, and diagnostics are prefixed with "PluginName:DiagnosticSeverity".

The linker keeps track of diagnostics emitted from the diagnostic framework. This is required to maintain extensive logs for diagnostic purposes. The framework also adjusts diagnostic behavior based on linker command-line options that affect diagnostics. For example, if the `--fatal-warnings` option is enabled, then warnings will be converted to fatal errors. If the `-Werror` option is used, warnings are promoted to regular errors instead of fatals.

The framework is thread-safe. Thus, diagnostics can be reported safely from multiple threads simultaneously. This is crucial in situations where thread-safety is neessary. In such cases, C++ `std::cout` and `std::cerr` output streams cannot be used, as they are not thread-safe.

That's enough about the theories. Now, let's take a look at how to use the Plugin Diagnostic Framework.

The linker assigns a unique ID to each diagnostic template. A diagnostic template has a diagnostic severity and a diagnostic format string. Diagnostic template unique IDs are necessary for creating and reporting diagnostics. The code below demonstrates how to obtain a diagnostic ID for a diagnostic template.

```
// There are similar functions for other diagnostic severities.
// Linker is a LinkerWrapper object.
DiagnosticEntry::DiagIDType errorID = Linker->getErrorDiagID("Error diagnostic with two args: %0 and %1");
DiagnosticEntry::DiagIDType warnID = Linker->getWarningDiagID("Warning diagnostic with one arg: %0");
```

`%0`, `%1` and so on in diagnostic format string are replaced by diagnostic arguments. Diagnostic arguments must be provided when reporting a diagnostic.

The below code demonstrates how to report a diagnostic using a diagnostic ID.

```
// arguments can be of any type. 'LinkerWrapper::reportDiag' is a
// variadic template function.
Linker->reportDiag(errorID, arg1, arg2);
Linker->reportDiag(warnID, arg1);
```

11.3.5.1. DiagnosticEntry

`DiagnosticEntry` packs complete diagnostic information, and can be used to pass diagnostics from one location to another. Many plugin framework APIs use `DiagnosticEntry` to return errors to the caller, where they can be properly handled.

```
// diag represents a complete diagnostic.
// diagID encodes diagnostic severity and diagnostic format string.
DiagnosticEntry diag(diagID, {arg1, arg2, ...});
```

11.3.5.2. eld::Expected<ReturnType>

Many plugin framework APIs return values of type `eld::Expected<ReturnType>`. At any given time, `eld::Expected<ReturnType>` holds either an expected value of type `ReturnType` or an unexpected value of type `std::unique_ptr<eld::plugin::DiagnosticEntry>`. Returning the error to the caller allows plugin authors to decide how to best handle a particular error for their use case. A typical usage pattern for this is.

```
eld::Expected<eld::plugin::INIFile> readFile = Linker->readINIFile(configPath);
if (!readFile) {
  if (readFile.error()->diagID() == eld::plugin::Diagnostic::errr_file_does_not_exit) {
    // handle this particular error
    // ...
  } else {
    // Or, simply report the error and return.
    Linker->reportDiagEntry(std::move(readFile.error()));
    return;
  }
}

eld::plugin::INIFile file = std::move(readFile.value());
```

11.3.5.3. Overriding Diagnostic Severity

Diagnostic severity can be overriden by using `DiagnosticEntry` subclasses. `DiagnosticEntry` has subclasses for each diagnostic severity level. Let's explore how to use them to override diagnostic severity.

```
eld::Expected<eld::plugin::INIFile> readFile = Linker->readINIFile(configPath);
eld::plugin::INIFile file;
if (readFile)
  file = std::move(readFile.value());
else {
  // Let's report the error as a Note, and move on by using a default INI file.
  eld::plugin::NoteDiagnositcEntry noteDiag(readFile.error()->diagID(), readFile.error()->args());
  Linker->reportDiagEntry(noteDiag);
  file = DefaultINIFile();
}
```

11.3.5.4. Diagnostic Framework types API reference

class DiagnosticBuilder

A light-weight helper class to produce diagnostics.

This is created by LinkerWrapper::getDiagnosticBuilder, and it allows to add arguments to the diagnostic in a cout style as described below:

```
Linker->getDiagnosticBuilder(id) << arg1 << arg2
```

DiagnosticBuilder object should always be used as a temporary object. This ensures that the diagnostics are emitted when intended. This is because diagnostics are emitted when the DiagnosicBuilder object is destructed, and temporary objects are destructed when their parent expression is fully evaluated. On the other hand, if DiagnosticBuilder object is stored in a variable, then the diagnostic won't be emitted until the variable goes out of scope. For example:

```
    {
      auto builder = Linker->getDiagnosticBuilder(id);
      builder << arg1 << arg2;  // diagnostic would not be emitted at this
      // ...                    // point.
      // ...
//  } // diagnostic is emitted at this point.
```

Public Functions

DiagnosticBuilder(const DiagnosticBuilder&) = delete

DiagnosticBuilder should not be copied. If it is copied, then multiple DiagnosticBuilder object would represent the same diagnostic. This would lead to multiple DiagnosticBuilder object trying to emit the same diagnostic.

const DiagnosticBuilder &operator<<(const std::string &pStr) const

Add string argument to the diagnostic.

const DiagnosticBuilder &operator<<(const char *pStr) const

Add C-style string argument to the diagnostic.

const DiagnosticBuilder &operator<<(int pValue) const

Add integer argument to the diagnostic.

const DiagnosticBuilder &operator<<(unsigned int pValue) const

Add unsigned int argument to the diagnostic.

const DiagnosticBuilder &operator<<(long pValue) const

Add long int argument to the diagnostic.

const DiagnosticBuilder &operator<<(long long pValue) const

Add long long argument to the diagnostic.

const DiagnosticBuilder &operator<<(unsigned long pValue) const

Add unsigned long argument to the diagnostic.

const DiagnosticBuilder &operator<<(unsigned long long pValue) const

Add unsigned long long argument to the diagnostic.

const DiagnosticBuilder &operator<<(bool pValue) const

Add bool argument to the diagnostic.

class DiagnosticEntry

DiagnosticEntry represents a diagnostic. It allows to conveniently pass diagnostics from one function to another.

Each diagnostic ID has an associated severity. ‘Severity’ specified in DiagnosticEntry overrides the default severity associated with a diagnostic ID. If no ‘Severity’ is specified, then the default severity associated with a diagnostic ID is used.

Subclassed by eld::plugin::ErrorDiagnosticEntry, eld::plugin::FatalDiagnosticEntry, eld::plugin::NoteDiagnosticEntry, eld::plugin::VerboseDiagnosticEntry, eld::plugin::WarningDiagnosticEntry

Public Functions

inline explicit operator bool() const

Returns true if the object contains an error.

inline DiagIDType diagID() const

Returns diagnostic ID.

inline std::vector<std::string> &args()

Returns diagnostic arguments.

bool isWarning() const

Returns true if the diagnostic entry represents a warning.

bool isVerbose() const

Returns true if the diagnostic entry represents a verbose message.

bool isNote() const

Returns true if the diagnostic entry represents a note.

bool isError() const

Returns true if the diagnostic entry represents an error.

bool isFatal() const

Returns true if the diagnostic entry represents a fatal error or fatal warning

class FatalDiagnosticEntry : public eld::plugin::DiagnosticEntry

FatalDiagnosticEntry subclass allows to easily create fatal diagnostic entry.

class ErrorDiagnosticEntry : public eld::plugin::DiagnosticEntry

ErrorDiagnosticEntry subclass allows to easily create error diagnostic entry.

class WarningDiagnosticEntry : public eld::plugin::DiagnosticEntry

WarningDiagnosticEntry subclass allows to easily create warning diagnostic entry.

class NoteDiagnosticEntry : public eld::plugin::DiagnosticEntry

NoteDiagnosticEntry subclass allows to easily create note diagnostic entry.

class VerboseDiagnosticEntry : public eld::plugin::DiagnosticEntry

VerboseDiagnosticEntry subclass allows to easily create verbose diagnostic entry.

12. Linker Optimization Features

- Plugins : More Information about linker plugins can be found at Linker Plugins
- Garbage Collection

  - Unused function and data sections are garbage collected by linker when we provide **–gc-sections** option at the link time
  - Garbage collected sections can be printed by providing **–print-gc-sections** option
  - –gc-sections decides which input sections are used by examining symbols and relocations.
  - The section containing the entry symbol and all sections containing symbols undefined on the command-line will be kept, as will sections containing symbols referenced by dynamic objects.
  - Note that when building shared libraries, the linker must assume that any visible symbol is referenced.
  - If the linker performs a partial link (-r linker option), then you will need to provide the entry point using the -e / –entry linker option.

13. ELF TOOLS

This sections documents llvm ELF command-line tools along with detailed examples of their usage

llvm-readelf <options> <file>

Inspect ELF files

- **-h** Display ELF header information:

```
$ llvm-readelf -h a.out
ELF Header:
  Magic:   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF32
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              EXEC (Executable file)
  Machine:                           QUALCOMM DSP6 Processor
  Version:                           0x1
  Entry point address:               0x0
  Start of program headers:          52 (bytes into file)
  Start of section headers:          228768 (bytes into file)
```

```
  Flags:                             0x68
  Size of this header:               52 (bytes)
  Size of program headers:           32 (bytes)
  Number of program headers:         4
  Size of section headers:           40 (bytes)
  Number of section headers:         17
  Section header string table index: 14
```

- **-S** List ELF Sections

```
$ llvm-readelf -S a.out
There are 17 section headers, starting at offset 0x37da0:

Section Headers:
  [Nr] Name              Type            Addr     Off    Size   ES Flg Lk Inf Al
  [ 0]                   NULL            00000000 000000 000000 00      0   0  0
  [ 1] .start            PROGBITS        00000000 001000 0046c8 00 WAX  0   0 64
  [ 2] .init             PROGBITS        00005000 006000 000064 00  AX  0   0 32
  [ 3] .text             PROGBITS        00006000 007000 018bcc 00  AX  0   0 4096
  [ 4] .fini             PROGBITS        0001ebe0 01fbe0 000030 00  AX  0   0 32
  ...
  [11] .sdata            PROGBITS        00027000 028000 000170 00 WAp  0   0 4096
  [12] .bss              NOBITS          00027170 028170 0017d8 00  WA  0   0  8
  [13] .comment          PROGBITS        00000000 028170 000083 00  MS  0   0  1
  [14] .shstrtab         STRTAB          00000000 0281f3 000081 00      0   0  1
  [15] .symtab           SYMTAB          00000000 028274 006d50 10     16 530  4
  [16] .strtab           STRTAB          00000000 02efc4 008dbd 00      0   0  1
```

- **-l** List ELF segments:

```
$ llvm-readelf -l a.out

Elf file type is EXEC (Executable file)
Entry point 0x0
There are 4 program headers, starting at offset 52

Program Headers:
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  LOAD           0x001000 0x00000000 0x00000000 0x046c8 0x046c8 RWE 0x1000
  LOAD           0x006000 0x00005000 0x00005000 0x1f658 0x1f658 R E 0x1000
  LOAD           0x026000 0x00025000 0x00025000 0x02170 0x03948 RW  0x1000
  GNU_RELRO      0x026000 0x00025000 0x00025000 0x00044 0x00044 RW  0x4

Section to Segment mapping:
  Segment Sections...
  00     .start
  01     .init .text .fini .rodata .eh_frame .gcc_except_table
  02     .ctors .dtors .data .sdata .bss
  03     .ctors .dtors
```

- **-s** Display the symbol table

```
$ llvm-readelf -s hello.o

  Symbol table '.symtab' contains 324 entries:
    Num:    Value  Size Type    Bind   Vis      Ndx Name
      0: 00000000     0 NOTYPE  LOCAL  DEFAULT  UND
      1: 00000000     0 FILE    LOCAL  DEFAULT  ABS hello.cc
      2: 00000090     0 NOTYPE  LOCAL  DEFAULT    7 GCC_except_table10
      3: 00000230     0 NOTYPE  LOCAL  DEFAULT    7 GCC_except_table100
      4: 00000240     0 NOTYPE  LOCAL  DEFAULT    7 GCC_except_table121
      5: 00000268     0 NOTYPE  LOCAL  DEFAULT    7 GCC_except_table128
      ...
```

- **-r** = **–relocations** Display relocation entries

```
$ cat 1.c
int foo() {
  printf("hello world\n");
  return 0;
}
$ clang -c 1.c
$ llvm-readelf -r 1.o
Relocation section '.rela.text' at offset 0x110 contains 3 entries:
Offset     Info    Type                Sym. Value  Symbol's Name + Addend
00000004  00000211 R_HEX_32_6_X           00000000   .rodata.str1.1 + 0
```

```
00000008  00000217 R_HEX_16_X             00000000   .rodata.str1.1 + 0
0000000c  00000401 R_HEX_B22_PCREL        00000000   printf
```

- **-string-dump** <section> Treat the contents of a section as a string and print it

```
$cat 1.c
int foo() {
  printf("hello world\n");
  return 0;
}
$clang 1.c -c
$llvm-readelf -S 1.o
There are 9 section headers, starting at offset 0x194:

Section Headers:
  [Nr] Name              Type            Address  Off    Size   ES Flg Lk Inf Al
  [ 0]                   NULL            00000000 000000 000000 00      0   0  0
  [ 1] .strtab           STRTAB          00000000 000131 000061 00      0   0  1
  [ 2] .text             PROGBITS        00000000 000040 000018 00  AX  0   0 16
  [ 3] .rela.text        RELA            00000000 00010c 000024 0c      8   2  4
  [ 4] .rodata.str1.1    PROGBITS        00000000 000058 00000d 01 AMS  0   0  1
...
$llvm-readelf --string-dump .rodata.str1.1 1.o
String dump of section '.rodata.str1.1':
[     0] hello world.
```

- **-hex-dump** <section> Print the contents of a section as hex

```
$cat 1.c
void foo() {printf("hello world\n")}
$clang 1.c -c
$llvm-readelf -S 1.o
There are 9 section headers, starting at offset 0x194:

Section Headers:
  [Nr] Name              Type            Address  Off    Size   ES Flg Lk Inf Al
  [ 0]                   NULL            00000000 000000 000000 00      0   0  0
  [ 1] .strtab           STRTAB          00000000 000131 000061 00      0   0  1
  [ 2] .text             PROGBITS        00000000 000040 000018 00  AX  0   0 16
  [ 3] .rela.text        RELA            00000000 00010c 000024 0c      8   2  4
  [ 4] .rodata.str1.1    PROGBITS        00000000 000058 00000d 01 AMS  0   0  1
...
$llvm-readelf --hex-dump .rodata.str1.1 1.o
Hex dump of section '.rodata.str1.1':
0x00000000 68656c6c 6f20776f 726c640a 00 hello world..
```

llvm-objdump <options> <file>

Disassembler

- **-d**

Show assembler contents of executable sections:

```
$ llvm-objdump -d a.out
a.out:  file format ELF32-hexagon
Disassembly of section .start:
0000000000000000 start:
0:       4c c0 00 58 5800c04c {  jump 0x98 }
4:       3e c0 00 58 5800c03e {  jump 0x80 }
8:       42 c0 00 58 5800c042 {  jump 0x8c }
...
```

- **-r**

Display relocation entries involved during linking:

```
$ llvm-objdump -d -r hello.o
hello.o:        file format ELF32-hexagon
Disassembly of section .text:
0000000000000000 __cxx_global_var_init:
      0:       01 c0 9d a0 a09dc001 {  allocframe(#8) }
      4:       00 40 00 00 00004000 {  immext(#0)
                        00000004:  R_HEX_32_6_X .bss
      8:       02 c0 00 78 7800c002    r2 = ##0 }
                        00000008:  R_HEX_16_X   .bss
      c:       00 c0 62 70 7062c000 {  r0 = r2 }
      10:       ff e2 9e a7 a79ee2ff {  memw(r30+#-4) = r2 }
      14:       00 c0 00 5a 5a00c000 {  call 0x14 }
                      00000014:  R_HEX_B22_PCREL      _ZNSt8ios_base4InitC1Ev
      ...
```

llvm-nm <options> <file>

Lists symbols in a file

- **no args** list regular symbols

```
$cat 1.c
int bar() { return 10; }
int foo() { return 1 + bar(); }
$clang -c 1.c
$llvm-nm 1.o
00000000 T bar
0000000c T foo
```

- **-D** = **–dynamic** List dynamic symbols

```
$cat 1.c
int bar() { return 10; }
int foo() { return 1 + bar(); }
$clang 1.c -o lib.so -fPIC -shared
$llvm-nm -D lib.so
000003a0 T _fini
00000240 T _init
00000370 T bar
0000037c T foo
```

- **-C** List symbols with demangling

llvm-objcopy <options> <input file> <output file>

Copy and translate binary files

- **no args** <input file> <output file> Create identical copy of one file to another, perform no translation if no output file is specified, input file will be modified in place

```
$llvm-objcopy 1.o 1.o.copy
```

- **–input-target** <bfdname>, **–output-target** <bfdname> translate from input target to output target

```
$file 1.o
1.o: ELF 32-bit LSB relocatable, Intel 80386, version 1 (SYSV), not stripped
$llvm-objcopy --input-target elf32-i386 --output-target elf32-x86-64 1.o 1.o.copy
$file 1.o.copy
1.o.copy: ELF 32-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
```

- **–add-section** <section=file> insert a section into the output file with name *section* and contents of *file* for ELF files, inserted section will be of type *SHT_NOTE* if section name begins with ".note" otherwise will have type *SHT_PROGBITS*

```
$cat insert
hello world
$llvm-objcopy 1.o 1.o.copy --add-section newSection=insert
$llvm-readelf --string-dump newSection
```

```
String dump of section 'newSection':
[     0] hello world
```

llvm-ar <options>

Combine several input files into an archive library that can be linked with a program

- **-cr** <output file> <input files> Create an archive file containing given input files

```
$ llvm-ar -cr lib.a 1.o 2.o 3.o
$ file lib.a
lib.a: current ar archive
```

- **-t** <archive> list archive objects in an archive file

```
$ llvm-ar -cr lib.a 1.o 2.o 3.o
$ llvm-ar -t lib.a
1.o
2.o
3.o
```

- **-x** <archive> <files> extract files from archive

```
$ ls
lib.a
$ llvm-ar -t lib.a
1.o
2.o
3.o
$ llvm-ar -x lib.a 1.o 2.o
$ ls
1.o 2.o lib.a
```

llvm-strings <options> <files>

Dump printable strings from binary files to stdout

no-args <files>

Dump printable strings from input file(s) to stdout. If no file is specified, stdin is used

```
$ cat 1.c
$ void foo() {printf("hello world\n");}
$ clang 1.c -c
hello world
clang version 8.0.0 (tags/RELEASE_800/final)
.rela.text
.comment
.note.GNU-stack
.llvm_addrsig
printf
.rela.eh_frame
.strtab
.symtab
.rodata.str1.1
```

-print-file-name <files>

Display the file containing the string before each string

```
$llvm-strings 1.o a.out
1.o: hello world
```

```
...
1.o: .strtab
1.o: .symtab
1.o: .rodata.str1.1
a.out: /lib64/ld-linux-x86-64.so.2
a.out: libc.so.6
a.out: printf
a.out: __libc_start_main
...
a.out: __frame_dummy_init_array_entry
```

14. LTO Support

14.1. Overview

ELD supports Link Time Optimization (LTO) using the LLVM LTO libraries. The linker can consume LLVM bitcode directly or use embedded bitcode inside ELF objects and drive Full LTO or ThinLTO code generation. This chapter describes the LTO model used by ELD and the linker switches that control it.

14.2. Background: LLVM LTO

LLVM LTO performs intermodular (whole program) optimization at link time. Compared to traditional per-object optimization, LTO enables cross-module inlining, dead code elimination, and whole-program analysis by merging or summarizing LLVM IR across inputs. The LLVM LTO design emphasizes transparency in the build system: compile with LTO enabled and link with an LTO-capable linker, and the linker handles optimization without special build steps.

ThinLTO is LLVM's scalable LTO mode. It uses a summary index to avoid a full merge of all IR, supports parallel code generation, and allows caching of imported summaries or produced objects to speed up incremental builds. ELD provides knobs for ThinLTO parallelism and caching, alongside Full LTO support when modules are compiled for full LTO.

14.3. ELD LTO model

14.3.1. Inputs

ELD triggers LTO when it sees bitcode inputs or when it is instructed to use embedded bitcode sections in ELF files:

- **Bitcode inputs**: `.bc` inputs are always treated as LTO inputs.
- **Embedded bitcode**:
  - ELD recognizes `.llvmbc` sections in ELF objects and can replace the native object with embedded bitcode when LTO input selection enables it.
  - ELD also supports `.llvm.lto` (`SHT_LLVM_LTO`) sections used by fat LTO objects when `--fat-lto-objects` is enabled.

14.3.2. Selecting LTO inputs

LTO is enabled in two ways:

- `-flto` enables LTO if any bitcode input or embedded bitcode is present.
- `--include-lto-filelist` can be used without `-flto` to select which ELF inputs with embedded bitcode should be upgraded to LTO inputs.
- `--fat-lto-objects` enables consuming embedded `.llvm.lto` sections from relocatable ELF inputs (alias: `--ffat-lto-objects`).

When `-flto` is used, embedded bitcode is used by default unless excluded using `--exclude-lto-filelist`. When `-flto` is not used, only file patterns listed in `--include-lto-filelist` are upgraded to LTO inputs.

By default, `.llvm.lto` sections are ignored. They are only consumed when `--fat-lto-objects` (or `--ffat-lto-objects`) is specified.

File lists accept one glob pattern per line (wildcards `*`, `?`, and `[]` are supported). Patterns are matched against the input path as seen by the linker.

14.3.3. LTO phases in ELD

ELD performs LTO in distinct phases:

1. **Pre-LTO**: Inputs are read, symbols are collected, and linker scripts are evaluated. Bitcode inputs are registered for LTO.
2. **LTO**: LLVM performs symbol resolution across bitcode modules and emits native object files (Full LTO) or partitioned outputs (ThinLTO).
3. **Post-LTO**: The generated objects are inserted back into the link and linked like normal ELF objects. Map files can include pre-LTO and post-LTO details.

When `--fat-lto-objects` (or `--ffat-lto-objects`) selects embedded `.llvm.lto` from a mixed object, the pre-LTO map input record for that file is annotated with `Fat LTO object selected`.

14.3.4. Linker scripts and LTO

ELD supports LTO with linker scripts. When a script uses `SECTIONS` ordering, ELD preserves input order for LTO-generated sections. If you need to disable this ordering for performance or to match non-script ordering, use `--flto-options=disable-linkorder`.

14.4. LTO switch reference

This section lists the LTO-related switches supported by ELD. Some switches also accept `--plugin-opt=...` aliases for compatibility with LLVMgold and lld.

14.4.1. Core LTO enablement and input selection

- `-flto` or `--flto` Enable LTO if a bitcode file or embedded bitcode section is present.
- `--fat-lto-objects` / `--no-fat-lto-objects` Enable/disable use of `.llvm.lto` embedded bitcode sections from fat LTO relocatable objects. Aliases: `--ffat-lto-objects`, `--fno-fat-lto-objects`.
- `--include-lto-filelist=<list>` Use this list to select which ELF inputs with embedded bitcode should be used for LTO when `-flto` is not present.
- `--exclude-lto-filelist=<list>` Use this list to exclude embedded bitcode inputs when `-flto` is present.

14.4.2. LTO output control and artifacts

- `--save-temps` Save intermediate LTO artifacts. Temporary files use the prefix `<output>.llvm-lto.*`.
- `--save-temps=<dir>` Save LTO temporary files under the specified directory.
- `--lto-emit-asm` Run LTO and emit assembly only. No final link is performed. Output files use the prefix `<output>.llvm-lto.<task>.s`. Alias: `--plugin-opt=emit-asm`.
- `--lto-emit-llvm` Run LTO and emit LLVM bitcode to the final output file (`-o`). No final link is performed. Alias: `--plugin-opt=emit-llvm`.
- `--lto-obj-path=<prefix>` Prefix for LTO-generated object files. When provided, output objects are not deleted after LTO. Alias: `--plugin-opt=obj-path=`.
- `--flto-options=lto-asm-file=<file[,file2,...]>` Use pre-generated LTO assembly files as inputs to the external assembler.
- `--flto-options=lto-output-file=<file[,file2,...]>` Use pre-generated LTO object files as outputs (bypasses LTO code generation).

14.4.3. LTO optimization control

- `--lto-O=<level>` Set the LTO optimization level (0-4). Alias: `--plugin-opt=O`.
- `--lto-partitions=<number>` Set the number of code generation partitions. Alias: `--plugin-opt=lto-partitions=`.
- `--thinlto-jobs=<number>` Set the number of ThinLTO backend jobs. Overrides `--threads=` for ThinLTO. Alias: `--plugin-opt=jobs=`.
- `--lto-sample-profile=<file>` Provide a sample PGO profile for LTO. Alias: `--plugin-opt=sample-profile=`.
- `--lto-cs-profile-generate` Enable context-sensitive PGO instrumentation during LTO. Alias: `--plugin-opt=cs-profile-generate`.
- `--lto-cs-profile-file=<file>` Provide a context-sensitive profile for LTO. Alias: `--plugin-opt=cs-profile-path=`.
- `--lto-debug-pass-manager` Enable debug output for the new pass manager. Alias: `--plugin-opt=debug-pass-manager`.
- `--disable-verify` Disable the LLVM IR verifier in the LTO pipeline. Alias: `--plugin-opt=disable-verify`.
- `--dwodir=<dir>` Directory for split DWARF `.dwo` files generated during LTO.

14.4.4. LLVM option passthrough

- `--plugin-opt=-<llvm-option>` Pass a raw LLVM option to LTO (LLVMgold compatibility). For example, `--plugin-opt=-debug-pass-manager`.

- `--flto-options=codegen=<llvm-arg>` Pass LLVM codegen options to LTO (supports `-O`, `-mcpu=`, `-mattr=`, and arbitrary LLVM options).
- `--flto-options=<string>` Passes additional LTO plugin options through. For example, tests use `--flto-options=no-merge-modules` and `--flto-options=codegen=-split-lto-cg`.

14.4.5. Assembler control

- `--flto-use-as` Use the external assembler instead of the integrated assembler for LTO.
- `--flto-options=asmopts=<arg>` Extra arguments passed to the external assembler during LTO.

14.4.6. LTO caching

- `--flto-options=cache` Enable ThinLTO caching. By default, cache output is written to `<output>.ltocache`.
- `--flto-options=cache=<dir>` Enable ThinLTO caching using the specified directory.

14.4.7. Symbol preservation and ordering

- `--flto-options=preserveall` Preserve all bitcode symbols during LTO.
- `--flto-options=preserve-sym=<sym1,sym2,...>` Preserve the specified symbols in LTO.
- `--flto-options=preserve-file=<file>` Preserve the symbols listed in the given file (one symbol per line).
- `--flto-options=disable-linkorder` Disable link order enforcement when linker scripts are present.
- `--flto-options=verbose` Enable verbose LTO diagnostics and trace output.

14.4.8. Diagnostics and tracing

- `--trace-lto` Trace LTO stages and emit additional diagnostics.
- `--opt-record-file` Emit LTO optimization remarks. ELD writes `-LTO.opt.yaml` alongside the output file.
- `--display-hotness` Display hotness information with optimization remarks.
- `--print-timing-stats`, `--emit-timing-stats` Emit or print timing stats, including LTO phases.

14.5. Examples

Full LTO with embedded bitcode:

```
clang -flto -c foo.c -o foo.o
clang -flto -c bar.c -o bar.o
ld.eld -flto foo.o bar.o -o app.elf
```

ThinLTO with parallel backends and cache:

```
clang -flto=thin -c foo.c -o foo.o
clang -flto=thin -c bar.c -o bar.o
ld.eld -flto --thinlto-jobs=8 --flto-options=cache=thinlto.cache \
  foo.o bar.o -o app.elf
```

Selective LTO on a subset of objects:

```
# lto_list.txt contains glob patterns like:
#   libfoo/*.o
#   *vector_math*.o
ld.eld --include-lto-filelist=lto_list.txt foo.o bar.o -o app.elf
```

Emit LTO assembly for inspection:

```
ld.eld -flto --lto-emit-asm foo.o bar.o -o app.elf
# Output files: app.elf.llvm-lto.0.s, app.elf.llvm-lto.1.s, ...
```

Use `.llvm.lto` from fat LTO objects:

```
# foo.fat.o carries native code plus a .llvm.lto section.
ld.eld --fat-lto-objects foo.fat.o bar.bc -o app.elf
```

14.6. References

- LLVM Link Time Optimization: Design and Implementation (`https://llvm.org/docs/LinkTimeOptimization.html`)

15. Getting Image Details

15.1. Overview

- The linker provides options for getting information about the files it generates.
- You can use following options to get information about how your file is generated by the linker, and also extract useful diagnostic information during the linking process.

15.2. Supported Diagnostic Options

- **–Map** : Displays the image memory map, and contains the address and the size of each load region, execution region, and input section in the image, including linker-generated input sections.
- **-MapStyle=[Text|YAML|Binary]** : Display the Map information in Text/YAML/Binary format.
- **–MapDetail <value>** : Detail information in the map file
- **–verbose** : Displays detailed information about the link operation, including the objects that are included and the libraries that contain them.
- **–trace** : Allows tracing of various information in linker.
  - **all-symbols** - tracing of all symbols
  - **reloc** - tracing of relocations
  - **symbol=<symbol name>** - Allows tracing of a particular symbol. It also supports regular expressions
  - **section=<section name>** - Allows tracing of a particular section. Supports regular expressions.
  - **GarbageCollection** - tracing of garrbage collected symbols
  - **Trampolines** - tracing of trampolines.
  - **LTO** - Allows tracing of LTO.
- **–color <value>** : Enable color output for diagnostics
- **-display_hotness <value>** : Display hotness information for optimization remarks
- **–emit-timing-stats <value>** : Emit time statistics of various linker operations to the specified file
- **–print-timing-stats** : Print time statistics of various linker operations to console
- **–time-region <value>** : - Emit time statistics for specified region of linker operation.
  - Users can provide <value> as “plugin” to display timing stats of all user plugins
  - Otherwise, we can provide particular plugin’s name as <value> of –time-region option to display stats of a particular plugin.
- **–version** : Print the Linker version and also lists the targets supported by the linker.

16. Command-line options

Here’s the list of command-line options accepted by the ELD:

16.1. Common options for all the backends

16.1.1. Compatibility Or Ignored Options

-EL

Link little endian objects.

-Qy

This option is ignored for SVR4 compatibility

–add-needed

Deprecated

–copy-dt-needed-entries

Add the dynamic libraries mentioned to DT_NEEDED

–ignore-unknown-opts

Disable warnings of unimplemented options

–nmagic

Turn off page alignment of sections, and disable linking against shared libraries

–no-add-needed

Deprecated

–no-allow-shlib-undefined

Do not allow undefined symbols from dynamic library when creating executables

–no-copy-dt-needed-entries

Turn off the effect of the –copy-dt-needed-entries

–plugin <pluginfile>

Specify a plugin file

–plugin-opt <plugin-opt>

Specify plugin options

-thin-archive-rule-matching-compatibility, –thin-archive-rule-matching-compatibility

Provides rule-matching compatibility when fat archives are converted to thin archives

16.1.2. DIAGNOSITC OPTIONS

-W<warn-type>

Print warnings which are OFF by default

- -Wall : print all warnings
- -W[no]linker-script : display warnings detected while parsing linker scripts
- -W[no]attribute-mix : display a warning if object files of RISC-V with different attributes are being linked
- -W[no-]archive-file : display warnings detected while reading archive files
- -W[no-]linker-script-memory : display warnings detected while processing linker script memory command
- -W[no-]bad-dot-assignments : display warnings for bad dot assignments
- -W[no-]error : treat warnings as errors
- -W[no-]whole-archive : display a warning when whole-archive is enabled for any archive file
- -W[no-]osabi : display a warning when linking objects with different values for OS/ABI
- -W[no-]version-script : display warnings detected while processing version scripts

–color <never/always/auto>

Enable color output for diagnostics

–cref

Print the references for a symbol or section

-t

Print all files processed by the linker

–display-hotness <value>

Display hotness information for optimization remarks

–emit-timing-stats <filename>

Emit time statistics of various linker operations to the specified file

–emit-timing-stats-in-output

Insert link-time stats in section .note.qc.timing

–error-limit <max-error-number>

Maximum number of errors to emit before stopping (0 = no limit)

–error-style <style>

Specify an error style, LLVM/GNU

–fatal-internal-errors

Enable fatal internal errors

–fatal-warnings

Enable fatal warnings

–gc-cref <symbol/section>

Print the references for a symbol or section when garbage collection is enabled

–no-fatal-internal-errors

Disable fatal internal errors

–no-fatal-warnings

Disable fatal warnings

–no-warn-mismatch

Do not Warn on incompatible files passed to the linker.

–opt-record-file

Create diagnostic yaml file

–print-timing-stats

Print time statistics of various linker operations to console

–progress-bar

Show Progress Bar

–summary

Display linker run summary at the end

–time-region <region>

Emit time statistics for specified region of linker operation

–trace <trace-type>

Allow tracing of

- –trace=linker-script : trace linker script
- –trace=files : trace input files
- –trace=reloc=<pattern> : trace relocations
- –trace=symbol=<symbol-name> : trace symbol
- –trace=all-symbols : trace all symbols
- –trace=LTO : trace LTO
- –trace=garbage-collection : trace linker garbage collection
- –trace=plugin : trace plugin
- –trace=plugin=<plugin-name> : trace a single plugin
- –trace=threads : trace threads
- –trace=assignments : trace symbol assignments
- –trace=command-line : trace header info
- –trace=live-edges : trace reachable sections when garbage collection is enabled
- –trace=merge-strings : trace linker string optimization
- –trace=trampolines : trace trampolines
- –trace=untar : trace tar extraction entries
- –trace=wrap-symbols : trace symbol wrap options
- –trace=symdef : trace symbol resolution from symdef files
- –trace=dynamic-linking : trace dynamic linking
- –trace=symbol-versioning : trace symbol versioning

–trace-linker-script

Trace stages of linker script processing

–trace-lto

Trace stages of lto

–trace-merge-strings <option>

Emit diagnostics describing how strings were merged. Options: all(default), allocatable_sections, <output section (regex)>

–trace-reloc <relocation>

trace a particular relocation

–trace-section <section_name>

Show metadata about a particular section

-y <symbol>

Trace a particular symbol

–verbose

Enable verbose output

–verbose=<verbose-level>

Enable verbose output

–verify-options <option>

Verify certain internal linker computations - currently supports reloc

–warn-common

Warn on common symbols

–warn-limit <maxwarnings>

Maximum number of warnings to emit (0 = no limit)

–warn-mismatch

Warn on incompatible files passed to the linker.

16.1.3. DYNAMIC LIBRARY OPTIONS

–Bgroup

Enable runtime linker to handle lookups in this object and its dependencies to be performed only inside this group

–Bsymbolic

Bind references to global symbols to the definition within the shared library

–Bsymbolic-functions

Bind references to global functions to the definition within the shared library

-g

Enable debug output when building shared libraries or executables

-fPIC

Enable PIC mode

–hash-size <size>

Specify a hash size when creating the hash sections for the dynamic loader

–hash-style <hashstyle>

Specify a hash style when creating the hash sections for the dynamic loader

–no-warn-shared-textrel

Don’t Warn if the linker adds a DT_TEXTREL

–soname <name>

Set the internal DT_SONAME field to the specified name

–warn-shared-textrel

Warn if the linker adds a DT_TEXTREL

16.1.4. DYNAMIC LIBRARY/EXECUTABLE OPTIONS

–default-symver

Create a default symbol version node, that is used for unversioned exported symbols

–dynamic-linker <path>

Set the path to the dynamic linker

–export-dynamic

Add all symbols to the dynamic symbol table when creating executables

–export-dynamic-symbol <symbol>

Export specified symbol to dynamic symbol table

–force-dynamic

Build executable as a force dynamic executable

–no-dynamic-linker

The program does not need a dynamic linker when creating static PIE executables

–no-export-dynamic

–rpath <path>

Add a path to the runtime library search path

–rpath-link <path>

Specifies the path to search

16.1.5. Extended Options

–about

Emit detailed information about the linker

–align-segments

Align segments to page boundaries

–allow-bss-conversion

Don’t produce an error when mixing NOBITS, and PROGBITS sections in the same segment

–copy-farcalls-from-file <filename>

Copy far calls instead of using trampolines

-z <extended-opts>

Extended Options or Non standard options.

Available options are:

- -z=combreloc : Combines multiple reloc sections and sorts them to make dynamic symbol lookup caching
- -z=now : Enables immediate binding
- -z=nocopyreloc : Disables Copy Relocation
- -z=text : Do not permit relocations in read-only segments (default)
- -z=notext : Allow relocations in read-only segments
- -z=separate-code : Create separate code segment
- -z=no-separate-code : Do not create separate code segment
- -z=separate-loadable-segments : Create separate loadable segments

–disable-linker-version

Disable the LINKER_VERSION linker script directive (default)

–disable-new-dtags

Disable new dynamic tags

–dump-mapping-file <outputfilename>

Dump mapping file to output file

–dump-response-file <outputfilename>

Dump response file to output file

–emit-relocs-llvm

Emit relocations sections

–enable-linker-version

Enable the LINKER_VERSION linker script directive

–enable-new-dtags

Enable new dynamic tags

–mapping-file <INI-file>

Reproduce link using a mapping file

–no-align-segments

Don’t align segments to page boundaries

–no-emit-relocs

Don’t emit relocations in the output file, applicable to –emit-relocs-llvm option too

–no-record-command-line

Do not record the linker command line in the comment section

–no-reuse-trampolines-file <filename>

Don’t reuse trampolines for symbols specified in file

–no-verify

Don’t verify link output

–record-command-line

Record the linker command line in the comment section

–reproduce <tarfilename>|default

Write a tar file containing input files and command line options to inspect and re-link

–reproduce-compressed <tarfilename>

Write compressed tar file in zlib format containing input files and command line options to inspect and re-link.

–reproduce-on-fail <tarfilename>|default

Write reproduce tar file when link fails containing input files and command line options to inspect and re-link.

–rosegment

Put read-only non-executable sections in their own segment

–script-options <matchtype>

Specify a match type, match-gnu/match-llvm

–use-old-rule-matching

Use the original (pre-optimization) linker script rule-matching algorithm

–use-old-style-trampoline-name

Use old style of naming trampolines

16.1.6. GENERAL OPTIONS

–build-id

Request creation of ".note.gnu.build-id" ELF note section

–build-id=fast/md5/sha1/tree/uuid/0x<hexstring>/none

Request creation of ".note.gnu.build-id" ELF note section

–no-warn-rwx-segments

Silence warnings for segments with RWX permissions

-o <outputfile>

Path to file to write output

–repository-version

Print the Linker Repository version

–sysroot <sysroot-path>

Set the system root

INPUT(file, file, …)

GROUP(file, file, …)

The files specified using the INPUT and GROUP command are searched inside the sysroot when:

- The file begins with the `/` character, and
- The script containing the INPUT/GROUP command is located within the sysroot directory.

Additionally, paths in INPUT/GROUP commands or on the command line can explicitly force sysroot expansion by using the `=` or `$SYSROOT` prefix:

- `=/path/to/file` expands to `<sysroot>/path/to/file`
- `$SYSROOT/path/to/file` expands to `<sysroot>/path/to/file`

This works for both linker script commands and command-line input files. If no sysroot is configured, the prefix is stripped and the remaining path is used.

`-L <search_directory>`

When `<search_directory>` is prefixed with `=`, then `=` is expanded to the sysroot.

-v

Print the Linker version and continue processing

–version

Print the Linker version

–warn-rwx-segments

Warn if the linker creates a segment with RWX permissions (default)

16.1.7. Help!

–help

Print option help

–help-hidden

Print hidden option help

16.1.8. Input Format

-b <input-format>

Specify input format for inputs following this option.

Supported values: binary,default

–remap-inputs <pattern=filename>

Rules are evaluated in command-line order; the first matching rule wins. The pattern supports wildcard matching.

Example:

–remap-inputs=foo.o=bar.o

–remap-inputs=*foo.o=build/alt/bar.o

Remapping is applied before path search/open, including linker-script `INPUT(...)`, `GROUP(...)`, and `INCLUDE` processing.

How to tell remapping is active:

–verbose

prints:

Remapping input file <old> to <new>

.

Text map output (

-MapStyle txt

) annotates remapped entries with

remapped from <original-name>

.

–remap-inputs-file <file>

Read remap rules from `<file>`.

Each non-empty line in the file is one rule in either format:

pattern=replacement

pattern replacement

Lines beginning with `#` and blank lines are ignored. Inline `#` comments are also supported.

Rules from this file are appended to the same ordered remap list used by `--remap-inputs`.

16.1.9. OUTPUT KIND

–Bdynamic

Link against dynamic library

–dynamic

Create dynamic executable (default)

–no-pie

Do not create a PIE executable

–pie

Create PIE executable

-r

Create relocatable object file

–shared

Create dynamic library

–static

Create static executable

16.1.10. Link Time Speedup

–enable-threads=<option>

make linker fully threaded, available options: all

–no-threads

Disable Threads at Link time

–thread-count <threadcount>

Specify the number of threads for all linker operations

–threads

Enable Threads at Link time

16.1.11. LLVM and Target Options

-m

Select target emulation

-mabi <mabi>

Target ABI

-march <march>

Target architecture

-mcpu <mcpu>

Target CPU

-mllvm <option>

Options to pass to LLVM

-mtriple <triple>

Target triple to link for

16.1.12. LTO Options

–disable-verify, -disable-verify

Do not run the verifier during the optimization pipeline

–dwodir <dir>

Directory to save .dwo files when split DWARF is used in LTO

–exclude-lto-filelist <list_of_files>

Specify a list of files that are to be disregarded while using embedded bitcode section for LTO. This has no effect if -flto switch is not used.

–fat-lto-objects

Use .llvm.lto section in fat LTO objects

–flto

Enable LTO if a Bitcode file is present.

–flto-options <option>

Specify various options with LTO

–flto-use-as

Use the standalone assembler instead of the integrated assembler for LTO

–include-lto-filelist <list_of_files>

Specify a list of files with embedded bitcode sections that are to be used for LTO. This has no effect if -flto switch is used

–lto-O<opt-level>

Optimization level for LTO

–lto-cs-profile-file=<value>

Context sensitive profile file path

–lto-cs-profile-generate

Perform context sensitive PGO instrumentation

–lto-debug-pass-manager

Debug new pass manager

–lto-emit-asm

Emit assembly code

–lto-emit-llvm

Emit LLVM-IR bitcode

–lto-obj-path=<prefix>

Specify file path used as a prefix for output LTO object files. Files created with this prefix will not be deleted after LTO finishes.

–lto-partitions=<number>

Number of LTO codegen partitions

–lto-sample-profile=<file>

Sample PGO profile file to be loaded for LTO

–no-fat-lto-objects

Ignore .llvm.lto section in fat LTO objects

–opt-remarks-filename <value>

YAML output file for optimization remarks

–opt-remarks-format <value>

The format used for serializing remarks (default: YAML)

–opt-remarks-hotness-threshold <value>

–opt-remarks-passes <value>

Regex for the passes that need to be serialized to the output file

–opt-remarks-with-hotness

Include hotness information in the optimization remarks file

–plugin-opt=-<value>, -plugin-opt=-<value>

Specify an LLVM option for compatibility with LLVMgold.so

–plugin-opt=stats-file=<value>, -plugin-opt=stats-file=<value>

Filename to write LTO statistics to

–save-temps

Save the temporary files produced by LTO

–save-temps=<dir>

Save the temporary files produced by LTO in the directory

–thinlto-jobs=<number>

Number of ThinLTO jobs. Default to –threads=

16.1.13. EXECUTABLE OPTIONS

-L

Path to search for libraries or linker scripts

-Y <path>

Add path to the default library search path

–emit-relocs

Make Emit relocs behave just like GNU.

-e <symbolname>

Name of entry point symbol

–fini <symbol>

Specify a finalizer function

–image-base <address>

–init <symbol>

Specify an initializer function

-l

Root name of library to use

–library-path <path>

Path to search for libraries or linker scripts

–library=<namespec>

library to use

–no-omagic

This option negates most of the effects of the -N option. Disable linking with shared libraries

–no-whole-archive

Restores the default behavior of loading archive members

–noinhibit-exec

Retain the executable output file whenever it is still usable

–nostdlib

Disable default search path for libraries

–omagic

Set the text and data sections to be readable and writable. Also, do not page-align the data segment, and disable linking against shared libraries.

–whole-archive

Force load of all members in a static library

16.1.14. Map Options

–Map <filename>

Dump the output layout to the map file

–MapDetail <option>

Detail information in the map file

–MapStyle <fileformat>

Dump the output layout to the map file in YAML/Text/Binary Form

-M

Emit the map file

–trampoline-map <filename>

Dump Trampoline Information in YAML format

–color-map

Color the map file

16.1.15. Misc Features

–archive-member-report <filename>

Emit JSON report for archive members pulled in by symbols

–allow-incompatible-section-mix

Allow incompatible section mix

16.1.16. OPTIMIZATION OPTIONS

–eh-frame-hdr

Create EH Frame Header section for faster exception handling

–gc-sections

Enable garbage collection

–no-gc-sections

Disable garbage collection

–no-merge-strings

Disable String Merging

–no-trampolines

Disable Trampolines

–print-gc-sections

Print sections that are garbage collected

–sframe-hdr

Process and emit .sframe sections for stack unwinding

16.1.17. Features From other Linkers

-R <filename>

Read symbol names and addresses from filename

–symdef

Output SymDef file to console

–symdef-file <filename>

Emit SymDef file

–symdef-style <style>

Determine how to handle symbol resolution for symbols from symdef file, Options available are provide

16.1.18. Plugin Options

–plugin-activity-file <filename>

Emit plugin activity JSON files

–no-default-plugins

Allow no plugins to be implicitly loaded

–plugin-config <config-file>

Specify a plugin configuration

16.1.19. SYMBOL RESOLUTION OPTIONS

–allow-multiple-definition

Allow multiple definitions

–allow-shlib-undefined

Allow undefined symbols from dynamic library when creating executables

–as-needed

This option affects ELF DT_NEEDED tags for dynamic libraries mentioned on the command line

–defsym symbol=<expression>

Create a global symbol in the output file containing the absolute address given by expression

–end-group

End a group

–end-lib

End a library

–end-lib-thin

End a thin library

–no-as-needed

This option restores the default behavior of adding DT_NEEDED entries

–no-undefined

Report unresolved symbol references from regular object files

–pop-state

Restore previously saved input handling flags

–push-state

Save current input handling flags (as-needed, whole-archive, etc)

–start-group

Start a group

–start-lib

Start a library

–start-lib-thin

Start a thin library

-u

Force symbol to be entered in the output file as an undefined symbol

–use-shlib-undefines

Resolve undefined symbols from dynamic libraries

–warn-once

Warn only once for every undefined reference

16.1.20. SCRIPT OPTIONS

-T <linkerscriptfile>

Use the given linker script in place of the default script.

–Tbss <address>

Specify an address for the .bss section

–Tdata <address>

Specify an address for the .data section

–Ttext <address>

Specify an address for the .text section

–Ttext-segment <address>

Specify an address for the .text-segment segment

–default-script <pluginfile>

Use the linker script as the default linker script.

–no-check-sections

Do not check section addresses for overlaps

–dynamic-list <list-of-symbols>

Specify a list of symbols if present will be exported

–check-sections

Check section addresses for overlaps(default)

–exclude-libs <libs>

Specifies a list of archive libraries from which symbols should not be automatically exported

–extern-list <list-of-symbols>

Specify a list of symbols that exists as external dependencies

–global-merge-non-alloc-strings

merge non-alloc strings across output sections

–map-section <section>

Specify a input section that maps to a output section

–orphan-handling <mode>

Specify how to handle orphan sections. Options available are place, error, warn

–print-memory-usage

Print memory usage when MEMORY linker script directive is used

–section-start <address>

Specify a virtual output section address for a specified section

–sort-section <sort_section>

–unique-output-sections

Place each input section in a unique output section

–unresolved-symbols <option>

Determine how to handle unresolved symbols, Options available are ignore-all, report-all(Default), ignore-in-object-files, ignore-in-shared-libs

–version-script <linkersciptfile>

Use the linker script as a version script.

16.1.21. SYMBOL OPTIONS

-d

Assign space to common symbols

–demangle

Demangle C++ symbols

–demangle-style <value>

Specify whether the linker should demangle symbols when emitting errors or emitting Map files

–discard-all

Discard all symbols

–discard-locals

Discard all local symbols

–no-demangle

Don’t demangle C++ symbols

–portable <symbol>

Specify symbol to be portable wrapped to

–sort-common

sort common symbols by alignment

–sort-common=<ascending|descending>

Sort common symbols by order of alignment

–strip-all

Omit all symbol information from output

–strip-debug

Omit all debug information from output

–wrap <symbol>

Specify symbol to be wrapped to

16.2. Input remapping options

`--remap-inputs` and `--remap-inputs-file` let you redirect input file names before ELD opens them.

16.2.1. `--remap-inputs`

Use `--remap-inputs=<pattern>=<replacement>` (or `--remap-inputs <pattern>=<replacement>`) to add one remap rule.

Rules are evaluated in command-line order and the first matching rule wins. `<pattern>` supports wildcard matching.

Example:

```
$ ld.eld main.o foo.o --remap-inputs=foo.o=bar.o -T script.t
```

In this example, any reference to `foo.o` is resolved as `bar.o`.

16.2.2. `--remap-inputs-file`

Use `--remap-inputs-file=<file>` (or `--remap-inputs-file <file>`) to load remap rules from a text file.

Each non-empty line in `<file>` defines one rule, either as:

- `pattern=replacement`
- `pattern replacement`

Lines starting with `#` (or text after `#` on a line) are treated as comments.

Example:

```
# remap.txt
*foo.o build/alt/bar.o
libold.a=libnew.a

$ ld.eld main.o foo.o libold.a --remap-inputs-file=remap.txt -T script.t
```

Notes:

- Remapping is applied before path search/open, including linker-script `INPUT(...)`, `GROUP(...)`, and `INCLUDE` processing.
- `--remap-inputs` and `--remap-inputs-file` rules share one ordered rule list; whichever matching rule appears first is applied.
- To confirm remapping happened:
  - Use `--verbose` to see diagnostics such as `Remapping input file <old> to <new>`.
  - In text map output (`-MapStyle txt`), input/load lines and linker-script include tracking lines are annotated with `# remapped from <original-name>` when remapping is applied.

16.3. Using `--start-lib` / `--end-lib`

`--start-lib` and `--end-lib` let you pass one or more object files and have ELD treat them as if they were members of an archive for symbol resolution purposes.

Conceptually, ELD packages the object files between `--start-lib` and `--end-lib` into an in-memory archive and then processes that archive like a normal `.a` input.

16.3.1. Thin variant

ELD also supports `--start-lib-thin` / `--end-lib-thin`, which packages the enclosed object files into an in-memory *thin* archive (member paths are stored instead of embedding member bytes).

16.3.2. Linker scripts with archive member patterns

ELD supports using archive member patterns in linker scripts to match inputs from a `--start-lib` region.

Internally, ELD assigns a synthetic archive name to each `--start-lib` region: `<start-lib:1>`, `<start-lib:2>`, etc., in command-line order. Thin regions use `<start-lib-thin:N>`. You can reference these synthetic archive names using the standard `archive:member` pattern syntax in an input section description.

Example:

```
$ ld.eld main.o --start-lib foo.o bar.o --end-lib -T script.t -MapStyle txt -Map out.map

/* script.t */
SECTIONS {
  .foo_out : { "*<start-lib:1>:*foo.o"(.text*) }
  .bar_out : { "*<start-lib:1>:*bar.o"(.text*) }
}
```

Notes:

- The `<start-lib:N>` names are ELD-specific (they do not correspond to a real on-disk archive).
- If you need a stable archive name for portability, create a real archive (e.g. with `ar cr libname.a ...`) and match against `*libname.a:*member.o` patterns in the script.

16.4. MEMORY-related options

When using linker scripts with `MEMORY` regions, these options are commonly useful for debugging and enforcing correctness:

- `--print-memory-usage` prints per-region usage when a `MEMORY` directive is present in the linker script.
- `-Wlinker-script-memory` enables warnings for suspicious `MEMORY` setups (for example, zero-sized regions or regions that end up unused).

16.5. PIE options

ELD accepts both GNU-style single-dash and double-dash forms for PIE toggles:

- `-pie` and `--pie`: create a position-independent executable.
- `-no-pie` and `--no-pie`: create a non-PIE executable.

16.6. ARM and AArch64 specific options

16.6.1. ARM/AArch64 Linker Options

–disable-bss-conversion

Don’t convert BSS to NonBSS when BSS/NonBSS Sections are mixed

–enable-bss-mixing

Enable mixing BSS/NonBSS sections

–use-mov-veneer

Use movt/movw to load address in veneers with absolute relocation

16.6.2. ARM Linker Option ONLY

-execute-only, –execute-only

Mark executable sections execute-only on AArch64

-fix-cortex-a53-843419, –fix-cortex-a53-843419

Fix the cortex a53 errata 843419

–fix-cortex-a8

Fix the Cortex A8 bug

-fropi, –fropi

Enable Read write data access relative to a static base registers.

-frwpi, –frwpi

Enable Read write data access relative to a static base registers.

–no-fix-cortex-a8

Dont Fix the Cortex A8 bug

-target2 <type>

Interpret R_ARM_TARGET2 as <type>, where <type> is one of abs, rel, or got-rel (default).

16.7. Hexagon specific options

16.7.1. Hexagon Options

–gpsize=<maxsize>

Set the maximum size of objects to be optimized using GP

16.8. RISCV specific options

16.8.1. RISCV Linker Options

–disable-bss-conversion

Don’t convert BSS to NonBSS when BSS/NonBSS Sections are mixed

–enable-bss-mixing

Enable mixing BSS/NonBSS sections

–keep-labels

Keep all local labels for debugging purposes

–no-relax-got

Disable relaxing GOT load instruction sequences

–no-relax-gp

Disable GP relaxation

–no-relax-tbljal

Disable conversion of call/jump instructions to table jump instructions (default behavior)

–no-relax-tlsdesc

Disable relaxing TLSDESC instruction sequences

–no-relax-zero

Disable zero-page relaxation

–no-relax

Disable relaxation

–no-relax-c

Disable relaxation to compressed instructions

–no-relax-xqci

Disable relaxing to/from Xqci instructions (default behavior)

–relax-tbljal

Enable conversion of call/jump instructions to Zcmt table jump instructions (cm.jt/cm.jalt)

–relax-tbljal=<zcmt|xqccmt>

Enable conversion of call/jump instructions to table jump instructions for zcmt or xqccmt (default: zcmt)

–relax

Enable relaxation (default behavior)

–relax-xqci

Enable relaxing to/from Xqci instructions

16.9. -z keyword options

The following `-z` keywords are supported by ELD:

`-z combreloc`

: Combine multiple dynamic relocation sections and sort them to improve dynamic symbol lookup caching.

`-z nocombreloc`

: Disable relocation section combining.

`-z global`

: When building a shared object, make its symbols available for resolution by subsequently loaded libraries.

`-z defs`

: Report unresolved symbol references from regular object files, even when creating a non-symbolic shared library.

`-z initfirst`

: When building a shared object, mark it to be initialized before other objects loaded at the same time, and finalized after them.

`-z muldefs`

: Allow multiple definitions.

`-z nocopyreloc`

: Disable linker-generated .dynbss variables used in place of shared-library variables. This may result in dynamic text relocations.

`-z nodefaultlib`

: Ignore default library search paths when resolving dependencies at runtime.

`-z relro`

: Create a PT_GNU_RELRO segment and request it be made read-only after relocation, if supported. This is disabled by `-z norelro`.

`-z norelro`

: Do not create a PT_GNU_RELRO segment.

`-z lazy`

: Mark the output for lazy binding (resolve function calls on first use).

`-z now`

: Mark the output for immediate binding (resolve all symbols at load time).

`-z origin`

: Require `$ORIGIN` handling in rpath/runpath entries.

`-z text`

: Report an error if DT_TEXTREL is set (dynamic relocations in read-only sections).

`-z notext`

: Allow dynamic relocations in read-only sections (do not report DT_TEXTREL).

`-z noexecstack`

: Mark the output as not requiring an executable stack.

`-z nognustack`

: Ignore `.note.GNU-stack` and do not create PT_GNU_STACK based on it.

`-z separate-code`

: Create a separate code PT_LOAD segment with instructions on pages disjoint from any other data. This is a no-op when a linker script with a `SECTIONS` command is used.

`-z separate-loadable-segments`

: Place every loadable segment on pages disjoint from all other segments.

`-z noseparate-code`

: Disable separate code segment handling. This is the default behavior.

`-z execstack`

: Mark the output as requiring an executable stack.

`-z nodelete`

: Mark a shared object as non-unloadable at runtime.

`-z force-bti`

: Force GNU property BTI feature bits to be recorded (AArch64).

`-z pac-plt`

: Force GNU property PAC feature bits to be recorded for PLT use (AArch64).

`-z common-page-size=<value>`

: Set the most common page size used for segment alignment and layout optimization.

`-z max-page-size=<value>`

: Set the maximum supported page size used for segment alignment.

16.10. Linker version directive

`--enable-linker-version`

: Enable the GNU linker-script `LINKER_VERSION` directive. When this option is active, every `LINKER_VERSION` statement encountered in a linker script emits a NUL-terminated string containing the eld version at that position in the output section. This matches the GNU ld directive and is useful for embedding the linker version directly into a binary. The option applies to the entire link once specified.

`--disable-linker-version`

: Disable the `LINKER_VERSION` directive. This is the default behaviour, so the directive is parsed but emits no data unless the feature has been explicitly enabled.

16.11. Record command line

`--record-command-line`

: Record the linker command line in the `.comment` section. This is disabled by default.

`--no-record-command-line`

: Do not record the linker command line in the `.comment` section.

17. Target Specific Features

17.1. Hexagon

- Plugins
  - Information can be found at Linker Plugins
- Small data
- Trampolines
- **–disable-guard-for-weak-undef**
  - Disable guard for weak undefined symbols
- **-gpsize=<value>**
  - Set the maximum size of objects to be optimized using GP

17.2. ARM

- Baremetal
- Thumb
- ARM
- Linux
- Android
- Supported errata fixes
  - `fix_cortex_a8`
  - `fix_cortex_a53_843419`
- **enable-bss-mixing**
  - Enable mixing of BSS and non-BSS sections
- **-execute-only**
  - Mark executable sections execute-only
- **–disable-bss-conversion**
  - Don’t convert BSS to NonBSS when BSS/NonBSS sections are mixed
- **–use-mov-veneer**
  - Use movt/movw to load address in veneers with absolute relocation

17.3. AArch64

- Baremetal
- AArch64
- Linux
- Android
- **enable-bss-mixing**
  - Enable mixing of BSS and non-BSS sections
- **–disable-bss-conversion**
  - Don’t convert BSS to NonBSS when BSS/NonBSS sections are mixed
- **–use-mov-veneer**
  - Use movt/movw to load address in veneers with absolute relocation
- **-z nognustack**
  - Do not create a `GNU_STACK` segment

17.4. RISCV

- Baremetal

- Linux
- **-gpsize=<value>**
  - Set the maximum size of objects to be optimized using GP
- Supports relaxation
  - `--relax` (enabled by default)
  - `-no-relax` to disable relaxation
- **enable-bss-mixing**
  - Enable mixing of BSS and non-BSS sections
- **-W[no]attribute-mix**
  - Warn about RISC-V attributes mix instead of failing to link
- **–disable-bss-conversion**
  - Don’t convert BSS to NonBSS when BSS/NonBSS sections are mixed

18. SFrame Support

18.1. Overview

SFrame (Simple Frame) is a lightweight debugging format that provides stack unwinding information for debuggers and profilers. ELD provides comprehensive support for processing `.sframe` sections and creating SFrame headers in linked executables and shared libraries.

The SFrame format is designed to be much simpler and more compact than DWARF debug information, focusing solely on the minimal information needed for stack unwinding:
- **Canonical Frame Address (CFA)**: The address of the call frame
- **Frame Pointer (FP)**: Location of the frame pointer
- **Return Address (RA)**: Location of the return address

ELD implements support for **SFrame version 2 (errata 1)** as specified in the SFrame Format documentation.

18.2. SFrame vs EhFrame

SFrame offers several advantages over the traditional EhFrame format:

Size Efficiency

: SFrame sections are significantly smaller than equivalent EhFrame sections, resulting in reduced binary size and memory usage.

Parsing Speed

: The simplified format allows for faster parsing during stack unwinding operations.

Reduced Complexity

: SFrame eliminates much of the complexity of DWARF-based unwinding while maintaining the essential functionality.

Better Cache Performance

: The compact format leads to better cache utilization during runtime stack unwinding.

18.3. Command Line Options

ELD provides the following command line option for SFrame support:

`--sframe-hdr`

: Create an SFrame header section and process `.sframe` sections in input files. This option enables:
- Processing of `.sframe` sections from input object files
- Creation of consolidated SFrame sections in the output
- Proper linking and address resolution for SFrame data
- Integration with ELD’s section layout and memory management

18.4. Usage Examples

18.4.1. Basic Linking with SFrame

To link object files containing `.sframe` sections and create an SFrame header:

```
# Create source file with SFrame section
cat > source1.s << 'EOF'
.text
.globl func1
func1:
    ret
.section .sframe,"a"              # Type automatically set to SHT_GNU_SFRAME (llvm-mc 22.x+)
.byte 0xe2, 0xde, 0x02, 0x08      # SFrame header with magic and version
EOF

# Assemble files containing SFrame sections (requires llvm-mc 22.x or newer)
llvm-mc -filetype=obj -triple=x86_64-linux-gnu source1.s -o source1.o
llvm-mc -filetype=obj -triple=x86_64-linux-gnu source2.s -o source2.o

# Link with SFrame header creation
eld --sframe-hdr source1.o source2.o -o executable
```

Note

For llvm-mc versions prior to 22.x, the section type must be specified explicitly:

```
.section .sframe,"a",@0x6ffffff4  # Explicit SHT_GNU_SFRAME type for older assemblers
```

18.4.2. Shared Library with SFrame

SFrame sections can also be included in shared libraries:

```
# Create shared library with SFrame information
eld --sframe-hdr -shared libfoo.o -o libfoo.so
```

18.4.3. Combining with Other Options

SFrame support works seamlessly with other ELD features:

```
# SFrame with garbage collection
eld --sframe-hdr --gc-sections input.o -o output

# SFrame with optimization
eld --sframe-hdr -O2 input.o -o output

# SFrame with debug symbols
eld --sframe-hdr --debug input.o -o output
```

18.5. Section Processing

18.5.1. Input Processing

ELD processes `.sframe` sections from input object files by:

1. **Recognition**: Identifying `.sframe` sections in ELF input files
2. **Parsing**: Validating SFrame headers and parsing FRE (Frame Row Entry) data
3. **Merging**: Combining multiple `.sframe` sections from different input files
4. **Address Resolution**: Resolving function addresses and updating SFrame data

18.5.2. Output Generation

When the `--sframe-hdr` option is specified, ELD creates:

- **Consolidated .sframe section**: Contains all SFrame data from input files
- **Proper section alignment**: Ensures correct alignment for runtime access
- **Address fixups**: Updates all address references to final linked addresses
- **Section headers**: Creates appropriate ELF section headers for SFrame data

18.6. Supported Architectures

ELD’s SFrame support is available for the architectures with ABI identifiers defined in the SFrame specification:

- **AArch64**: ARM 64-bit architecture (little-endian and big-endian)
- **AMD64 (x86_64)**: AMD/Intel 64-bit architecture (little-endian)

Other architectures are not yet assigned ABI identifiers in the SFrame specification and are not currently supported.

18.7. Integration with Debugging Tools

SFrame sections created by ELD are compatible with tools that support the SFrame format:

- **GDB**: GNU Debugger (version 16+) can use SFrame information for stack unwinding
- **libsframe**: The libsframe library (from GNU binutils) can parse SFrame sections

18.8. Garbage Collection Behavior

When using `--gc-sections` with SFrame support:

- **Preservation**: `.sframe` sections are not subject to garbage collection and are preserved in the output

18.9. Error Handling

ELD provides comprehensive error detection for SFrame processing:

Invalid Magic Number

: Reports errors for SFrame sections with incorrect magic numbers (expected: 0xdee2)

Unsupported Version

: Detects and reports unsupported SFrame versions (ELD supports version 2)

Truncated Sections

: Identifies and reports SFrame sections with incomplete data

Address Resolution Failures

: Reports errors when SFrame function addresses cannot be resolved

Example error output:

```
Error: SFrame Read Error : invalid SFrame magic number from file input.o
Error: SFrame Read Error : unsupported SFrame version from file input.o
Error: SFrame Read Error : section too small for SFrame header from file input.o
```

18.10. Performance Considerations

SFrame support in ELD is designed for optimal performance:

Link Time : - Minimal overhead during linking
- Efficient parsing of SFrame sections
- Fast address resolution and fixups

Runtime : - Compact sections reduce memory usage
- Fast parsing during stack unwinding
- Improved cache locality

Binary Size : - Significantly smaller than equivalent DWARF information
- Optional inclusion based on build requirements

18.11. Best Practices

1. **Consistent Usage**: Use `--sframe-hdr` consistently across all objects in a project
2. **Testing**: Verify SFrame functionality with debugging tools after linking
3. **Architecture Specific**: Ensure SFrame generation tools target the correct architecture
4. **Integration**: Combine with appropriate optimization levels for best results
5. **Validation**: Use readelf or similar tools to verify SFrame section contents

18.12. Troubleshooting

Missing SFrame Sections

: Ensure input objects were compiled with SFrame generation enabled

Incorrect Function Addresses

: Verify that all input objects use consistent compilation flags

Debugger Issues

: Check that debugging tools support SFrame format version 2

Performance Problems

: Consider the trade-off between SFrame inclusion and binary size requirements

For additional support and troubleshooting, consult the ELD diagnostic output when using the `--verbose` option along with `--sframe-hdr`.

19. Editor Support

19.1. Overview

ELD provides editor integration files that add syntax highlighting for ELD linker scripts and map files. The integration is currently available for **Vim** (version 9.1 or later).

19.2. Vim Syntax Highlighting

19.2.1. Supported File Types

The filetype-detection file recognises the following files automatically:

Pattern	Description
*.ld, *.lds, *.lcs, *.lcs.template, *.t, *.x	ELD linker scripts
*.map (first line matches # Linker)	ELD map files

To set the filetype manually inside an open Vim buffer:

```
:set filetype=eld
```

19.2.2. Installation

Copy the two Vim files into your Vim runtime directories:

```
cp etc/vim/eld.vim          ~/.vim/syntax/eld.vim
cp etc/vim/ftdetect_eld.vim ~/.vim/ftdetect/eld.vim
```

Vim resolves syntax files by filetype name (`filetype=eld` → `syntax/eld.vim`), so the filenames must be kept exactly as shown.

After copying the files, open any linker script or map file and Vim will apply syntax highlighting automatically.

20. Linker support and frequently asked questions!!!

20.1. General Linker Questions

20.1.1. How do I pass arguments to linker using clang/clang++ driver ?

clang/clang++ driver provides **“-Wl,<args>”** option to pass args to linker.

-Wl,<args>

Pass the comma separated arguments in args to the linker

E.g. clang <compiler_options> **-Wl,-Map=test.map,-e=main**

where -Map and -e are linker specific options/args.

These args will be passed to the underlying linker used for processing.

Alternatively, user can also use **“-Xlinker <arg>”** option to pass argument one at a time to linker.

-Xlinker <arg>

Pass arg to the linker.

E.g. clang <compiler_options> **-Xlinker -Map=test.map -Xlinker -e=main**

20.1.2. How to remove unused functions from the final ELF ?

This can be achieved in two ways:

- With LTO : - All source files should be compiled with compiler option "-flto".
  - During linking stage, enable LTO optimization by specifying "-flto" option. E.g. clang <compiler_options> **-flto -c** foo.c -o foo.o

    Note
    When -flto option is used during compilation, the output object file (say, foo.o) is not in ELF format.It will be in LLVM-IR bitcode binary format.

```
clang <compiler_options> -flto -c bar.c -o bar.o
ld.eld <other_linker_options> -flto foo.o bar.o -e <entry_point> -o final.out
```

- Without LTO : - All source files should be compiled with compiler option "-ffunction-sections".
  - During linking stage, enable garbage collection by specifying "–gc-sections" option. Example:

```
clang <compiler_options> -ffunction-sections -c foo.c -o foo.o
clang <compiler_options> -ffunction-sections -c bar.c -o bar.o
ld.eld <other_linker_options> --gc-sections -e <entry_point> foo.o bar.o -o final.out
```

20.1.3. How do I trace a symbol ?

The user can trace a symbol with –trace-symbol <symbolname> or –trace=symbol=<symbolname>

Example:

```
$ cat 1.c
int foo() {return 0;}
int bar() {return 1;}
$ cat 2.c
int baz() {return foo() + bar ();}
$ ld.eld 1.o 2.o --trace-symbol bar # equivalent to --trace=symbol=bar
Note: Symbol `bar' from Input file `1.o' with info `(ELF)(FUNCTION)(DEFINE)[Global]{DEFAULT}' being added
to Namepool
Note: Symbol `bar' from Input file `2.o' with info `(ELF)(NOTYPE)(UNDEFINED)[Global]{DEFAULT}' being added
to Namepool
Note: Symbol `bar' from Input file `1.o' with info `(ELF)(FUNCTION)(DEFINE)[Global]{DEFAULT}' being resolved
from Namepool
Symbol bar, application site: 0x2c
```

20.1.4. How do I trace a relocation ?

The user can trace a relocation with the option –trace=reloc="<relocname>"

20.1.5. Figure out garbage collected symbols

The user can find the list of sections that the linker garbage collected using the option *–print-gc-sections* option.

The user then needs to go over the sections in the input files, and list the symbols using objdump tool.

20.1.6. Linker is garbage collecting a symbol

The linker can garbage collect the symbol only if the symbol is not referenced. You can use the option –trace=live-edges and see if you are finding a reference to the section that contains the symbol.

If you don't see the section that contains the symbol, there might be a missing link.

You might want to use the option

- –entry=<entry symbol>
- Use the KEEP linker script directive to add to the list of sections that needs to be kept.
- -u *symbol_name* to keep a symbol from being garbage collected

20.1.7. Print Timing Information

Linker can print timing information with the option –print-timing-stats. This option can be used to see where the linker is spending most of its time.

20.1.8. Linker is taking a long time to link

The linker may be taking a long time to link due to the following patterns
- A lot of linker script rules : - Linker script rules with EXCLUDE_FILE
- A lot of object files
- Bad image layout

For figuring out any of the above, you should check the Map file and see the statistics when the link is successful.

20.1.9. How to figure out why a archive member is being loaded by the linker

If you are trying to figure out why an archive member is being loaded you need to look at the archive records.

Example:

```
cat > 1.c << \!
int foo() { return bar(); }
!

cat > 2.c << \!
int bar() { return baz(); }
!

cat > 3.c << \!
int baz() { return bar(); }
!


clang -target hexagon -c 1.c -ffunction-sections
clang -target hexagon -c 2.c 3.c -ffunction-sections
hexagon-ar cr mylib.a 2.o 3.o
hexagon-link 1.o mylib.a -Map x
```

If you see the map file section for archive reference records, you will find this information :-

```
Archive member included because of file (symbol)
mylib.a(2.o)
                1.o (bar)
mylib.a(3.o)
                mylib.a(2.o) (baz)
```

You can interpret the information as below :-
- There was a reference in 1.o for bar, hence mylib.a(2.o) was loaded.
- There was a reference in 2.o for baz, hence mylib.a(3.o) was loaded.

20.1.10. Controlling Page size

You can use the option z max-page-size=<x> to set the page size used by the linker.

20.1.11. How to fail a build for linker warnings

A linker warning may be considered fatal when the switch –fatal-warnings is used.

You can turn off this behavior –no-fatal-warnings or removing the switch –fatal-warnings.

20.1.12. Weak references and definitions

Symbols can be given weak binding by the compiler and assembler. Weak references and definitions are typically references to library functions.

The linker does not load an object from a library to resolve a weak reference. It is able to resolve the weak reference only if the definition is included in the image explicitly.

This can be done by specifying the object file containing the definition on the link command line.

Alternatively, using “–whole-archive <archive-file> –no-whole-archive” linker options includes all objects files in the archive to the link.

Also, removing the weak attribute on the symbol will make it a normal or non-weak. For non-weak symbols, linker will scan the library/archive and loads the required member(s).

> **Note**
> An unresolved weak function call is replaced with a no-operation instruction, NOP (only for aarch64 and if the linker has not reserved a PLT).

An example showing that the linker does not load an object from a library to resolve a weak reference.

By inspecting the disassembly of “a.out”, user can observe that call to bar() in main() is replaced with a NOP.

```
cat > 1.c << \!
__attribute__((weak))  int bar();
int main() {
 bar();
 return 0;
}
!

cat > 2.c << \!
int bar() { return 0; }
!

clang -c 1.c 2.c -target aarch64
rm -f lib2.a
llvm-ar cr lib2.a 2.o
ld.eld 1.o lib2.a -t -march aarch64
```

20.1.13. Reproducing a failure

If you are having an issue, and you want to pass the link step for someone else to debug, you can use the `--reproduce` option.

`--reproduce <tarball>` creates a tarball with all the inputs that were fed to the linker and rewrites the link command so the problem can be reproduced elsewhere. Use `--reproduce=default` to automatically name the tarball after the output file (`<output>.tar`, where `<output>` is the `-o` filename, or `a.out.tar` if `-o` is not given).

> **Note**
> If the tarball is big and you want the linker to compress the tarball automatically, you can use the switch –reproduce-compressed.

> **Note**
> If you are having a link time failure and the problem does not reproduce everytime, but in specific builds, it may be because of non determinism in the builds. In that case use the option `--reproduce-on-fail <tarball>`. Use `--reproduce-on-fail=default` to name the tar after the output file.

The `--reproduce-on-fail` switch only creates a tarball when the link step fails.

20.1.14. Multiple ways to invoke ELD linker

- arm-link can be invoked since it is a symbolic link to ld.eld

```
> cat bar.c
int main(){return 0;}

clang -target  armv7-none-eabi  -g  -c bar.c  -o bar.o
arm-link -m armelf_linux_eabi --dynamic-linker /lib/ld-linux.so.3 -o bar.elf --strip-debug bar.o
```

- Pass -fuse-ld=eld flag to clang driver, so that eld is used for linking

```
clang -target armv7-none-eabi -fuse-ld=eld -fuse-baremetal-sysroot bar.c -o bar.elf -fno-use-baremetal-crt
```

Note
ELD linker flags can be passed to clang driver using -Wl prefix. Example : clang …… -Wl,-shared -Wl,–gc-sections bar.o -o bar.elf

- Pass -v (verbose) when linking using clang binary, pick the link command line, edit and execute it

```
$ clang -target armv7-none-eabi -fuse-ld=eld -fuse-baremetal-sysroot bar.o -o bar.elf -fno-use-baremetal-crt -v
clang version 16.0.0
Snapdragon LLVM ARM Compiler 16.0.0
Target: armv7-none-unknown-eabi
Thread model: posix
InstalledDir: /pkg/qct/software/llvm/release/arm/16.0.0/bin
 "/pkg/qct/software/llvm/release/arm/16.0.0/bin/ld.eld" -EL -X --eh-frame-hdr -m armelf_linux_eabi --dynamic-linker /lib/ld-linux.so.3 -o bar.elf bar.o -L/pkg/qct/software/llvm/release/arm/16.0.0/armv7-none-eabi/libc/lib -L/afs/localcell/cm/gv2.6/sysname/pkg.@sys/qct/software/llvm/release/arm/16.0.0/lib/clang/16.0.0/lib/baremetal --start-group -lc -lclang_rt.builtins-armv7 --end-group -lc

$ ld.eld -EL -X --eh-frame-hdr -m armelf_linux_eabi --dynamic-linker /lib/ld-linux.so.3 -o bar.elf bar.o -L/pkg/qct/software/llvm/release/arm/16.0.0/armv7-none-eabi/libc/lib  -L/afs/localcell/cm/gv2.6/sysname/pkg.@sys/qct/software/llvm/release/arm/16.0.0/lib/clang/16.0.0/lib/baremetal --start-group -lc -lclang_rt.builtins-armv7 --end-group -lc --strip-debug
```

20.1.15. Ignore multiple definition errors and continue with the link

Sometimes a developer might want to ignore all multiple definition errors and continue with the link.

This can be achived by using –allow-multiple-definition switch.

Example:

```
cat > 1.c << \!
int foo() { return 0; }
!

cat > 2.c << \!
int foo() { return 1; }
!

#compile
clang -target hexagon -c 1.c 2.c
#produces error
hexagon-link 1.o 2.o
#produces a successful link
hexagon-link 1.o 2.o --allow-multiple-definition
```

20.1.16. Multiple ways to provide entry point to linker

- Using linker flag : -e <value> –> Name of entry point symbol
- Specifying ENTRY(symbol) command in a linker script
- Initialising the value of linker symbol “start”
- Specifying the start address for the first input section in linker script (eg: .text : AT(0))

20.1.17. How to obtain a non-executable stack

Non-executable stack (NX) is a virtual memory protection mechanism to block shell code injection from executing on the stack by restricting a particular memory and implementing the NX bit.

ELD has the following equivalent option :

-z noexecstack : Mark the executable as not requiring an executable stack

20.1.18. What is the difference between section type NOBITS and PROGBITS

NOBITS section do not occupy size on the disk.

PROGBITS section occupies space on disk.

20.1.19. How to link libraries to resolve circular dependencies

The order in which the libraries are loaded matter. Incorrect order leads to “undefined reference” errors.

Keeping the libraries to linked within “–start-group” and “–end-group” linker flags takes care of the circular dependencies, so that the user need not worry about the order in which libs are loaded.

Circular Dependies:

```
cat > 1.c << \!
extern int bar();
int foo() { return bar(); }
!

cat > 2.c << \!
extern int fred();
int bar() { return fred(); }
!

cat > 3.c << \!
int baz() { return 0; }
!

cat > 4.c << \!
extern int baz();
int fred() { return baz(); }
!

clang -target hexagon -c 1.c 2.c 3.c 4.c
# lib2.a creates a dependency of fred in lib4.a in a different group
hexagon-ar cr lib2.a 2.o 3.o
# lib3.a creates a reverse-dependency to lib2.a which was previously visited
hexagon-ar cr lib3.a 4.o
# problem
hexagon-link 1.o --start-group lib2.a --end-group --start-group lib3.a --end-group
# solution 1
hexagon-link 1.o --start-group lib2.a --end-group --start-group lib3.a lib2.a --end-group
# solution 2
hexagon-link 1.o --start-group 2.o 3.o --end-group --start-group lib3.a --end-grou
```

20.1.20. How to disable use of standard system startup files

“-nostartfiles” : Does not use the standard system startup files when linking.

If this option is specified, user has to specify their own program entry point and write their own start files

20.1.21. How to disable use of standard system libraries

“-nostdlib” : Disable default search path for libraries

This option will prevent the compiler from picking the standard libraries, rtlibs provided by the toolchain. User has to come up with their own definitions or libs so that there are no undefined reference errors reported.

The compiler rt libraries have to be specified explicitly if there are calls to __aeabi* functions and are not defined by the users.

20.1.22. How to remove a section from a OBJ file

llvm-objcopy tool can be used to remove a section from an ELF file.

Example:

```
llvm-objcopy --remove-section=<section_name>
```

20.1.23. How to extract OBJ files from an archive

llvm-ar is the utility to extract obj files from the archive

Example:

```
llvm-ar -x <archive_file>
```

20.1.24. How to obtain a deterministic output from linker

By default deterministic behaviour is supported. To get maximum throughput from linker, –enable-threads=all linker option must be passed.

20.1.25. How to obtain memory statistics from an ELF file or OBJ file

llvm-size utility can provide details of size of .text, .data, .rodata, .bss, etc.

20.1.26. How to compare layout of two output images

Mapfile contains the layout of the image. You can compare the layout section of mapfiles of the two output images which you are interested in.

By default, mapfiles layout section also contains virtual addresses associated with the layout. This brings in too much unnecessary noise when comparing the two layouts, because a small change in the layout can result in change of virtual address of many sections. This can be fixed by using '–MapDetail only-layout' option.

```
// Run link commands with '--MapDetail only-layout' when generating the two output images.
vim -d image1.map image2.map # To compare the two output images layouts.
```

20.1.27. How to embed a binary and use it in 'C' code

You can include a binary file and use it in 'C' code to reference the binary content.

Here is an example which builds a simple table and embeds that table in 'C' code.

20.1.28. Simple python code to create a binary file

The below python code when run creates a binary file by name table.bin

```
f = open('table.bin', 'w+b')
byte_arr = [1,2,3,4,5]
binary_format = bytearray(byte_arr)
f.write(binary_format)
f.close()
```

20.1.29. Create a simple assembly file to include this binary file

This snippet of assembly creates a section named table and includes

```
.section "table","a",@progbits
.incbin "table.bin"
```

20.1.30. Include this assembly file to build a static executable

The program snippet below prints the contents of the binary file to stdout.

```
extern char __start_table;
extern char __stop_table;
int main() {
  char *s = &__start_table;
  char *e = &__stop_table;
  while (s < e) {
    printf("%d\\n", *s);
    s++;
  }
  return 0;
}
```

Linker defines a variable __start_<section> and __stop_<section> if the linker is going to create an output section that matches a ‘C’ identifier.

Using the snippet above, and a simple assembly file, you will be able to iterate over the binary file in ‘C’ code

20.1.31. Where should .note.gnu.property section map to in elf

The .note.gnu.property section is mapped to PT_NOTE segment by default which should have only read permission.

20.1.32. How to garbage-collect symbols in a shared library?

One way to garbage-collect symbols in a shared library is to mark the symbols as hidden. Hidden symbols are garbage collected from a shared library if they are not used from any of the entry symbols in the shared library (entry symbols are symbols that are marked with visibility default).

Example:

```
cat >1.c <<\EOF
// can not be garbage-collected
int foo() {
  return 1;
}

// can be garbage-collected
__attribute__((visibility("hidden")))
int bar() {
  return 2;
}

int baz() {
  return 0;
}
EOF

clang -target hexagon -o 1.o 1.c -c -fPIC -ffunction-sections
hexagon-link -o 1.elf 1.o --gc-sections --print-gc-sections -e baz
```

Running the above script gives the below output:

```
cat
clang -target hexagon -c 1.c -ffunction-sections -fPIC
hexagon-link 1.o -shared --gc-sections --print-gc-sections -e baz
Trace: GC : 1.o[.text]
Trace: GC : 1.o[.text.foo]
```

We can see that ‘foo’, a hidden symbol, got garbage-collected during the link process as it was not reachable from the entry point (baz).

20.1.33. How are zero-sized sections handled?

- A relocation refers to the zero sized section
- A symbol is present in the zero sized section

Note

Releases

This change is observed on the below toolchain / patch releases on Hexagon:
- Hexagon 8.7.04

Releases that will be published after summer of 2023 will also have this support.

20.1.33.1. Relocation referring to a zero sized section

In the below example, the section .text.baz has size zero and is a relocation target for .text.c1, hence .text.baz is useful and is made a part of the layout.

Example:

The example illustrates a reference to the section .text.baz references by the section .text.c1

```
cat > x.s << \EOF
.section .text.foo
.global foo
.set foo, bar
.section .text.bar
bar:
.word 100
.section .text.baz
.section .text.c1
.word .text.baz
.section .text.empty
EOF

cat > main.c << \EOF
int main() { return foo(); }
EOF

cat > script.t << \EOF
SECTIONS {
 . = 0x10000;
 .foo : {
  *(.text.*)
  *(.text)
 }
}
EOF
```

Compile and Link step

```
$ clang -c x.s  main.c
$ ld.eld main.o x.o -T script.t -Map x
```

Image layout

The image layout is adjusted to keep **.text.baz** in the output, as there is a relocation that is referred from the section .text.c1

```
# Output Section and Layout
.(0x10000) = 0x10000; # . = 0x10000; script.t

.foo     0x10000 0x24 # Offset: 0x1000, LMA: 0x10000, Alignment: 0x10, Flags: SHF_ALLOC|SHF_EXECINSTR, Type:
SHT_PROGBITS
*(.text.*) #Rule 1, script.t [9:0]
.text.foo       0x10000 0x0     x.o     #SHT_PROGBITS,SHF_ALLOC|SHF_EXECINSTR,1
.text.bar       0x10000 0x4     x.o     #SHT_PROGBITS,SHF_ALLOC|SHF_EXECINSTR,1
        0x10000         bar
        0x10000         foo
.text.baz       0x10004 0x0     x.o     #SHT_PROGBITS,SHF_ALLOC|SHF_EXECINSTR,1
.text.c1        0x10004 0x4     x.o     #SHT_PROGBITS,SHF_ALLOC|SHF_EXECINSTR,1
*(.text) #Rule 2, script.t [2:0]
PADDING_ALIGNMENT       0x10008 0x8     0x0
.text   0x10010 0x14    main.o  #SHT_PROGBITS,SHF_ALLOC|SHF_EXECINSTR,16
        0x10010         main
*(.foo) #Rule 3, Internal-LinkerScript (Implicit rule inserted by Linker) [0:0]
```

20.1.33.2. The section has associated symbols

Sections containing symbols are preserved in the layout. Such sections are not considered to be empty sections.

> **Warning**
> Such cases should be avoided at all times due to the following reasons
>
> - The code is very sensitive to layout for it to work properly.

- When link time garbage collection is enabled, this may end up garbage collecting sections which would rather be needed at runtime.

Example:

Warning

In the below example

- The code is very sensitive to layout for it to work properly because the program relies on .text.foo and .text.bar to be placed together
- When link time garbage collection is enabled, this may end up garbage collecting sections which would rather be needed at runtime

  - In the example below the linker may end up garbage collecting .text.bar

```
cat > x.s << \!
.section .text.foo
.type foo, @function
.global foo
foo:
.section .text.bar
.type bar, @function
.global bar
bar:
nop
!

cat > main.c << \!
int main() { return foo(); }
!

cat > script.t << \!
SECTIONS {
 . = 0x10000;
 .text: {
  *(.text.*)
  *(.text)
 }
}
!
```

Compile and link step

```
$ clang -c x.s  main.c
$ ld.eld main.o x.o -T script.t -Map x
```

Image Layout

```
# Output Section and Layout
.(0x10000) = 0x10000; # . = 0x10000; script.t

.text    0x10000 0x24 # Offset: 0x1000, LMA: 0x10000, Alignment: 0x10, Flags: SHF_ALLOC|SHF_EXECINSTR, Type:
SHT_PROGBITS
*(.text.*) #Rule 1, script.t [6:0]
.text.foo       0x10000 0x0     x.o     #SHT_PROGBITS,SHF_ALLOC|SHF_EXECINSTR,1
        0x10000         foo
.text.bar       0x10000 0x4     x.o     #SHT_PROGBITS,SHF_ALLOC|SHF_EXECINSTR,1
        0x10000         bar
*(.text) #Rule 2, script.t [2:0]
PADDING_ALIGNMENT       0x10004 0xc     0x0
.text   0x10010 0x14    main.o  #SHT_PROGBITS,SHF_ALLOC|SHF_EXECINSTR,16
        0x10010         main
*(.text) #Rule 3, Internal-LinkerScript (Implicit rule inserted by Linker) [0:0]
```

20.1.33.3. Can a zero-sized section ever affect the layout? - Yes it can

Example:

```
cat > x.s << \EOF
.section .text.foo
.global foo
.set foo, bar
.section .text.bar
bar:
.word 100
.section .text.baz
.p2align 4
.section .text.baz1
.section .text.baz2
.section .text.c1
.word .text.baz
.section .text.empty
EOF

cat > main.c << \EOF
int main() { return foo(); }
EOF

cat > script.t << \EOF
SECTIONS {
    . = 0x10003;
    .baz :  {
        . = 0x10005;
        *(.text.baz1)
        . = . + 16;
        *(.text.baz)
        . = . + 28;
        *(.text.baz2)
        . = . + 40;
    }
    .foo : {
        *(.text.foo)
        *(.text*)
    }
}
EOF
```

Compile and Link Step:

```
$clang -c x.s  main.c
$ld.eld main.o x.o -T script.t -Map x.map
```

Image Layout

Image Layout from xmap:

```
# Output Section and Layout
.(0x10003) = 0x10003; # . = 0x10003; script.t

.baz     0x10010 0x54 # Offset: 0x1010, LMA: 0x10010, Alignment: 0x10, Flags: SHF_ALLOC|SHF_EXECINSTR, Type:
SHT_PROGBITS
        .(0x10005) = 0x10005; # . = 0x10005; script.t
*(.text.baz1) #Rule 1, script.t [1:0]
        .(0x100010015) = .(0x100010005) + 0x10; # . = . + 0x10; script.t
*(.text.baz) #Rule 2, script.t [1:0]
PADDING_ALIGNMENT       0x10015 0xb     0x0
.text.baz       0x10020 0x0     x.o     #SHT_PROGBITS,SHF_ALLOC|SHF_EXECINSTR,16
        .(0x1003c) = .(0x10020) + 0x1c; # . = . + 0x1c; script.t
*(.text.baz2) #Rule 3, script.t [1:0]
PADDING 0x100010005     0x10    0x0
PADDING 0x1003c 0x28    0x0
```

```
        .(0x10064) = .(0x1003c) + 0x28; # . = . + 0x28; script.t
*(.baz) #Rule 4, Internal-LinkerScript (Implicit rule inserted by Linker) [0:0]

.foo     0x10070 0x1c # Offset: 0x1070, LMA: 0x10070, Alignment: 0x10, Flags: SHF_ALLOC|SHF_EXECINSTR, Type:
SHT_PROGBITS
*(.text.foo) #Rule 5, script.t [1:0]
.text.foo       0x10070 0x0     x.o     #SHT_PROGBITS,SHF_ALLOC|SHF_EXECINSTR,1
*(.text*) #Rule 6, script.t [9:0]
.text   0x10070 0x14    main.o  #SHT_PROGBITS,SHF_ALLOC|SHF_EXECINSTR,16
        0x10070         main
.text.bar       0x10084 0x4     x.o     #SHT_PROGBITS,SHF_ALLOC|SHF_EXECINSTR,1
        0x10084         bar
        0x10084         foo
.text.c1        0x10088 0x4     x.o     #SHT_PROGBITS,SHF_ALLOC|SHF_EXECINSTR,1
```

Here because of p2align directive for .text.baz the PADDING_ALIGNMENT of size 0xb at address 0x100015 was added

20.1.33.4. Debugging zero-sized sections related issues

ELD has 2 ways that help in debugging issues wrt to zero-sized sections

- Linker diagnostics as warnings tied to options `-Wall` and `-Wzero-sized-sections`. Look for traces containing `Warning: Zero sized fragment`.

  ```
  cat > x.s << \EOF
  .section .text.baz
  .type baz, %function
  .global baz
  baz:
  .section .text.bar
  .type bar, %function
  .global bar
  bar:
  nop
  .section .foo,"a",%progbits
  .local sym
  sym:
  .size sym, 40
  EOF

  cat > main.c << \EOF
  int main() { return baz(); }
  EOF

  cat > script.t << \EOF
  SECTIONS {
   . = 0x10000;
   .text: {
    *(.text.*)
    *(.text)
   }
  }
  EOF

  $ clang -c x.s  main.c
  $ ld.eld main.o x.o -T script.t -Map x
  Warning: Zero sized fragment .foo for non zero sized symbol sym from input file x.o
  ```

- Linker stats emitted in the map file as * ZeroSizedSections * ZeroSizedSectionsGarbageCollected

```
cat > x.s << \!
.section .text.foo
.global foo
.set foo, bar
.section .text.bar
bar:
.word 100
.section .text.baz
```

```
.section .text.c1
.word .text.baz
.section .text.empty
 !

cat > main.c << \!
int main() { return foo(); }
!

cat > script.t << \!
SECTIONS {
 . = 0x10000;
 .text: {
  *(.text.*)
  *(.text)
}
}
!

$ clang -c x.s  main.c
$ ld.eld main.o x.o -T script.t -Map x
$ cat x
...
# LinkStats Begin
# ObjectFiles : 2
# LinkerScripts : 2
# ThreadCount : 64
# NumInputSections : 29
# ZeroSizedSections : 17
# SectionsGarbageCollected : 5
# ZeroSizedSectionsGarbageCollected : 4
# NumLinkerScriptRules : 2
# NumOutputSections : 6
# NumOrphans : 5
# NoRuleMatches : 2
# LinkStats End
...
```

20.1.34. How to interpret the new trampoline naming convention?

The Linker trampoline naming convention has been updated to the following format

trampoline_for_<target symbol name>_from_<source input section name>_<Input file ordinality of the source input section>#<Optional: relocation addend>_<Optional: Additional trampoline count>

The trampoline naming convention can be broadly classified into 4 cases:

20.1.34.1. Case 1:

Format: trampoline_for_<target symbol name>_from_<source input section name> _<Input file ordinality of the source input section>

Example:

```
cat > main.c << \!
int baz() {
  return 0;
}
int main ()
{
  return baz() ;
}
!
cat > script.t << \!
SECTIONS
{
  .text          :
  {
```

```
    *(.text.main)
  } =0x00c0007f
  . = 0x08000000;
  .text.baz :
  {
    *(.text.baz)
  }
}
!
```

Compile and Link Steps:

```
$ clang -c  main.c -ffunction-sections
$ ld.eld main.o -T script.t -Map x
```

Symbols from readelf:

```
$ llvm-readelf -s a.out
Symbol table '.symtab' contains 9 entries:
   Num:    Value  Size Type    Bind   Vis       Ndx Name
     0: 00000000     0 NOTYPE  LOCAL  DEFAULT   UND
     1: 00000000     0 SECTION LOCAL  DEFAULT     1 .text
     2: 00000000     0 SECTION LOCAL  DEFAULT     3 .comment
     3: 08000000     0 SECTION LOCAL  DEFAULT     2 .text.baz
     4: 00000000     0 FILE    LOCAL  DEFAULT   ABS main.c
     5: 00000014     8 FUNC    LOCAL  DEFAULT     1 trampoline_for_baz_from_.text.main_25
     6: 00000000    20 FUNC    GLOBAL DEFAULT     1 main
     7: 08000000    12 FUNC    GLOBAL DEFAULT     2 baz
     8: 08000011     0 NOTYPE  GLOBAL DEFAULT   ABS __end
```

Trampoline Symbol: `trampoline_for_baz_from_.text.main_25` → call baz from main

This is the most vanilla case where the target symbol name, source input section and input file are used to coin the trampoline name.

20.1.34.2. Case2:

Format: trampoline_for_<target symbol name>_from_<source input section name>_ <Input file ordinality of the source input section>_<Additional trampoline count>

Example:

```
cat > main.c << \!
int far() {
  return 0;
}
int callfar() {
  return far() + far();
}
int main ()
{
  return callfar();
}
!

cat > noreuse << \!
{
    far;
}
!

cat > script.t << \!
SECTIONS
{
  .text          :
  {
```

```
    *(.text.main)
    *(.text.callfar)
  } =0x00c0007f
  . = 0x08000000;
  .text.far :
  {
    *(.text.far)
  }
}
!
```

Compile and Link Steps:

```
$ clang -c  main.c -ffunction-sections
$ ld.eld main.o -T script.t -Map x --no-reuse-trampolines-file=noreuse
```

Symbols from readelf:

```
$ llvm-readelf -s a.out
Symbol table '.symtab' contains 11 entries:
   Num:    Value  Size Type    Bind   Vis       Ndx Name
     0: 00000000     0 NOTYPE  LOCAL  DEFAULT   UND
     1: 00000000     0 SECTION LOCAL  DEFAULT     1 .text
     2: 00000000     0 SECTION LOCAL  DEFAULT     3 .comment
     3: 08000000     0 SECTION LOCAL  DEFAULT     2 .text.far
     4: 00000000     0 FILE    LOCAL  DEFAULT   ABS main.c
     5: 00000040     8 FUNC    LOCAL  DEFAULT     1 trampoline_for_far_from_.text.callfar_25
     6: 00000048     8 FUNC    LOCAL  DEFAULT     1 trampoline_for_far_from_.text.callfar_25_1
     7: 00000000    20 FUNC    GLOBAL DEFAULT     1 main
     8: 00000020    32 FUNC    GLOBAL DEFAULT     1 callfar1
     9: 08000000    12 FUNC    GLOBAL DEFAULT     2 far
    10: 08000011     0 NOTYPE  GLOBAL DEFAULT   ABS __end
```

Trampoline Symbol: `trampoline_for_far_from_.text.callfar1_25_1` → call far from callfar1

The additional trampoline count is added as a suffix in cases where the reuse of the existing trampoline is not possible.

> **Note**
> The reuse was not possible because of `--no-reuse-trampolines-file=noreuse`.

20.1.34.3. Case3:

Format: trampoline_for_<target symbol name>_from_<source input section name>_#(relocation addend)

Example:

```
cat > main.c << \!
static int baz() {
  return 0;
}
int main ()
{
  return baz();
}
!

cat > script.t << \!
SECTIONS
{
  .text           :
  {
    *(.text.main)
  } =0x00c0007f
  . = 0x08000000;
  .text.baz :
```

```
  {
    *(.text.baz)
  }
}
!
```

Compile and Link Steps:

```
$ clang -c  main.c -ffunction-sections
$ ld.eld main.o -T script.t -Map x
```

Symbols from readelf:

```
$ llvm-readelf -s a.out
Symbol table '.symtab' contains 9 entries:
   Num:    Value  Size Type    Bind   Vis       Ndx Name
     0: 00000000     0 NOTYPE  LOCAL  DEFAULT   UND
     1: 00000000     0 SECTION LOCAL  DEFAULT     1 .text
     2: 00000000     0 SECTION LOCAL  DEFAULT     3 .comment
     3: 08000000     0 SECTION LOCAL  DEFAULT     2 .text.baz
     4: 00000000     0 FILE    LOCAL  DEFAULT   ABS main.c
     5: 00000014     8 FUNC    LOCAL  DEFAULT     1 trampoline_for_.text.baz_from_.text.main_25#(0)
     6: 08000000    12 FUNC    LOCAL  DEFAULT     2 baz
     7: 00000000    20 FUNC    GLOBAL DEFAULT     1 main
     8: 08000011     0 NOTYPE  GLOBAL DEFAULT   ABS __end
```

Trampoline Symbol: `trampoline_for_.text.bar_from_.text.main_25#(0)` → call to bar from main here (`#(0)` represents the relocation addend)

The relocation addend 0 is added to the trampoline symbol name in case the trampoline jumps to the section symbol for the symbol

20.1.34.4. Case4:

Format: trampoline_for_<target symbol name>_from_<source input section name> _<Input file ordinality of the source input section>#<relocation addend> _<Additional trampoline count>

Example:

```
cat > main.c << \!
static int baz() {
  return 0;
}
int main ()
{
  return baz() + baz();
}
!

cat > noreuse << \!
{
    baz;
    .text.baz;
}
!

cat > script.t << \!
SECTIONS
{
  .text           :
  {
    *(.text.main)
  } =0x00c0007f
  . = 0x08000000;
  .text.baz :
  {
```

```
    *(.text.baz)
  }
}
!
```

Compile and Link Steps:

```
$ clang -c  main.c -ffunction-sections
$ ld.eld main.o -T script.t -Map x --no-reuse-trampolines-file=noreuse
```

Symbols from readelf:

```
$ llvm-readelf -s a.out
Symbol table '.symtab' contains 10 entries:
   Num:    Value  Size Type    Bind   Vis       Ndx Name
     0: 00000000     0 NOTYPE  LOCAL  DEFAULT   UND
     1: 00000000     0 SECTION LOCAL  DEFAULT     1 .text
     2: 00000000     0 SECTION LOCAL  DEFAULT     3 .comment
     3: 08000000     0 SECTION LOCAL  DEFAULT     2 .text.baz
     4: 00000000     0 FILE    LOCAL  DEFAULT   ABS main.c
     5: 00000028     8 FUNC    LOCAL  DEFAULT     1 trampoline_for_.text.baz_from_.text.main_25#(0)
     6: 00000030     8 FUNC    LOCAL  DEFAULT     1 trampoline_for_.text.baz_from_.text.main_25#(0)_1
     7: 08000000    12 FUNC    LOCAL  DEFAULT     2 baz
     8: 00000000    40 FUNC    GLOBAL DEFAULT     1 main
     9: 08000011     0 NOTYPE  GLOBAL DEFAULT   ABS __end
```

Trampoline Symbol: `trampoline_for_.text.bar_from_.text.main_25#(0)_1` → 2nd call for bar from main

The duplicate trampoline count was added as a prefix since the symbol name .text.bar was added to the noreuse list.

Benefits of the new trampoline naming convention:

The new format has benefits as follows
- Helps determine/infer a huge amount of info about the target symbol and the source sections inherently
- Helps make diff between map files easier since the naming convention is now more context-based

20.1.35. Understanding “loadable segments are unsorted by virtual address” warning from llvm-readelf

The llvm-readelf can sometimes emit a warning: “loadable segments are unsorted by virtual address”. This occurs in very specific scenarios where below items are involved
- Dynamic sections
- “-shared” in linker commandline
- The virtual addresses of dynamic sections not in increasing order

Below is a small example to illustrate this:

Example:

```
cat > 1.c << \!
int foo() { return 0; }
!

cat > script.t << \!
PHDRS {
    A PT_LOAD;
    B PT_LOAD;
    DYN PT_DYNAMIC;
}

SECTIONS {
      .dynsym (0x900) : { *(.dynsym) } :A
      .dynamic (0x800) : { *(.dynamic) } :B :DYN
}
!
```

```
clang -target hexagon -c 1.c -ffunction-sections
hexagon-link 1.o -T script.t -shared
llvm-readelf -S -W a.out
```

Important thing to note in above script is the non-increasing order of virtual addresses of the dynamic sections

When the *"llvm-readelf"* in above script runs, a specific code branch for dynamic images in ELF.cpp emits this warning

Output:

```
llvm-readelf: warning: 'a.out': loadable segments are unsorted by virtual address
```

This warning in its current form is rather vague and very general as it doesn't inform the user about the specific dynamic builds this warning is meant for.

Currently, there is **no** option available to suppress this warning.

20.1.36. Linker overlap checks

Linker has been improved to detect overlaps in the image layout. Linker does this by default. The following overlaps are detected by the linker. * Virtual address overlaps * Physical address overlaps * File offset overlaps

If the overlaps are known and you want to turn this behavior OFF, you can use *–no-check-sections* flag.

20.1.37. Link errors

20.1.37.1. Why am I receiving the error 'cannot recognize the format of file YourObjectFile.o'. object format or given target machine (…) is incompatible.'?

Fix: Ensure that all object files being linked use the same object format and target architecture.

But what is the cause of the error?

A linker can only combine object files that are compatible with each other. This means every input file must match in:

- Object format (e.g. ELF)
- Target architecture (e.g. Hexagon)
- Endianness and ABI

If any object file differs, the linker cannot interpret it correctly and will produce this error.

How to verify: You can inspect the object file using by using tools like llvm-readelf. You can enable/disable this message by using the *–[no-]warn-mismatch* flag.

20.2. Input File to Linker

The linker takes the following kinds of input files as input :-
- Object files
- Shared libraries
- Linker scripts
- Multiple command line options
- Other kinds of input files such as : - Extern list
  - dynamic list
  - version scripts
  - linker plugin configuration files

Most recently the linker also was modified to support taking fully built static executables as part of the link step.

20.2.1. ELF executable as input files to the linker

Linker allows a fully built static executable as input to the linker.

The LLVM community linker does not support this option. GNU linker used to support but now does not as per the below bug

https://sourceware.org/bugzilla/show_bug.cgi?id=26223

ELD allows fully built static executables as input to the linker as per the below example.

```
cat > script.t << \!
SECTIONS {
  .text : {
    *(.text)
  }
  . = 0x2000;
  anotherelf : {
    anotherexec.elf(.bar)
  }
}
!

cat > foo.c << \!
extern int bar();
int foo() { return bar(); }
!

cat > bar.c << \!
__attribute__((section(".bar"))) int bar() { return 0; }
!

$clang -O2 -c bar.c -fno-exceptions -fno-asynchronous-unwind-tables
$clang -c foo.c -fno-exceptions -fno-asynchronous-unwind-tables
$link bar.o -o anotherexec.elf -Ttext=0x2000
$link foo.o anotherexec.elf -T script.t
```

Warning
This is strongly discouraged by the community and got accidentally removed in the GNU linker.

It is upto the user to make sure that the linker script places the executable code previously linked at the same address.

Linking dynamic executables is not possible with this functionality and will not be supported

Note
If you really need to link with executables, we recommend to use the –just-symbols option

20.3. Linker Script General Questions

20.3.1. Debugging section placements

20.3.2. Linker Script Rules

Input sections can be stored in files or archives. They are specified in the SECTIONS command with the following syntax:

[path][archive:][file](section...)

The standard wildcard characters (*, ?, etc.) can be used anywhere in an input section specification to do the following:
- Specify multiple paths, archives, or files where sections will be searched for
- Specify multiple sections as input sections

Specification	Description
dir/subdir/init.lib:init.o(.text.*)	Specify one or more .text. sections from a specific object file (init.o) in a specific archive (init.lib)
dir/subdir/init.lib:(.text.*)	Specify one or more .text. sections from any object file in thespecified archive (init.lib)
dir/subdir/:(.text.)	Specify one or more .text. sections from any archive in the specified directory (dir/subdir)
(.text.)	Specify one or more .text. sections from any archive or object file in the entire file system

20.3.3. Fill Values

20.3.3.1. Output Section Fill

You can set the fill pattern for an entire section by using '=fillexp'. fillexp is an expression.

Any otherwise unspecified regions of memory within the output section (for example, gaps left due to the required alignment of input sections) will be filled with the value, repeated as necessary.

In all cases, the number specified by the fillexp is big-endian.

Here is a simple example:

```
SECTIONS { .text : { *(.text) } =0x90909090 }
```

20.3.4. I am getting an error, no linker script rule for ".bss"

The way that you would investigate this is to figure out if there is an actual .bss section that is present in the input files, not selected by any linker script rule. Emit the Map file and look at the section .bss and see what kind of sections are present.

- If it shows that a input file with a particular section is being listed and not present in any of the patterns, you will likely need to go and add a rule such as `*(.bss)` so that you are sure of where you want to place it.
- If all the input file and sections are selected, you should go and look at if the section corresponds to a common symbol (common symbol). You will most probably be missing a rule such as `*(COMMON)`.
- Try to do the link step again with any of the above changes and your error should have gone.

20.3.5. I am getting an error, no linker script rule for ".data.bar"

This is most often a problem if there is a compiler flag that has changed in your environment or a change of tools. You would need to fix the linker script and add a rule as below.

- *(.data.bar)

20.3.6. Is there a way to find all used/unused rules in the Linker script ?

You can use a simple grep pattern to check for used/unused rules in the Map file.

20.3.6.1. UnUsed Rules

```
grep "Rule.*\[0:" link.map | grep -v Implicit
```

20.3.6.2. Used Rules

```
grep "Rule.*\[[1-9]*:" link.map  | grep -v Implicit
```

20.3.7. How do I exclude sections from object files/archive files

20.3.7.1. Excluding sections from object files

The example illustrates users trying to exclude all text sections from being placed in section .text1 and when the input file is an object file.

```
cat > a.c << \!
int foo() { return 0; }
int bar() { return 0; }
```

```
!


cat > script.t << \!
SECTIONS {
  .text1 : {
    *(EXCLUDE_FILE(a.o) .text.*)
  }
  .text2 : {
    *(.text*)
  }
}
!

clang -target hexagon -ffunction-sections -fdata-sections -c a.c -G0
rm -f lib1.a
hexagon-ar cr lib1.a a.o
hexagon-link -T script.t a.o -Map x
```

20.3.7.2. Exclude all patterns of text section from one file

The example illustrates an user trying to exclude all text sections from being placed in section .text1.

In this example, all code from lib1.a/a.o is not emitted to the the .text1 section

```
cat > a.c << \!
int foo() { return 0; }
int bar() { return 0; }
!


cat > script.t << \!
SECTIONS {
  .text1 : {
    *lib1.a:(EXCLUDE_FILE(a.o) .text.*)
  }
  .text2 : {
    *(.text*)
  }
}
!

clang -target hexagon -ffunction-sections -c a.c
rm -f lib1.a
hexagon-ar cr lib1.a a.o
hexagon-link -T script.t --whole-archive lib1.a
```

20.3.7.3. Excluding all patterns of text section from more than one file

The example illustrates an user trying to exclude all text sections from being placed in section .text1.

In this example, all code from lib1.a/a.o and lib1.a/b.o is not emitted to the .text1 section

```
cat > a.c << \!
int foo() { return 0; }
int bar() { return 0; }
!

cat > b.c << \!
int baz() { return 0; }
!


cat > script.t << \!
SECTIONS {
  .text1 : {
    *lib1.a:(EXCLUDE_FILE(a.o b.o) .text.*)
```

```
  }
  .text2 : {
    *(.text*)
  }
}
!

clang -target hexagon -ffunction-sections -fdata-sections -c a.c -G0
clang -target hexagon -ffunction-sections -fdata-sections -c b.c -G0
rm -f lib1.a
hexagon-ar cr lib1.a a.o b.o
hexagon-link -T script.t --whole-archive lib1.a -Map x
```

20.3.7.4. Specifying multiple exclude patterns

You can specify more than one EXCLUDE_FILE pattern in a linker script rule.

This example below illustrates a way that the user can specify multiple EXCLUDE_FILE linker script commands.

```
cat > a.c << \!
int foo() { return 0; }
int bar() { return 0; }
!

cat > b.c << \!
int baz() { return 0; }
!


cat > script.t << \!
SECTIONS {
  .text1 : {
    *lib1.a:(EXCLUDE_FILE(a.o) .text.foo EXCLUDE_FILE(a.o) .text.bar EXCLUDE_FILE(b.o) .text.baz)
  }
  .text2 : {
    *(.text*)
  }
}
!

clang -target hexagon -ffunction-sections -fdata-sections -c a.c -G0
clang -target hexagon -ffunction-sections -fdata-sections -c b.c -G0
rm -f lib1.a
hexagon-ar cr lib1.a a.o b.o
hexagon-link -T script.t --whole-archive lib1.a -Map x
```

20.3.7.5. Common mistakes with exclude files

Specifying more than one pattern

> **Warning**
> A common mistake users do is when they specify more than one pattern with one occurrence of an EXCLUDE_FILE rule. Only the first pattern following the EXCLUDE_FILE rule is used for matching

In the above example if the user did the following

```
SECTIONS {
  .text1 : {
    *lib1.a:(EXCLUDE_FILE(a.o) .text.foo .text.bar EXCLUDE_FILE(b.o) .text.baz)
  }
  .text2 : {
    *(.text*)
  }
}
```

Note
The function specified by .text.bar would be placed in .text1 since .text.bar is following the pattern .text.foo.

20.3.7.6. Not specifying all the files in the EXCLUDE_FILE pattern

When the file pattern does not match the filename specified in EXCLUDE_FILE, pattern following the EXCLUDE_FILE is used to select input sections that do not originate from that file.

Below example illustrates that. In this example .text.baz from file b.o is selected by the rule, which results in .text1 housing .text.baz.

```
cat > a.c << \!
int foo() { return 0; }
int bar() { return 0; }
!

cat > b.c << \!
int baz() { return 0; }
!


cat > script.t << \!
SECTIONS {
  .text1 : {
    *lib1.a:(EXCLUDE_FILE(a.o) .text.*)
  }
  .text2 : {
    *(.text*)
  }
}
!

clang -target hexagon -ffunction-sections -fdata-sections -c a.c -G0
clang -target hexagon -ffunction-sections -fdata-sections -c b.c -G0
rm -f lib1.a
hexagon-ar cr lib1.a a.o b.o
hexagon-link -T script.t --whole-archive lib1.a -Map x
```

20.3.7.7. Specifying EXCLUDE_FILE directive that applies to multiple section patterns

We can specify EXCLUDE_FILE directive that applies to multiple section patterns by placing EXCLUDE_FILE before the corresponding <file-patterns>(<section-patterns>).

For Example:

```
#!/usr/bin/env bash

cat >1.c <<\EOF
int foo() { return 1; }
int baz() { return 3; }
EOF

cat >2.c <<\EOF
int bar() { return 3; }
EOF

cat >script.t <<\EOF
SECTIONS {
  FOO_BAZ : { EXCLUDE_FILE(2.o) *(.text.foo .text.baz .text.bar) }
  BAR : { *(*.text*) }
}
EOF

clang -target hexagon -o 1.o 1.c -c -ffunction-sections
clang -target hexagon -o 2.o 2.c -c -ffunction-sections
hexagon-link -o a.out 1.o 2.o -T script.t
```

In the above example, EXCLUDE_FILE(2.o) applies to all the section patterns present in the sub-rule: *(.text.foo .text.baz .text.bar)

20.3.7.8. NOCROSSREFS

The NOCROSSREFS command takes a list of output section names. If the linker detects any cross references between the sections, it reports an error and returns a non-zero exit status. Note that the NOCROSSREFS command uses output section names, not input section names.

```
cat > 1.c << \!
extern int bar();
int foo() { return bar(); }
!

cat > 2.c << \!
int bar() { return 0; }
!

cat > script.t << \!
NOCROSSREFS(.foo .bar)
SECTIONS {
  .foo : { *(.text.foo) }
  .bar : { *(.text.bar) }
}
!

clang -c 1.c 2.c -ffunction-sections
ld.eld 1.o 2.o -T script.t
```

The above linker script usage of **NOCROSSREFS** produces an error because content in output section foo is calling into bar.

Example error:

```
Error: 1.o:(.text.foo:0x8): prohibited cross reference from .foo to `bar'(2.o) in .bar
```

20.3.8. How to discard an input section?

Input sections can be explicitly discarded from the output image by assigning the input section to a special output section named '/DISCARD/'. It can also discard sections marked with the ELF flag SHF_GNU_RETAIN which would otherwise have been saved from linker garbage collection.

For Example:

```
cat >1.c <<\EOF
__attribute__((retain)) int foo() { return 0; }
int bar() { return 0; }
int main() { return 0; }
EOF

cat >1.linker.script <<\EOF
SECTIONS {
  /DISCARD/ : { *(.text.foo) }
}
EOF

clang -target hexagon -o 1.o -c 1.c -ffunction-sections
hexagon-link -o 1.elf 1.o --gc-sections --print-gc-sections -e main
hexagon-link -o 1.with_script.elf 1.o -T 1.linker.script --gc-sections --print-gc-sections -e main --trace-
section .text.foo
```

hexagon-link -o 1.elf 1.o –gc-sections –print-gc-sections -e main

Running the above command gives the following output:

Trace: GC : 1.o[.text] | Trace: GC : 1.o[.text.bar]

Please note that '.text.foo' is preserved even though there are no references to the function foo from the entry point. It is because '.text.foo' is marked with the ELF flag SHF_GNU_RETAIN using '__attribute__((retain))'.

The above command can be run with the verbose option to see the 'retain section…' diagnostic for '.text.foo':

Verbose: Retaining section .text.foo from file 1.o

'.text.foo' can be discarded by assigning it to /DISCARD/ special output section.

hexagon-link -o 1.with_script.elf 1.o -T 1.linker.script –gc-sections –print-gc-sections -e main –trace-section .text.foo

Running the above command gives the following output:

Note: Input Section : .text.foo InputFile : 1.o Alignment : 0x10 Size : 0xc Flags : SHF_ALLOC|SHF_EXECINSTR | Note: Input Section : .text.foo Symbol : foo | Note: Section : .text.foo is being discarded. Section originated from input : 1.o | Trace: GC : 1.o[.text] | Trace: GC : 1.o[.text.bar]

We can see in the 'Note' diagnostic that '.text.foo' has been discarded.

20.3.9. Marking a loadable output section as non loadable

Linker now supports loadable sections to be listed after non loadable sections, so the linker is able to place them in a loadable segment.

There have been various assumptions in the past that customers relied on their linker scripts, about linker not supporting loadable sections after non loadable sections.

If these assumptions are still valid for your use case(s) :

- Make sure to list the ouput section to be of type INFO * Example : OutputSection **(INFO)** * This will mark the output section non loadable
- Always inspect the output of your section to segment mapping before loading the image * llvm-readelf -l -W *outputfile*

If you have been waiting for this feature, remove the virtual address assigned to the section.

Most users have these output sections listed in the linker script to use a virtual address of 0. You will need to remove it.

Always inspect the output image to check for any discrepancies.

20.3.10. What does KEEP mean for input section descriptions in /DISCARD/? (Added in HEX 8.8)

KEEP marks all the input sections matched by an input section description as an entry section. Entry sections are not subject to garbage-collection and are used to calculate live-edges for garbage-collection. However, entry sections can still be discarded. Therefore, behavior of KEEP is the same for input section description of all output sections, including the /DISCARD/ output section. Important thing to remember is when KEEP is used for input section descriptions in the/DISCARD/ output section then the matched input sections are treated as entry sections for live-edge computation but are still discarded nonetheless.

Example:

```
#!/usr/bin/env bash

cat >1.c <<\EOF
int bar() {
  return 1;
}

int foo() {
  return bar();
}

int baz() {
  return 3;
}

int abc() {
```

```
  return 5;
}

int main() {
  return baz();
}
EOF

cat >script.t <<\EOF
SECTIONS {
  /DISCARD/ : { KEEP(*(*.foo)) }
  .text : { *(.text.*) }
}
EOF

clang -target hexagon -o 1.o 1.c -c -ffunction-sections -fdata-sections
hexagon-link -o 1.elf 1.o -T script.t --gc-sections --print-gc-sections -e main -Map 1.map.tx
```

In this example, bar is not garbage-collected because it is reachable by foo. foo is an entry section due to KEEP specifier even though it is discarded. Map file can be used to confirm the behavior.

20.3.11. How to build executables by linking symbols from an another ELF file?

Users can build executables by linking symbols from another ELF file by using the SYMDEF feature.

ELD mimics the functionality available in the ARM linker for this.

This method is also used by ARM baremetal builds for linking against symbols that exist in another image.

20.3.11.1. SymDef file

A symdef file is a file that consists of all global symbols that can be used in future link steps.

The linker produces a symdefs file during a successful final link stage. It is not produced for partial linking or for unsuccessful final linking.

The symdef file is produced by using the linker symdef option.

Example:

```
cat >1.c << \!
int foo() {return bar(); }
!

cat > 2.c << \!
int boo() { return baz(); }
!

cat > 3.c << \!
int bar() { return 0; }
int baz() { return 0; }
!

cat > script.t << \!
SECTIONS {
  .foo : { *(.text.*) }
}
!

cat > image.t << \!
SECTIONS {
  . = 0xF0000000;
  .text : { *(.text*) }
}
!

$CLANG -c 1.c -ffunction-sections
$CLANG -c 2.c -ffunction-sections
$CLANG -c 3.c -ffunction-sections
```

```
$LINK -o otherimage.elf 3.o --symdef-file r.symdef -T image.t
# otherimage.elf should be preserved for the below link command to work
$LINK 1.o 2.o r.symdef -T script.t -o out.elf
```

The symdef file gets produced that contains the address of bar and baz in the image.

```
#<SYMDEFS>#
#DO NOT EDIT#
0xf0000000      FUNC    bar
0xf0000010      FUNC    baz
```

Later the file is used in the secondary link to generate the complete binary.

20.3.12. How to specify Load Memory Address (LMA) of a section?

Load Memory Address (LMA) of a section can be specified using the ‘AT’ command. Please note that it is different from the virtual memory address.

```
FOO 0x1000 : AT(0x4000) {
   *(*foo*)
 }
```

In the above example, ‘0x1000’ is the virtual memory address (VMA) of the output section ‘FOO’, and ‘0x4000’ is the load memory address (lma).

A concrete example:

```
#!/usr/bin/env bash

cat >1.c <<\EOF
int foo() { return 1; }
int bar() { return 3; }
EOF

cat >script.t <<\EOF
SECTIONS {
  FOO 0x1000 : AT(0x4000) {
    *(*foo*)
  }

  BAR 0x2000 : AT(0x6000) {
    *(*bar*)
  }
}
EOF

cat >script.without_at.t <<\EOF
SECTIONS {
  FOO 0x1000 : { *(*foo*) }
  BAR 0x2000 : { *(*bar*) }
}
EOF

CCs=("clang -target hexagon" clang-15 clang-15)
LDs=(hexagon-link ld.bfd ld.lld)
SFs=(eld bfd lld)

clang -target hexagon -o 1.o 1.c -c -ffunction-sections
hexagon-link -o 1.elf 1.o -T script.t
hexagon-llvm-objdump -h 1.elf

# Output:
# 1.eld.elf:     file format elf32-hexagon
#
# Sections:
# Idx Name          Size     VMA      LMA      Type
```

```
#  0               00000000 00000000 00000000
#  1 FOO           0000000c 00001000 00004000 TEXT
#  2 BAR           0000000c 00002000 00006000 TEXT
#  3 .comment      00000106 00000000 00000000
#  4 .shstrtab     0000002c 00000000 00000000
#  5 .symtab       00000080 00000000 00000000
#  6 .strtab       00000024 00000000 00000000
```

```
Please note the VMA and LMA of output sections FOO and BAR:

FOO           0000000c 00001000 00004000
BAR           0000000c 00002000 00006000
```

20.3.13. Image layout using LMA

When sections are placed in a segment, the LMA address of the section is calculated as the sum of LMA address of the segment and virtual address offset of the section from the beginning of the segment.

User can query the LMA address of the section using the linker script keyword LOADADDR.

```
cat > 1.c << \!
int bss[100] = { 0 };
int data[100] = { 0 };
int foo() { return 0; }
!

cat > script.t << \!
PHDRS {
  A PT_LOAD;
}

SECTIONS
  .foo : AT(0x1000) { *(.text.foo) }  :A
  .bar : { *(.bss.bss*) }  :A
  .baz : { *(.bss.data*) }  :A
  .nothing : { } :A
  load_addr_baz = LOADADDR(.nothing);
}
!

$clang -c 1.c -ffunction-sections -fdata-sections -G0 -fno-asynchronous-unwind-tables
$link 1.o -T script.t -Map x
```

With the above example you can see that the linker sets the value of load_addr_baz to the load address of the section .nothing.

For Hexagon architecture, you can see that the load address thats calculated accounts to be 0x1330.

```
Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
LOAD           0x001000 0x00000000 0x00001000 0x0000c 0x00330 RWE 0x1000
```

This change in behavior is observed from linkers on release
- Hexagon 8.9 and above
- RISC-V 18.0 release

If you have a linker script that assumes the behavior of LOADADDR, you might want to fix that with the recent change to behavior.

20.3.14. What are BYTE/SHORT/LONG/QUAD/SQUAD linker script commands?

These commands allow including explicit bytes of data in an output section. Each of these commands is followed by an expression in parentheses providing the value to store. The value is stored at the current value of the location counter

The BYTE, SHORT, LONG, and QUAD commands store one, two, four, and eight bytes (respectively). After storing the bytes, the location counter is incremented by the number of bytes stored.

An example demonstrating these commands:

```
#!/usr/bin/env bash

cat >1.c <<\EOF
int foo() { return 1; }
EOF

cat >script.t <<\EOF
SECTIONS {
  FIVE: { BYTE(0x5) }
  SIXTEEN : { LONG(0x10) }
}
EOF

clang -target hexagon -o 1.o 1.c -c
hexagon-link -o 1.elf 1.o -T script.t
hexagon-readelf -x 'FIVE' 1.elf
hexagon-readelf -x 'SIXTEEN' 1.elf
```

20.3.15. What is the order of linker script assignment evaluations?

Linker script contains assignment expressions. Assignment expressions can be categorized into 3 distinct categories: before-sections, in-sections and after-sections. We will use the term outside-sections to collectively refer before-sections and after-sections categories. When an assignment expression gets evaluated depends on the assignment expression category.

First, all the assignment expressions outside the SECTIONS commands are evaluated and then the assignment expressions within the SECTIONS command are evaluated. There are few catches, which we will soon explore.

Let’s look at some concrete examples to understand assignment evaluation order behavior.

20.3.15.1. Basic

```
#!/usr/bin/env bash

cat >script.t <<\EOF
bar = 0x3;
SECTIONS {
  baz = bar;
}
EOF

touch 1.c
clang -target hexagon -o 1.o 1.c -c
hexagon-link -o 1.out 1.o -T script.t
hexagon-readelf -s 1.out
# Output:
# Symbol table '.symtab' contains 7 entries:
#   Num:    Value  Size Type    Bind   Vis       Ndx Name
#     // ...
#     // ...
#     5: 00000003     0 NOTYPE  GLOBAL DEFAULT   ABS bar
#     6: 00000003     0 NOTYPE  GLOBAL DEFAULT   ABS baz
```

Both bar and baz have the value 0x3 as expected.

20.3.15.2. Use an after-sections variable to initialize an in-sections variable

```
#!/usr/bin/env bash

cat >script.t <<\EOF
SECTIONS {
  baz = bar;
}
```

```
bar = 0x3;
EOF

touch 1.c
clang -target hexagon -o 1.o 1.c -c
hexagon-link -o 1.out 1.o -T script.t
hexagon-readelf -s 1.out
# Output:
# Symbol table '.symtab' contains 7 entries:
#   Num:    Value  Size Type    Bind   Vis       Ndx Name
#     // ...
#     // ...
#     5: 00000003     0 NOTYPE  GLOBAL DEFAULT   ABS bar
#     6: 00000003     0 NOTYPE  GLOBAL DEFAULT   ABS baz
```

Both bar and baz have the value 0x3 even though at first glance it seems that bar is defined later than baz.

That's not true. Remember that all outside-sections assignments are evaluated before in-sections assignments. Hence, bar is evaluated before baz.

20.3.15.3. Use a variable, that is defined in both before-sections and after-sections assignment, to initialize an in-sections variable

```
#!/usr/bin/env bash

cat >script.t <<\EOF
bar = 0x3;
SECTIONS {
  baz = bar;
}
bar = 0x5;
EOF

touch 1.c
clang -target hexagon -o 1.o 1.c -c
hexagon-link -o 1.out 1.o -T script.t
hexagon-readelf -s 1.out
# Output:
# Symbol table '.symtab' contains 7 entries:
#   Num:    Value  Size Type    Bind   Vis       Ndx Name
#     // ...
#     // ...
#     5: 00000005     0 NOTYPE  GLOBAL DEFAULT   ABS bar
#     6: 00000005     0 NOTYPE  GLOBAL DEFAULT   ABS baz
```

Now, that's a little interesting. If the assignments were evaluated in the natural-order then we would have seen baz value as 0x3 and bar value as 0x5. However, this is not what we have observed.

The reason is that outside-sections assignments are evaluated before in-sections assignments. Hence, the order of evaluations is:

1. bar = 0x3
2. bar = 0x5
3. baz = bar

So remember that the golden rule is outside-sections assignments are evaluated before in-sections assignments.

20.3.15.4. Using a `--defsym` symbol in linker script assignment expression

```
#!/usr/bin/env bash

cat >script.t <<\EOF
SECTIONS {
  baz = bar;
}
```

```
EOF

touch 1.c
clang -target hexagon -o 1.o 1.c -c
hexagon-link -o 1.out 1.o -T script.t --defsym bar=0x5
hexagon-readelf -s 1.out
# Output:
# Symbol table '.symtab' contains 7 entries:
#    Num:    Value  Size Type    Bind   Vis       Ndx Name
#      // ...
#      // ...
#      5: 00000005     0 NOTYPE  GLOBAL DEFAULT   ABS bar
#      6: 00000005     0 NOTYPE  GLOBAL DEFAULT   ABS baz
```

`--defsym` symbols are treated as outside-sections symbols, and hence they are always evaluated before in-sections symbols. This explains the readelf output that we see.

20.3.15.5. What linker script changes are required for supporting thread-local storage (TLS)?

If any of the linker inputs use thread-local storage (TLS), then some linker script changes are required to correctly support thread-local storage functionality.

To properly understand these linker script changes, it is helpful to understand the TLS functionality and how TLS is allocated and initialized.

Thread-local storage functionality makes a variable local to each thread. This means that each thread has its own copy of the variable. This is unlike ordinary global/static variables that are shared across all threads. The runtime library allocates thread-local storage for each thread and it initializes the thread-local storage to the region pointed by the `PT_TLS` segment. Among other things, `PT_TLS` segment describes to the runtime library where to find the initial contents of the thread-local storage and the alignment requirements.

The runtime library needs `PT_TLS` segment for properly initializing TLS region for each block. Thus, the linker needs to emit `PT_TLS` segment for the images that are utilizing TLS functionality. The linker script changes are required for instructing the linker how to properly emit `PT_TLS` segment. In particular, we need to add a `PT_TLS` program header and put the TLS sections (`.tdata` and `.tbss`) into both the `PT_TLS` and a `PT_LOAD` section. For example:

```
PHDRS {
  ...
  DATA PT_LOAD;
  TLS PT_TLS;
  ...
}

SECTIONS {
  ...
  .tbss : { *(*.tbss) } :DATA :TLS
  .tdata : { *(*.tdata) } :DATA :TLS
  .data : { *(*.data) } :DATA // It is important to specify :DATA here
                               // otherwise ':DATA :TLS' will be assumed
  ...
}
```

20.3.16. Why am I getting the warning ‘Space between archive:member file pattern is deprecated’

Linker script allows to specify archive members in input section descriptions using the `<archive-pattern>:<member-pattern>` syntax. For example in the below linker script `SECTIONS` command snippet, `foo_text` output section contains the `.text*` sections from the `foo.o` member of `libfoobar.a` archive.

```
SECTIONS {
  foo_text : { libfoobar.a:foo.o(.text*) }
}
```

Please note that there should be no space between the `<archive-pattern>:` and the `<member-pattern>`. Previously, the linker accepted the space between the two patterns for convenience. This behavior is now deprecated and will be removed in the future.

The below linker script snippet demonstrates some of the cases where the warning will be emitted and why.

```
cat > 1.c << \!
int data = 10;
!

cat > script.t << \!
SECTIONS {
  .data : {
     /* The warning will be displayed because there is a space between
        '*lib1.a:' and '1.o' */
    *lib1.a: 1.o(.*data*)
     /* No warning. When member-pattern is missing, all members of the
        matched archives are matched. */
    /tmp/lib1.a*: (.*data*)
    *lib1.a: (.*data*)
    /* The warning will be displayed because there is a space between
       '*lib1.a:' and '*' */
    *lib1.a: *(.*data*)
    /* No warning. '*lib1.a:' and '*(.data*)' are considered as separate
       input section descriptions. */
    *lib1.a:
    *(.data*)
  }
}
!

clang -c   -c 1.c -g
llvm-ar cr lib1.a 1.o
ld.eld --whole-archive lib1.a -T script.t -Map x -Wlinker-script
```

20.4. Linker Script PHDRS

20.4.1. When should I use PHDRS command

PHDRS command should be used only when you want to control how segments are created by the linker.

If you don’t list a segment in the linker script when using PHDR’s, the segment will not be created by the linker.

What this means, is that the user needs to explicit specify a segment in the linker script for any specific needs of the loader or the image at runtime.

20.4.2. My output file is big!

An output file may become huge or big depending on how you have created the linker script file. Most use cases

- Will show that the user uses the linker script directive PHDRS.
- The user has specified a virtual address hardcoded in the linker script

For figuring out the issue, its always useful to look at the Map file.

Look for the section where the Map file shows a bigger offset than expected. Most often you will need to change the linker script to remove the hardcoded virtual address for the output section.

20.4.3. Loadable section not in any load segment

This error is experienced when the user has a loadable section, but the user has not assigned a PT_LOAD segment for it. You might want to look at the list of segments and the segment assignments for each output section.

20.4.4. Segments need to be mentioned contiguously in the linker script

Segments need to be contiguous in the linker script, or the virtual address assigned to the segment need to be correctly assigned based on where the previous segment ended for that particular segment.

This is because file size and memory size of the segment needs to be calculated correctly, and the segment needs to be declared contiguously.

The file size / memory size is calculated by the difference between the first output section in the segmet and the last output section in the segment.

If there are segments declared in between, they will create these kinds of duplicate sections.

Below is an example that has duplicate sections in the segment list.

```
cat > 1.c << \!
int bss[10000] = { 0 };
int data = 100;
int foo() { return 0; }
!cat > script.t << \!
PHDRS {
  A PT_LOAD;
  B PT_LOAD;
}
SECTIONS {
  .text : { *(.text*) } :A
  .bss : { *(.bss*) } :B
  .data : { *(.data*) } :A
}
!
clang -target hexagon -c 1.c -ffunction-sections -fdata-sections -G0
hexagon-link 1.o -T script.t
```

readelf output:

```
$ readelf -l -W a.outElf file type is EXEC (Executable file)
Entry point 0x0
There are 2 program headers, starting at offset 52Program Headers:
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  LOAD           0x001000 0x00000000 0x00000000 0x09c54 0x09c54 RWE 0x1000
  LOAD           0x00b010 0x00000010 0x00000010 0x00000 0x09c40 RW  0x1000 Section to Segment mapping:
  Segment Sections...
   00     .text .bss .data
   01     .bss
```

20.4.5. Getting file header and program header to be loaded

Often times, it may be required by the runtime loader to inspect the file header and the program header.

You can use FILEHDRS and PHDRS as part of using PHDRS command.

```
cat > 1.c << \!
int data = 20;
int foo() { return 0; }
!

cat > script.t << \!
PHDRS {
  text PT_LOAD FILEHDR PHDRS;
  data PT_LOAD;
}

SECTIONS {
  . = SIZEOF_HEADERS;
  .foo : { *(.text.foo) } :text
  .data : { *(.data.data) } :data
}
!

$CLANG -c 1.c -ffunction-sections -fdata-sections -G0
$LD 1.o -T script.t
```

To verify that the linker did the right thing, you can use readelf output to figure out.

readelf output

```
Elf file type is EXEC (Executable file)
Entry point 0x0
There are 2 program headers, starting at offset 52

Program Headers:
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  LOAD           0x000000 0x00000000 0x00000000 0x0008c 0x0008c R E 0x1000
  LOAD           0x00008c 0x0000008c 0x0000008c 0x00004 0x00004 RW  0x1000

 Section to Segment mapping:
  Segment Sections...
   00     .foo
   01     .data
```

If you see the above, the first load segment starts from offset 0, which means the file header gets loaded when the executable runs.

Often times this may be required for building shared libraries, as the dynamic loader needs the file header and program header for relocating it.

20.4.6. Using AT along with loading file headers and program headers

Users need to be careful when including file headers and program headers to be loaded using PHDRS linker script command, and also set a physical address for the section.

Since the file headers and program headers are loaded, linker needs to account for the physical address when creating the segments.

Below is an example that shows an image built with a linker script and uses AT linker script directive.

PHDRS with AT Command

```
cat > 1.c << \!
int foo() { return 0; }
!

cat > script.t << \!
PHDRS {
  A PT_LOAD FILEHDR PHDRS;
}

SECTIONS {
  . = 0xd820000;
  . = . + SIZEOF_HEADERS;
  .text : AT(0xd820000) { *(.text*) } :A
}
!

clang -c -target aarch64 -c 1.c  -ffunction-sections -fno-asynchronous-unwind-tables
ld.eld -march aarch64 1.o -T script.t
```

Here the linker script shows that the text section has a physical address of 0xd820000, but the user also has said that the FILEHDR and PHDRS to be loaded as part of the load segment.

Linker automatically moves the physical address of the segment to satisfy that the text section has a physical address that can be met as per user needs.

To fix the problem and account for the adjustment of physical addresses, the user needs to make sure that the physical address assigned accounts for the SIZEOF_HEADERS increase.

Recommendation : The user also can drop the usage of AT which seems to be unnecessary to simplify the build step.

Linker Script Fix

```
cat > script.t << \!
PHDRS {
  A PT_LOAD FILEHDR PHDRS;
}

SECTIONS {
  . = 0xd820000;
  . = . + SIZEOF_HEADERS;
  .text : AT(0xd820080) { *(.text*) } :A
}
!
```

20.4.7. Using PHDR flags

Developers can use PHDR flags to convey information from the elf image built to the loader.

The ELF specification provides a convenient way to record this information using phdr flags.

```
#define PF_MASKOS       0x0ff00000      /* OS-specific */
#define PF_MASKPROC     0xf0000000      /* Processor-specific */
```

All bits included in the *PF_MASKOS* mask are reserved for operating system-specific semantics.

All bits included in the PF_MASKPROC mask are reserved for processor-specific semantics. If meanings are specified, the processor supplement explains them.

An example of setting this flag is as below :-

```
PHDRS {
  A PT_LOAD FLAGS (0x03000000);
}

SECTIONS {
.text : { *(.text.*) } :A
}
```

Using the above linker script, you have a segment “A” that contains loadable text but with a OS specific property recorded in the program header.

20.4.8. Loading only the program header

In the above example, we illustrated how an user can load the file header and the program header.

Sometimes it may be required that the user does not want file header but only program header to be loaded.

Below example illustrates the behavior:

Example:

```
cat > 1.c << \!
int data = 20;
int foo() { return 0; }
!

cat > script.t << \!
PHDRS {
  text PT_LOAD PHDRS;
  data PT_LOAD;
}

SECTIONS {
  . = SIZEOF_HEADERS;
  .foo : { *(.text.foo) } :text
  .data : { *(.data.data) } :data
}
!
```

```
$CC -c 1.c -ffunction-sections -fdata-sections -G0
$LD 1.o -T script.t
```

You can inspect the resulting executable to see if program headers have been loaded.

readelf output

```
$ readelf -l -W a.out

Elf file type is EXEC (Executable file)
Entry point 0x0
There are 2 program headers, starting at offset 52

Program Headers:
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  LOAD           0x000034 0x00000034 0x00000034 0x00058 0x00058 R E 0x1000
  LOAD           0x00008c 0x0000008c 0x0000008c 0x00004 0x00004 RW  0x1000

 Section to Segment mapping:
  Segment Sections...
   00     .foo
   01     .data
```

If you see the above, the first load segment starts from offset 34, which means the program header gets loaded when the executable runs.

20.5. MEMORY command: common issues and fixes

20.5.1. Why did an orphan section end up in a different region than GNU ld/LLD?

If your script uses `MEMORY` and a section is not explicitly matched by any rule in `SECTIONS`, it becomes an *orphan output section*. In ELD, allocatable orphans (and other allocatable output sections without an explicit `>REGION`) are auto-assigned based on region attributes (for example, `(rx)` vs `(!rwx)`), scanning regions in `MEMORY` order.

Some other linkers may keep an orphan section in the “current” region/cursor instead of re-running attribute matching, which can produce different results.

Fix:
- Add an explicit output section rule and a `>REGION` assignment for the section (for example, `.rodata` or `.eh_frame`), or discard it explicitly if you do not want it in the image.

See also: https://github.com/qualcomm/eld/issues/127

20.5.2. Why do I get `Error: No memory region assigned to section <name>`?

This occurs when `MEMORY` is present and an allocatable output section is not explicitly assigned to a region and does not match any region’s attribute constraints.

Fix checklist:
- Assign the section explicitly (`.data : { *(.data*) } >RAM`).
- Ensure you have a region whose attributes match the section type (for example, a writable region for `.data`/`.bss`).
- If you are using `PHDRS`, remember that segment assignment (`:phdr`) does not imply a memory region assignment (`>REGION` / `AT>REGION`).

20.5.3. Why does a MEMORY region overflow even though LENGTH looks sufficient?

Region usage is based on placed output section sizes and their addresses, which can include padding due to:
- section alignment,
- page/segment alignment (especially with separate-code / permissions changes),
- header space when `FILEHDR PHDRS` is used,
- gaps introduced by explicit addresses in the script.

Fix:
- Inspect the map file (`-MapStyle txt -Map out.map`) to see per-section addresses and which regions they were charged against.
- Use `--print-memory-usage` to see per-region usage totals.
- Increase the region `LENGTH` or reduce alignment/padding requirements.

20.6. String Merging

20.6.1. Merging order

On 8.7, the linker deduplicates strings according to link order. On >= 8.8 duplicate strings are decided by script rule order. Example below:

```
cat > 1.s << \!
.section .rodata1.str1.1,"aMS",@progbits,1
.string "abc"
!

cat > 2.s << \!
.section .rodata1.str1.2,"aMS",@progbits,1
.string "abc"
!

cat > script.t << \!
SECTIONS {
  .rodata : {
      2.o(.rodata*)
      1.o(.rodata*)
   }
}
!

# link order 1.o, 2.o
# rule order 2.o, 1.o
ld.eld 1.o 2.o
```

On 8.7, the string “abc” from 1.o will be included and the string from 2.o will be merged with it (according to link order). On >=8.8, the string from 2.o will be included and string from 1.o merged with it (according to rule order). This difference is reflected in the map file.

20.7. Hexagon

20.7.1. Convert binary file to Hexagon

If you want to convert a binary file file1.bin to hexagon, you can use objcopy.

```
hexagon-llvm-objcopy -I binary file1.bin -O elf32-hexagon test.o
```

20.7.2. Hexagon specific behavior

20.7.2.1. PHDRS

Hexagon static link and dynamic linked executables dont rely on PHDR’s to be available for the dynamic loader.

If there is a need for you to load PHDR’s, you need to look at Gettingfileheaderandprogramheadertobeloaded

> **Warning**
> **PHDRS not covered by load segment**
>
> Often times, if you are using readelf on a dynamic executable for hexagon, you may get an error
>
> readelf: Error: the PHDR segment is not covered by a LOAD segment
>
> This is because of the above assumption that Hexagon dynamic executables dont rely on PHDR’s to be loaded and available to the dynamic loader.
>
> Use hexagon-llvm-readelf to overcome this error.

20.7.2.2. Why *COMMON* input section description does not affect all the common symbols?

For the hexagon target, by default, the linker maps small common symbols to internal sections, .scommon.x, where x can be 1, 2, 4 and 8. The x represents the size of the common symbol in bytes. For example, .scommon.4 will match common symbols having size 4 bytes. Therefore, to write rules for small common symbols, .scommon.x input section description should be used. This allows writing rules affecting common symbols with greater precision

For Example, to match:

- all common symbols of size less than or equal to 2 bytes to the output section *.small_commons_2*
- all common symbols of size greater than 2 bytes but less than or equal to 8 bytes to the output section *.small_commons_8.*

The following linker script SECTIONS command can be used:

```
SECTIONS {
    .text : {*(.text*) }
    .small_commons_2: { *(.scommon.1 .scommon.2) }
    .small_commons_8: { *(.scommon.4 .scommon.8) }
    .bss: { *(COMMON) }
}
```

*(COMMON) input section description will match all common symbols of size greater than 8 bytes.

To verify the linker script

Test C code

```
char a;
short b;
int c;
long long d;
double e[100];
```

Compile and Link Step:

```
clang -target hexagon -c test.c -o test.o
hexagon-link test.o -o test.elf -T test.linker.script
```

Verify:

```
hexagon-readelf -Ss common.elf
```

readelf output:

```
There are 8 section headers, starting at offset 0x11c0:

Section Headers:
  [Nr] Name              Type            Address  Off    Size   ES Flg Lk Inf Al
  [ 0]                   NULL            00000000 000000 000000 00      0   0  0
  [ 1] .small_commons_2  NOBITS          00000000 001000 000004 00  WA  0   0  2
  [ 2] .small_commons_8  NOBITS          00000008 001000 000010 00  WA  0   0  8
  [ 3] .bss              NOBITS          00000018 001000 000320 00  WA  0   0  8
  [ 4] .comment          PROGBITS        00000000 001000 000065 01  MS  0   0  1
  [ 5] .shstrtab         STRTAB          00000000 001065 00004b 00      0   0  1
  [ 6] .symtab           SYMTAB          00000000 0010b0 0000c0 10      7   6  4
  [ 7] .strtab           STRTAB          00000000 001170 00004a 00      0   0  1
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
  L (link order), O (extra OS processing required), G (group), T (TLS),
  C (compressed), x (unknown), o (OS specific), E (exclude),
  R (retain), p (processor specific)

Symbol table '.symtab' contains 12 entries:
   Num:    Value  Size Type    Bind   Vis       Ndx Name
     0: 00000000     0 NOTYPE  LOCAL  DEFAULT   UND
     1: 00000000     0 SECTION LOCAL  DEFAULT     1 .small_commons_2
     2: 00000000     0 SECTION LOCAL  DEFAULT     4 .comment
     3: 00000008     0 SECTION LOCAL  DEFAULT     2 .small_commons_8
     4: 00000018     0 SECTION LOCAL  DEFAULT     3 .bss
     5: 00000000     0 FILE    LOCAL  DEFAULT   ABS common.c
```

```
     6: 00000000     1 OBJECT  GLOBAL DEFAULT     1 a
     7: 00000002     2 OBJECT  GLOBAL DEFAULT     1 b
     8: 00000008     4 OBJECT  GLOBAL DEFAULT     2 c
     9: 00000010     8 OBJECT  GLOBAL DEFAULT     2 d
    10: 00000018   800 OBJECT  GLOBAL DEFAULT     3 e
    11: 00000339     0 NOTYPE  GLOBAL DEFAULT   ABS __end
```

readelf output shows what we expected:

- symbols a, and b are matched to the output section *.small_commons_2*
- symbols c, and d are matched to the output section *.small_commons_8*
- symbol e is matched to the output section *.bss.*

Note

The linker option -G<size> can be used to specify the maximum size for the small common symbols. For example, -G4 option specifies the maximum size for the small common symbols to be 4 bytes. By default, the linker assumes the maximum size of the small common symbols to be 8 bytes.

20.7.2.3. How to disable small common symbol functionality?

Small common symbol functionality can effectively be disabled by using -G0 linker command-line option. -G<size> option specifies the maximum size for the small common symbols.

-G0 will have the following effects:

- Small common symbols will map to COMMON input section description instead of .scommon.x input section descriptions in the linker script
- If not specified otherwise, small common symbols will be stored in .bss output section instead of .sdata output section.

20.8. RISC-V

20.8.1. Show Linker relaxation output

The linker shows relaxation output by using the option –verbose. In future there will be a better option to annotate where linker performed relaxation.

20.8.2. Disable Relaxation

You can disable relaxation in the linker using the option, –no-relax. This option disables all of linker relaxation except handling of alignment.

20.8.3. Disable compressed relaxation

You can disable linker relaxing to compressed instructions by using –no-c-relax flag.

20.9. Usage

Response Files

Response files let you pass linker arguments via a file referenced with `@`.

Example:

```
ld.eld @x.cmd -o <elf>
```

Notes:

- The response file contains command-line arguments. Any valid linker argument can be passed via the file and is expanded in place of `@x.cmd`.
- Response files help avoid host command-line length limits for very long argument lists.
- A `@response-file` can appear inside another response file (recursive expansion).
- Unrecognized arguments in the file are ignored with a warning (for example `Warning: Unrecognized option '--bad'`).
- Missing response files or invalid arguments cause the link to fail.

Reference: https://llvm.org/docs/CommandLine.html#response-files

20.10. Linker Plugin Framework

20.10.1. What are the differences between SectionMatcherPlugin and SectionIteratorPlugin?

- SectionMatcherPlugin is run before rule-matching and garbage-collection whereas SectionIteratorPlugin is run after rule-matching and garbage-collection.As a consequence of this, SectionIteratorPlugin callbacks have access to garbage-collected sections, discarded sections and rule matching information among other things.
- SectionIteratorPlugin does not call 'SectionIteratorPlugin::Process' callback hook for garbage-collected sections. It is though called for discarded sections. On the other hand, SectionMatcherPlugin call 'SectionMatcherPlugin::Process' callback hook for each input section.

20.11. Symbol Resolution

20.11.1. Symbol wrapping

20.11.1.1. What is symbol wrapping and how to use it?

Symbol wrapping allows to use a wrapper symbol in-place of the original symbol, without any modification to the source code. When symbol wrapping is used for the symbol 'symbol', then all undefined references to 'symbol' is resolved to '__wrap_symbol', and all undefined references to '__real_symbol' is resolved to 'symbol'. To enable symbol wrapping, use '–wrap=symbol' option.

20.11.1.2. Where is symbol wrapping helpful?

Symbol wrapping is typically used for wrapping system/standard function calls. For example, a 'malloc' wrapper function can be used to keep track of bytes allocated by 'malloc', and a 'printf' wrapper function can be used to add a timestamp in the printf output.

20.11.1.3. Example

The below program provides a wrapper function for 'my_malloc' function. The wrapper function prints the number of bytes that is requested by user using 'my_malloc', and then calls 'my_malloc' to do the actual memory allocation.

```
#!/usr/bin/env bash

cat >1.c <<\EOF
#include <stdio.h>

void *my_malloc(size_t sz);

int main() {
  int *p = my_malloc(sizeof(int)); // Resolves to __wrap_my_malloc
  *p = 11;
  printf("p: %d\n", *p);
  return *p;
}
EOF

cat >2.c <<\EOF
#include <stdlib.h>
#include <stdio.h>

void *my_malloc(size_t sz) { return malloc(sz); }

void *__real_my_malloc(size_t sz);
void *__wrap_my_malloc(size_t sz) {
  printf("'%u' bytes requested using my_malloc\n", sz);
  return __real_my_malloc(sz); // Resolves to 'my_malloc'
}
EOF

clang -target hexagon -o 1.wrapped.elf 1.c 2.c 3.c -ffunction-sections -g -Wl,--wrap=my_malloc
hexagon-sim 1.wrapped.elf
```

```
# Output:
# '4' bytes requested using my_malloc
# p: 11
```

20.12. Build time issues

20.12.1. Common to all targets

20.12.1.1. LTO is showing undefined symbol when symbol is defined

The questions to answer when there is an issue like this is to, see if the symbol is in an archive library or an object file.

If its an archive library, make sure that the library is built using **llvm-ar**

The default archiver which handles ELF files does not handle **Bitcode** which is the format that LLVM uses.

This is a must for LTO use.

If you have a reproducer using the reproduce option, you can use this snippet to replace all your archive files using llvm-ar

```
for file in `grep "\.lib" mapping.ini`; do file=`echo $file | sed 's/.*=//'`; echo $file; mkdir -p x; cd x;
cp ../$file .; ar x $file; rm -f $file; llvm-ar cr $file *.o; cp $file ..; cd ..; done
```

20.12.1.2. LTO is showing could not set section name for the symbol

This note is emitted while reading the bitcode files (only part of LTO flow, as in non-LTO flow object files are read) for symbols defined using the '.set' directive.

> **Note**
> extern void someSymbol_relocated (void) __attribute__((__visibility__("hidden")));
>
> __asm(".set someSymbol_relocated, 0x00005730");

This note means that no input section was found for this symbol in the input bitcode file as these symbols are found to be undefined, due to assembler directive usage

The LTO flow, though, happens at link time but doesn't have visibility to mapping symbol address set with .set directive to symbols.

20.13. Hexagon

20.13.1. Relocation overflows to function symbols

A relocation overflow can show up when the linker cannot insert a trampoline to reach that symbol.

Most often the problems would be that the symbol may be *Undefined Weak.*

The programmer would have specified a *__attribute__((weak))* while declaring the function in the header file.

Also look at Weakreferencesanddefinitions why the linker was not seeing the definition.

> **Note**
> A common work around for this sort of problem is to remove the __attribute__((weak)) from the prototype of the function.

20.13.2. Debugging Linker crash issues

20.13.2.1. Step 1

Look for the error message produced by the linker, if it says Unexpected Linker behavior continue to the next step.

If it shows the problem was in the user plugin, debug the plugin and figure out what the issue is.

20.13.2.2. Step 2

Check to see if there any system environment variables set. Important variables to note are :-
- LD_LIBRARY_PATH
- DYLD_LIBRARY_PATH (macOS)
- PATH (Windows)

Remove the values set and see if the error disappears

20.13.2.3. Step 3

If the linker is still crashing, check to see if the cause is due to multithreading plugins.

You can then use –no-threads and see if the error disappears

20.13.2.4. Step 4

Sometimes the plugin may write some data into linker memory and corrupt some data structures.

Build the plugin with sanitizer enabled, and see if you can spot any sanitizer issues in the plugin.

20.13.2.5. Step 5

If it still is crashing then contact the support teams for help debugging the issue.

20.14. ARM

20.14.1. SBREL relocations : Fixing relocation error when applying relocation ‘R_ARM_SBREL32’

SBREL relocation stands for segment base relative relocations. There needs to be only one segment defined that is segment base relative in the image.

All variables and access to the variables need to be placed in that segment.

Example and how to fix :-

```
cat > 1.c << \!
int foo;
int bar = 20;
int main() { return foo + bar; }
!

cat > script.t << \!
PHDRS {
  TEXT PT_LOAD;
  SBREL_SEGMENT PT_LOAD;
  SOMETHING_ELSE PT_LOAD;
}

SECTIONS {
  .text : { *(.text) } :TEXT
  .ARM.exidx : { *(.ARM.exidx*) }
  .data : { *(.data.bar) } :SBREL_SEGMENT
  .bss : { *(COMMON) } :SOMETHING_ELSE
}
!

clang -c -frwpi 1.c -target arm -fdata-sections -g3
ld.eld -march arm 1.o -T script.t -Map x
```

Example error output:

```
Error: R_ARM_SBREL32 Relocation Mismatch for symbol bar defined in 1.o[.text] has a different load segment
Error: Relocation error when applying relocation `R_ARM_SBREL32' for symbol `bar' referred from 1.o[.text] symbol
defined in 1.o[.data.bar]
Error: R_ARM_SBREL32 Relocation Mismatch for symbol bar defined in 1.o[.debug_info] has a different load segment
Error: Relocation error when applying relocation `R_ARM_SBREL32' for symbol `bar' referred from 1.o[.debug_info]
symbol defined in 1.o[.data.bar]
Fatal: Linking had errors (a.out)
```

Fixed linker script for this example

```
PHDRS {
  TEXT PT_LOAD;
  SBREL_SEGMENT PT_LOAD;
  SOMETHING_ELSE PT_LOAD;
}

SECTIONS {
  .text : { *(.text) } :TEXT
  .ARM.exidx : { *(.ARM.exidx*) }
  .data : { *(.data.bar) } :SBREL_SEGMENT
  .bss : { *(COMMON) } :SBREL_SEGMENT
}
```

20.15. Runtime issues

20.15.1. ARM

20.15.2. Crash with load / stores

Things to look at when there is a crash :-

- Look at the point of crash, and the memory from where the instruction is loading from or storing to
- If the load or store is happening with a variable stored in the stack, bounce the problem to the compiler.
- Otherwise, check:
  - Where the variable is located (from the linker map file).
  - Alignment restrictions.
  - TLB permissions.
  - Whether the linker script overrides default alignment for the output section that contains the variable.
  - Page alignment. Some operating systems may require a higher page alignment; you can change the default page size using `-z max-page-size`.

20.16. Improving your image/build

The below section documents how users can improve the image / build by looking into build time warning messages emitted by the linker.

20.16.1. Warnings

20.16.1.1. Convert warning to error

Users can add `-Werror` to convert all warnings that the linker emits into non-fatal errors. A stronger form is `--fatal-warnings`, which promotes warnings to fatal errors.

This is synonymous to using `-Wall -Werror` with the compiler

20.16.1.2. Non allocatable section assigned to an output section

Users might want to avoid assigning a non allocatable section to an output section which is allocatable.

Though this is not a serious issue to be looked into, some of the issues which this will result in are :-

- It may be that tools may be not be able to decode or disassemble contents from the object files properly when there is a bug.
- The build also may become flaky : - When section which converts the output section to be allocatable garbage collected, which will make the section that user added as a non allocatable section.
- Non allocatable sections are not loaded by the linker
- Relocations in non allocatable sections are not performed in the way allocatable sections are treated.

Linker detects that a non allocatable section is being assigned to an output section which is allocatable.

```
cat > code.s << \!
.section .foo
sym:
nop
.section .text
call sym
!
```

```
cat > script.t << \!
SECTIONS {
  .text : {
    *(.text)
    *(.foo)
  }
}
!

$clang -c code.s
$link code.o -T script.t -o code.out
```

Linker would issue the below warning in this case.

> **Warning**
> Warning: Non-allocatable section '.foo' from input file 'code.o' is being merged into loadable output section '.text'

These warnings can be converted to errors with –fatal-warnings switch.

```
$link code.o -T script.t --fatal-warnings
```

> **Error**
> Fatal: Non-allocatable section '.foo' from input file 'code.o' is being merged into loadable output section '.text'

20.17. LinkerPlugin API

20.17.1. Build Errors

20.17.1.1. Why am I receiving the error 'functions that differ only in their return type cannot be overloaded' error while building plugins?

Fix: Build the plugin with '-std=c++14' (or greater C++ standard).

But what is the cause of the error?

The plugin framework overloads member functions based on the reference-qualifier of 'this' argument to provide optimal performance. However, C++11 does not support specifying reference-qualifier for the 'this' argument of member functions. This means that when C++11 is used, C++ cannot distinguish between member function prototypes that only differ due to the reference-qualifier of the 'this' argument.

20.17.1.2. Why am I getting an error about "Plugin Error referenced chunk <seen from last rule> deleted from Output section"

This is a very common error, when a plugin takes chunks from linker script rules, and sometimes the chunk are not put back by the linker plugins.

These chunks may be referred from other locations in the image, and is needed to satisfy image layout requirements.

When such an error happens
- Please debug the plugin by inspecting calls to : - addChunk
  - removeChunk
  - updateChunks

You may also rely on linker map file to see that chunks are added and removed properly by inspecting plugin behavior.

20.17.2. Runtime errors

20.17.2.1. Why am I receiving the error 'Unable to load library ${libYourPlugin.so}: undefined symbol: …'?

Fix: Build the plugin with '-stdlib=libc++'.

But what is the cause of this runtime error?

Plugin framework is compiled with 'libc++' implementation of standard library. Therefore, the mangled names of standard library components in the plugin framework are as per 'libc++' implementation. By default, Clang uses 'libstdc++' implementation of standard library, which has different mangled names of standard library components as compared to 'libc++' implementation. This difference of C++ standard library implementation causes the conflict in the symbol names which in turn causes undefined symbol error.

20.17.3. Concepts

20.17.3.1. What are the differences between SectionMatcherPlugin and SectionIterator-Plugin?

- *SectionMatcherPlugin* interface is run before garbage-collection whereas SectionIteratorPlugin interface is run after rule-matching and garbage-collection. As a consequence of this, SectionIteratorPlugin callbacks have access to the information which sections are garbage-collected and discarded.
- *SectionIteratorPlugin* interface does not call 'SectionIteratorPlugin::Process' callback hook for garbage-collected sections. It is though called for discarded sections. On the other hand, SectionMatcherPlugin call 'SectionMatcherPlugin::Process' callback hook for each input section.

20.17.3.2. What is the search order for plugin invocation configuration file?

Plugin invocation configuration file is the YAML file that is specified to the linker using *–plugin-config <file>* option.

The linker searches this file as follows:
- If the *<file>* is an absolute path, then no search is performed and the path is taken as it is.
- If the *<file>* is a relative path, then the search is performed as follows: : - In the current working directory. i.e. the directory from which linker is invoked.
  - In the search directories specified by the -L option, in the order they are listed in the link command.

20.17.3.3. What is the search order for LinkerWrapper::findConfigFile function?

LinkerWrapper::findConfigFile searches the file as follows:
- If the *<file>* is an absolute path, then no search is performed and the path is taken as it is.
- If the *<file>* is a relative path, then the search is performed as follows: : - In the current working directory. i.e. the directory from which linker is invoked
  - In the search directories specified by the -L option, in the order they are listed in the link command.
  - Plugin default configuration file directory that is shipped with the toolchain. This directory is different for each plugin that is shipped with the toolchain. This directory is searched so that it is convenient to use the default plugin configuration files.

20.17.4. Debugging

20.17.4.1. How to see which plugins are successfully loaded?

Plugin workflow can be traced using the option *'–trace=plugin'.* This option tells the linker to print logs describing the plugin workflow, including information about plugin loading.

20.17.4.2. Is it expected to see decrease in link-time performance when plugins are used?

Yes, user can is expected to see a decrease in link-time performance when
- Plugins are built with a wrong compiler optimization options. : - Using -g in release builds
- Plugins are developed without using Multi-threading features available.

If you are developing plugins and plan to deploy it, use –print-timing-stats to figure out which plugin / what part of the plugin is taking more time during the link step. The plugin framework is feature rich and does not cause link time degradation in any way.

20.17.4.3. How to verify which plugin config file was selected by the linker?

Run the link command with –verbose option, and search the build logs for 'Trying to open. *<plugin config file name>' and '<plugin config file name>*.: found'. These two searches will tell you which directories were searched by the linker (and in which order), and where the plugin configuration file was finally found.

20.17.5. Plugin compatibility

20.17.5.1. Plugin API version

Starting with version 19, linker verifies that the loaded plugin is build using a compatible API version. Linker will refuse to load a plugin which API version is incompatible or the plugin does not report its API version.

Plugin API version consists of major and minor version numbers. For example, the current Plugin API version is 3.2, which can be inspected with the –version linker option:

```
$ ld.eld --version
Supported Targets: hexagon
Linker from QuIC LLVM Hexagon Clang version Version
Linker based on LLVM version: 19.0
Linker Plugin Support Enabled
Linker Plugin Interface Version 3.2
LTO Support Enabled
```

A plugin is compatible with given Plugin API version if their major version numbers are equal and plugin's minor API version is the same or lower than the current minor API version. This means that a plugin compiled for a certain linker API is compatible with future linkers as long as the major version number stays the same.

Plugins must be recompiled to be used with linker with a different major API version.

20.17.5.2. Reporting API version by plugins

Each plugin must report its API version by exporting the following function from the plugin shared library:

```
extern "C" void getPluginAPIVersion(unsigned int *Major, unsigned int *Minor);
```

Plugin API library provides a helper implementation of this function, which as defined in the PluginVersion.h file. To report the plugin API, a plugin developer can include this header file into one of the C++ source files:

```
#include <PluginVersion.h>
```

20.17.6. Linker Plugin Config Search Paths

20.17.6.1. What are the search paths for linker plugin config files?

The *LinkerWrapper::findConfigFile()* API searches for a config file in the following directories in the following order:

- Try to find the file path directly
- Search directories passed to the linker via -L. Some directories are implicitly included, such as the current working directory, and /lib and /usr/lib on linux.
- Default path for config files: *${ORIGIN}/../etc/ELD/Plugins/<Plugin name>*

Note
${ORIGIN} is the directory in which the linker binary resides

20.17.6.2. How to verify that the plugin search config was found and debug related issues?

- The –verbose option will print extra logs to help determine if the plugin config file was found
- To debug issues related to search path for linker plugin file analyze the traces obtained via the below 2 steps: : - To get the list of paths at which the plugin config file path was searched for grep for “Trying to open.*<plugin config file name>” on verbose logs.
  - To see if the linker plugin config file was found grep for “<plugin config file name>.*: found” on verbose logs.

Example:

An example of using the API and verbose option to debug search paths is shown below for linux and for windows:

```
eld::Expected<std::string> expConfigPath =
  getLinker()->findConfigFile("foo.ini");
```

Link command (linux):

```
ld.eld -L/tmp/ ...
```

```
// Search for the file directly
Verbose: Trying to open `foo.ini' for plugin configuration INI file `foo.ini' (file path): not found
// look in -L directories
Verbose: Trying to open `/tmp/foo.ini' for plugin configuration INI file `foo.ini' (search path): not found
// implicit directories (cwd, /lib, /usr/lib)
Verbose: Trying to open `/usr2/aregmi/foo.ini' for plugin configuration INI file `foo.ini'
Verbose: Trying to open `/lib/foo.ini' for plugin configuration INI file `foo.ini' (search path): not found
Verbose: Trying to open `/usr/lib/foo.ini' for plugin configuration INI file `foo.ini' (search path): not found
// try the default config directory
Note: Using default config path for linker plugins
Verbose: Trying to open `/local/mnt/workspace/aregmi/llvm-project/llvm/build/bin/../etc/ELD/Plugins/ConfigFile/
foo.ini' for plugin configuration INI file `foo.ini' (search path): found
Verbose: Found plugin config file /local/mnt/workspace/aregmi/llvm-project/llvm/build/etc/ELD/Plugins/
ConfigFile/foo.ini
```

Link command (Windows):

```
ld.eld -L/ C:\Users\ ...
```

Verbose logs:

```
// Search for the file directly
Verbose: Trying to open `foo.ini' for plugin configuration INI file `foo.ini' (file path): not found
// look in -L directories
Verbose: Trying to open `C:\Users\foo.ini' for plugin configuration INI file `foo.ini' (search path): not found
// implicit directories (cwd)
Verbose: Trying to open `C:\Users\aregmi\tmp\foo.ini' for plugin configuration INI file `foo.ini' (search path):
not found
Note: Using default config path for linker plugins
// default config directory
Verbose: Trying to open `C:\Users\aregmi\install\Tools\bin/../etc/ELD/Plugins/ConfigFile\foo.ini' for plugin
configuration INI file
`foo.ini' (search path): found
Verbose: Found plugin config file C:\Users\aregmi\install\Tools\etc\ELD\Plugins\ConfigFile\config\foo.ini
```

20.17.7. Linker Script NOLOAD handling

20.17.7.1. Description

NOLOAD sections are used as a way to tell the loader to reserve one or more regions in memory and not refresh its contents for repeated loads of the same image.

Assumptions of the regions are
- The region has a reserved virtual address space
- The region has a reserved physical address space
- The region gets translated to “BSS”

This means that the image when it runs, can write some content to the region and re-read their contents.

Examples of use cases are:
- Store state of the application before the application crashes, resume from where it left off
- Store state of why the application crashed for inspection

20.17.7.2. References

https://mcuoneclipse.com/2014/04/19/gnu-linker-can-you-not-initialize-my-variable/

20.17.7.3. Usecases

20.17.7.3.1. NOLOAD region assigned to PT_NULL segment

Code:

```
cat > fb.c << \!
int foo() { return 0; }
int bar() { return 0; }
int baz() { return 0; }
!

cat > noload.lcs << \!
PHDRS {
  A PT_LOAD;
  B PT_NULL;
}

SECTIONS {
  .foo : { *(.text.foo) } :A
  .bar (NOLOAD) : { *(.text.bar) } :B
  .baz (NOLOAD) : { *(.text.baz) } :B
}
!
```

The above test places bar and baz in NOLOAD regions and places them in a segment that is non loadable (PT_NULL).

Readelf Output:

```
$ llvm-readelf -l -W a.out

Elf file type is EXEC (Executable file)
Entry point 0x0
There are 2 program headers, starting at offset 52

Program Headers:
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  LOAD           0x001000 0x00000000 0x00000000 0x0000c 0x0000c R E 0x1000
  NULL           0x000000 0x00000010 0x00000010 0x00000 0x00320 RW  0x1000

 Section to Segment mapping:
  Segment Sections...
   00     .foo
   01     .bar .baz
   None   .comment .shstrtab .symtab .strtab
```

! **Warning**

PT_NULL segments in this case will have the virtual address set and the segment memory size appropriately set. If you do not want to see virtual addresses assigned or memory size set, remove the segment assignment(s).

20.17.7.3.2. Placing a loadable section to PT_NULL

Code:

```
cat > fb.c << \!
int foo() { return 0; }
int bar[100] = { 0 };
int baz[100] = { 0 };
!

cat > noload.lcs << \!
PHDRS {
  A PT_LOAD;
  B PT_NULL;
}

SECTIONS {
  .foo : { *(.text.foo) } :A
  .bar (NOLOAD) : { *(.bss.bar) } :B
  .baz  : { *(.bss.baz) } :B
```

```
}
!
```

The test places baz which is loadable into a PT_NULL segment

Output:

Error
Error: Loadable section .baz not in any load segment Fatal: Linking had errors (a.out)

20.17.7.3.3. Placing NOLOAD region to LOAD segment

Code:

```
cat > fb.c << \!
int foo() { return 0; }
int bar[100] = { 0 };
int baz[100] = { 0 };
!

cat > noload.lcs << \!
PHDRS {
  A PT_LOAD;
  B PT_LOAD;
}

SECTIONS {
  .foo : { *(.text.foo) } :A
  .bar (NOLOAD) : { *(.bss.bar) } :B
  .baz (NOLOAD) : { *(.bss.baz) } :B
}
!
```

This tests produces valid results with a section that is set to NOLOAD and placed in a PT_LOAD segment.

Output:

```
$ llvm-readelf -l -W a.out

Elf file type is EXEC (Executable file)
Entry point 0x0
There are 2 program headers, starting at offset 52

Program Headers:
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  LOAD           0x001000 0x00000000 0x00000000 0x0000c 0x0000c R E 0x1000
  LOAD           0x001010 0x00000010 0x00000010 0x00000 0x00320 RW  0x1000

 Section to Segment mapping:
  Segment Sections...
   00     .foo
   01     .bar .baz
```

20.17.7.3.4. Mixing NOLOAD and LOAD regions

Code:

```
cat > fb.c << \!
__attribute__((aligned(4))) int foo() { return 0; }
__attribute__((aligned(4))) int bar() { return 0; }
__attribute__((aligned(4))) int baz() { return 0; }
__attribute__((aligned(4))) int bad() { return 0; }
!

cat > noload.lcs << \!
PHDRS {
```

```
  A PT_LOAD;
  B PT_LOAD;
}

SECTIONS {
  .foo : { *(.text.foo) } :A
  .bar (NOLOAD) : { *(.text.bar) } :B
  .baz (NOLOAD) : { *(.text.baz) } :B
  .bad  : { *(.text.bad) } :B
}
!
```

This places text.bar and text.baz as NOLOAD and .text.bad as a loadable section but all of them in a LOAD segment.

These are the things that happens
- The text section gets turned into a BSS section (without any warning)
- The text.bad section gets assigned a proper address

Output:

```
$ llvm-readelf -l -W a.out

Elf file type is EXEC (Executable file)
Entry point 0x0
There are 2 program headers, starting at offset 52

Program Headers:
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  LOAD           0x001000 0x00000000 0x00000000 0x0000c 0x0000c R E 0x1000
  LOAD           0x001010 0x00000010 0x00000010 0x0000c 0x0002c R E 0x1000

 Section to Segment mapping:
  Segment Sections...
   00     .foo
   01     .bar .baz .bad
```

20.17.7.3.5. NOLOAD section in the middle of a PT_LOAD segment

```
cat > 1.c << \!
int foo() {
  return 0;
}
int mybss[100] = { 0 };

int bar() {
  return 0;
}
!

cat > script.t << \!
PHDRS {
 A PT_LOAD;
}
SECTIONS {
 .foo : {
   *(.text.foo)
 } :A
 .bss (NOLOAD) : {
   *(.data*)
   *(.bss*)
 } :A
 .bar : {
  *(.text.bar)
 } :A
}
!
```

The example places foo and bar in .foo and .bar output sections respectively. The linker script also reserves a .bss region of type NOLOAD, which is followed by a loadable section .bar. All of the sections are located in a loadable segment by name ‘A’.

The layout is adjusted so that, the section .bar is placed after .bss in the file. The offset for .bar is calculated as the offset of bss and the size of bss.

Output:

```
Program Headers:
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  LOAD           0x001000 0x00000000 0x00000000 0x001ac 0x001ac RWE 0x1000

 Section to Segment mapping:
  Segment Sections...
   00     .f .bss .bar
```

Warning

Without the NOLOAD directive, Linker would have produced this error message

Mixing BSS and non-BSS sections in segment. non-BSS ‘.bar’ is after BSS ‘.bss’ in linker script

20.17.7.3.6. NOLOAD sections start of the segment and PT_NULL segment

Code:

```
int foo() {
  return 0;
}
__attribute__((aligned(16384))) int bss1[100] = { 0 };
__attribute__((aligned(16384))) int bss2[100] = { 0 };
__attribute__((aligned(16384))) int bss3[100] = { 0 };
__attribute__((aligned(16384))) int abss1[100] = { 0 };
__attribute__((aligned(16384))) int abss2[100] = { 0 };
__attribute__((aligned(16384))) int abss3[100] = { 0 };

int bar() {
  return 0;
}
```

LinkerScript:

```
PHDRS {
 A PT_NULL;
}
SECTIONS {
 .abss1 (NOLOAD) : {
   *(.bss.abss1)
 } :A
 .abss2 (NOLOAD) : {
   *(.bss.abss2)
 } :A
 .abss3 (NOLOAD) : {
   *(.bss.abss3)
 } :A
 .f (NOLOAD) : {
   *(.text.foo)
 } :A
 .bss1 (NOLOAD) : {
   *(.bss.bss1)
 } :A
 .bss2 (NOLOAD) : {
   *(.bss.bss2)
 } :A
 .bss3 (NOLOAD) : {
   *(.bss.bss3)
 } :A
```

```
 .bar (NOLOAD) : {
  *(.text.bar)
 } :A
}
```

Commands:

```
$clang -c 1.c -ffunction-sections -fdata-sections -G0
$ld.eld 1.o -T script.t
```

Output:

```
$ readelf -l -W a.out

Elf file type is EXEC (Executable file)
Entry point 0x0
There is 1 program header, starting at offset 52

Program Headers:
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  LOAD           0x000000 0x00000000 0x00000000 0x00000 0x1419c RWE 0x1000

 Section to Segment mapping:
  Segment Sections...
   00     .abss1 .abss2 .abss3 .f .bss1 .bss2 .bss3 .bar
```

Note
Note: the initial offset assigned to the section. This results in a bug with GNU linker.

20.17.7.3.7. NOLOAD sections start of the segment

Code:

```
int foo() {
  return 0;
}
__attribute__((aligned(16384))) int bss1[100] = { 0 };
__attribute__((aligned(16384))) int bss2[100] = { 0 };
__attribute__((aligned(16384))) int bss3[100] = { 0 };
__attribute__((aligned(16384))) int abss1[100] = { 0 };
__attribute__((aligned(16384))) int abss2[100] = { 0 };
__attribute__((aligned(16384))) int abss3[100] = { 0 };

int bar() {
  return 0;
}
```

LinkerScript:

```
PHDRS {
 A PT_LOAD;
}
SECTIONS {
 .abss1 (NOLOAD) : {
   *(.bss.abss1)
 } :A
 .abss2 (NOLOAD) : {
   *(.bss.abss2)
 } :A
 .abss3 (NOLOAD) : {
   *(.bss.abss3)
 } :A
 .f : {
   *(.text.foo)
 } :A
```

```
 .bss1 (NOLOAD) : {
   *(.bss.bss1)
 } :A
 .bss2 (NOLOAD) : {
   *(.bss.bss2)
 } :A
 .bss3 (NOLOAD) : {
   *(.bss.bss3)
 } :A
 .bar : {
  *(.text.bar)
 } :A
}
```

Commands:

```
$clang -c 1.c -ffunction-sections -fdata-sections -G0
$ld.eld 1.o -T script.t
```

Output:

```
$ readelf -l -W a.out

Elf file type is EXEC (Executable file)
Entry point 0x0
There is 1 program header, starting at offset 52

Program Headers:
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  LOAD           0x004000 0x00000000 0x00000000 0x1419c 0x1419c RWE 0x1000

 Section to Segment mapping:
  Segment Sections...
   00     .abss1 .abss2 .abss3 .f .bss1 .bss2 .bss3 .bar
```

20.17.7.3.7.1. How the offset of the section is calculated

- The offset of the first loadable section starts at an offset = previous offset + (current section address - first section in the segment)
- The offset of the first lodable section starts at an offset = previous offset + (current section address - previous section which occupies file space)

20.17.7.3.8. NOLOAD sections start of the segment with first load section starting at a virtual address

Code:

```
  int foo() {
  return 0;
}
__attribute__((aligned(16384))) int bss1[100] = { 0 };
__attribute__((aligned(16384))) int bss2[100] = { 0 };
__attribute__((aligned(16384))) int bss3[100] = { 0 };
__attribute__((aligned(16384))) int abss1[100] = { 0 };
__attribute__((aligned(16384))) int abss2[100] = { 0 };
__attribute__((aligned(16384))) int abss3[100] = { 0 };

int bar() {
  return 0;
}
```

LinkerScript:

```
 PHDRS {
A PT_LOAD;
```

```
}
SECTIONS {
 .abss1 (NOLOAD) : {
   *(.bss.abss1)
 } :A
 .abss2 (NOLOAD) : {
   *(.bss.abss2)
 } :A
 .abss3 (NOLOAD) : {
   *(.bss.abss3)
 } :A
 . = 0x80000000;
 .f : {
   *(.text.foo)
 } :A
 .bss1 (NOLOAD) : {
   *(.bss.bss1)
 } :A
 .bss2 (NOLOAD) : {
   *(.bss.bss2)
 } :A
 .bss3 (NOLOAD) : {
   *(.bss.bss3)
 } :A
 .bar : {
  *(.text.bar)
 } :A
}
```

Commands:

```
$clang -c 1.c -ffunction-sections -fdata-sections -G0
$ld.eld 1.o -T script.t
```

Output:

```
$ readelf -l -W a.out

Elf file type is EXEC (Executable file)
Entry point 0x0
There is 1 program header, starting at offset 52

Program Headers:
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  LOAD           0x004000 0x00000000 0x00000000 0x8000c19c 0x8000c19c RWE 0x1000

 Section to Segment mapping:
  Segment Sections...
   00     .abss1 .abss2 .abss3 .f .bss1 .bss2 .bss3 .ba
```

20.17.7.3.9. NOLOAD sections placed at beginning of segment

When NOLOAD sections are placed at the beginning of the segment, they are not assigned to any segment.

The sections are assigned NOBITS section property.

```
cat > script.t << \!
PHDRS {
  A PT_LOAD;
}

SECTIONS {
  .foo (NOLOAD) : {
            *(.text.foo)
            . = . + 0x4000;
         }
```

```
  .bar (NOLOAD) : { *(.text.bar) }
  .baz : { *(.text.baz) } :A
  /DISCARD/ : { *(.eh_frame) *(.ARM.exidx*) }
}
!

cat > 1.c << \!
int foo() { return 0; }
int bar() { return 0; }
int baz() { return 0; }
!
```

```
$clang -c 1.c -ffunction-sections -fdata-sections -G0
$ld.eld 1.o -T script.t
```

```
$ readelf -l -W a.out

Elf file type is EXEC (Executable file)
Entry point 0x0
There is 1 program header, starting at offset 52

Program Headers:
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  LOAD           0x001020 0x00004020 0x00004020 0x0000c 0x0000c R E 0x1000

 Section to Segment mapping:
  Segment Sections...
   00     .baz
```

20.17.7.3.10. LOAD sections following NOLOAD sections

If there is a LOAD section and the the previous section was a NOLOAD section, the load section is not assigned program headers automatically.

```
cat > script.t << \!
PHDRS {
  A PT_LOAD;
}

SECTIONS {
  .foo (NOLOAD) : {
            *(.text.foo)
            . = . + 0x4000;
         }
  .bar (NOLOAD) : { *(.text.bar) }
  .baz : { *(.text.baz) }
  /DISCARD/ : { *(.eh_frame) *(.ARM.exidx*) }
}
!

cat > 1.c << \!
int foo() { return 0; }
int bar() { return 0; }
int baz() { return
!
```

```
$clang -c 1.c -ffunction-sections
$ld.eld 1.o -T script.t
```

> **Warning**
> Section .baz does not have segment assignment in linker script.

Error
Loadable section .baz not in any load segment

Error
Linking had errors (a.out)

20.17.7.3.11. NOLOAD sections at the beginning of the LOAD segment

NOLOAD sections when assigned to a LOAD segment gets loaded. The section is marked NOBITS.

```
cat > script.t << \!
PHDRS {
  A PT_LOAD;
}

SECTIONS {
  .foo (NOLOAD) : {
            *(.text.foo)
            . = . + 0x4000;
         } :A
  .bar (NOLOAD) : { *(.text.bar) } :A
  .baz : { *(.text.baz) } :A
  /DISCARD/ : { *(.eh_frame) *(.ARM.exidx*) }
}
!

cat > 1.c << \!
int foo() { return 0; }
int bar() { return 0; }
int baz() { return 0; }
!
```

Commands:

```
$clang -c 1.c -ffunction-sections
$ld.eld 1.o -T script.t
```

Output:

```
Section Headers:
  [Nr] Name              Type            Addr     Off    Size   ES Flg Lk Inf Al
  [ 0]                   NULL            00000000 000000 000000 00      0   0  0
  [ 1] .foo              NOBITS          00000000 001000 00400c 00  AX  0   0 16
  [ 2] .bar              NOBITS          00004010 001000 00000c 00  AX  0   0 16
  [ 3] .baz              PROGBITS        00004020 005020 00000c 00  AX  0   0 16
  [ 4] .comment          PROGBITS        00000000 00502c 000106 01  MS  0   0  1
  [ 5] .shstrtab         STRTAB          00000000 005132 000033 00      0   0  1
  [ 6] .symtab           SYMTAB          00000000 005168 0000a0 10      7   6  4
  [ 7] .strtab           STRTAB          00000000 005208 00002f 00      0   0  1
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
  L (link order), O (extra OS processing required), G (group), T (TLS),
  C (compressed), x (unknown), o (OS specific), E (exclude),
  p (processor specific)

Program Headers:
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  LOAD           0x001000 0x00000000 0x00000000 0x0402c 0x0402c R E 0x1000

 Section to Segment mapping:
  Segment Sections...
   00     .foo .bar .baz
```

1. AArch64 Relocation Reference

1.1. Introduction

AArch64 relocation operators such as `:abs_g0_nc:`, `:pg_hi21:`, and `:lo12:` instruct the assembler and linker how to encode symbol references into instruction immediate fields.

1.2. Relocation Conventions

Symbol	Meaning
S	Runtime address of referenced symbol
A	Relocation addend
P	Address of relocation site
X	Relocation result before masking
Page(expr)	expr & ~0xFFF
GOT	Global Offset Table

1.3. Group Relocations

1.3.1. Unsigned Absolute Relocations

Operator	Relocation	Operation	Inst	Bits	Range Check
:abs_g0:	MOVW_UABS_G0	S + A	movz	[15:0]	0 <= X < 2^16
:abs_g0_nc:	MOVW_UABS_G0_NC	S + A	movk	[15:0]	none
:abs_g1:	MOVW_UABS_G1	S + A	movz	[31:16]	0 <= X < 2^32
:abs_g1_nc:	MOVW_UABS_G1_NC	S + A	movk	[31:16]	none
:abs_g2:	MOVW_UABS_G2	S + A	movz	[47:32]	0 <= X < 2^48
:abs_g2_nc:	MOVW_UABS_G2_NC	S + A	movk	[47:32]	none
:abs_g3:	MOVW_UABS_G3	S + A	movz/movk	[63:48]	none

1.3.2. Signed Absolute Relocations

Operator	Relocation	Operation	Inst	Bits	Range Check
:abs_g0_s:	MOVW_SABS_G0	S + A	movz/movn	[15:0]	−2^16 <= X < 2^16
:abs_g1_s:	MOVW_SABS_G1	S + A	movz/movn	[31:16]	−2^32 <= X < 2^32
:abs_g2_s:	MOVW_SABS_G2	S + A	movz/movn	[47:32]	−2^48 <= X < 2^48

1.3.3. Example: Building a 64-bit Constant

```
movz x1, #:abs_g3:u64
movk x1, #:abs_g2_nc:u64
movk x1, #:abs_g1_nc:u64
movk x1, #:abs_g0_nc:u64
```

1.4. PC-Relative Address Relocations

Operator	Relocation	Expression	Instruction	Encoded Bits	Range Check	Alignment
implicit	LD_PREL_LO19	S+A-P	literal ldr	[20:2]	−2^20 <= X < 2^20	X must be a multiple of 4
implicit	LD_PREL_LO21	S+A-P	adr	[20:0]	−2^20 <= X < 2^20	none
:pg_hi21:	ADR_PREL_PG_HI21	Page(S+A)-Page(P)	adrp	[31:12]	−2^32 <= X < 2^32	none
:pg_hi21_nc:	ADR_PREL_PG_HI21_NC	Page(S+A)-Page(P)	adrp	[31:12]	none	none

`LD_PREL_LO19` **alignment:** The literal `ldr` instruction addresses 4-byte-aligned literals; `X` (`S+A-P`) must therefore be divisible by 4. The linker encodes `X >> 2` into the 19-bit field and emits a warning if `X & 3 != 0`.

1.4.1. Typical ADRP + ADD Sequence

```
adrp x0, :pg_hi21:foo
add  x0, x0, #:lo12:foo
```

1.5. Low 12-bit Relocations

Operator	Relocation	Used By	Immediate Extracted
:lo12:	ADD_ABS_LO12_NC	add	X[11:0]
:lo12:	LDST8_ABS_LO12_NC	byte loads/stores	X[11:0]
:lo12:	LDST16_ABS_LO12_NC	halfword loads/stores	X[11:1]
:lo12:	LDST32_ABS_LO12_NC	word loads/stores	X[11:2]
:lo12:	LDST64_ABS_LO12_NC	doubleword loads/stores	X[11:3]
:lo12:	LDST128_ABS_LO12_NC	128-bit accesses	X[11:4]

1.6. Control Flow Relocations

Relocation	Instruction	Expression	Encoded Bits	Reach	Alignment	Out-of-range
TSTBR14	tbz, tbnz	S+A-P	[15:2]	−2^15 <= X < 2^15	X must be a multiple of 4	error
CONDBR19	b.<cond>	S+A-P	[20:2]	−2^20 <= X < 2^20	X must be a multiple of 4	error
JUMP26	b	S+A-P	[27:2]	−2^27 <= X < 2^27	X must be a multiple of 4	trampoline
CALL26	bl	S+A-P	[27:2]	−2^27 <= X < 2^27	X must be a multiple of 4	trampoline

Alignment: All four control-flow relocations encode `X >> 2` into their immediate field, so `X` (`S+A-P`) must be divisible by 4. The linker emits an error if `X & 3 != 0`.

Out-of-range (`JUMP26`/`CALL26`): When the branch target is beyond ±128 MB the linker inserts a trampoline stub for range extension rather than reporting an overflow error. `TSTBR14` and `CONDBR19` have no trampoline support; exceeding their range is a hard error.

1.7. Common Prefix Cheat Sheet

Prefix	Meaning
:abs_g0:	Bits 15:0
:abs_g1:	Bits 31:16
:abs_g2:	Bits 47:32
:abs_g3:	Bits 63:48
_nc	No overflow check
_s	Signed relocation
:pg_hi21:	ADRP page-relative high 21 bits
:lo12:	Low 12 bits

1.8. Large Memory Model Relocations

The small and medium code models typically use ADRP+ADD sequences and are limited by the reach of ADRP. Large memory models use MOVZ/MOVK relocation sequences to construct arbitrary 64-bit addresses.

Relocation	Operator	Instruction	Bits Loaded	Range Check
R_AARCH64_MOVW_UABS_G0	:abs_g0:	movz	[15:0]	0 <= X < 2^16
R_AARCH64_MOVW_UABS_G0_NC	:abs_g0_nc:	movk	[15:0]	none
R_AARCH64_MOVW_UABS_G1	:abs_g1:	movz	[31:16]	0 <= X < 2^32
R_AARCH64_MOVW_UABS_G1_NC	:abs_g1_nc:	movk	[31:16]	none
R_AARCH64_MOVW_UABS_G2	:abs_g2:	movz	[47:32]	0 <= X < 2^48
R_AARCH64_MOVW_UABS_G2_NC	:abs_g2_nc:	movk	[47:32]	none
R_AARCH64_MOVW_UABS_G3	:abs_g3:	movz/movk	[63:48]	none

1.8.1. Example

```
movz x0, #:abs_g3:foo
movk x0, #:abs_g2_nc:foo
movk x0, #:abs_g1_nc:foo
movk x0, #:abs_g0_nc:foo
```

Resulting relocation sequence:

```
R_AARCH64_MOVW_UABS_G3
R_AARCH64_MOVW_UABS_G2_NC
R_AARCH64_MOVW_UABS_G1_NC
R_AARCH64_MOVW_UABS_G0_NC
```

1.8.2. GOT-Based Large Model Relocations

Relocation	Purpose
R_AARCH64_ADR_GOT_PAGE	Compute GOT page address
R_AARCH64_LD64_GOT_LO12_NC	Load GOT entry offset
R_AARCH64_GLOB_DAT	Dynamic linker fills GOT entry
R_AARCH64_JUMP_SLOT	PLT/GOT function resolution

1.8.3. Reach Comparison

Scheme	Reach
ADR	Approximately ±1 MB
ADRP + ADD	Approximately ±4 GB
MOVZ/MOVK + MOVW_UABS_G*	Full 64-bit address space
GOT-based sequence	Full 64-bit address space

1.9. AArch64 Code Models

1.9.1. Small Code Model

Assumptions:
- Code and static data are reachable using PC-relative addressing.
- Symbols are typically accessed using `ADRP + ADD` or `ADRP + LDR/STR`.

Common relocations:

Relocation	Purpose
R_AARCH64_ADR_PREL_PG_HI21	ADRP page calculation
R_AARCH64_ADD_ABS_LO12_NC	ADD low 12 bits
R_AARCH64_LDST8_ABS_LO12_NC	8-bit load/store offset
R_AARCH64_LDST16_ABS_LO12_NC	16-bit load/store offset
R_AARCH64_LDST32_ABS_LO12_NC	32-bit load/store offset
R_AARCH64_LDST64_ABS_LO12_NC	64-bit load/store offset

Example:

```
adrp x0, :pg_hi21:global
add  x0, x0, #:lo12:global
```

1.9.2. Medium Code Model

Assumptions:
- Code remains reachable through branch relocations.
- Data may reside outside the ADRP reach window.
- Large objects are frequently accessed through the GOT.

Common relocations:

Relocation	Purpose
R_AARCH64_ADR_GOT_PAGE	Compute GOT page
R_AARCH64_LD64_GOT_LO12_NC	Access GOT entry
R_AARCH64_GLOB_DAT	Dynamic linker fills GOT
R_AARCH64_JUMP_SLOT	Function resolution via PLT/GOT

Example:

```
adrp x0, :got:symbol
ldr  x0, [x0, #:got_lo12:symbol]
```

1.9.3. Large Code Model

Assumptions:
- Code and data may be located anywhere in the 64-bit address space.
- Full-width addresses are materialized using MOVZ/MOVK relocation sequences.

Common relocations:

Relocation
R_AARCH64_MOVW_UABS_G0
R_AARCH64_MOVW_UABS_G0_NC
R_AARCH64_MOVW_UABS_G1
R_AARCH64_MOVW_UABS_G1_NC
R_AARCH64_MOVW_UABS_G2
R_AARCH64_MOVW_UABS_G2_NC
R_AARCH64_MOVW_UABS_G3

Example:

```
movz x0, #:abs_g3:symbol
movk x0, #:abs_g2_nc:symbol
movk x0, #:abs_g1_nc:symbol
movk x0, #:abs_g0_nc:symbol
```

1.10. References

1.10.1. Official AArch64 ELF ABI Specification

1.10.1.1. Current Source (GitHub)

- ELF for the Arm® 64-bit Architecture (AArch64)
  - https://github.com/ARM-software/abi-aa/blob/main/aaelf64/aaelf64.rst

1.10.1.2. AAELF64 Repository Directory

- https://github.com/ARM-software/abi-aa/tree/main/aaelf64

1.10.2. Recommended Sections

When implementing linker relocations, the most useful sections are:
- Relocation Processing

- Static AArch64 Relocations
- Dynamic Relocations
- GOT and PLT Relocations
- TLS Relocations
- Code Models
- Program Loading and Dynamic Linking

2. ARM Relocation Reference

2.1. Introduction

ARM (32-bit) supports two instruction set architectures: the 32-bit ARM ISA and the 16/32-bit Thumb/Thumb-2 ISA. Relocations encode symbol references into instruction immediate fields and, for branch relocations, into the encoded offset. The linker selects ARM or Thumb encoding based on whether the target symbol is a Thumb function (the T bit).

2.2. Relocation Conventions

Symbol	Meaning
S	Runtime address of the referenced symbol
A	Relocation addend
P	Address of the relocation site (place)
T	1 if the target symbol is a Thumb function, 0 otherwise
B(S)	Base address of the segment containing symbol S
GOT_ORG	Address of the Global Offset Table
GOT(S)	Address of the GOT entry for symbol S
TLS	TLS segment base address
TP	Thread pointer

2.2.1. The T Bit

ARM and Thumb functions are distinguished by the least-significant bit of a branch target. When `T = 1`, the expression `(S + A) \| T` sets bit 0, signalling an interworking branch to a Thumb function. Plain data relocations (`R_ARM_ABS16`, `R_ARM_ABS8`) do not OR in `T`.

2.3. Absolute Relocations

Relocation	Expression	Bits	Range Check
R_ARM_ABS32	(S + A) | T	32	none
R_ARM_ABS16	S + A	16	none
R_ARM_ABS8	S + A	8	none
R_ARM_MOVW_ABS_NC	(S + A) | T	[15:0]	none
R_ARM_MOVT_ABS	S + A	[31:16]	none
R_ARM_THM_MOVW_ABS_NC	(S + A) | T	[15:0]	none
R_ARM_THM_MOVT_ABS	S + A	[31:16]	none

`R_ARM_MOVW_ABS_NC` and `R_ARM_MOVT_ABS` are used in pairs: `MOVW` loads bits `[15:0]` and `MOVT` loads bits `[31:16]` of a 32-bit address.

2.3.1. Example: Loading a 32-bit Address (ARM)

```
movw r0, #:lower16:symbol   @ R_ARM_MOVW_ABS_NC
movt r0, #:upper16:symbol   @ R_ARM_MOVT_ABS
```

2.3.2. Example: Loading a 32-bit Address (Thumb)

```
movw r0, #:lower16:symbol   @ R_ARM_THM_MOVW_ABS_NC
movt r0, #:upper16:symbol   @ R_ARM_THM_MOVT_ABS
```

2.4. PC-Relative Relocations

Relocation	Expression	Bits	Range Check
R_ARM_REL32	((S + A) | T) - P	32	none
R_ARM_SBREL32	((S + A) | T) - P	32	none
R_ARM_PREL31	((S + A) | T) - P	31	none
R_ARM_BASE_PREL	B(S) + A - P	32	none
R_ARM_MOVW_PREL_NC	((S + A) | T) - P	[15:0]	none
R_ARM_MOVT_PREL	S + A - P	[31:16]	none
R_ARM_THM_MOVW_PREL_NC	((S + A) | T) - P	[15:0]	none
R_ARM_THM_MOVT_PREL	S + A - P	[31:16]	none
R_ARM_THM_MOVT_BREL	S + A - P	[31:16]	none
R_ARM_THM_MOVW_BREL_NC	((S + A) | T) - B(S)	[15:0]	none
R_ARM_THM_MOVW_BREL	((S + A) | T) - B(S)	[15:0]	none
R_ARM_ALU_PC_G0	((S + A) | T) - P	top 8 bits, 4-bit rotation	overflow checked
R_ARM_ALU_PC_G0_NC	((S + A) | T) - P	top 8 bits, 4-bit rotation	none (truncates)
R_ARM_ALU_PC_G1	((S + A) | T) - P	top 8 bits, 4-bit rotation	overflow checked
R_ARM_ALU_PC_G1_NC	((S + A) | T) - P	top 8 bits, 4-bit rotation	none (truncates)
R_ARM_ALU_PC_G2	((S + A) | T) - P	top 8 bits, 4-bit rotation	overflow checked
R_ARM_LDR_PC_G2	S + A - P	imm12 (bits 11:0)	[0, 4095]
R_ARM_LDR_PC_G0	S + A - P	imm12 (bits 11:0)	[0, 4095]

`R_ARM_SBREL32` uses the same handler as `R_ARM_REL32` but produces a segment-base-relative offset. `R_ARM_PREL31` is used in ARM exception table entries.

2.5. Branch Relocations

ARM branch instructions encode a signed byte offset divided by 4 into a 24-bit field, giving a range of ±32 MB. Thumb branch instructions encode the offset divided by 2.

2.5.1. ARM Branch

Relocation	Expression	Instruction	Encoded bits	Range
R_ARM_PC24	((S + A) | T) - P	b	24-bit signed (X >> 2)	±32 MB
R_ARM_PLT32	((S + A) | T) - P	b	24-bit signed (X >> 2)	±32 MB
R_ARM_JUMP24	((S + A) | T) - P	b/bx	24-bit signed (X >> 2)	±32 MB
R_ARM_CALL	((S + A) | T) - P	bl/blx	24-bit signed (X >> 2)	±32 MB

2.5.2. Thumb Branch

Relocation	Expression	Instruction	Encoded bits	Range
R_ARM_THM_JUMP8	S + A - P	b (cond)	8-bit signed (X >> 1)	±256 B
R_ARM_THM_JUMP11	S + A - P	b (uncond)	11-bit signed (X >> 1)	±2 KB
R_ARM_THM_JUMP19	((S + A) | T) - P	b.cond (Thumb-2)	21-bit signed (X >> 1)	±1 MB
R_ARM_THM_CALL	((S + A) | T) - P	bl/blx (Thumb-2)	25-bit signed (X >> 1)	±16 MB
R_ARM_THM_JUMP24	((S + A) | T) - P	b (Thumb-2)	25-bit signed (X >> 1)	±16 MB

2.6. GOT-Relative Relocations

Relocation	Expression	Bits	Range Check
R_ARM_GOTOFF32	((S + A) | T) - GOT_ORG	32	none
R_ARM_GOT_BREL	GOT(S) + A - GOT_ORG	12	none
R_ARM_GOT_PREL	GOT(S) + A - P	32	none

2.7. TLS Relocations

Relocation	Expression	Notes
R_ARM_TLS_GD32	GOT(S) + A - P	General Dynamic — two-GOT-entry sequence
R_ARM_TLS_LDM32	GOT(S) + A - P	Local Dynamic — module GOT entry
R_ARM_TLS_IE32	GOT(S) + A - P	Initial Exec — GOT-indirect TP offset
R_ARM_TLS_LDO32	S + A - TLS	Local Dynamic offset within TLS block
R_ARM_TLS_LE32	S + A + 2*WordSize + …	Local Exec — TP-relative offset, static only

2.8. Unsupported Relocations

The table below lists every relocation that ELD’s ARM backend maps to the `unsupport` handler. Applying any of these to a non-dynamic object causes a link error. Relocations not listed here are handled. The **ABI category** column records how the ARM ELF ABI classifies the entry, and the **TODO** column records cases where the linker could reasonably add support.

Type	Relocation	ABI category	TODO
5	R_ARM_ABS16	16-bit absolute	Implement 16-bit absolute
6	R_ARM_ABS12	12-bit absolute (LDR/STR immediate)	Implement ABS12
7	R_ARM_THM_ABS5	Thumb 5-bit absolute (LDR/STR)	
8	R_ARM_ABS8	8-bit absolute	
11	R_ARM_THM_PC8	Thumb 8-bit PC-relative (LDR literal)	
12	R_ARM_BREL_ADJ	Dynamic — adjustment for R_ARM_TLS_DESC	Dynamic only
13	R_ARM_TLS_DESC	Dynamic TLS descriptor	Dynamic only
14	R_ARM_THM_SWI8	Obsolete	
15	R_ARM_XPC25	Obsolete	
16	R_ARM_THM_XPC22	Obsolete	
17	R_ARM_TLS_DTPMOD32	Dynamic TLS module index	Dynamic only
18	R_ARM_TLS_DTPOFF32	Dynamic TLS symbol offset	Dynamic only
19	R_ARM_TLS_TPOFF32	Dynamic TLS thread-pointer offset	Dynamic only
20	R_ARM_COPY	Dynamic copy reloc	Dynamic only
21	R_ARM_GLOB_DAT	Dynamic GOT fill	Dynamic only
22	R_ARM_JUMP_SLOT	Dynamic PLT fill	Dynamic only
23	R_ARM_RELATIVE	Dynamic base-relative	Dynamic only
31	R_ARM_BASE_ABS	Segment-base absolute	
32	R_ARM_ALU_PCREL_7_0	Deprecated (ELF B02 name for ALU_PC_G0_NC)	
33	R_ARM_ALU_PCREL_15_8	Deprecated (ELF B02 name for ALU_PC_G1_NC)	
34	R_ARM_ALU_PCREL_23_15	Deprecated (ELF B02 name for ALU_PC_G2)	
35	R_ARM_LDR_SBREL_11_0_NC	Deprecated section-base relative LDR	
36	R_ARM_ALU_SBREL_19_12_NC	Deprecated section-base relative ALU	
37	R_ARM_ALU_SBREL_27_20_CK	Deprecated section-base relative ALU	
39	R_ARM_SBREL31	Deprecated 31-bit section-relative	
52	R_ARM_THM_JUMP6	Thumb 6-bit branch (CBZ/CBNZ)	
53	R_ARM_THM_ALU_PREL_11_0	Thumb-2 ALU PC-relative 11:0	
54	R_ARM_THM_PC12	Thumb-2 LDR/STR 12-bit PC-relative	
55	R_ARM_ABS32_NOI	32-bit absolute, no interworking bit	
56	R_ARM_REL32_NOI	32-bit PC-relative, no interworking bit	
62	R_ARM_LDR_PC_G1	Group reloc — LDR PC-relative G1	Implement LDR PC-group
64	R_ARM_LDRS_PC_G0	Group reloc — LDRD/STRD PC-relative G0	Implement LDRS PC-group
65	R_ARM_LDRS_PC_G1	Group reloc — LDRD/STRD PC-relative G1	Implement LDRS PC-group
66	R_ARM_LDRS_PC_G2	Group reloc — LDRD/STRD PC-relative G2	Implement LDRS PC-group
67	R_ARM_LDC_PC_G0	Group reloc — LDC/STC PC-relative G0	
68	R_ARM_LDC_PC_G1	Group reloc — LDC/STC PC-relative G1	
69	R_ARM_LDC_PC_G2	Group reloc — LDC/STC PC-relative G2	
70	R_ARM_ALU_SB_G0_NC	Group reloc — ALU section-base G0, no overflow	Implement ALU_SB_G group
71	R_ARM_ALU_SB_G0	Group reloc — ALU section-base G0	Implement ALU_SB_G group

2.9. ABI Comparison Notes

The following discrepancies or gaps were found when comparing this document against the AAELF32 ABI specification:

Area	Observed difference	TODO
R_ARM_TLS_LE32 expression	Document shows S + A + 2*WordSize + …; ABI defines it as S + A - tp (offset from thread pointer)	Fix expression in TLS table to S + A - tp
R_ARM_GOT_BREL bits column	Document shows 12; ABI says the relocation places a 32-bit GOT-relative offset — the 12 refers to LDR's 12-bit immediate encoding, which is an instruction constraint, not the relocation size	Clarify as 32 with a note that the offset must fit in 12 bits
R_ARM_SBREL32	Documented as identical to R_ARM_REL32; ABI marks it as using the segment base B(S) rather than P, so the formula is ((S+A)|T) - B(S) not -P	Fix expression to ((S + A) | T) - B(S)
Thumb branch ranges	R_ARM_THM_CALL and R_ARM_THM_JUMP24 are shown with ±16 MB range; ABI specifies the range depends on whether J1J2 encoding is available (±4 MB without it)	Add conditional range note matching the Thumb Branch table in the Veneers section
R_ARM_TARGET2	Handled by ELD (as target2) but not documented in any section of this file	Add R_ARM_TARGET2 to the PC-relative or absolute table with a note that its behaviour (ABS32 or GOT-relative) is controlled by a linker option
Deprecated relocations	R_ARM_PC24 and R_ARM_PLT32 are listed in the Branch section without noting they are deprecated in favour of R_ARM_CALL/ R_ARM_JUMP24	Add deprecation note

2.10. Veneers (Branch Island Stubs)

When a branch target is out of range or requires an ISA mode switch (ARM ↔ Thumb), the linker inserts a small stub function called a **veneer** (also called a trampoline or branch island). The veneer is placed near the caller and jumps to the real target using an absolute or PC-relative address.

2.10.1. When the Linker Creates Veneers

Caller ISA	Target ISA	Relocation	Condition	Notes
ARM	ARM	R_ARM_PC24, R_ARM_CALL, R_ARM_JUMP24, R_ARM_PLT32	Target out of ±32 MB range	No stub if target is Thumb (T bit = 1)
ARM	Thumb	R_ARM_PC24, R_ARM_JUMP24, R_ARM_PLT32	Always — instruction cannot be rewritten as blx	b/bl/b.w cannot encode ISA switch directly
ARM	Thumb	R_ARM_CALL	Target out of ±32 MB range	bl → blx rewrite is possible in range; stub used only if out of range
Thumb	ARM	R_ARM_THM_JUMP24	Always — Thumb b.w cannot switch ISA	Disabled on microcontroller profiles (Cortex-M0)
Thumb	ARM	R_ARM_THM_CALL	Target out of ±4 MB range (23-bit) or ±16 MB range (25-bit J1J2)	Disabled on microcontroller profiles
Thumb	Thumb	R_ARM_THM_JUMP24, R_ARM_THM_CALL	Target out of ±4 MB range (23-bit) or ±16 MB range (25-bit J1J2)	No stub if target is ARM (T bit = 0)

Microcontroller profile: For CPUs such as Cortex-M0 (and those with the M-profile CPU attribute), Thumb→ARM veneers are never created — the ARM ISA is not available. Thumb→Thumb veneers still use Thumb-1-safe templates (see below).

J1J2 encoding: Thumb-2 processors with J1J2 branch encoding support a 25-bit offset (±16 MB); older Thumb-2 without J1J2 use a 23-bit offset (±4 MB). The linker detects this from the object attribute section.

2.10.2. Branch Range Reference

Caller ISA	Branch encoding	Range
ARM	24-bit signed × 4, bias 8	±32 MB
Thumb	23-bit signed × 2, bias 4	±4 MB
Thumb (J1J2)	25-bit signed × 2, bias 4	±16 MB

2.10.3. Veneer Templates

The linker selects a veneer template based on link-time options and the target CPU profile:

Template	Selected when	Instruction sequence
ABS (default)	Static link, no -fPIC	ldr ip, [pc, #0]; bx ip; dcd target
PIC	Position-independent code (-fPIC / -fPIE)	ldr ip, [pc, #4]; add ip, pc, ip; bx ip; dcd offset
MOV	–use-mov-veneer or microcontroller with MOVT/ MOVW support	movw ip, #lo16; movt ip, #hi16; bx ip
THUMB1	Microcontroller without MOVT/MOVW	push {r0,r1}; ldr r0, [pc, #4]; str r0, [sp,#4]; pop {r0,pc}; dcd target

For Thumb→Thumb veneers the stub body first switches to ARM mode (`bx pc; nop`) before executing the ABS or PIC template, because the ARM load/branch sequence is simpler. The MOV and THUMB1 templates operate entirely in Thumb-2 or Thumb-1 respectively and therefore do not need that prefix.

Because the ABS/PIC Thumb→Thumb stub body switches ISA partway through, the fragment must carry correct ARM-ELF mapping symbols: `$t` at offset 0 (the `bx pc; nop` preamble, still Thumb) and `$a` at offset 4 (where the ARM-mode `ldr`/`add`/`bx` sequence actually begins). Getting these swapped doesn't corrupt the emitted code (the CPU executes the real bytes regardless) but it does corrupt mode-aware disassembly (`objdump -d`, debuggers) for the stub, since those tools trust the mapping symbols rather than re-deriving the mode.

2.11. References

- ELF for the Arm Architecture (AAELF32)
  - https://github.com/ARM-software/abi-aa/blob/main/aaelf32/aaelf32.rst

3. Adding a New Backend to ELD

3.1. Overview

ELD is a modular linker framework supporting multiple target architectures. The **Template backend** (`eld/lib/Target/Template`) is a minimal reference implementation you can copy and adapt for new architectures.

A complete ELD backend consists of three main components:

1. **Backend** - Core linking logic (relocations, GOT/PLT, symbol resolution)
2. **Driver** - Command-line interface and option parsing (optional)
3. **Wrapper** - Plugin API for extending the linker (optional)

This guide focuses on the backend implementation. The driver and wrapper are optional for basic functionality.

3.2. Quick Start

1. Copy the Template backend directory to your new target name
2. Rename all `Template*` files and classes to your target name
3. Implement the key methods in each component

4. Register your backend in the target registry
5. (Optional) Create a driver for command-line support
6. (Optional) Implement wrapper hooks for plugin support

3.3. Template Backend Structure

The Template backend contains the minimum necessary hooks:

```
Template/
├── TemplateInfo.h/cpp              # Target machine type and flags
├── TemplateLDBackend.h/cpp         # Core linker backend
├── TemplateRelocator.h/cpp         # Relocation processing
├── TemplateGOT.h/cpp               # Global Offset Table
├── TemplatePLT.h/cpp               # Procedure Linkage Table
├── TemplateELFDynamic.h/cpp        # Dynamic section
├── TemplateRelocationInfo.h        # Relocation type definitions
├── TemplateRelocationFunctions.h   # Relocation helpers
├── TemplateRelocationCompute.cpp   # Relocation implementations
└── TargetInfo/TemplateTargetInfo.cpp  # Target registration
```

3.4. Available Hooks

The Template backend implements the **minimum necessary hooks**. Additional hooks are available in base classes:

- `eld/Target/GNULDBackend.h` - Backend base class
- `eld/Target/Relocator.h` - Relocator base class
- `eld/Target/TargetInfo.h` - Target info base class
- `eld/Fragment/GOT.h` - GOT base class
- `eld/Fragment/PLT.h` - PLT base class
- `eld/Target/ELFDynamic.h` - Dynamic section base class

Refer to these headers for advanced functionality.

3.5. References

- ELF Specification
- Your target's ABI specification
- Template backend: `eld/lib/Target/Template/`
- Driver: `eld/include/eld/Driver/`
- Wrapper: `eld/lib/LinkerWrapper/`

4. ELD Code Coverage Guide

4.1. LLVM Native Coverage (llvm-cov source-based)

This pipeline uses Clang's source-based coverage instrumentation, which provides finer-grained (expression-level) coverage data.

> **Toolchain requirement**: This pipeline requires the **LLVM/Clang toolchain** (`clang`/`clang++`). It is not compatible with GCC or the GNU toolchain.

The pipeline relies on two artifact types:

- **`.profraw`** — Generated at **runtime** by each executed binary. Contains raw profile counters per function region.
- **`.profdata`** — Merged from one or more `.profraw` files using `llvm-profdata merge`. This is the indexed form that `llvm-cov` consumes.

4.2. Prerequisites

Ensure `llvm-cov` and `llvm-profdata` are available:

```
llvm-cov --version
llvm-profdata --version
```

Export the paths of clang and clang++

```
export CC=clang
export CXX=clang++
```

4.3. Code Coverage Build

Note:

- We use the CMake option `ELD_COVERAGE=ON` which enables the coverage flags (`-fprofile-instr-generate -fcoverage-mapping`).
- `CMAKE_BUILD_TYPE=Release` can also be used for source-based coverage — Clang's instrumentation is inserted before the optimizer runs and is immutable, so optimizations do not affect report accuracy. See Clang source-based coverage docs.

```
cmake -G Ninja \
  -DCMAKE_BUILD_TYPE=Debug \
  -DLLVM_ENABLE_ASSERTIONS:BOOL=ON \
  -DLLVM_ENABLE_PROJECTS='llvm;clang' \
  -DLLVM_EXTERNAL_PROJECTS=eld \
  -DLLVM_EXTERNAL_ELD_SOURCE_DIR=${PWD}/llvm-project/eld \
  -DLLVM_TARGETS_TO_BUILD='ARM;AArch64;RISCV;Hexagon;X86' \
  -DCMAKE_CXX_FLAGS='-stdlib=libc++' \
  -DLLVM_USE_LINKER=lld \
  -DELD_ENABLE_SYMBOL_VERSIONING=ON \
  -DELD_ENABLE_RUN_TESTS=ON \
  -DELD_COVERAGE=ON \
  -B ./obj \
  -S ./llvm-project/llvm
```

Build:

```
cmake --build obj
```

4.4. Generating Code Coverage Report

4.4.1. Step 1 — Run Tests with Profile Output

Set `LLVM_PROFILE_FILE` so every test process writes its raw profile data to a unique file. The `%p` token expands to the process ID, preventing parallel test processes from overwriting each other:

```
LLVM_PROFILE_FILE="$PWD/obj/eld-%p.profraw" cmake --build obj -- check-eld
```

4.4.2. Step 2 — Merge Profile Data

Merge all `.profraw` files into a single indexed `.profdata` file. The `-sparse` flag omits zero counters to keep the output compact:

```
llvm-profdata merge -sparse obj/eld-*.profraw -o obj/eld.profdata
```

4.4.3. Step 3 — Generate HTML Report

Run `llvm-cov show` to produce an annotated HTML report. We pass `libLW.so` as the instrumented object — all ELD lib, include, and tools source code is bundled into `libLW.so` at link time.

The `--include-filename-regex` filter includes only source files whose path contains `/eld/`, covering both `llvm-project/eld/` and cmake-generated headers in `obj/tools/eld/`.

> **Note**: `--include-filename-regex` was introduced in LLVM on 2026-01-27 (#175779) and will be available in LLVM 23 and later. For older toolchains use `--ignore-filename-regex` instead to exclude non-ELD paths. For LLVM < 23, replace `--include-filename-regex='/eld/'` with `--ignore-filename-regex='llvm-project/(llvm|clang)/|third-party/|/obj/tools/[^e]|/obj/tools/e[^l]|/obj/tools/el[^d]|/obj/(include|lib|bin)/|llvm-project/eld/test/|llvm-project/eld/Plugins/HelloWorldPlugin/'`.

```
llvm-cov show \
  --format=html \
  --instr-profile=obj/eld.profdata \
  --object obj/lib/libLW.so \
  --show-branches=count \
  --show-expansions \
  --show-instantiations \
  --include-filename-regex='/eld/' \
  -o coverage-report
```

4.4.4. Step 4 — Quick Coverage Summary

Print line, function, region, and branch coverage percentages:

```
llvm-cov report \
  --instr-profile=obj/eld.profdata \
  --object obj/lib/libLW.so \
  --include-filename-regex='/eld/'
```

5. Hexagon Relocation Reference

5.1. Introduction

Hexagon is a VLIW DSP architecture where all instructions are 32 bits wide. Relocations encode symbol references into instruction immediate fields. The Hexagon ABI supports an extended relocation pair mechanism for large offsets: a non-extended relocation encodes the low 6 bits, while a paired `_X`-suffix relocation encodes the upper bits of the value.

5.2. Relocation Conventions

Symbol	Meaning
S	Runtime address of the referenced symbol
A	Relocation addend
P	Address of the relocation site (place)
GP	Global pointer value
GOT	Address of the Global Offset Table
GOT(S)	Address of the GOT entry for symbol S
TP	Thread pointer

5.2.1. Extended Relocation Pair Mechanism

For branch or absolute references that exceed the reach of a single immediate field, Hexagon uses a two-relocation sequence:

- **`R_HEX_B32_PCREL_X`** (or `R_HEX_32_6_X` for absolute) encodes bits `[31:6]` of the value into a 32-bit instruction immediate using a 26-bit field with shift 6.
- **`R_HEX_B*_PCREL_X`** / **`R_HEX_*_X`** (the non-extended counterpart) encodes bits `[5:0]` of the value into a 6-bit field with no shift.

Relocations with the `_X` suffix are the upper-bits half of such a pair. The lower-bits half uses `R_HEX_6_PCREL_X` (PC-relative) or a corresponding `R_HEX_*_X` variant (absolute).

5.2.2. Table Columns

Column	Meaning
EffectiveBits	Number of significant bits checked after the shift is applied
Shift	Right-shift applied to the value before encoding and before the range check
Signed	Whether the field is treated as a signed two’s-complement value
Range check	Exact inequality the linker enforces on the raw value X; — means no check
Alignment check	Exact divisibility condition the linker enforces on the raw value X; — means no check

How the checks work. Let `X` be the raw relocation value (e.g. `S+A-P`).

- Range (signed, N bits, shift S): `-(2^(N-1) × 2^S) ≤ X < 2^(N-1) × 2^S` — i.e. `X >> S` fits in a signed N-bit field.

- Range (unsigned, N bits, shift S): `0 ≤ X < 2^N × 2^S` — i.e. `X >> S` fits in an unsigned N-bit field.
- Alignment: `X % A == 0` where `A` is the per-relocation alignment requirement shown in the check expression.

5.3. Absolute Relocations

Relocation	Expression	EffectiveBits	Shift	Signed	Range check	Alignment check
R_HEX_32	S + A	32	0	no	—	—
R_HEX_16	S + A	16	0	no	—	—
R_HEX_8	S + A	8	0	no	—	—
R_HEX_LO16	S + A	16	0	no	—	—
R_HEX_HI16	S + A	16	16	no	—	—

`R_HEX_LO16` encodes bits `[15:0]` and `R_HEX_HI16` encodes bits `[31:16]` of `S + A` into an instruction immediate. `R_HEX_HL16` is not supported by ELD.

5.4. PC-Relative Branch Relocations

Expression: `S + A - P`

5.4.1. Non-Extended

Relocation	EffectiveBits	Shift	Signed	Range check	Alignment check
R_HEX_B22_PCREL	22	2	yes	−2^23 ≤ X < 2^23	X % 4 == 0
R_HEX_B15_PCREL	15	2	yes	−2^16 ≤ X < 2^16	X % 4 == 0
R_HEX_B13_PCREL	13	2	yes	−2^14 ≤ X < 2^14	X % 4 == 0
R_HEX_B9_PCREL	9	2	yes	−2^10 ≤ X < 2^10	X % 4 == 0
R_HEX_B7_PCREL	7	2	yes	−2^8 ≤ X < 2^8	X % 4 == 0
R_HEX_32_PCREL	32	0	yes	—	—
R_HEX_6_PCREL_X	6	0	no	—	—

`R_HEX_32_PCREL` is a 32-bit PC-relative relocation used in data (not in instruction immediates). `R_HEX_6_PCREL_X` encodes the low 6 bits of an extended PC-relative pair.

5.4.2. Extended (_X variants — upper bits of a two-relocation sequence)

Relocation	EffectiveBits	Shift	Signed	Range check	Alignment check
R_HEX_B32_PCREL_X	26	6	yes	—	—
R_HEX_B22_PCREL_X	22	0	yes	−2^21 ≤ X < 2^21	—
R_HEX_B15_PCREL_X	15	0	yes	−2^14 ≤ X < 2^14	—
R_HEX_B13_PCREL_X	13	0	yes	−2^12 ≤ X < 2^12	—
R_HEX_B9_PCREL_X	9	0	yes	−2^8 ≤ X < 2^8	—
R_HEX_B7_PCREL_X	7	0	yes	−2^6 ≤ X < 2^6	—

5.4.2.1. Example: Extended B22 Pair

```
; Upper bits: R_HEX_B32_PCREL_X encodes (S+A-P)[31:6]
; Lower bits: R_HEX_6_PCREL_X encodes (S+A-P)[5:0]
call target
```

5.5. GP-Relative Relocations

Expression: `S + A - GP`

Relocation	EffectiveBits	Shift	Signed	Range check	Alignment check
R_HEX_GPREL16_0	16	0	no	0 ≤ X < 2^16	X % 1 == 0 (any)
R_HEX_GPREL16_1	16	1	no	0 ≤ X < 2^17	X % 2 == 0
R_HEX_GPREL16_2	16	2	no	0 ≤ X < 2^18	X % 4 == 0
R_HEX_GPREL16_3	16	3	no	0 ≤ X < 2^19	X % 8 == 0

The suffix `_0` through `_3` reflects the access size: byte, halfword, word, and doubleword respectively.

5.6. Absolute Extended/Truncated Relocations

Expression: `S + A`

`R_HEX_32_6_X` encodes bits `[31:6]` into a 26-bit instruction field (shift 6). The `_X` variants encode only the low 6 bits of the value with no shift, forming the lower half of an absolute extended pair.

Relocation	EffectiveBits	Shift	Signed	Range check	Alignment check
R_HEX_32_6_X	26	6	no	0 ≤ X < 2^32	—
R_HEX_16_X	6	0	no	—	—
R_HEX_12_X	6	0	no	—	—
R_HEX_11_X	6	0	no	—	—
R_HEX_10_X	6	0	no	—	—
R_HEX_9_X	6	0	no	—	—
R_HEX_8_X	6	0	no	—	—
R_HEX_7_X	6	0	no	—	—
R_HEX_6_X	6	0	no	—	—

5.7. Dynamic Relocations

These relocations are resolved by the dynamic linker at load time.

Relocation	Purpose
R_HEX_COPY	Copy relocation — copy symbol from shared library to executable's BSS
R_HEX_GLOB_DAT	Fill a GOT entry with the symbol's runtime address
R_HEX_JMP_SLOT	PLT stub — dynamic linker fills with function address
R_HEX_RELATIVE	Base-relative relocation — add load bias to a stored address

5.8. GOT-Relative Relocations

5.8.1. GOT-offset (Expression: `GOT(S) + A - GOT_ORG`)

Relocation	EffectiveBits	Shift	Signed	Range check	Alignment check
R_HEX_GOTREL_LO16	16	0	yes	—	—
R_HEX_GOTREL_HI16	16	16	yes	—	—
R_HEX_GOTREL_32	32	0	yes	—	—
R_HEX_GOTREL_32_6_X	26	6	yes	—	—
R_HEX_GOTREL_16_X	6	0	no	—	—
R_HEX_GOTREL_11_X	6	0	no	—	—

5.8.2. GOT entry PC-relative (Expression: `GOT(S) + A - P`)

Relocation	EffectiveBits	Shift	Signed	Range check	Alignment check
R_HEX_GOT_LO16	16	0	yes	—	—
R_HEX_GOT_HI16	16	16	yes	—	—
R_HEX_GOT_32	32	0	yes	—	—
R_HEX_GOT_16	16	0	yes	−2^15 ≤ X < 2^15	—
R_HEX_GOT_32_6_X	26	6	yes	—	—
R_HEX_GOT_16_X	6	0	yes	—	—
R_HEX_GOT_11_X	6	0	no	—	—

5.9. TLS Relocations

5.9.1. General Dynamic (GD) and Local Dynamic (LD)

These relocations use a GOT-entry-based indirection. Expression: `GOT(S) + A - P`.

5.9.1.1. GD PLT Call

Relocation	EffectiveBits	Shift	Signed	Range check	Alignment check
R_HEX_GD_PLT_B22_PCREL	22	2	yes	−2^23 ≤ X < 2^23	X % 4 == 0
R_HEX_GD_PLT_B22_PCREL_X	22	0	yes	—	—
R_HEX_GD_PLT_B32_PCREL_X	26	6	yes	—	—

5.9.1.2. GD GOT Access

Relocation	EffectiveBits	Shift	Signed	Range check	Alignment check
R_HEX_GD_GOT_LO16	16	0	yes	—	—
R_HEX_GD_GOT_HI16	16	16	yes	—	—
R_HEX_GD_GOT_32	32	0	yes	—	—
R_HEX_GD_GOT_16	16	0	yes	−2^15 ≤ X < 2^15	X % 4 == 0
R_HEX_GD_GOT_32_6_X	26	6	yes	—	—
R_HEX_GD_GOT_16_X	6	0	no	—	—
R_HEX_GD_GOT_11_X	6	0	no	—	—

The LD variants follow the same structure:

Relocation	EffectiveBits	Shift	Signed	Range check	Alignment check
R_HEX_LD_PLT_B22_PCREL	22	2	yes	−2^23 ≤ X < 2^23	X % 4 == 0
R_HEX_LD_PLT_B22_PCREL_X	22	0	yes	—	—
R_HEX_LD_PLT_B32_PCREL_X	26	6	yes	—	—
R_HEX_LD_GOT_LO16	16	0	yes	—	—
R_HEX_LD_GOT_HI16	16	16	yes	—	—
R_HEX_LD_GOT_32	32	0	no	—	—
R_HEX_LD_GOT_16	16	0	no	0 ≤ X < 2^16	X % 4 == 0
R_HEX_LD_GOT_32_6_X	26	6	yes	—	—
R_HEX_LD_GOT_16_X	6	0	no	—	—
R_HEX_LD_GOT_11_X	6	0	no	—	—

5.9.2. Initial Exec (IE)

Relocation	EffectiveBits	Shift	Signed	Range check	Alignment check
R_HEX_IE_LO16	16	0	yes	—	—
R_HEX_IE_HI16	16	16	yes	—	—
R_HEX_IE_32	32	0	yes	—	—
R_HEX_IE_32_6_X	26	6	yes	—	—
R_HEX_IE_16_X	6	0	no	—	—
R_HEX_IE_GOT_LO16	16	0	yes	—	—
R_HEX_IE_GOT_HI16	16	16	yes	—	—
R_HEX_IE_GOT_32	32	0	yes	—	—
R_HEX_IE_GOT_32_6_X	26	6	yes	—	—
R_HEX_IE_GOT_16_X	6	0	no	—	—
R_HEX_IE_GOT_11_X	6	0	no	—	—
R_HEX_IE_GOT_16	16	0	yes	−2^15 ≤ X < 2^15	X % 4 == 0

5.9.3. Local Exec (LE) — TP-Relative

Relocation	EffectiveBits	Shift	Signed	Range check	Alignment check
R_HEX_TPREL_LO16	16	0	yes	—	—
R_HEX_TPREL_HI16	16	16	yes	—	—
R_HEX_TPREL_32	32	0	yes	—	—
R_HEX_TPREL_16	16	0	yes	−2^15 ≤ X < 2^15	—
R_HEX_TPREL_32_6_X	26	6	yes	—	—
R_HEX_TPREL_16_X	6	0	no	—	—
R_HEX_TPREL_11_X	6	0	no	—	—

5.9.4. DTPREL — Module-Relative (Local Dynamic)

Relocation	EffectiveBits	Shift	Signed	Range check	Alignment check
R_HEX_DTPMOD_32	—	—	—	—	—
R_HEX_DTPREL_LO16	16	0	yes	—	—
R_HEX_DTPREL_HI16	16	16	yes	—	—
R_HEX_DTPREL_32	32	0	yes	—	—
R_HEX_DTPREL_16	16	0	yes	−2^15 ≤ X < 2^15	X % 4 == 0
R_HEX_DTPREL_32_6_X	26	6	yes	—	—
R_HEX_DTPREL_16_X	6	0	no	—	—
R_HEX_DTPREL_11_X	6	0	no	—	—

`R_HEX_DTPMOD_32` is a dynamic relocation; the dynamic linker fills it with the module ID.

5.10. Register Relocations

These relocations encode an address into an instruction’s register-index field.

Relocation	EffectiveBits	Shift	Signed	Range check	Alignment check
R_HEX_23_REG	21	2	no	0 ≤ X < 2^23	—
R_HEX_27_REG	25	2	no	0 ≤ X < 2^27	—

5.11. PLT Relocations

Relocation	EffectiveBits	Shift	Signed	Range check	Alignment check
R_HEX_PLT_B22_PCREL	22	2	yes	−2^23 ≤ X < 2^23	X % 4 == 0

`R_HEX_PLT_B22_PCREL` is used in PLT stubs to call the PLT resolver.

5.12. References

- ELF for the Hexagon Architecture (Hexagon V73 ABI)
  - https://docs.qualcomm.com/bundle/publicresource/topics/80-N2040-1/ELF_for_the_Hexagon_Architecture.html

6. GNU IFunc support

GNU IFunc functionality enables a developer to provide multiple implementations for a function and a resolver function that selects at runtime which implementation to use. It is typically used for selecting the most optimized implementation for a given CPU / runtime. The resolver function is called once at the start of the application runtime and then the selected implementation is fixed. Resolver function is always called eagerly (at program start) instead of lazily (when first needed).

Example:

```
__attribute__((ifunc("foo_resolver")))
int foo(int);

int foo_impl(int u) {
  return u;
}

int (*foo_resolver())(int) {
  return foo_impl;
}

int bar(int u) {
  int v = foo(u); // Calls the function that is returned by foo_resolver
}
```

6.1. Static linking

eld generates a PLT slot for each ifunc symbol. Each PLT slot has a corresponding GOTPLT slot. This model is very similar to how preemptible dynamic symbols are handled. However, instead of doing symbol resolution at runtime to fill the GOTPLT slot, the runtime calls the corresponding resolver function to fill the GOTPLT slot.

For the case of static executables, libc plays the role of runtime and takes on a task that is typically unusual for a libc – resolve symbols and patch the binary at runtime with the resolution information.

eld emits `R_<TARGET>_IRELATIVE` relocations in `.rel[a].plt` section, and `__rel[a]_iplt_start` and `__rela_iplt_end` symbols that store the start and end addresses of `.rel[a].plt` section.

The tiny-loader in the libc process the `R_<TARGET>_IRELATIVE` relocations by iterating over the [`__rel[a]_iplt_start`, `__rel[a]_iplt_end`) range.

```
Relocation section '.rela.plt' at offset 0x190 contains 6 entries:
    Offset             Info             Type               Symbol's Value
0000000000490000  0000000000000408 R_<TARGET>_IRELATIVE              4358c0
0000000000490008  0000000000000408 R_<TARGET>_IRELATIVE              40db20
```

The symbol value of `IRELATIVE` relocations contains the IFunc resolver address, and the relocation offset points to the GOTPLT slot for the IFunc symbol. For each relocation, the tiny-loader in the libc calls the IFunc resolver and stores the result in the GOTPLT slot.

Note that there is no lazy binding here.

6.1.1. Direct reference to an IFunc symbol

Direct references to an IFunc symbol are resolved to the PLT slot of the IFunc symbol.

```
// global variable!
// Absolute relocation, for example, `R_AARCH64_ABS64`, `R_RISCV_64`.
int (*foo_gp)(int) = foo;
```

eld will resolve the absolute relocation to the address of PLT[foo].

6.2. GOT references to an IFunc symbol

GOT references to an IFunc symbol are resolved to the GOTPLT slot of the IFunc symbol when there are no direct references to the IFunc symbol. When there is a direct reference to the IFunc symbol, then the GOT references to the IFunc symbols gets resolved to the GOT slot of the IFunc symbol. It is to ensure pointer equality. Let's understand this in detail:

Case 1: PIC code and no direct reference

```
// foo is an ifunc symbol!
int main() {
  // adrp x0, :got:foo ;  R_AARCH64_GOT_PAGE
  // ldr x0, [x0, :got_lo12:foo] ; R_AARCH64_LD64_GOT_LO12_NC
  int (*foo_lp)(int) = foo;
}
```

Here eld will resolve `adrp + ldr` relocation pair to the address of GOTPLT[foo]. Hence, `ldr` will load the address that is stored in the GOTPLT[foo]. Hence, `foo_lp` will store the address of the resolved function. With this design, there is no indirection penalty for calls to `foo_lp`.

Case 2: PIC code and direct reference

```
// foo is an ifunc symbol!

// R_AARCH64_ABS64
int (*foo_gp)(int) = foo;

int main() {
  // adrp x0, :got:foo ;  R_AARCH64_GOT_PAGE
  // ldr x0, [x0, :got_lo12:foo] ; R_AARCH64_LD64_GOT_LO12_NC
  int (*foo_lp)(int) = foo;
}
```

Here eld will resolve `ABS64` to the address of PLT[foo]. With this, calls to `foo_gp` will work as expected. Note that we cannot resolve `ABS64` to the address of the resolved function because at link time we cannot know the resolved function.

Now, if we resolve `adrp + ldr` relocation pair as before, then `foo_lp` will store the address of the resolved function. This is a problem because `foo_gp` and `foo_lp`, both pointers to the same function `foo`, have different values.

To resolve this, whenever their is a direct reference to an ifunc symbol `foo`, eld creates an additional GOT slot for `foo`, and fill that with the address of the PLT[foo], and resolve all GOT references of `foo` to the GOT[foo] instead of the GOTPLT[foo]. With this design, `foo_lp` will store the address of PLT[foo], the same as `foo_gp`. Hence, no pointer inequality issue.

6.3. IFunc behaviour across all relocations

To describe IFunc behavior for all relocations, we categorize the relocations into the following categories:

- Absolute / PC-relative data relocations
- GOT-related data relocations
- Control flow relocations
- Regular GOT-related instruction relocations
- Regular absolute / PC-relative address-forming relocations
- Address forming relocations used to compute both GOT and non-GOT addresses.
- Absolute / PC-relative load/store relocations

Regular in the relocation category name here means that the relocations are non-TLS non-call relocations.

Any relocation against an IFunc symbol that does not fall into one of the above categories is invalid and produces a warning.

The relocations which are not supported by GNU for IFunc symbols are annotated with NotSupportedInGNULDForIFunc. The GNU toolchain that is used for verifying this is: aarch64-none-linux-gnu-gcc (Arm GNU Toolchain 15.2.Rel1 (Build arm-15.86)) 15.2.1 20251203, and riscv64-unknown-linux-ld (GNU ld (GNU Binutils) 2.43.1).

6.3.1. Absolute / PC-relative data relocations

Resolves to PLT[IFuncSymbol]. Sets HasDirectReference[IFuncSymbol] to true.

- R_AARCH64_ABS{16, 32, 64}
- R_RISCV_{32, 64}
- R_ARM_ABS32
- R_ARM_ABS32_NOI
- R_ARM_TARGET1 (treated as R_ARM_ABS32)

The below relocations are currently unsupported by eld. For IFunc symbols, these should be resolved to the PLT[IFuncSymbol] as well.

- R_AARCH64_PREL{16, 32, 64} (NotSupportedInGNULDForIFunc)
- R_AARCH64_PLT32
- R_RISCV_PLT32
- R_ARM_ABS8
- R_ARM_ABS16
- R_ARM_PREL{16, 32, 64}

6.3.2. GOT-related data relocations

The below relocations are currently unsupported by eld.

- R_AARCH64_GOTREL{32, 64}
- R_AARCH64_GOTPCREL32
- R_RISCV_GOT32_PCREL

6.3.3. Control flow relocations

Resolves to PLT[IFuncSymbol].

- R_AARCH64_TSTBR14
- R_AARCH64_CONDBR19
- R_AARCH64_JUMP26
- R_AARCH64_CALL26
- R_RISCV_CALL
- R_RISCV_CALL_PLT
- R_RISCV_JAL
- R_RISCV_BRANCH
- R_RISCV_RVC_BRANCH
- R_RISCV_RVC_JUMP
- R_ARM_CALL
- R_ARM_JUMP24
- R_ARM_PC24
- R_ARM_PLT32
- R_ARM_THM_CALL
- R_ARM_THM_JUMP24

The below relocations should not be used with IFunc functions because these relocations are used with thumb instructions that does not support interworking (correctly transferring control between thumb-state and ARM-state) and PLT slot is typically always in the ARM state.

- R_ARM_THM_JUMP6 (UNSUPPORTED) (NotSupportedInGNULDForIFunc)
- R_ARM_THM_JUMP19
- R_ARM_THM_JUMP11
- R_ARM_THM_JUMP8

6.3.4. Regular GOT-related instruction relocations

Resolves-to / uses GOTPLT[IFuncSymbol] if there is no direct reference to IFuncSymbol; otherwise uses GOT[IFuncSymbol].

- R_AARCH64_ADR_GOT_PAGE
- R_AARCH64_LD{32,64}_GOT_LO12_NC (LD32 variant UNSUPPORTED)
- R_AARCH64_LD{32,64}_GOTPAGE_LO15 (LD32 variant UNSUPPORTED)
- R_RISCV_GOT_HI20
- R_ARM_GOT_BREL
- R_ARM_GOT_PREL

The below relocations are unsupported by eld:

- R_AARCH64_GOT_LD_PREL19

- AUTH-ABI GOT relocations
- R_AARCH64_MOVW_GOTOFF_G{0,1}{_NC}
- R_ARM_GOT_ABS

6.3.5. Regular absolute / PC-relative address-forming relocations

Resolves to PLT[IFuncSymbol]. Sets HasDirectReference[IFuncSymbol] to true.
- R_AARCH64_ADR_PREL_LO21 (NotSupportedInGNULDForIFunc)
- R_AARCH64_ADR_PREL_PG_HI21{_NC}
- R_AARCH64_ADD_ABS_LO12_NC
- R_RISCV_HI20
- R_RISCV_PCREL_HI20
- R_RISCV_LO12_I
- R_RISCV_LO12_S
- R_ARM_REL32
- R_ARM_PREL31
- R_ARM_MOVW_ABS_NC
- R_ARM_MOVT_ABS
- R_ARM_MOVW_PREL_NC
- R_ARM_MOVT_PREL
- R_ARM_THM_MOVW_ABS_NC
- R_ARM_THM_MOVT_ABS
- R_ARM_THM_MOVW_PREL_NC
- R_ARM_THM_MOVT_PREL

The below relocations are resolved to the IFunc symbol:
- R_AARCH64_MOVW_UABS_G{0,1,2,3}{_NC} (NotSupportedInGNULDForIFunc)
- R_AARCH64_SABS_G{0,1,2,3} (NotSupportedInGNULDForIFunc)

> [!IMPORTANT] FIXME: We should perhaps resolve these relocations to the PLT[IFuncSymbol] instead of the IFuncSymbol for ensuring pointer equality.

The below relocations are currently unsupported in eld:
- R_AARCH64_MOVW_PREL_G{0, 1, 2, 3}{_NC}

6.3.6. Address forming relocations used to compute both GOT and non-GOT addresses

These are the relocations that can be used to form both GOT and non-GOT addresses.

The behavior of these relocations depend upon the corresponding Hi-relocation pair.
- R_RISCV_PCREL_LO12_I
- R_RISCV_PCREL_LO12_S
- There is no relevant AArch64 relocation here.

6.3.7. Absolute / PC-relative load/store relocations

These relocations do not make sense with IFunc symbols. Loading a value at a function address is invalid behavior, and so is storing a value at a function address.
- LD_PREL_LO19 (NotSupportedInGNULDForIFunc)
- LDST{8, 16, 32, 64, 128}_ABS_LO12_NC (NotSupportedInGNULDForIFunc)

> [!IMPORTANT] FIXME: It should be an error to use these relocations with IFunc symbols.

7. RISC-V Relocation Reference

7.1. Introduction

This document describes the RISC-V relocations handled by ELD, including their encoding types, alignment requirements, range checks, and the relaxations that transform instruction sequences at link time.

The official ABI specification is the RISC-V ELF psABI: https://github.com/riscv-non-isa/riscv-elf-psabi-doc/blob/master/riscv-elf.adoc

---

7.2. Relocation Conventions

Symbol	Meaning
S	Runtime address of the referenced symbol
A	Relocation addend
P	Address of the relocation site
G	Address of the global pointer (__global_pointer$)
GOT	Global Offset Table base address
X	Relocation result before masking/encoding

7.3. Encoding Types

Each relocation carries an encoding type that describes which instruction immediate field is written and how the value is packed into it. The "Immediate bits written" column shows which bits of the computed relocation value `X` are placed into which instruction word bits, matching the `encode*` helpers in `RISCVHelper.h`.

7.3.1. Unscrambled (linear) encodings

Encoding	Instruction format	Value bits → instruction bits	Width	Inst size
EncTy_None	No field written	—	—	—
EncTy_8	Data byte	X[7:0] → inst[7:0]	8	8-bit
EncTy_16	Data halfword	X[15:0] → inst[15:0]	16	16-bit
EncTy_32	Data word	X[31:0] → inst[31:0]	32	32-bit
EncTy_64	Data doubleword	X[63:0] → inst[63:0]	64	64-bit
EncTy_6	Byte, low 6 bits	X[5:0] → inst[5:0]	6	8-bit
EncTy_LEB128	ULEB128 data	Variable-length unsigned LEB128	var	variable

7.3.2. 32-bit instruction encodings

Encoding	Instruction format	Value bits → instruction bits	Width
EncTy_I	I-type (addi, lw, …)	X[11:0] → inst[31:20]	12
EncTy_S	S-type (sw, sb, …)	X[11:5] → inst[31:25]X[4:0] → inst[11:7]	12
EncTy_SB	B-type (branch)	X[12] → inst[31]X[10:5] → inst[30:25]X[4:1] → inst[11:8]X[11] → inst[7]	13 (signed, 2-byte aligned)
EncTy_UJ	J-type (jal)	X[20] → inst[31]X[10:1] → inst[30:21]X[11] → inst[20]X[19:12] → inst[19:12]	21 (signed, 2-byte aligned)
EncTy_U_HI20	U-type (lui, auipc)	(X + 0x800)[31:12] → inst[31:12] (carry-adjusted upper 20 bits)	20
EncTy_U_ABS20	Xqci 20-bit U-type	X[19] → inst[31]X[14:0] → inst[30:16]X[18:15] → inst[15:12]	20

`EncTy_U_HI20` carry adjustment: `0x800` is added before masking so that when the low 12-bit `addi` immediate is sign-extended, it correctly reconstructs the original address together with the upper 20 bits.

7.3.3. 16-bit (RVC) instruction encodings

Encoding	Instruction format	Value bits → instruction bits	Width
EncTy_CB	C.B-type (RVC branch)	X[8] → inst[12]X[4:3] → inst[11:10]X[7:6] → inst[6:5]X[2:1] → inst[4:3]X[5] → inst[2]	9 (signed, 2-byte aligned)
EncTy_CJ	C.J-type (RVC jump)	X[11] → inst[12]X[4] → inst[11]X[9:8] → inst[10:9]X[10] → inst[8]X[6] → inst[7]X[7] → inst[6]X[3:1] → inst[5:3]X[5] → inst[2]	12 (signed, 2-byte aligned)
EncTy_CI	C.I-type (c.lui, c.li)	X[5+shift] → inst[12]X[4+shift:shift] → inst[6:2] (shift=12 for c.lui, shift=0 for c.li)	6

7.3.4. 48-bit (Xqci/Xqcilo) instruction encodings

Encoding	Instruction format	Value bits → instruction bits	Width
EncTy_QC_EI	Xqcilo E-I-type load	X[9:0] → inst[29:20]X[25:10] → inst[47:32]	26 (signed)
EncTy_QC_ES	Xqcilo E-S-type store	X[4:0] → inst[11:7]X[9:5] → inst[31:25]X[25:10] → inst[47:32]	26 (signed)
EncTy_QC_EB	Xqci E-B-type branch	same scramble as EncTy_SB in inst[31:0] (the low 32 bits of the 48-bit instruction)	13 (signed, 2-byte aligned)
EncTy_QC_EJ	Xqci E-J-type call	X[31:16] → inst[63:48]X[12] → inst[31]X[10:5] → inst[30:25]X[15:13] → inst[20:17]X[4:1] → inst[11:8]X[11] → inst[7]	32 (signed)
EncTy_QC_EAI	Xqci E-AI-type absolute imm	X[31:0] → inst[47:16]	32 (signed)

7.4. Standard Relocations

7.4.1. Dynamic / Runtime Relocations

These are written by the dynamic linker at load time.

Relocation	Type	Encoding	Operation	Size
R_RISCV_NONE	0	none	—	—
R_RISCV_32	1	EncTy_32	S + A	32
R_RISCV_64	2	EncTy_64	S + A	64
R_RISCV_RELATIVE	3	none	B + A	32
R_RISCV_COPY	4	none	Copy from shared object	32
R_RISCV_JUMP_SLOT	5	none	PLT entry	32
R_RISCV_TLS_DTPMOD32	6	none	TLS module index (32-bit)	32
R_RISCV_TLS_DTPMOD64	7	none	TLS module index (64-bit)	64
R_RISCV_TLS_DTPREL32	8	EncTy_32	TLS DTP offset (32-bit)	32
R_RISCV_TLS_DTPREL64	9	EncTy_64	TLS DTP offset (64-bit)	64
R_RISCV_TLS_TPREL32	10	EncTy_32	TLS TP offset (32-bit)	32
R_RISCV_TLS_TPREL64	11	EncTy_64	TLS TP offset (64-bit)	64

7.4.2. Control Flow Relocations

Relocation	Type	Encoding	Operation	Inst size	Alignment	Range check
R_RISCV_BRANCH	12	EncTy_SB	S + A - P	32-bit	2 bytes	−4096 <= X < 4096, X must be even
R_RISCV_JAL	13	EncTy_UJ	S + A - P	32-bit	2 bytes	−1048576 <= X < 1048576, X must be even
R_RISCV_CALL	14	EncTy_None	S + A - P	32-bit	2 bytes	none (32-bit range via auipc+jalr)
R_RISCV_CALL_PLT	15	EncTy_None	S + A - P (via PLT)	32-bit	2 bytes	none (32-bit range via auipc+jalr)
R_RISCV_RVC_BRANCH	44	EncTy_CB	S + A - P	16-bit	2 bytes	−256 <= X < 256, X must be even
R_RISCV_RVC_JUMP	45	EncTy_CJ	S + A - P	16-bit	2 bytes	−2048 <= X < 2048, X must be even

`R_RISCV_CALL` / `R_RISCV_CALL_PLT`: Applied to an `auipc`+`jalr` pair (8 bytes). The linker splits `X` into a 20-bit upper part (encoded in `auipc`) and a 12-bit signed lower part (encoded in `jalr`), with a carry adjustment when bit 11 of the lower part is set. These relocations are the primary candidates for call relaxation (see Relaxations).

7.4.3. PC-Relative Address Relocations

These always appear in pairs: a HI20 relocation on `auipc` followed by a LO12 relocation on the instruction that uses the result.

Relocation	Type	Encoding	Operation	Inst size	Range check
R_RISCV_PCREL_HI20	19	EncTy_U_HI20	S + A - P	32-bit	none
R_RISCV_PCREL_LO12_I	20	EncTy_I	low 12 bits of paired HI20	32-bit	none
R_RISCV_PCREL_LO12_S	21	EncTy_S	low 12 bits of paired HI20 (stores)	32-bit	none

Pair invariant: The symbol of `R_RISCV_PCREL_LO12_I/S` must point to the address of the corresponding `R_RISCV_PCREL_HI20` relocation site, not the target symbol. The linker resolves the HI20 first, then propagates its computed value to the LO12.

7.4.4. Absolute Address Relocations

Relocation	Type	Encoding	Operation	Inst size	Range check
R_RISCV_HI20	22	EncTy_U_HI20	S + A	32-bit	−2^31 <= X < 2^31 (32-bit range)
R_RISCV_LO12_I	23	EncTy_I	S + A low 12 bits	32-bit	none
R_RISCV_LO12_S	24	EncTy_S	S + A low 12 bits (stores)	32-bit	none

7.4.5. TLS Relocations

Relocation	Type	Encoding	Operation	Inst size
R_RISCV_TPREL_HI20	25	EncTy_U_HI20	S + A - tp	32-bit
R_RISCV_TPREL_LO12_I	26	EncTy_I	low 12 bits of TP offset	32-bit
R_RISCV_TPREL_LO12_S	27	EncTy_S	low 12 bits of TP offset (stores)	32-bit
R_RISCV_TPREL_ADD	28	none	Relaxation hint (no bits written)	32-bit
R_RISCV_TLS_GOT_HI20	17	EncTy_U_HI20	GOT(S) - P	32-bit
R_RISCV_TLS_GD_HI20	18	EncTy_U_HI20	GOT(S) - P (GD model)	32-bit
R_RISCV_TLSDESC_HI20	58	EncTy_U_HI20	TLSDESC(S) - P	32-bit
R_RISCV_TLSDESC_LOAD_LO12	59	EncTy_I	low 12 bits of TLSDESC	32-bit
R_RISCV_TLSDESC_ADD_LO12	60	EncTy_I	low 12 bits of TLSDESC	32-bit
R_RISCV_TLSDESC_CALL	61	none	TLSDESC call hint	32-bit

7.4.6. GOT Relocations

Relocation	Type	Encoding	Operation	Inst size
R_RISCV_GOT_HI20	16	EncTy_U_HI20	GOT(S) - P	32-bit
R_RISCV_GOT32_PCREL	37	EncTy_None	GOT(S) - P (32-bit)	32-bit

7.4.7. Arithmetic / Addend Relocations

Used in debug sections and compact unwind tables to encode label differences.

Relocation	Type	Encoding	Operation	Size
R_RISCV_ADD8	29	EncTy_8	*(loc) += S + A	8
R_RISCV_ADD16	30	EncTy_16	*(loc) += S + A	16
R_RISCV_ADD32	31	EncTy_32	*(loc) += S + A	32
R_RISCV_ADD64	32	EncTy_64	*(loc) += S + A	64
R_RISCV_SUB8	33	EncTy_8	*(loc) -= S + A	8
R_RISCV_SUB16	34	EncTy_16	*(loc) -= S + A	16
R_RISCV_SUB32	35	EncTy_32	*(loc) -= S + A	32
R_RISCV_SUB64	36	EncTy_64	*(loc) -= S + A	64
R_RISCV_SUB6	52	EncTy_6	*(loc)[5:0] -= S + A	8
R_RISCV_SET6	53	EncTy_6	*(loc)[5:0] = S + A	8
R_RISCV_SET8	54	EncTy_8	*(loc) = S + A	8
R_RISCV_SET16	55	EncTy_16	*(loc) = S + A	16
R_RISCV_SET32	56	EncTy_32	*(loc) = S + A	32
R_RISCV_32_PCREL	57	EncTy_32	S + A - P (32-bit)	32
R_RISCV_SET_ULEB128	62	EncTy_LEB128	*(loc) = S + A (ULEB128)	variable
R_RISCV_SUB_ULEB128	63	EncTy_LEB128	*(loc) -= S + A (ULEB128)	variable

7.4.8. Relaxation Hint Relocations

These carry no data; they mark sites eligible for relaxation.

Relocation	Type	Meaning
R_RISCV_RELAX	51	Paired with another relocation to indicate the site may be relaxed
R_RISCV_ALIGN	43	Requests NOP padding to meet an alignment requirement; bytes may be deleted by relaxation
R_RISCV_TPREL_ADD	28	Marks the add in a TPREL_HI20+add+TPREL_LO12 TLS-LE sequence; deleted during relaxation
R_RISCV_VENDOR	255	Vendor extension marker; the following relocation is vendor-defined

7.5. Internal (Linker-Only) Relocations

These relocations are created by ELD during relaxation and are never present in input object files. They are used to apply the final encoding after an instruction sequence has been rewritten.

Relocation	Encoding	Operation	Inst size	Notes
R_RISCV_RVC_LUI	EncTy_CI	S	16-bit	LUI compressed to c.lui
R_RISCV_RVC_LI	EncTy_CI	S	16-bit	Load immediate compressed to c.li
R_RISCV_GPREL_I	EncTy_I	S - G	32-bit	GP-relative I-type load/compute
R_RISCV_GPREL_S	EncTy_S	S - G	32-bit	GP-relative S-type store
R_RISCV_TPREL_I	EncTy_I	S - tp	32-bit	TP-relative I-type after TLS-LE relaxation
R_RISCV_TPREL_S	EncTy_S	S - tp	32-bit	TP-relative S-type after TLS-LE relaxation
R_RISCV_TBJAL	none	—	16-bit	Table-jump (cm.jt/cm.jalt) after call relaxation
R_RISCV_QC_ABS26_I	EncTy_QC_EI	S + A	48-bit	Xqcilo absolute load (26-bit imm)
R_RISCV_QC_ABS26_S	EncTy_QC_ES	S + A	48-bit	Xqcilo absolute store (26-bit imm)
R_RISCV_QC_GPREL26_I	EncTy_QC_EI	S - G	48-bit	Xqcilo GP-relative load
R_RISCV_QC_GPREL26_S	EncTy_QC_ES	S - G	48-bit	Xqcilo GP-relative store
R_RISCV_QC_ABS20_U	EncTy_U_ABS20	S + A	32-bit	Xqci 20-bit absolute
R_RISCV_QC_E_BRANCH	EncTy_QC_EB	S + A - P	48-bit	Xqci extended branch
R_RISCV_QC_E_32	EncTy_QC_EAI	S + A	48-bit	Xqci extended 32-bit immediate
R_RISCV_QC_E_CALL_PLT	EncTy_QC_EJ	S + A - P	48-bit	Xqci extended call (via PLT)
R_RISCV_QC_ACCESS_16	none	—	16-bit	Xqcilo 16-bit access marker
R_RISCV_QC_ACCESS_32	none	—	32-bit	Xqcilo 32-bit access marker

7.6. Relaxations

Relaxation is an iterative pass that shrinks instruction sequences when the final link addresses allow it. Each pass may reduce code size; the linker repeats until no further changes occur.

7.6.1. Relaxation Flags

Flag	Default	Description
–relax / –no-relax	enabled	Master switch; disables all relaxations when off
–no-relax-c	off	Disable compressed-instruction relaxations (c.j, c.jal, c.lui, c.li)
–no-relax-gp	off	Disable GP-relative relaxations (requires __global_pointer$, non-PIC only)
–no-relax-zero	off	Disable zero-page relaxations (symbols with value in signed 12-bit range)
–no-relax-got	off	Disable GOT load relaxations for non-preemptible symbols
–no-relax-tlsdesc	off	Disable TLS descriptor relaxations
–relax-xqci / –no-relax-xqci	disabled	Enable Xqci instruction relaxations (qc.e.j, qc.e.jal)
–relax-tbljal[=zcmt|xqccmt] / –no-relax-tbljal	disabled	Enable table-jump relaxations (cm.jt/cm.jalt)

7.6.2. Call Relaxation (`R_RISCV_CALL` / `R_RISCV_CALL_PLT`)

An `auipc`+`jalr` pair (8 bytes) is shrunk in priority order.

→ **`c.j` / `c.jal` (−6 bytes):** offset fits 12 bits signed, `--no-relax-c` not set; `c.j` when `rd=x0`, `c.jal` when `rd=x1` (RV32 only).

```
# Before (8 bytes)
auipc   ra, %pcrel_hi(func)     # R_RISCV_CALL_PLT
jalr    ra, %pcrel_lo(1b)(ra)

# After (2 bytes)
c.jal   func                    # or c.j func  when rd=x0
```

→ **`cm.jt` / `cm.jalt` (−6 bytes):** `--relax-tbljal` enabled, target has a valid JVT entry, non-PIC.

```
# Before (8 bytes)
auipc   ra, %pcrel_hi(func)     # R_RISCV_CALL_PLT
jalr    ra, %pcrel_lo(1b)(ra)

# After (2 bytes)
cm.jalt  <jvt_index>            # or cm.jt when rd=x0
```

→ **`jal` (−4 bytes):** offset fits 21 bits signed (±1 MiB).

```
# Before (8 bytes)
auipc   ra, %pcrel_hi(func)     # R_RISCV_CALL_PLT
jalr    ra, %pcrel_lo(1b)(ra)

# After (4 bytes)
jal     ra, func                # R_RISCV_JAL
```

→ **`qc.e.j` / `qc.e.jal` (−2 bytes):** `--relax-xqci` enabled, RV32 only.

```
# Before (8 bytes)
auipc   ra, %pcrel_hi(func)
jalr    ra, %pcrel_lo(1b)(ra)

# After (6 bytes)
qc.e.jal  func                  # 48-bit Xqci extended call
```

7.6.3. JAL Relaxation (`R_RISCV_JAL`)

→ **`cm.jt` / `cm.jalt` (−2 bytes):** `--relax-tbljal` enabled, valid JVT entry.

```
# Before (4 bytes)
jal    ra, func              # R_RISCV_JAL

# After (2 bytes)
cm.jalt  <jvt_index>
```

7.6.4. LUI Relaxation (R_RISCV_HI20 + R_RISCV_LO12_*)

A `lui`+LO12 pair is shrunk in priority order.

→ **zero-base (−4 bytes):** symbol value fits signed 12 bits; `--no-relax-zero` not set.

```
# Before (8 bytes)
lui   a0, %hi(var)           # R_RISCV_HI20
addi  a0, a0, %lo(var)       # R_RISCV_LO12_I

# After (4 bytes)
addi  a0, x0, var            # immediate encodes full value
```

→ **GP-relative (−4 bytes):** symbol within `gp ± 2048` `--no-relax-gp` not set, non-PIC.

```
# Before (8 bytes)
lui   a0, %hi(var)           # R_RISCV_HI20
lw    a0, %lo(var)(a0)       # R_RISCV_LO12_I

# After (4 bytes)
lw    a0, var(gp)            # R_RISCV_GPREL_I
```

→ **`c.lui` (−2 bytes):** immediate fits 6-bit non-zero CI field; `rd ≠ x0`, `rd ≠ x2` `--no-relax-c` not set.

```
# Before (4 bytes)
lui   a0, %hi(var)           # R_RISCV_HI20

# After (2 bytes)
c.lui a0, %hi(var)           # R_RISCV_RVC_LUI
```

7.6.5. PC-Relative to GP Relaxation (R_RISCV_PCREL_HI20 + R_RISCV_PCREL_LO12_I)

→ **GP-relative (−4 bytes):** symbol within `gp ± 2048` `--no-relax-gp` not set, non-PIC.

```
# Before (8 bytes)
auipc a0, %pcrel_hi(var)     # R_RISCV_PCREL_HI20
lw    a0, %pcrel_lo(1b)(a0)  # R_RISCV_PCREL_LO12_I

# After (4 bytes)
lw    a0, var(gp)            # R_RISCV_GPREL_I
```

7.6.6. GOT Relaxation (R_RISCV_GOT_HI20 + R_RISCV_PCREL_LO12_I)

For non-preemptible symbols the GOT indirection is bypassed.

→ **`c.li` (−6 bytes):** symbol value fits 6-bit signed CI field.

```
# Before (8 bytes)
auipc a0, %pcrel_hi(sym@GOT)   # R_RISCV_GOT_HI20
lw    a0, %pcrel_lo(1b)(a0)    # R_RISCV_PCREL_LO12_I  (GOT load)

# After (2 bytes)
c.li  a0, sym                  # R_RISCV_RVC_LI
```

→ **`addi x0, imm` (−4 bytes):** symbol value fits 12-bit signed.

```
# Before (8 bytes)
auipc a0, %pcrel_hi(sym@GOT)
lw    a0, %pcrel_lo(1b)(a0)
```

```
# After (4 bytes)
addi  a0, x0, sym              # immediate encodes full value
```

→ `auipc` + `addi` PCREL (0 bytes): symbol fits PC-relative 32-bit range; GOT load converted to direct address calculation.

```
# Before (8 bytes)
auipc a0, %pcrel_hi(sym@GOT)   # R_RISCV_GOT_HI20
lw    a0, %pcrel_lo(1b)(a0)    # GOT load

# After (8 bytes)
auipc a0, %pcrel_hi(sym)       # R_RISCV_PCREL_HI20
addi  a0, a0, %pcrel_lo(1b)    # R_RISCV_PCREL_LO12_I  (no GOT access)
```

7.6.7. TLSDESC Relaxation

For symbols with local/static TLS, the TLSDESC call sequence is replaced with a direct TP-relative access. Disabled by `--no-relax-tlsdesc`.

```
# Before: GD/TLSDESC (16 bytes, calls runtime)
auipc  a0, %tlsdesc_hi(x)      # R_RISCV_TLSDESC_HI20
lw     t0, %tlsdesc_lo(1b)(a0) # R_RISCV_TLSDESC_LOAD_LO12
addi   a0, a0, %tlsdesc_lo(1b) # R_RISCV_TLSDESC_ADD_LO12
jalr   t0, t0(a0)              # R_RISCV_TLSDESC_CALL

# After: TLS-LE (8–12 bytes, no runtime call)
lui    a0, %tprel_hi(x)        # R_RISCV_TPREL_HI20   (omitted if fits 12-bit)
add    a0, a0, tp              # R_RISCV_TPREL_ADD
lw     a0, %tprel_lo(x)(a0)    # R_RISCV_TPREL_LO12_I
```

7.6.8. Zcmt / Xqccmt Table-Jump Relaxation (`--relax-tbljal`)

The Zcmt extension introduces a 64-entry Jump Vector Table (`.riscv.jvt`), a 64-byte aligned array of function-pointer-sized words. At link time ELD builds this table from all call targets that appear more than once, then replaces each call site with a 2-byte `cm.jalt N` (return via `ra`) or `cm.jt N` (tail call, no link register written). The index `N` is the zero-based slot in the JVT where the target address is stored.

Xqccmt uses the same 16-bit encoding under the names `qc.cm.jalt` / `qc.cm.jt`, and additionally supports writing the return address into `t0` (x5) by setting bit 0 in the JVT entry.

`.riscv.jvt` section layout (4 entries shown, 32-bit target):

```
.section .riscv.jvt,"a",@progbits
.align 6                         # 64-byte alignment (Zcmt requirement)
jvt_base:
  .word  foo                     # entry 0 — target of cm.jalt 0 / cm.jt 0
  .word  bar                     # entry 1
  .word  baz                     # entry 2
  .word  qux                     # entry 3
  ...                            # up to 64 entries
```

Call site relaxation (−6 bytes per site, from `auipc`+`jalr` to `cm.jalt`):

```
# Before (8 bytes): called at many sites throughout the binary
auipc  ra, %pcrel_hi(foo)        # R_RISCV_CALL_PLT
jalr   ra, %pcrel_lo(1b)(ra)

# After (2 bytes): JVT entry 0 holds foo's address
cm.jalt  0                       # R_RISCV_TBJAL — loads PC from jvt_base[0], links ra
```

Tail-call relaxation (−6 bytes, `cm.jt` when `rd=x0`):

```
# Before (8 bytes)
auipc  x0, %pcrel_hi(baz)
jalr   x0, %pcrel_lo(1b)(x0)

# After (2 bytes): JVT entry 2 holds baz's address
cm.jt  2                        # tail call — no link register written
```

JAL → `cm.jalt` (−2 bytes):

```
# Before (4 bytes): already within ±1 MiB but target is in the JVT
jal    ra, bar                  # R_RISCV_JAL

# After (2 bytes)
cm.jalt  1                      # R_RISCV_TBJAL
```

The linker only builds JVT entries for targets where the total bytes saved across all call sites exceeds the 4-byte cost of the JVT entry itself plus any required padding to meet the 64-byte section alignment.

7.6.9. ALIGN Relaxation (`R_RISCV_ALIGN`)

NOP padding inserted by the assembler to satisfy `.align` directives is removed by the linker after relaxation shrinks preceding code. The linker recalculates required padding and deletes any excess NOP bytes.

```
# Before (assembler-inserted NOPs, e.g. .align 4 after a 2-byte instruction)
c.j    target                 # 2 bytes
nop                           # 2 bytes — R_RISCV_ALIGN padding
nop                           # 2 bytes — R_RISCV_ALIGN padding
aligned_label:                # now at 4-byte boundary

# After (if relaxation has already re-aligned the section)
c.j    target                 # 2 bytes
nop                           # 2 bytes — only one NOP needed
aligned_label:
```

7.7. Common Instruction Sequences

7.7.1. Absolute Address (HI20 + LO12)

```
lui   a0, %hi(symbol)        # R_RISCV_HI20
addi  a0, a0, %lo(symbol)    # R_RISCV_LO12_I
```

7.7.2. PC-Relative Address (AUIPC + LO12)

```
auipc a0, %pcrel_hi(symbol)  # R_RISCV_PCREL_HI20
addi  a0, a0, %pcrel_lo(1b)  # R_RISCV_PCREL_LO12_I
```

7.7.3. Function Call (CALL → relaxed to JAL or compressed)

```
auipc ra, %pcrel_hi(func)    # R_RISCV_CALL / R_RISCV_CALL_PLT
jalr  ra, %pcrel_lo(1b)(ra)  # (paired with auipc)
```

After relaxation (if target within ±1 MiB):

```
jal   ra, func               # R_RISCV_JAL
```

7.7.4. TLS Local-Exec (TPREL)

```
lui   a0, %tprel_hi(x)       # R_RISCV_TPREL_HI20
add   a0, a0, tp, %tprel_add(x)  # R_RISCV_TPREL_ADD
lw    a0, %tprel_lo(x)(a0)   # R_RISCV_TPREL_LO12_I
```

7.8. References

- RISC-V ELF psABI specification: https://github.com/riscv-non-isa/riscv-elf-psabi-doc/blob/master/riscv-elf.adoc
- RISC-V ISA specification: https://github.com/riscv/riscv-isa-manual
- RISC-V Compressed extension (RVC) — Volume I, Chapter 16
- Zcmt table-jump extension: https://github.com/riscv/riscv-isa-manual

8. Symbol versioning

All symbol versioning related functionality is guarded by the `ELD_ENABLE_SYMBOL_VERSIONING` build macro.

8.1. `--default-symver`

`--default-symver` assigns the output's default version to any exported dynamic symbol that does not already have an explicit version. The default version name is the output `DT_SONAME` for shared libraries when `-soname` is used; otherwise it is the basename of the output file.

When `--default-symver` is active for a supported link mode, eld always creates the synthetic default version node. This is true even when no regular dynamic symbol is assigned to that version, such as a hidden-only shared library, a dynamic executable without `-E`, or a static PIE without `-E`.

The ordinary base version definition is emitted by the existing version definition machinery whenever named version definitions are present. This option additionally adds the synthetic default version node.

eld models this option as an internal version script node named after the default version name with `global: *;`. The node is registered before user version scripts so explicit user version-script rules take precedence.

eld creates the default symbol version node for link modes that can produce dynamic symbol/version information. These include:

- shared libraries;
- dynamic executables, including links forced dynamic with `--force-dynamic`
- code-independent executables, including PIE and static PIE.

Regular static non-PIE executables do not create the node.

If `ELD_ENABLE_SYMBOL_VERSIONING` is disabled, eld warns that `--default-symver` is unsupported and does not synthesize the node.

8.2. Symbol resolution of versioned symbols

A versioned symbol is of two types:

- **default** versioned symbol (`bar@@V1`): a default versioned symbol satisfies undefined references of both the versioned and unversioned references of the symbol. `bar@@V1` definition can satisfy undefined references of both `bar@V1` and plain `bar`.
- **non-default** versioned symbol (`bar@V1`): a non-default versioned symbol only satisfies undefined references of the versioned references of the symbol. `bar@V1` definition can only satisfy undefined references of `bar@V1`.

eld supports symbol resolution of versioned symbols by inserting two symbols for default-versioned symbols into the already-existing symbol resolution machinery and adding a symbol normalization phase that runs after symbol resolution to combine the two symbol nodes for the default-versioned symbols into one, if required.

Whenever eld sees a non-default-versioned symbol defintion `bar@V1`, then eld inserts `bar@V1` to the symbol resolution machinery, and it automatically gets used to resolve `bar@V1` undefined references.

Here is the interesting part, whenever eld sees a default-versioned symbol definition `bar@@V1`, then eld inserts two symbols to the symbol resolution machinery: a canonical symbol `bar@V1` and a non-canonical symbol `bar`. With this, `bar@@V1` definition resolves undefined references to both `bar` and `bar@V1`, as it should.

One fundamental rule of versioned-symbol resolution is that, for a default versioned symbol, both the canonical symbol (`foo@V1`) and the non-canonical (unversioned symbol `foo`) must resolve to the same definition. Allowing them to resolve to different symbols is incorrect because, the unversioned symbol is simply an alias for the versioned symbol (`foo@V1`), and both are originally the same symbol (`foo@@V1`).

8.2.1. Why is symbol normalization required?

As we saw above, for a default-versioned symbol definition `bar@@V1`, eld creates two symbols `bar@V1` and `bar`. Some undefined symbol references may have resolved to `bar@V1` definition and some may have resolved to `bar` definition. This is a problem because in reality there is only one symbol `bar@@V1`. If we keep two symbol nodes per default-versioned symbol definition, then we will need to also emit both these symbols into the symbol table and create duplicate GOT/PLT slots for these symbols.

Symbol normalization resolves this duplicate symbol issue by rewriting all the references to non-canonical symbols (`bar`) into the canonical symbols (`bar@V1`). A key thing to note here is that we only require the non-canonical aliases during symbol resolution phase.

LLD does not always normalize the symbols and thus creates duplicate GOT/PLT slots for the same symbol when a default versioned symbol is accessed as both plain unversioned reference and versioned reference.

Reproducer:

```
#!/usr/bin/env bash

cat >1.c <<\EOF
__asm__(".symver foo1,foo@@V1");
int foo1() {
  return 1;
}
EOF

cat >vs.t <<\EOF
V1 {
  global:
    foo;
};
EOF

cat >main.c <<\EOF
#include <stdio.h>

int foo();

__asm__(".symver foov1, foo@V1");
int foov1();

int main() {
  int u = foo() + foov1();
  printf("u: %d\n",u);
  return 0;
}
EOF

clang-20 -o 1.o 1.c -c -fPIC
clang-20 -o main.o main.c -c -fPIC

LDs=(ld.eld ld.lld ld.bfd)
SFs=(eld lld bfd)

for i in "${!SFs[@]}"; do
  ${LDs[$i]} -o lib1.${SFs[$i]}.so 1.o -shared --version-script vs.t
  clang-20 -o main.${SFs[$i]}.out main.o lib1.${SFs[$i]}.so --ld-path=$(which ${LDs[$i]})
  echo "${SFs[$i]}:"
  llvm-readelf -r main.${SFs[$i]}.out | grep foo
  echo ""
done
```

Output:

```
eld:
0000000000002230  0000000700000007 R_X86_64_JUMP_SLOT     0000000000000000 foo@V1 + 0

lld:
```

```
0000000000003a58  0000000600000007 R_X86_64_JUMP_SLOT     0000000000000000 foo@V1 + 0
0000000000003a68  0000000800000007 R_X86_64_JUMP_SLOT     0000000000000000 foo@V1 + 0

bfd:
0000000000004008  0000000500000007 R_X86_64_JUMP_SLOT     0000000000000000 foo@V1 + 0
```

8.2.2. Versioned symbols symbol-resolution examples

Symbol resolution of versioned-symbols is largely the same as non-versioned symbols. However, there are (corner) cases that may seem surprising. Here we will see some examples of symbol resolution of versioned symbols. We will also note differences from GNU ld (2.46.50) and lld (22.0) wherever relevant.

In these examples (A) and (B) are simply used to differentiate symbols and to avoid any confusion when referring to them. Please note that (A) and (B) does not mean that the symbols come from different input files.

The order matters in these examples, (A) `bar@@V1`, (B) `bar@V2`, means that the linker first sees `bar@@V1` and then sees `bar@V2`.

The symbols originates from regular object files unstated otherwise.

1. (A) `bar@@V1`, (B) `bar@V1`

multiple definition error.

1. (A) `bar@@V1`, (B) weak `bar@V1`

(A) `bar@@V1` is used to resolve undefined references to `bar@V1` and `bar`.

1. (A) weak `bar@@V1`, (B) `bar@V1`

(B) `bar@V1` is used to resolve undefined references to `bar@V1` and `bar`.

(B) `bar@V1` becomes a default-versioned symbol, and appears as `bar@@V1` in the dynamic symbol table.

1. (A) weak `bar@@V1`, (B) `bar`

(B) `bar` is used to resolve undefined references to `bar@V1` and `bar`.

(B) `bar` becomes a default-versioned symbol, and appears as `bar@@V1` in the dynamic symbol table.

1. (A) weak `bar@@V1`, (B) `bar@@V1`

(B) `bar@@V1` is used to resolve undefined references to `bar@V1` and `bar`.

1. (A) `bar@@V1`, (B) `bar@@V2`

eld and bfd errors out with multiple definition error.

lld incorrectly links correctly. Reproducer:

```
#!/usr/bin/env bash

cat >1.c <<\EOF
__asm__(".symver foo1,foo@@V1");
int foo1() {
  return 1;
}

__asm__(".symver foo2,foo@@V2");
int foo2() {
  return 3;
}

__asm__(".symver bar,bar@V2");
int bar() {
  return 5;
}
EOF

cat >main.c <<\EOF
```

```
#include <stdio.h>

int foo();
int bar();

int main() {
  int u = foo() + bar();
  printf("u: %d\n",u);
  return 0;
}
EOF

cat >vs.t <<\EOF
V1 {
  global:
    foo;
};

V2 {
  global:
    foo;
};
EOF

clang-20 -o 1.o 1.c -c -fPIC
clang-20 -o main.o main.c -c

LDs=(ld.eld ld.lld ld.bfd)
SFs=(eld lld bfd)

for i in "${!SFs[@]}"; do
  clang-20 -o lib.${SFs[$i]}.so 1.o --ld-path=$(which ${LDs[$i]}) -shared -fPIC -Wl,--version-script,vs.t
done
```

1. (A) weak `bar@@V1`, (B) `bar`, (C) `bar@V1`

multiple definition error.

1. (A) weak `bar@@V1`, (B) weak `bar`, (C) `bar@V1`

(C) `bar@V1` is used to resolve undefined references to both `bar` and `bar@V1`.

1. (A) weak `bar@@V1`, (B) `bar`, and version script that assigns `V2` to `bar`.

(B) `bar` becomes a default versioned symbol and appears as `bar@@V1` in the dynamic symbol table. **There will be no `bar@@V2` in the symbol table!**

reproducer:

```
#!/usr/bin/env bash

cat >1.c <<\EOF
__asm__(".symver bar1, bar@@V1");
__attribute__((weak))
int bar1() {
  return 1;
}
EOF

cat >2.c <<\EOF
int bar() {
  return 3;
}
EOF

cat >3.c <<\EOF
int bar();

__asm__(".symver bar3v1, bar@V1");
```

```
int bar3v1();

int b() {
  return bar() + bar3v1();
}
EOF

cat >vs.t <<\EOF
V2 {
  global:
    bar;
};

V1 {
  global:
    bar;
};
EOF

cat >main.c <<\EOF
#include <stdio.h>
#define show(x) printf("%s: %d\n", #x, x);

int b();
int bar();

__asm__(".symver bar1, bar@V1");
int bar1();

int b_main() {
  return bar() + bar1();
}

int main() {
 show(b());
 show(b_main());
 return 0;
}
EOF

TARGET="x86_64-linux-gnu"

clang -target ${TARGET} -o 1.o 1.c -c -fPIC
clang -target ${TARGET} -o 2.o 2.c -c -fPIC
clang -target ${TARGET} -o 3.o 3.c -c -fPIC
clang -target ${TARGET} -o main.o main.c -c

LDs=(ld.eld ld.lld ld.bfd)
SFs=(eld lld bfd)

for i in "${!SFs[@]}"; do
  ${LDs[$i]} -o lib12.${SFs[$i]}.so 1.o 2.o 3.o --version-script vs.t -shared
  clang -target ${TARGET} --ld-path=$(which ${LDs[$i]}) -o main.${SFs[$i]}.out main.o lib12.${SFs[$i]}.so
done
```

1. (A) `bar`, (B) weak `bar@@V1`, and version script that assigns `V2` to `bar`.

With GNU ld, the dynamic symbol table contains both `bar@@V1` and `bar@@V2`, and with lld and eld, the dynamic symbol table only contains `bar@@V1`.

Additionally, GNU seems to incorrectly resolves `bar@V1` undefined references to weak `bar@@V1`, instead of the plain `bar`. LLD and eld correctly behaves here.

Reproducer:

```
#!/usr/bin/env bash

cat >1.c <<\EOF
```

```
__asm__(".symver bar1, bar@@V1");
__attribute__((weak))
int bar1() {
  return 1;
}

int bar() {
  return 5;
}
EOF

cat >2.c <<\EOF
int baz() {
  return 3;
}
EOF

cat >3.c <<\EOF
int bar();

__asm__(".symver bar3v1, bar@V1");
int bar3v1();

int b() {
  return bar() + bar3v1();
}
EOF

cat >vs.t <<\EOF
V2 {
  global:
    bar;
};

V1 {
  global:
    bar;
};
EOF

cat >main.c <<\EOF
#include <stdio.h>
#define show(x) printf("%s: %d\n", #x, x);

int b();
int bar();

__asm__(".symver bar1, bar@V1");
int bar1();

int b_main() {
  return bar() + bar1();
}

int main() {
 show(b());
 show(b_main());
 return 0;
}
EOF

TARGET="x86_64-linux-gnu"

clang -target ${TARGET} -o 1.o 1.c -c -fPIC
clang -target ${TARGET} -o 2.o 2.c -c -fPIC
clang -target ${TARGET} -o 3.o 3.c -c -fPIC
clang -target ${TARGET} -o main.o main.c -c
```

```
LDs=(ld.eld ld.lld ld.bfd)
SFs=(eld lld bfd)

for i in "${!SFs[@]}"; do
  ${LDs[$i]} -o lib12.${SFs[$i]}.so 1.o 2.o 3.o --version-script vs.t -shared
  clang -target ${TARGET} --ld-path=$(which ${LDs[$i]}) -o main.${SFs[$i]}.out main.o lib12.${SFs[$i]}.so
done
```

1. (A) bar, (B) `bar@@V1`, and version script that assigns `bar` to the version node `V2`.

With GNU ld, the dynamic symbol table contains both `bar@@V1` and `bar@@V2`, and with lld, the dynamic symbol table only contains `bar@@V1`.

> [!IMPORTANT] eld currently reports multiple definition error.

1. (A) `bar@@V1`, (B) `bar`, and version script that assigns `bar` to the version node `V2`.

Multiple definition error.

9. x86-64 Relocation Reference

9.1. Introduction

This document describes the x86-64 relocations handled by ELD, including their encoding types, operations, range checks, and dynamic relocation behaviour.

The official ABI specification is the System V AMD64 ABI: https://gitlab.com/x86-psABIs/x86-64-ABI

9.2. Relocation Conventions

Symbol	Meaning
S	Runtime address of the referenced symbol
A	Relocation addend
P	Address of the relocation site
B	Base address of the shared object (for RELATIVE)
GOT	Address of the symbol's entry in the Global Offset Table
PLT	Address of the symbol's entry in the Procedure Linkage Table
TP	Thread pointer (base of the TLS block for the current thread)
DTP	Dynamic thread pointer (base of the TLS module block)
TLS	Size of the TLS template (used to compute TPOFF)

9.3. Encoding Types

x86-64 uses only flat-field encodings — no bit-scrambling.

Encoding	Value bits written	Field size
EncTy_None	none	—
EncTy_8	X[7:0] → inst[7:0]	8-bit
EncTy_16	X[15:0] → inst[15:0]	16-bit
EncTy_32	X[31:0] → inst[31:0]	32-bit
EncTy_64	X[63:0] → inst[63:0]	64-bit

9.4. Range Checks

Two distinct checks are applied, depending on the relocation:

`VerifyRange` (hard error): Enabled only for specific relocations (see table). Checks whether the computed value `X = result >> Shift` fits in the encoding field. Signed relocations use `llvm::isInt<N>`, unsigned use `llvm::isUInt<N>`.

Truncation check (`forceVerify`, diagnostic only): Enabled when the user passes `--verify-reloc <name>`. Checks whether the unsigned result overflows the field width. No error is raised unless explicitly requested.

The table below lists the exact bounds for each relocation.

Relocation	Encoding	Signed	VerifyRange	Effective bound
R_X86_64_NONE	EncTy_None	—	no	—
R_X86_64_64	EncTy_64	no	no	X truncated to 64 bits (no check)
R_X86_64_32	EncTy_32	no	no	X truncated to 32 bits; overflow possible
R_X86_64_32S	EncTy_32	yes	no	X truncated to 32 bits; sign-extended on load
R_X86_64_16	EncTy_16	no	no	X truncated to 16 bits; overflow possible
R_X86_64_8	EncTy_8	no	no	X truncated to 8 bits; overflow possible
R_X86_64_PC32	EncTy_32	no	no	X truncated to 32 bits
R_X86_64_PC16	EncTy_16	no	no	X truncated to 16 bits
R_X86_64_PC8	EncTy_8	no	no	X truncated to 8 bits
R_X86_64_PC64	EncTy_64	no	no	X truncated to 64 bits
R_X86_64_PLT32	EncTy_32	yes	no	X truncated to 32 bits
R_X86_64_GOT32	EncTy_32	no	no	X truncated to 32 bits
R_X86_64_GOTPCREL	EncTy_32	yes	no	X truncated to 32 bits
R_X86_64_GOTPCRELX	EncTy_32	yes	no	X truncated to 32 bits
R_X86_64_REX_GOTPCRELX	EncTy_32	yes	no	X truncated to 32 bits
R_X86_64_TLSGD	EncTy_32	yes	yes	−2^31 ≤ X ≤ 2^31 − 1
R_X86_64_TLSLD	EncTy_32	yes	yes	−2^31 ≤ X ≤ 2^31 − 1
R_X86_64_DTPOFF32	EncTy_32	yes	yes	−2^31 ≤ X ≤ 2^31 − 1
R_X86_64_GOTTPOFF	EncTy_32	yes	yes	−2^31 ≤ X ≤ 2^31 − 1
R_X86_64_TPOFF32	EncTy_32	yes	yes	−2^31 ≤ X ≤ 2^31 − 1
R_X86_64_DTPOFF64	EncTy_64	yes	no	X truncated to 64 bits
R_X86_64_TPOFF64	EncTy_64	yes	no	X truncated to 64 bits
R_X86_64_DTPMOD64	EncTy_64	no	no	filled by dynamic linker

Why only TLS 32-bit relocations have `VerifyRange`: These relocations encode offsets within a TLS block or a PC-relative offset to a GOT entry, both of which must fit in a sign-extended 32-bit immediate field in the instruction encoding. A value outside `[-2^31, 2^31 - 1]` would silently truncate and produce a wrong address at runtime. For general data and PC-relative relocations the ABI permits truncation; for TLS it is always an error.

Implicit truncation for unsigned relocations: `R_X86_64_32`, `R_X86_64_16`, and `R_X86_64_8` silently truncate the result to the field width. Use `--verify-reloc R_X86_64_32` (etc.) to promote truncation to a diagnostic.

9.5. Relocation Table

9.5.1. Absolute Relocations

The computed value `S + A` is written into the relocation site.

Relocation	Type	Encoding	Operation	Signed	Range check
R_X86_64_NONE	0	EncTy_None	—	—	—
R_X86_64_64	1	EncTy_64	S + A	no	truncated to 64 bits
R_X86_64_32	10	EncTy_32	S + A	no	truncated to 32 bits
R_X86_64_32S	11	EncTy_32	S + A	yes	truncated to 32 bits, sign-extended on load
R_X86_64_16	12	EncTy_16	S + A	no	truncated to 16 bits
R_X86_64_8	14	EncTy_8	S + A	no	truncated to 8 bits

`R_X86_64_32` vs `R_X86_64_32S`: Both write 32 bits. `R_X86_64_32` zero-extends (result must fit in `[0, 2^32)`); `R_X86_64_32S` sign-extends (result must fit in `[-2^31, 2^31)`). The compiler emits `32S` for addresses that are sign-extended when loaded into 64-bit registers, such as data or code in the low 2 GiB.

9.5.2. PC-Relative Relocations

The computed value `S + A - P` is written. `P` is the address of the relocation site; the caller's instruction already adds `P` back at runtime, so the net effect is a direct reference to `S + A`.

Relocation	Type	Encoding	Operation	Signed	Range check
R_X86_64_PC32	2	EncTy_32	S + A - P	no	truncated to 32 bits
R_X86_64_PC16	13	EncTy_16	S + A - P	no	truncated to 16 bits
R_X86_64_PC8	15	EncTy_8	S + A - P	no	truncated to 8 bits
R_X86_64_PC64	24	EncTy_64	S + A - P	no	truncated to 64 bits
R_X86_64_PLT32	4	EncTy_32	PLT(S) + A - P	yes	truncated to 32 bits

`R_X86_64_PLT32`: Used for function calls (`call foo`). If `foo` is non-preemptible the linker writes `S + A - P` directly. If `foo` is preemptible a PLT entry is created and the formula becomes `PLT(S) + A - P`.

9.5.3. GOT-Relative Relocations

These embed the PC-relative offset to the symbol's GOT entry. At runtime the instruction loads the actual symbol address from the GOT.

Relocation	Type	Encoding	Operation	Signed	Range check
R_X86_64_GOT32	3	EncTy_32	GOT(S) + A	no	truncated to 32 bits
R_X86_64_GOTPCREL	9	EncTy_32	GOT(S) + A - P	yes	truncated to 32 bits
R_X86_64_GOTPCRELX	41	EncTy_32	GOT(S) + A - P	yes	truncated to 32 bits
R_X86_64_REX_GOTPCRELX	42	EncTy_32	GOT(S) + A - P	yes	truncated to 32 bits

`GOTPCRELX` / `REX_GOTPCRELX`: Relaxable variants of `GOTPCREL`. `GOTPCRELX` is used without a REX prefix; `REX_GOTPCRELX` is used with one. When the symbol is non-preemptible the linker may rewrite the instruction sequence to avoid the GOT load entirely (see Relaxations).

9.5.4. Dynamic / Runtime Relocations

Written by the dynamic linker at load time; not applied statically by ELD.

Relocation	Type	Encoding	Operation
R_X86_64_COPY	5	EncTy_None	Copy symbol value from DSO into the executable
R_X86_64_GLOB_DAT	6	EncTy_64	S — resolve GOT entry to symbol address
R_X86_64_JUMP_SLOT	7	EncTy_64	S — lazy PLT resolution
R_X86_64_RELATIVE	8	EncTy_64	B + A — base-relative fixup for non-preemptible symbols in PIC

9.5.5. TLS Relocations

9.5.6. General Dynamic (GD) — `R_X86_64_TLSGD`

Calls `__tls_get_addr` with a two-word GOT descriptor (module ID + DTP offset). Works in all contexts.

Relocation	Type	Encoding	Operation	Signed	Range check
R_X86_64_TLSGD	19	EncTy_32	GOT(S) + A - P	yes	$-2^{31} \le X \le 2^{31} - 1$ (hard error)

```
leaq  x@tlsgd(%rip), %rdi      # R_X86_64_TLSGD; GOT holds {dtpmod, dtpoff}
call  __tls_get_addr@PLT
# %rax now holds &x in the current thread's TLS block
```

9.5.7. Local Dynamic (LD) — `R_X86_64_TLSLD`

Like GD but for symbols local to the module; one `__tls_get_addr` call covers all LD symbols in the module.

Relocation	Type	Encoding	Operation	Signed	Range check
R_X86_64_TLSLD	20	EncTy_32	GOT(S) + A - P	yes	−2^31 ≤ X ≤ 2^31 − 1 (hard error)

```
leaq  x@tlsld(%rip), %rdi     # R_X86_64_TLSLD; GOT holds {dtpmod, 0}
call  __tls_get_addr@PLT
# %rax is the base of this module's TLS block
movl  x@dtpoff(%rax), %eax    # R_X86_64_DTPOFF32: DTP offset added by linker
```

9.5.8. Initial Exec (IE) — R_X86_64_GOTTPOFF

Loads the TP-relative offset from the GOT; no runtime call.

Relocation	Type	Encoding	Operation	Signed	Range check
R_X86_64_GOTTPOFF	22	EncTy_32	GOT(S) + A - P	yes	−2^31 ≤ X ≤ 2^31 − 1 (hard error)

```
movq  x@gottpoff(%rip), %rax   # R_X86_64_GOTTPOFF; GOT[x] = tpoff(x)
movl  %fs:(%rax), %eax         # load x relative to TP
```

9.5.9. Local Exec (LE) — R_X86_64_TPOFF32 / R_X86_64_TPOFF64

Encodes the TP-relative offset directly; fastest model, only valid in executables.

Relocation	Type	Encoding	Operation	Signed	Range check
R_X86_64_TPOFF32	23	EncTy_32	S + A - TLS	yes	−2^31 ≤ X ≤ 2^31 − 1 (hard error)
R_X86_64_TPOFF64	18	EncTy_64	S + A - TLS	yes	truncated to 64 bits

```
movl  %fs:x@tpoff, %eax        # R_X86_64_TPOFF32: offset baked in at link time
```

9.5.10. DTP Offset Relocations

Used inside GOT entries created for GD/LD access.

Relocation	Type	Encoding	Operation	Signed	Range check
R_X86_64_DTPMOD64	16	EncTy_64	TLS module index (filled by dynamic linker)	no	—
R_X86_64_DTPOFF64	17	EncTy_64	S + A — DTP-relative offset	yes	truncated to 64 bits
R_X86_64_DTPOFF32	21	EncTy_32	S + A — DTP-relative offset (32-bit)	yes	−2^31 ≤ X ≤ 2^31 − 1 (hard error)

9.6. Dynamic Relocation Behaviour

ELD emits dynamic relocations during link when static resolution is not possible. The table below summarises when each dynamic relocation type is created.

Dynamic relocation	Trigger condition
R_X86_64_RELATIVE	Non-preemptible symbol referenced by R_X86_64_64 in PIC/PIE; or non-preemptible symbol referenced by GOTPCREL*
R_X86_64_GLOB_DAT	Preemptible symbol with a GOT entry (GOTPCREL*)
R_X86_64_64	Preemptible symbol referenced by R_X86_64_64 in a shared object
R_X86_64_JUMP_SLOT	Symbol referenced via PLT (PLT32) in a shared object
R_X86_64_COPY	Global data symbol referenced from a non-PIC executable
R_X86_64_TPOFF64	TLS IE/LD symbol in a shared object (GOTTPOFF)
R_X86_64_DTPMOD64	TLS GD/LD first GOT entry (module ID)
R_X86_64_DTPOFF64	TLS GD second GOT entry for preemptible symbol

9.7. Relaxations

x86-64 supports GOT-load relaxation for `GOTPCRELX` and `REX_GOTPCRELX` when the referenced symbol is non-preemptible. The linker rewrites the instruction in-place; no separate flag is required.

9.7.1. GOTPCRELX Relaxation (no REX prefix)

Applies to instructions that load or call through the GOT without a REX prefix (typically 32-bit operand-size or short-form instructions).

`call *foo@GOTPCREL(%rip)` → `call foo`

```
# Before (FF /2 — indirect call through GOT, 6 bytes)
call   *foo@GOTPCREL(%rip)      # R_X86_64_GOTPCRELX
                                # encodes: FF 15 <pcrel32 to GOT slot>

# After (E8 — direct call, 5 bytes + 1 NOP)
call   foo                      # direct; R_X86_64_PC32 applied to offset
nop                             # 1-byte filler to preserve instruction length
```

`jmp *foo@GOTPCREL(%rip)` → `jmp foo`

```
# Before (FF /4 — indirect jump, 6 bytes)
jmp    *foo@GOTPCREL(%rip)      # R_X86_64_GOTPCRELX

# After (E9 — direct jump, 5 bytes + 1 NOP)
jmp    foo
nop
```

`mov foo@GOTPCREL(%rip), %reg` → `lea foo(%rip), %reg`

```
# Before: load address from GOT
movq   foo@GOTPCREL(%rip), %rax # R_X86_64_GOTPCRELX

# After: compute address directly (non-preemptible, no GOT needed)
leaq   foo(%rip), %rax          # R_X86_64_PC32
```

9.7.2. REX_GOTPCRELX Relaxation (REX prefix present)

Applies to 64-bit forms of the same instructions (REX.W prefix).

`movq foo@GOTPCREL(%rip), %reg` → `leaq foo(%rip), %reg`

```
# Before (REX.W + 8B /5 — 64-bit GOT load, 7 bytes)
movq   foo@GOTPCREL(%rip), %rax # R_X86_64_REX_GOTPCRELX

# After (REX.W + 8D /5 — 64-bit LEA, 7 bytes)
leaq   foo(%rip), %rax          # R_X86_64_PC32
```

`movq foo@GOTPCREL(%rip), %reg` → `movq $imm32, %reg` (small absolute)

```
# Before (REX.W + 8B — GOT load, 7 bytes)
movq   foo@GOTPCREL(%rip), %rax # R_X86_64_REX_GOTPCRELX

# After (REX.W + C7 — sign-extended 32-bit immediate, 7 bytes)
movq   $foo, %rax               # R_X86_64_32S; valid when foo fits in 32 bits
```

9.8. Common Instruction Sequences

9.8.1. Absolute Address (64-bit)

```
movabsq  $symbol, %rax          # R_X86_64_64
```

9.8.2. Absolute Address (32-bit zero-extend)

```
movl   $symbol, %eax            # R_X86_64_32; valid when symbol < 2^32
```

9.8.3. PC-Relative Data Access

```
leaq  symbol(%rip), %rax       # R_X86_64_PC32
```

9.8.4. Function Call (direct or via PLT)

```
call  foo                      # R_X86_64_PLT32
                               # linker uses direct offset if non-preemptible,
                               # PLT stub otherwise
```

9.8.5. GOT-Indirect Load

```
movq  foo@GOTPCREL(%rip), %rax # R_X86_64_REX_GOTPCRELX (or GOTPCREL)
movq  (%rax), %rax             # dereference the loaded address
```

9.8.6. TLS Local-Exec

```
movl  %fs:x@tpoff, %eax        # R_X86_64_TPOFF32
```

9.8.7. TLS Initial-Exec

```
movq  x@gottpoff(%rip), %rax   # R_X86_64_GOTTPOFF
movl  %fs:(%rax), %eax
```

9.9. References

- System V AMD64 ABI specification: https://gitlab.com/x86-psABIs/x86-64-ABI
- Intel 64 and IA-32 Architectures Software Developer Manuals: https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html

User guide

- ELD User Guide (PDF)

Indices and tables

- Index
- Module Index
- Search Page
- Build Dashboard

Index

E